Graph Hunter

Algoritmos

Lucas Sotomayor
Base4Security

Mayo 2026

Prefacio

Lo que vas a leer.

Este libro es la guía pedagógica de los algoritmos, modelos y trucos de sistemas que hacen que Graph Hunter —el motor de *threat hunting* basado en grafos temporales que desarrollamos en Base4Security— corra sub-10 ms sobre grafos de 100K aristas y sobreviva sesiones de 28 GB de EVTX en una notebook. La meta no es documentar el código (de eso se ocupan los libros *Arquitectura* y *Especificación*), ni enseñar a operarlo (*Manual*), ni defenderlo académicamente (*Paper*). La meta es que un lector con *background* de CS de primer o segundo año pueda entender, desde cero, *por qué* cada decisión es la que es.

¿A quién está dirigido?

A tres audiencias bien delimitadas:

- **Estudiantes de Ciencias de la Computación** que ya saben qué es un grafo dirigido, big-O, recursión, *hash tables*, y han programado en algún lenguaje de sistemas (C, C++, Rust o Go). Para vos este libro es un *textbook* de un sistema real: no es un toy-example didáctico, es la mecánica interna de un motor que sostiene casos de fraude reales.
- **Ingenieros de *platforms* de seguridad** (*detection engineers*, *SIEM* engineers, *XDR* devs) que vienen de full-text search y quieren entender qué cambia cuando el modelo de datos pasa de tabla a grafo temporal.
- **Devs nuevos al equipo de Graph Hunter** que necesitan ramp-up rápido sin tener que leer 50.000 líneas de Rust antes de proponer una optimización. Este libro es la versión larga de la onboarding chat.

Prerequisitos

Asumimos que:

- Sabés qué es un *grafo dirigido* y un *árbol*, distinguís *path* de *walk*, y entendés *depth-first search* y *breadth-first search* a nivel pseudocódigo.
- Manejás notación big-O y entendés que "$O(m \cdot \deg^k)$" significa algo concreto sobre cómo escala un algoritmo.
- Conocés *hash tables* ($O(1)$ amortizado, función de hash, colisiones) y *arrays*.
- Programaste alguna vez en un lenguaje de sistemas y sabés qué es un puntero, qué pasa cuando se desreferencia, y por qué asignar memoria no es "gratis".

No asumimos:

- Conocimiento previo de bases de datos relacionales o álgebra relacional. Cuando aparezca un *join*, lo definimos (capítulo 4).

- Familiaridad con *worst-case-optimal joins*, AGM bound, Yannakakis, GYO, hipergrafos. Construimos cada uno desde cero.
- Conocimiento de *memory-mapped files*, *cache lines*, *branch prediction*. El capítulo 5 los cubre antes de que los necesites.
- Familiaridad con Rust. Los *listings* de código son auto-explicativos para alguien acostumbrado a un *C-family language*; cuando una construcción de Rust (⟨ `Arc`, `RwLock`, `repr(C)` ⟩) sea relevante para el argumento, la explicamos en el momento.

Cómo está organizado el libro

Parte I — Fundamentos. Cinco capítulos que cubren todo lo que necesitás saber *para poder leer el resto del libro*: el paradigma de threat hunting, la formalización de grafos temporales, el problema de subgraph matching, los joins relacionales (con derivación didáctica del AGM bound), y los fundamentos de sistemas que justifican las decisiones de *layout* y memoria del motor.

Partes II–III — Arquitectura y modelo de datos. Tres capítulos de arquitectura (capas, pipeline) y cuatro del modelo de datos: `Entity`, `Relation`, el *string interner*, `CompactRelation` de 32 bytes, y la migración del `HuntResult` a `Vec<Arc<str>>`.

Parte IV — Pipeline de ingesta. De *log* a grafo temporal: parsers, *streaming mirror*, *lazy build* de índices, NLF incremental.

Parte V — El *matcher*, paso a paso. El núcleo algorítmico del libro. Diez capítulos que construyen el zoológico de operadores (DFS legacy con *prunes*, LFTJ-K_4, Yannakakis chain, Yannakakis tree-walk) desde el DFS recursivo más básico hasta los *worst-case-optimal joins*.

Parte VI — Índices auxiliares. NLF y *k-hop reachability* en profundidad: *layouts dense+sparse*, *mmap-spill*, *lazy build*, extensión temporal.

Parte VII — La DSL. Diseño de una *domain-specific language* chica: forma *chain* legacy, forma *graph* con el separador `;`, parser, AST, dispatch al *matcher*.

Parte VIII — Diseño futuro (L3). Las dos piezas pendientes de la *roadmap* L3: *delta khop_reach* usando propagación inversa estilo Kankanamge SIGMOD 2017, y *streaming* como *canonical store*.

Parte IX — Plug-and-play. La tesis transversal: cada optimización que entrega valor lo hace sin que el usuario tenga que tocar *config*. Cómo se materializa esa promesa en el *frontend* y la *telemetría*.

Apéndices. Notación, glosario, soluciones a ejercicios selectos, bibliografía.

Cómo leer este libro

- **Lectura lineal.** Si no conocés el motor, leélo de principio a fin. Cada capítulo asume los anteriores; los *cross-refs* son explícitos cuando un argumento depende de algo de otro capítulo.
- **Lectura por reference.** Si ya sabés de qué se trata y querés entender un componente concreto, andá directo al capítulo correspondiente. Cada uno arranca con una caja *Intuición* que resume el contenido en un párrafo, y termina con una sección *Ejercicios*; podés saltearte el cuerpo y volver a él si la intuición no alcanza.

- **Lectura para enseñar.** Cada capítulo está pensado para ser una clase de 50–90 minutos. Los ejercicios marcados con ★ tienen solución en el Apéndice C; los demás son abiertos para que el lector se trabe un poco antes de pedir ayuda.

Convenciones tipográficas

- Las cajas **Intuición** (verde claro) preceden a la formalización. Si entendés la intuición, podés saltearte la prueba.
- Las cajas **Ejemplo** (borde verde claro, fondo blanco) contienen ejemplos trabajados extensos paso a paso.
- `Texto monoespaciado` es código o nombre de tipo / función / archivo del repo.
- *Texto en cursiva* marca términos técnicos en su primera aparición o préstamos lingüísticos del inglés (*path*, *join*, *spill*).
- $\mathcal{V}$, $\mathcal{E}$, $\mathcal{T}$, $\mathcal{L}$ son los conjuntos de vértices, aristas, tiempo y etiquetas respectivamente. Convenciones completas en Apéndice A.
- `core/graph-engine/src/lftj.rs` (con fuente monoespaciada) cita un archivo del repo. Las cajas **En el código** apuntan del concepto al fichero real cuando eso ayuda al lector a auditar la implementación.
- Los *algorithm blocks* usan pseudocódigo estilo CLRS: `for`, `if`, `while`, indentación por bloque, asignación con ←. Convenciones completas en la sección *Convenciones* (a continuación) y en el Apéndice A.

Relación con los otros libros

GraphHunter tiene seis libros. Cada uno cubre una porción distinta:

Libro	Cubre
Algoritmos	(este libro) — explicación pedagógica desde lo básico, con ejemplos y ejercicios.
Diseño	resumen ejecutivo del diseño en $\sim$ 25 páginas, asumiendo que el lector ya conoce el dominio.
Arquitectura	estructura de *crates*, *layering*, ADRs, decisiones de *repo organization*.
Especificación	*wire contracts* (HTTP API, MCP, API pública del matcher en Rust, formatos de *log*, *env vars*).
Manual	guía operacional para el analista que usa el motor.
Paper	justificación académica con *benchmarks* y comparaciones contra el estado del arte.

Si una decisión te resulta opaca acá, casi seguro la otra cara (*wire contract*, *benchmark*, ADR) está en uno de los otros cinco libros, y el *cross-ref* explícito te lleva ahí.

Lucas Sotomayor
Base4Security
Mayo 2026

Convenciones

Este capítulo no se lee de corrido: es una referencia para volver cuando aparezca una notación que no reconozcas. Si querés arrancar con el contenido, saltá al capítulo 1.

Notación matemática

- $G = (\mathcal{V}, \mathcal{E})$ — un grafo dirigido, donde $\mathcal{V}$ es el conjunto de vértices y $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ el de aristas. Para grafos temporales, las aristas son ternas (u, v, t) con $t \in \mathcal{T}$; el *multigrafo* permite que existan varias aristas entre el mismo par (u, v) con distintos t.
- $|\mathcal{V}| = n$, $|\mathcal{E}| = m$. Convención CLRS estándar.
- $\mathcal{L}$ — alfabeto de etiquetas (en GraphHunter: `EntityType` para vértices, `RelationType` para aristas).
- $\deg(v)$, $\text{in-deg}(v)$, $\text{out-deg}(v)$ — grado, grado de entrada y de salida del vértice v.
- $N(v)$, $N^+(v)$, $N^-(v)$ — vecindad, vecindad saliente y vecindad entrante.
- $\text{NLF}(v, L)$ — *Neighborhood Label Frequency*: número de vecinos salientes de v con etiqueta L. Definida formalmente en el capítulo 27.
- $\text{bits}[v]$ — bitmap de $|\mathcal{L}|$ bits asociado a v, donde el bit L está prendido sii existe un camino válido de v a algún vértice de tipo L. Definido en el capítulo 28.
- $\text{AGM}(Q)$ y $\rho^*(Q)$ — *Atserias-Grohe-Marx bound* y *fractional edge cover number* de la consulta Q. Definidos y derivados en el capítulo 20.

Notación del pseudocódigo

Los algoritmos siguen convención CLRS, ligeramente adaptada:

- Asignación: $x \leftarrow y$. Igualdad: $x = y$.
- Acceso a campos: v.campo (notación punto, igual que un *struct* en C/Rust).
- Bloques `for/while/if/else` delimitados por indentación más `end` explícito (igual que algorithm2e).
- `return` sale del procedimiento.
- `emit` agrega un resultado a la salida del enumerador (sin retornar).
- Conjuntos: $\{x : P(x)\}$ (comprensión), $A \cup B$, $A \cap B$, $A \setminus B$, $\emptyset$.
- Operaciones *bitwise*: $a \mid b$ (*or*), $a \mid\!= b$ (*or-assign*), $a \,\&\, b$ (*and*). Cuando aparezcan, los explicamos en el momento.

Cajas y entornos visuales

Intuición

Los párrafos importantes en lenguaje plano arrancan con una caja *Intuición* verde clara. Su propósito es que entiendas el *qué* y el *por qué* antes de leer el *cómo* formal. Si la prueba o el pseudocódigo se vuelven difíciles, releé la intuición —suele alcanzar para reconstruirlos.

Ejemplo

Las cajas *Ejemplo* (borde verde claro, fondo blanco) contienen un ejemplo trabajado paso a paso. Suelen incluir un diagrama TikZ y la traza de un algoritmo sobre una entrada chica.

En el código.

Los párrafos **En el código** apuntan del concepto al fichero real del repo donde está implementado. Por ejemplo: `core/graph-engine/src/lftj.rs`.

Nota. Las notas (sin numeración) son aclaraciones puntuales: una diferencia con un libro de referencia, una pista para el lector avanzado, una elección de diseño que vale la pena justificar.

Advertencia. Las advertencias señalan errores frecuentes. Léelas con cuidado: por ejemplo, "no confundir *path* con *walk*" es una trampa clásica del threat hunting (¿se permite repetir vértices?).

Teoremas, lemas, definiciones

Definición 0.1 (Plantilla)**.** Cada definición se enuncia con cuantificadores explícitos y la referencia al objeto matemático preciso. La numeración es por capítulo: Definicion $X.Y$ significa la Y-ésima definición del capítulo X.

Teorema 0.2 (Plantilla)**.** *Cada teorema enuncia formalmente una propiedad y la prueba sigue inmediatamente debajo. El símbolo* □ *cierra la prueba.*

Demostración. Esta es la prueba. □ □

Los *Lemas* son resultados auxiliares; los *Corolarios* consecuencias inmediatas; las *Proposiciones*, enunciados de medio peso (más fuertes que un lema, menos que un teorema).

Ejercicios

Ejercicios

Cada capítulo termina con 3–6 ejercicios. Tres tipos:

1. **Warm-up**: confirma que entendiste las definiciones. Suelen ser de 1–3 minutos.
2. **Derivaciones**: pedís *trabajar* un ejemplo nuevo o probar una variante chica del teorema. 10–30 minutos.
3. **Extensiones**: la solución requiere ir más allá del capítulo. Pueden ser problemas abiertos del motor; los abordamos en el capítulo siguiente o quedan como *food for thought*.

Los ejercicios marcados con ★ tienen solución en el Apéndice C. Los demás quedan abiertos por diseño.

Referencias cruzadas

- Los `\ref`s a capítulos son del estilo “capítulo 1”.
- Los `\ref`s a apéndices son del estilo “Apéndice A”.
- Las figuras se referencian con “figura X.Y” y los algoritmos con “algoritmo X.Y”.
- Las citas a literatura externa son nombre+año entre paréntesis: “(Veldhuizen, 2014)”. Las referencias completas están en la bibliografía (Apéndice D).

Índice general

Prefacio ii

Convenciones v

I Fundamentos 1

1. Threat hunting y por qué grafos 2
1.1. ¿Qué busca un *threat hunter*? . . . 2
1.2. Por qué un SIEM clásico no alcanza . . . 3
1.3. El cambio de modelo: de tablas a grafo temporal . . . 3
1.4. Por qué la representación importa . . . 4
1.5. Pero modelar como grafo no es magia . . . 4
1.6. Qué viene . . . 5

2. Grafos temporales: definiciones 6
2.1. Grafos dirigidos: el caso base . . . 6
2.2. Multigrafos . . . 6
2.3. Etiquetas en vértices y aristas . . . 7
2.4. Grafos temporales . . . 7
2.5. Caminos: *paths* vs *walks* . . . 7
2.6. Restricciones temporales . . . 8
2.6.1. Monotonicidad . . . 8
2.6.2. Ventana temporal . . . 8
2.6.3. Composición: *walk* de hipótesis . . . 8
2.7. Representación: *adjacency list* . . . 9
2.8. Resumen de notación . . . 9

3. Subgraph matching: el problema 11
3.1. Tres niveles de pregunta . . . 11
3.2. Patrón P versus dato G . . . 11
3.3. Embeddings: los matches del patrón . . . 12
3.3.1. Homomorfismo . . . 12
3.3.2. Isomorfismo (subgraph isomorphism) . . . 12
3.3.3. ¿Cuál es la convención de GraphHunter? . . . 12
3.4. Por qué el problema es duro en general . . . 12
3.4.1. Por qué entonces es tractable en la práctica . . . 13
3.5. Variante: subgraph matching temporal . . . 13
3.6. Operadores: qué algoritmo usa el motor . . . 13
3.7. Por qué el AGM bound importa . . . 14
3.8. Resumen . . . 14

4. Joins relacionales y su conexión con grafos **16**
4.1. Joins desde cero . . . 16
4.1.1. Relaciones . . . 16
4.1.2. Join binario natural . . . 16
4.1.3. Multi-way join . . . 17
4.2. Equivalencia: subgraph matching es multi-way join . . . 17
4.2.1. Por qué la equivalencia importa . . . 18
4.3. El AGM bound . . . 18
4.3.1. Motivación . . . 18
4.3.2. Cobertura fraccionaria . . . 18
4.3.3. El teorema . . . 19
4.3.4. Worst-case optimal joins . . . 19
4.4. Por qué entonces no toda hipótesis usa WCOJ . . . 20
4.5. Resumen . . . 20

5. Sistemas y memoria **22**
5.1. La jerarquía de memoria . . . 22
5.1.1. Cache lines . . . 22
5.1.2. Branch prediction . . . 23
5.1.3. SIMD . . . 23
5.2. Memoria virtual y mmap . . . 23
5.2.1. Memoria virtual: la abstracción del SO . . . 23
5.2.2. mmap . . . 24
5.3. Concurrencia: atomics, locks, y wait-free . . . 24
5.3.1. Lectura concurrente: el problema . . . 24
5.3.2. Operaciones atómicas . . . 25
5.3.3. Memory ordering . . . 25
5.3.4. Wait-free para el streaming backend . . . 25
5.4. Layout de datos en Rust . . . 25
5.4.1. `#[repr(C)]` y predictibilidad . . . 25
5.4.2. `Arc<str>` y *refcounting* . . . 26
5.5. Cómo todo esto vuelve en el resto del libro . . . 26

II Arquitectura del motor **28**

6. El pipeline en cuatro pasos **29**
6.1. Vista de pájaro . . . 29
6.2. Etapa 1: Ingesta . . . 29
6.3. Etapa 2: Hipótesis (parsing de la DSL) . . . 30
6.4. Etapa 3: Matching . . . 30
6.4.1. Por qué cuatro operadores y no uno . . . 31
6.5. Etapa 4: Scoring y paginación . . . 31
6.6. Walkthrough: “apreté Run” . . . 31
6.7. Resumen . . . 32

7. El stack de *crates* **34**
7.1. Diagrama de capas . . . 34
7.2. Por qué este orden . . . 35
7.3. Heurística: ¿en qué crate va esto? . . . 35
7.4. Cómo se enforce la regla . . . 35
7.5. Crates en detalle . . . 36
7.5.1. `core/` . . . 36

7.5.2. `platform/` . . . 36
7.5.3. `apps/` . . . 36
7.6. Por qué *platform* y no *infra* o *shared* . . . 36
7.7. Resumen . . . 37

III Modelo de datos 38

8. `Entity`, `Relation` y los tipos públicos 39
8.1. Tipos públicos . . . 39
8.2. `EntityType` y `RelationType`: alfabetos cerrados . . . 40
8.3. De evento Sysmon a triples . . . 40
8.4. El contrato del parser . . . 41
8.5. Idempotencia y deduplicación . . . 42
8.6. Metadata: cuándo materializa, cuándo no . . . 42
8.7. Resumen . . . 42

9. El *string interner* 44
9.1. El problema . . . 44
9.2. Definición . . . 44
9.3. Implementación: tabla bidireccional . . . 45
9.4. Análisis de costos . . . 45
9.4.1. `intern`: amortizado $O(1)$ . . . 45
9.4.2. `resolve`: $O(1)$ . . . 46
9.4.3. Espacio . . . 46
9.4.4. Comparaciones rápidas . . . 46
9.5. Variantes de diseño . . . 46
9.5.1. Por qué *no* usamos `Symbol` de string-interner crate . . . 46
9.5.2. Por qué *no* usamos un *trie* de prefix sharing . . . 46
9.6. Concurrencia . . . 46
9.7. Por qué importa para el resto del libro . . . 47
9.8. Resumen . . . 47

10. `CompactRelation`: 32 bytes por arista 49
10.1. El layout . . . 49
10.2. Por qué exactamente 32 bytes (y no 24, no 40) . . . 50
10.3. `#[repr(C)]`: por qué es obligatorio . . . 50
10.4. Conversión: `Relation` → `CompactRelation` . . . 51
10.5. Por qué el matcher itera `CompactRelation`, no `Relation` . . . 51
10.6. Resumen . . . 52

11. `HuntResult` y la migración a `Arc<str>` 53
11.1. La forma del resultado . . . 53
11.2. El bench que disparó el cambio . . . 53
11.3. Por qué `Vec<String>` era el problema . . . 54
11.4. Por qué la materialización diferida fracasó . . . 54
11.5. La solución: `Vec<Arc<str>>` . . . 55
11.6. Por qué `Arc<str>` en vez de `Rc<str>` . . . 55
11.7. Por qué `Arc<str>` en vez de `Arc<String>` . . . 55
11.8. Implicaciones para el resto del sistema . . . 55
11.8.1. Serialización . . . 55
11.8.2. FFI (histórico) . . . 56
11.8.3. Memoria total . . . 56

11.9. Cuándo *no* vale la pena `Arc<str>` . . . 56
11.10Resumen . . . 56

IV Pipeline de ingesta **58**

12. Anatomía del *ingest* **59**
12.1. Diagrama del pipeline . . . 59
12.2. Etapa 1: `LogSource` . . . 59
12.2.1. Implementaciones . . . 59
12.2.2. Watermarks . . . 60
12.3. Etapa 2: `LogParser` . . . 60
12.3.1. El parser es *event-to-N-triples* . . . 60
12.4. Etapa 3: `GraphWriter` . . . 60
12.5. El `SourcePoller` . . . 61
12.5.1. Concurrencia: ¿múltiples writers? . . . 61
12.6. Memoria del ingestor: peak vs total . . . 62
12.7. Provenance . . . 62
12.8. Resumen . . . 62

13. El *streaming backend*: *canonical store* **64**
13.1. Por qué un *streaming backend* y no un `Vec` . . . 64
13.2. La estructura: `NeighborList` degree-adaptive . . . 64
13.2.1. Por qué este shape y no la antigua `Btree` . . . 65
13.3. Promotion + auto-spill . . . 65
13.4. Wait-free reads: `neighbors_snapshot` . . . 66
13.4.1. Costo . . . 66
13.4.2. Memory reclamation . . . 66
13.5. ¿Cómo se usa hoy? . . . 67
13.6. Resumen . . . 67

14. Finalización: `sort_edges_by_timestamp` **69**
14.1. Por qué hace falta finalizar . . . 69
14.2. La operación `sort_edges_by_timestamp` . . . 69
14.2.1. Paso 1: sort per-vertex . . . 69
14.2.2. Paso 2: `edges_sorted = true` . . . 70
14.2.3. Paso 3: reset de `OnceLock` para lazy build . . . 70
14.3. Lazy build: el cambio que destrabó 28 GB . . . 70
14.3.1. La nueva versión . . . 70
14.3.2. Cómo `OnceLock` garantiza thread-safety . . . 71
14.4. Auto-spill por-vértice (cuando no entra en heap) . . . 71
14.5. ¿Por qué sort por (source_sid, timestamp) y no (timestamp, source_sid)? . . . 72
14.6. Resumen . . . 72

15. NLF incremental **73**
15.1. El problema . . . 73
15.2. La solución: `try_record_edge` . . . 73
15.2.1. Algoritmo: `add_relation` post stage 2a . . . 74
15.3. ¿Por qué *saturating add*? . . . 74
15.4. ¿Por qué `heap_slice_mut` y no acceso directo? . . . 74
15.4.1. Por qué no se hace mmap-mut . . . 75
15.5. Soundness . . . 75
15.6. El impacto medible . . . 76

15.7. Resumen 76

V El matcher, paso a paso 77

16. DFS recursivo: el *baseline* 78
16.1. DFS clásico sobre un grafo 78
16.1.1. Complejidad 79
16.2. De *visit-all* a *enumerate-paths* 79
16.2.1. Backtracking 80
16.3. Subgraph matching: agregar etiquetas 80
16.3.1. Complejidad 80
16.4. DFS temporal de GraphHunter 81
16.4.1. Las nuevas piezas 81
16.4.2. Por qué iterativo y no recursivo en producción 81
16.5. De DFS naive a DFS bien-podado 82
16.6. Resumen 82

17. Subgraph matching ingenuo: por qué no escala 83
17.1. El peor caso teórico 83
17.1.1. El gráfico que vale más que el álgebra 84
17.2. ¿Por qué scale-free empeora todo? 84
17.2.1. El *output* también explota 84
17.3. Tres caminos para escalar 85
17.3.1. Camino 1: Reducir *starts* con prunes 85
17.3.2. Camino 2: Cambiar el algoritmo (WCOJ) 85
17.3.3. Camino 3: Reducir antes de enumerar (Yannakakis) 85
17.4. ¿Cuándo el DFS naive es suficiente? 85
17.5. Resumen 85

18. Poda con NLF 87
18.1. Definición formal 87
18.2. El prune 88
18.3. Soundness 88
18.3.1. Prune más fuerte: chequear varios tipos 88
18.3.2. NLF en pasos intermedios 88
18.4. Cuánto poda en la práctica 88
18.4.1. Escenario 1: hipótesis con primer paso muy selectivo 89
18.4.2. Escenario 2: starts "hojas" 89
18.4.3. Escenario 3: hipótesis multi-hijo (shape graph) 89
18.5. Implementación: `NlfTable` 89
18.6. Construcción 90
18.7. Limitaciones 90
18.8. Resumen 90

19. Poda con *k-hop reachability* 92
19.1. Definición 92
19.2. El prune 92
19.3. Soundness 93
19.3.1. Cuando $k >$ `MAX_DEPTH` 93
19.4. Extensión temporal 93
19.4.1. El prune temporal 93
19.4.2. Cuándo aplica 93

19.5. Storage layout: dense + sparse . . . 94
19.6. Construcción . . . 94
19.7. Resumen . . . 95

20. *Worst-case-optimal joins* y el AGM *bound* **96**
20.1. Repaso del AGM bound . . . 96
20.2. Por qué los joins binarios pierden . . . 96
20.3. La idea del WCOJ: prefix-extension multi-way . . . 97
20.3.1. Aplicado al triángulo . . . 97
20.4. LFTJ: la versión implementable . . . 97
20.5. Por qué K_4 y no triángulo o cadena . . . 98
20.6. Resumen . . . 98

21. *Leapfrog Triejoin* para K_4 **100**
21.1. El patrón K_4 en GraphHunter . . . 100
21.2. La estructura del problema . . . 100
21.2.1. El DFS naive . . . 100
21.2.2. El plan LFTJ . . . 101
21.3. La operación *leapfrog* . . . 101
21.3.1. Por qué leapfrog es $O(\sum |L_i|)$ y no peor . . . 101
21.3.2. SIMD intersection . . . 102
21.4. El trie sobre las aristas . . . 102
21.4.1. Overhead vs DFS . . . 102
21.5. LFTJ para K_4: el algoritmo completo . . . 103
21.5.1. Análisis: $O(m^2)$ . . . 103
21.6. Symmetry breaking . . . 103
21.7. Cuándo LFTJ-K_4 gana en la práctica . . . 103
21.8. Resumen . . . 104

22. Hipergrafos, α-aciclicidad y *join trees* **105**
22.1. Hipergrafos: del join al *query graph* . . . 105
22.2. α-aciclicidad . . . 106
22.2.1. El algoritmo GYO . . . 106
22.3. El *join tree* . . . 106
22.4. Por qué α-aciclicidad importa . . . 107
22.5. La pieza estructural en GraphHunter . . . 107
22.6. Construcción concreta del join tree . . . 108
22.7. Resumen . . . 108

23. Yannakakis: cadena **110**
23.1. Setup: *bags* y la chain . . . 110
23.2. La semi-join reducer . . . 110
23.2.1. Pasada bottom-up . . . 110
23.2.2. Pasada top-down . . . 111
23.2.3. Diagrama . . . 111
23.3. Enumerar lo que quedó . . . 111
23.4. Correctitud . . . 112
23.5. Análisis de complejidad . . . 112
23.5.1. Costo del semi-join reducer . . . 112
23.5.2. Costo del enumerator . . . 112
23.5.3. Total . . . 112
23.6. Comparación con DFS . . . 112
23.6.1. Ejemplo concreto . . . 112

23.7. Cuando Yannakakis pierde . . . 113
23.8. Resumen . . . 113

24. Yannakakis: *tree-walk* **114**
24.1. El setup: variables, bindings, join tree . . . 114
24.2. El join tree para *shape graph* . . . 114
24.2.1. La estructura `JoinTree` . . . 114
24.3. El algoritmo . . . 115
24.4. Por qué el switch (origin_bound, dest_bound) . . . 116
24.4.1. Caso (None, Some(d)) . . . 116
24.4.2. Caso (Some, Some) . . . 116
24.5. Bindings y unbinding . . . 116
24.6. Análisis . . . 117
24.7. Cuándo se dispatcha . . . 117
24.8. Resumen . . . 117

25. El *planner* **119**
25.1. El `DispatchPlan` . . . 119
25.2. Las tres reglas . . . 119
25.2.1. Regla 1: `name_marker_dispatch` (back-compat S3) . . . 119
25.2.2. Regla 2: `structural_dispatch` (M1) . . . 120
25.2.3. Regla 3: `yannakakis_dispatch` (L1) . . . 120
25.3. ¿Por qué este orden? . . . 121
25.4. Cuándo el planner "se equivoca" . . . 121
25.5. Surface en UI: "Perf: M+L1" . . . 121
25.6. Agregar una regla nueva: el procedimiento . . . 121
25.7. Resumen . . . 122

VI Índices auxiliares en profundidad **123**

26. Diseño de índices auxiliares **124**
26.1. El contrato general . . . 124
26.2. Por qué "no source of truth" . . . 124
26.3. Soundness por sobre-aproximación . . . 125
26.4. Lazy build con `OnceLock` . . . 125
26.4.1. Invalidation . . . 125
26.5. La dualidad heap / mmap-spilled . . . 126
26.5.1. Cuándo se decide spillear . . . 126
26.6. El lifecycle completo de un índice . . . 127
26.7. Ejemplo concreto: cuánto cuesta cada estado . . . 127
26.8. Resumen . . . 127

27. NLF en detalle **129**
27.1. Layout . . . 129
27.1.1. Por qué `[u32; 9]` y no `HashMap<EntityType, u32>` . . . 129
27.2. El builder . . . 130
27.2.1. ¿Por qué `dest_type_tag` y no `source_type_tag`? . . . 130
27.3. Lookup: `can_match` . . . 130
27.3.1. Cómo lo invoca el matcher . . . 130
27.4. Memoria . . . 131
27.4.1. Cuándo spillea . . . 131
27.5. Incremental update revisitado . . . 131

27.6. ¿Y si en el futuro hace falta más resolución? . . . 132
27.7. Resumen . . . 132

28. *k-hop reachability* en detalle **133**
28.1. La estructura . . . 133
28.1.1. Por qué dense+sparse . . . 133
28.2. El builder: BFS reverse multi-source . . . 134
28.2.1. Por qué *reverse* BFS . . . 134
28.3. Lookup y prune . . . 134
28.4. Ejemplo numérico . . . 136
28.5. El `RwLock` en min/max_first . . . 136
28.6. Memoria efectiva . . . 137
28.7. Resumen . . . 137

VII La DSL **138**

29. Diseñando una DSL chica **139**
29.1. Por qué una DSL en vez de SQL / Cypher / KQL . . . 139
29.2. Principios de diseño . . . 139
29.2.1. Principio 1: legibilidad de izquierda a derecha . . . 139
29.2.2. Principio 2: un solo separador . . . 140
29.2.3. Principio 3: predicados como cláusulas post-fix . . . 140
29.2.4. Principio 4: defaults razonables . . . 140
29.3. La gramática (informal) . . . 140
29.4. Lo que la DSL *no* hace . . . 141
29.5. La sintaxis no es la semántica . . . 141
29.6. Resumen . . . 141

30. Sintaxis *chain* (legacy) **143**
30.1. Anatomía de una hipótesis chain . . . 143
30.2. Type bindings . . . 143
30.2.1. Casos donde el binding importa . . . 144
30.3. Predicados sobre la arista . . . 144
30.4. Predicados sobre el vértice destino . . . 144
30.5. Cláusulas trailing . . . 144
30.5.1. `WITHIN` . . . 144
30.5.2. `HAVING` . . . 144
30.5.3. `{k=N}` . . . 145
30.6. Ejemplos completos . . . 145
30.7. Cómo el parser construye la `Hypothesis` . . . 145
30.8. Resumen . . . 146

31. Sintaxis *graph*: el separador `;` **147**
31.1. El cambio mínimo: agregar `;` . . . 147
31.2. Reglas de las cláusulas tras `;` . . . 147
31.2.1. Bare binding reference . . . 147
31.2.2. Fresh type declaration . . . 148
31.2.3. Disambiguación . . . 148
31.3. Estructura interna del parser . . . 148
31.3.1. Cómo se lee cada nodo . . . 148
31.4. Ejemplos . . . 149
31.5. Limitaciones . . . 149

31.6. Resumen . . . 150

32. Del AST al *matcher* 151
32.1. La struct `Hypothesis` . . . 151
32.2. El `VarId` . . . 152
32.3. El campo `shape` . . . 152
32.4. Cómo lo construyen otros transports . . . 153
32.4.1. HTTP REST . . . 153
32.4.2. MCP . . . 153
32.4.3. Tauri (frontend) . . . 153
32.5. Estabilidad de la `Hypothesis` . . . 153
32.6. Validación . . . 154
32.7. Resumen . . . 154

VIII Diseño futuro (L3) 155

33. Por qué *khop_reach* delta es difícil 156
33.1. El problema en una frase . . . 156
33.2. Ejemplo: una arista, muchos cambios . . . 157
33.3. Por qué invalidar-y-rebuild es lento . . . 157
33.4. Por qué la propagación naive falla . . . 157
33.5. Por qué min/max_first es aún más difícil . . . 158
33.6. Soundness es la salvaguarda . . . 158
33.7. El plan: Delta WCOJ Propagation . . . 158
33.8. Resumen . . . 159

34. *Delta WCOJ Propagation* 160
34.1. El algoritmo . . . 161
34.2. Análisis paso a paso . . . 161
34.2.1. Round 1: efecto directo . . . 161
34.2.2. Rounds 2..MAX_DEPTH: propagación reverse . . . 162
34.2.3. Cascade budget . . . 162
34.3. Soundness . . . 162
34.3.1. Falsos positivos OK; falsos negativos NO . . . 162
34.4. Costo en mmap-spilled . . . 162
34.5. Sincronización readers/writer . . . 163
34.6. Tests de equivalencia . . . 163
34.7. Cuándo el delta supera al rebuild . . . 163
34.8. Cronograma estimado de implementación . . . 163
34.9. Resumen . . . 163

35. *Streaming* como almacenamiento canónico 165
35.1. El problema . . . 165
35.2. Por qué no se hizo en stage 2a . . . 166
35.3. El plano: tres pasos . . . 166
35.3.1. Paso 1: Adapter layer . . . 166
35.3.2. Paso 2: Re-write del hot path . . . 166
35.3.3. Paso 3: Drop del `edge_store` . . . 167
35.4. Beneficios esperados . . . 167
35.5. Riesgos y mitigaciones . . . 167
35.5.1. Regresión en matcher hot path . . . 167
35.5.2. Compatibilidad de *snapshot/load* . . . 167

35.5.3. Cache behavior de NeighborList variants . . . 167
35.6. Cronograma estimado . . . 168
35.7. Resumen . . . 168

IX Plug-and-play como tesis transversal 169

36.La promesa *plug-and-play* 170
36.1. La promesa, en una frase . . . 170
36.2. Por qué la regla es deliberada . . . 170
36.3. La regla en cinco pasos . . . 171
36.3.1. Paso 1: default-on al startup . . . 171
36.3.2. Paso 2: surface en UI . . . 171
36.3.3. Paso 3: default-off sólo con causa . . . 171
36.3.4. Paso 4: doc obliga . . . 171
36.3.5. Paso 5: capabilities de build . . . 171
36.4. Cuándo se rompe la promesa . . . 172
36.5. Por qué *auditable* . . . 172
36.6. Resumen . . . 172

37.Instrumentación del frontend 173
37.1. Las pildoras del toolbar . . . 173
37.2. `cmd_get_perf_status` . . . 173
37.2.1. Render de la pildora . . . 174
37.3. `cmd_get_memory_status` . . . 174
37.4. Telemetría adicional en `run_hunt_batch` . . . 174
37.5. Doc autocompliance . . . 175
37.6. El cierre del libro . . . 175
37.7. Resumen . . . 175

A. Notación 177
A.1. Grafos y vértices . . . 177
A.2. Cardinalidades, grados y vecindarios . . . 177
A.3. Hipótesis, joins, y enumeración . . . 178
A.4. Índices y ventanas temporales . . . 178
A.5. Tipos del motor (Rust) que aparecen recurrentemente . . . 179
A.6. Convenciones del pseudocódigo . . . 179

B. Glosario 181

C. Soluciones a ejercicios selectos 187

D. Bibliografía 195

E. Mapa interno del motor (referencia técnica) 199
E.1. Sistemas de algoritmos . . . 199
E.1.1. Panorama y dispatch . . . 199
E.1.2. DFS genérico y *smart* top-K . . . 199
E.1.3. Familia LFTJ canónica . . . 200
E.1.4. Yannakakis para α-acíclicos . . . 202
E.1.5. Filtros pre-match . . . 202
E.2. Mapeos de memoria . . . 202
E.2.1. Identificadores y string interner . . . 202

E.2.2. La estructura `CompactRelation` . . . 202
E.2.3. Caches columnares (SoA) . . . 203
E.2.4. CSR trie y claves `u64` empaquetadas . . . 203
E.2.5. Streaming, mmap y *hugepages* . . . 203
E.2.6. `HuntResult = Vec<Arc<str>>` . . . 203
E.3. Trucos del sistema . . . 204
E.3.1. Hot-path . . . 204
E.3.2. SIMD plug-and-play . . . 204
E.3.3. Allocator y locks . . . 205
E.3.4. Tricks algorítmicos . . . 205
E.3.5. Tricks de memoria . . . 205
E.3.6. Build y profile . . . 206
E.3.7. Filosofía: plug-and-play . . . 206
E.4. Referencias cruzadas a documentos de performance . . . 206

Parte I

Fundamentos

Capítulo 1

Threat hunting y por qué grafos

Intuición

Threat hunting es el oficio de buscar atacantes que ya están adentro de la red, antes de que detonen. Los *SIEM* clásicos (*Security Information and Event Management*) son motores de búsqueda full-text sobre *logs*: andan bien cuando uno sabe qué buscar ("decime todos los logins de `admin`"), pero fallan cuando la señal es *multi-hop*: un usuario inocuo, una máquina levemente sospechosa, una conexión saliente extraña. Cada evento por separado no dispara nada; sólo la *cadena* entre ellos delata al atacante. Modelar el ambiente como un grafo dirigido temporal hace que esa cadena sea *expresable* en una sola consulta y *ejecutable* sin recorrer joins manuales.

1.1. ¿Qué busca un *threat hunter*?

Imaginá una empresa con 5.000 empleados, 3.000 hosts, una semana de *logs* de Windows Sysmon, Sentinel y Fortinet. En esa semana hubo:

- ∼10 millones de logins exitosos.
- ∼20 millones de procesos lanzados.
- ∼80 millones de conexiones de red salientes.

La pregunta del *threat hunter* no es "¿quién se logueó hoy?" (la respuesta sería decenas de miles de filas y ninguna sería sospechosa por sí misma). La pregunta es:

> *"¿Hay algún usuario que se autenticó en un host que después, en menos de 30 minutos, se conectó a una IP nunca vista antes en nuestra red, y desde esa IP hubo* beaconing *regular en las horas siguientes?"*

Esta es una hipótesis **de tres saltos** (*usuario*, *host*, *IP destino*) con **restricciones temporales** ("en menos de 30 minutos") y **predicados de comportamiento** ("IP nunca vista antes", "*beaconing* regular"). Cada uno de los eventos por separado pasa los filtros del SIEM como ruido normal. La cadena no.

Ejemplo

Un caso real (en GraphHunter, lo llamamos el "Sysmon W0").

- t_0: el usuario `m.gomez` se loguea en `db-01` con credenciales válidas. Login normal: ese usuario se loguea acá todos los lunes para sacar reportes.
- $t_0 + 7$ min: `db-01` establece una conexión TCP saliente a `185.159.X.Y:443`. Conexión TLS válida, certificado emitido por una CA legítima. La IP no figura en la *baseline* de los últimos 30 días.
- $t_0 + 11$ min: la conexión se mantiene activa con tráfico bajo y regular, ∼2 KB cada 60 segundos. Pasa el *IDS* sin alertar.

Cada uno de esos tres eventos es *individualmente* normal. La cadena —usuario válido, host de DB, IP nueva, beacon de bajo volumen— es la firma de un *C2* reverse-shell que un atacante plantó la semana anterior. Para detectarlo, el SIEM tendría que correr tres queries y *joinar* los resultados a mano. GraphHunter lo encuentra con una sola hipótesis.

1.2. Por qué un SIEM clásico no alcanza

Los SIEM operan sobre el modelo *tabla por fuente*: una tabla para Sysmon, una para Sentinel, una para Fortinet. Cada tabla tiene sus columnas y sus índices. Una consulta full-text es eficiente sobre una tabla. Pero "cadena de tres saltos" significa *tres joins* entre tres tablas distintas, donde el atributo que los une es a veces explícito (`username`, `host_id`) y a veces implícito (la IP que aparece como `remote_ip` en una conexión saliente debe matchear con la IP que aparece como `src_ip` en el log del firewall).

Tres problemas concretos:

1. **Joins multi-tabla son caros.** Cada *join* requiere un índice; el optimizador del SIEM hace lo mejor que puede pero la cardinalidad explota. Para tres tablas con 20M filas cada una, un *nested-loop join* naive es 8×10^{21} comparaciones.
2. **Filtros temporales son post-join.** El SIEM materializa primero el producto cartesiano, después aplica el `WHERE t2 - t1` $\leq$ `30 min`. Para cuando el filtro actúa, el optimizador ya gastó memoria en filas que iban a descartarse.
3. **La hipótesis no se expresa en una línea.** Escribir el join de tres tablas en SQL requiere ∼15 líneas, con `JOIN …ON …AND`, alias, *sub-queries* para el "IP nunca vista antes". La hipótesis *conceptual* es de tres saltos; la representación SQL no lo refleja.

Nota. Esto no es una crítica al SIEM como producto. Los SIEM están optimizados para *detección por reglas* ("alertame si veo `evt_id=4625` con más de 5 fallas en 1 minuto") —ahí brillan. Lo que no resuelven bien es la *caza*: encontrar cadenas que el atacante diseñó precisamente para no disparar reglas individuales.

1.3. El cambio de modelo: de tablas a grafo temporal

GraphHunter modela la misma información —usuarios, hosts, procesos, IPs, conexiones, autenticaciones— como un **grafo dirigido temporal**:

- Los **nodos** son entidades: `User`, `Host`, `Process`, `IP`, `File`, `Domain`.
- Las **aristas** son eventos temporales: "`User` se autenticó en `Host` en t_1", "`Host` se conectó a `IP` en t_2".




```
hipótesis: User -[Auth]-> Host -[Connect]-> IP -[Beacon]-> Domain
           WITHIN 30m {first_seen_ip < t1 - 30d}
```

Figura 1.1: El ejemplo Sysmon W0 representado como grafo. Cada arista lleva un *timestamp*; la hipótesis exige $t_1 < t_2 < t_3$ y $t_3 - t_1 \leq 30$ minutos.

La pregunta de antes —"¿hay alguna cadena de tres saltos con estas restricciones?"— se vuelve, en este modelo, *enumerar todos los caminos del grafo* que matchean cierta forma y cumplen ciertos predicados. El motor que lo hace se llama un *subgraph matching engine*; el capítulo 3 lo define con precisión.

1.4. Por qué la representación importa

Tres ventajas concretas del modelo de grafo:

1. **La hipótesis se escribe igual que se piensa.** La DSL de GraphHunter (capítulo 30) deja escribir:

   ```
   User -[Auth]-> Host -[Connect]-> IP -[Beacon]-> Domain WITHIN 30m
   ```

   Una línea. La estructura del texto refleja la estructura del *path* buscado.

2. **El *join* es local.** Recorrer "vecinos salientes de v con etiqueta `Connect`" es $O(\deg(v))$ en una *adjacency list*. No hay producto cartesiano. El matcher avanza arista por arista y descarta caminos en cuanto violan una restricción temporal o un predicado.

3. **Las restricciones temporales son *in-line*.** Cuando el matcher elige una arista (h, ip, t_2) para extender el *path* parcial (u, h, t_1), chequea $t_1 < t_2$ y $t_2 - t_1 \leq 30$ min *antes* de recursar. Las ramas que violan la ventana mueren temprano.

1.5. Pero modelar como grafo no es magia

Hay tres preguntas que el resto del libro responde:

1. **¿De dónde sale el grafo?** Un *log* de Sysmon es texto JSON; convertirlo a vértices y aristas es la *ingesta* (Parte IV).

2. **¿Cómo se almacena un grafo de 100M aristas en una notebook con 16 GB de RAM?** Spoiler: con un `CompactRelation` de 32 bytes por arista, *spill* a disco vía mmap, e índices auxiliares *lazy* (Partes III y VI).

3. **¿Cómo se ejecuta el subgraph matching en sub-10 ms sobre eso?** Con un zoológico de operadores —DFS recursivo con *prunes*, LFTJ-K_4, Yannakakis chain, Yannakakis tree-walk— y un *planner* que elige el adecuado (Parte V).

1.6. Qué viene

El próximo capítulo formaliza los grafos temporales: definiciones precisas de vértice, arista, etiqueta, *path*, *walk*, ventana temporal, monotonicidad. El capítulo 3 formaliza el problema de *subgraph matching*, distingue isomorfismo de homomorfismo, y muestra por qué es NP-duro en general. El capítulo 4 construye la conexión con *joins* relacionales (también desde cero) y deriva el AGM bound, que es la pieza teórica que justifica los algoritmos *worst-case-optimal*. El capítulo 5 cierra la Parte I con los fundamentos de sistemas (*cache lines*, *branch prediction*, mmap, atomics) que justifican las decisiones de *layout* y memoria del motor.

Una vez completados esos cinco capítulos, el resto del libro construye el motor pieza por pieza, con todas las definiciones ya disponibles.

Ejercicios

1. ★ Escribí en una sola frase —en castellano, sin DSL— una hipótesis de *lateral movement* de cuatro saltos. Identificá los tipos de los vértices y las relaciones entre ellos.

2. Para la hipótesis del ejemplo Sysmon W0, ¿qué predicados tendría que evaluar el motor para descartar usuarios legítimos? Listá al menos tres y explicá si cada uno es local al vértice o requiere mirar la *baseline* histórica.

3. ★ Considerá un *SIEM* clásico con tres tablas de 20M filas cada una. Si el optimizador elige un *nested-loop join* naive para una consulta de tres saltos, ¿cuántas comparaciones hace en el peor caso? ¿Y si elige un *hash join* con cada índice cargado en memoria?

4. Pensá en una hipótesis multi-hop sobre tu propio dominio (web, mobile, e-commerce, IoT) que sea fácil de expresar como grafo y difícil de expresar como query SQL. Escribí ambos.

5. Imaginá que tu grafo tiene 10^8 aristas. ¿Cuánta memoria ocuparía si cada arista ocupara 200 bytes (un `Relation` "humano" con strings)? ¿Y si pesara 32 bytes? ¿Por qué no se puede almacenar todo el log original y armar el grafo *on-demand*?

Capítulo 2

Grafos temporales: definiciones

Intuición

Antes de hablar de *matching* necesitamos un vocabulario preciso. Un *grafo dirigido temporal* es un conjunto de vértices más un multiconjunto de aristas dirigidas con *timestamp*. "Multi" porque puede haber varias aristas entre los mismos dos nodos (el mismo proceso lanzando otro varias veces); "dirigido" porque cada arista tiene origen y destino fijos (no son simétricas); "temporal" porque el orden temporal de las aristas es parte de la estructura, no un metadato. Este capítulo define cada una de esas piezas con cuidado, distingue *path* de *walk* (una trampa frecuente del threat hunting), y formaliza las restricciones temporales que más tarde el matcher chequea *in-line*.

2.1. Grafos dirigidos: el caso base

Definición 2.1 (Grafo dirigido)**.** Un *grafo dirigido* es un par $G = (\mathcal{V}, \mathcal{E})$ donde $\mathcal{V}$ es un conjunto finito de *vértices* y $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ es un conjunto finito de *aristas dirigidas*. Una arista $e = (u, v)$ tiene *origen* u y *destino* v. Si $(u, v) \in \mathcal{E}$, v es un *vecino saliente* de u; u es un *vecino entrante* de v.

Convención de tamaño: $n = |\mathcal{V}|$ y $m = |\mathcal{E}|$.

Definición 2.2 (Vecindarios y grados)**.** Para $v \in \mathcal{V}$:

- $N^+(v) = \{w : (v, w) \in \mathcal{E}\}$ es la *vecindad saliente* de v.
- $N^-(v) = \{u : (u, v) \in \mathcal{E}\}$ es la *vecindad entrante* de v.
- $\text{out-deg}(v) = |N^+(v)|$ y $\text{in-deg}(v) = |N^-(v)|$.
- $\deg(v) = \text{out-deg}(v) + \text{in-deg}(v)$.

2.2. Multigrafos

En el caso base $\mathcal{E}$ es un conjunto: existe a lo sumo una arista entre cualesquiera dos vértices. En el threat hunting eso no alcanza: el mismo proceso p puede crear el mismo archivo f cinco veces en una hora; cada creación es un evento distinto.

Definición 2.3 (Multigrafo dirigido)**.** Un *multigrafo dirigido* es un par $G = (\mathcal{V}, \mathcal{E})$ donde $\mathcal{E}$ es un *multiconjunto* sobre $\mathcal{V} \times \mathcal{V}$. La *multiplicidad* $\mu(u, v)$ es la cantidad de veces que el par (u, v) aparece en $\mathcal{E}$.

Nota. En la práctica, modelar GraphHunter como multigrafo es lo correcto: distintos eventos entre las mismas dos entidades son distintos por su *timestamp* y su *metadata*, aunque compartan (origen, destino, tipo). El motor representa cada evento como una arista distinta con su propio `CompactRelation` (capítulo 10), no como un contador sobre una arista compartida.

2.3. Etiquetas en vértices y aristas

Hasta acá los vértices y aristas son indistinguibles entre sí. En el threat hunting cada uno tiene un *tipo*: un vértice puede ser un `User`, un `Host`, una `IP`; una arista puede ser una `Auth`, una `Connect`, un `Spawn`.

Definición 2.4 (Grafo etiquetado)**.** Sean $\mathcal{L}_V$ un alfabeto finito de *etiquetas de vértice* y $\mathcal{L}_E$ un alfabeto finito de *etiquetas de arista*. Un *grafo dirigido etiquetado* es una tupla $(G, \text{type}_V, \text{type}_E)$ donde $G = (\mathcal{V}, \mathcal{E})$ es un multigrafo dirigido, $\text{type}_V \colon \mathcal{V} \to \mathcal{L}_V$ asigna una etiqueta a cada vértice, y $\text{type}_E \colon \mathcal{E} \to \mathcal{L}_E$ asigna una etiqueta a cada arista.

En GraphHunter $\mathcal{L}_V$ es `EntityType` (`User`, `Host`, `Process`, `IP`, `File`, `Domain`, `Service`, `Resource`, `Group` — nueve tipos con *tag* $[0, 9)$) y $\mathcal{L}_E$ es `RelationType` (`Auth`, `Connect`, `Spawn`, `Read`, `Write`, `Beacon`, etc.).

2.4. Grafos temporales

Llegamos a la pieza que define al motor: el tiempo.

Definición 2.5 (Grafo dirigido temporal)**.** Un *grafo dirigido temporal etiquetado* es una tupla $(\mathcal{V}, \mathcal{E}, \text{type}_V, \text{type}_E, t)$ donde $(\mathcal{V}, \mathcal{E}, \text{type}_V, \text{type}_E)$ es un multigrafo dirigido etiquetado y $t \colon \mathcal{E} \to \mathcal{T}$ asigna un *timestamp* $t(e)$ a cada arista. $\mathcal{T}$ es el dominio temporal: en GraphHunter, $\mathcal{T} = \mathbb{Z}$ (segundos epoch).

Cuando hablemos de *una arista* en un grafo temporal usaremos la notación $e = (u, v, \tau)$, donde $\tau = t(e)$. Igualdad de aristas: dos aristas son distintas si difieren en type_E, τ, o (en multigrafos) en cualquier metadata adicional.

Ejemplo

Un grafo de seis vértices y siete aristas, todos con etiquetas y *timestamps*.

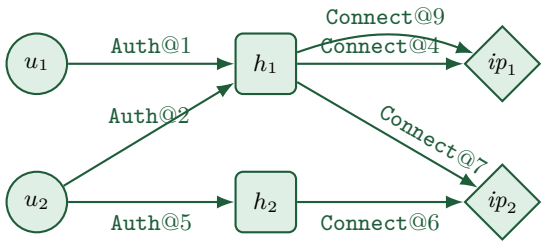


Figura 2.1: Grafo dirigido temporal etiquetado de ejemplo. $\mathcal{L}_V = \{\texttt{User}, \texttt{Host}, \texttt{IP}\}$, $\mathcal{L}_E = \{\texttt{Auth}, \texttt{Connect}\}$. Notar la **arista múltiple** $h_1 \to ip_1$: hay dos conexiones distintas, en $\tau = 4$ y $\tau = 9$.

2.5. Caminos: *paths* vs *walks*

Es la distinción más importante del capítulo, y la trampa clásica.

Definición 2.6 (Walk dirigido)**.** Un *walk* dirigido de longitud k en G es una secuencia de vértices $v_0, v_1, \ldots, v_k$ tal que para cada $i \in \{0, \ldots, k-1\}$, existe una arista $(v_i, v_{i+1}, \tau_i) \in \mathcal{E}$. Los vértices *pueden repetirse*.

Definición 2.7 (Path simple)**.** Un *path simple* (o sólo *path*) es un *walk* en el que todos los vértices son distintos.

Advertencia. En el threat hunting, *walks* y *paths* responden preguntas distintas. "¿Hay un usuario que se conectó a una IP y volvió a conectarse a sí mismo?" es una pregunta sobre *walks* (el usuario aparece en posiciones 0 y 2). "¿Hay tres entidades distintas encadenadas?" es una pregunta sobre *paths*. La DSL de GraphHunter expresa esta elección con la cláusula `{k=N}` (capítulo 30): *k-simplicidad* controla cuántas veces puede repetirse un mismo vértice.

Definición 2.8 (k-simplicidad)**.** Un *walk* $v_0, \ldots, v_k$ es *k-simple* sii ningún vértice aparece más de k veces en la secuencia. Caso particular: 1-simple = *path simple*.

2.6. Restricciones temporales

2.6.1. Monotonicidad

Definición 2.9 (Walk temporalmente válido)**.** Un *walk* $(v_0, e_0, v_1, e_1, \ldots, v_k)$ —donde $e_i = (v_i, v_{i+1}, \tau_i)$— es *temporalmente válido* sii

$$\tau_0 < \tau_1 < \cdots < \tau_{k-1}.$$

Es decir: las aristas se atraviesan en orden cronológico estricto. Si en el grafo de la figura 2.1 consideramos $u_2 \to h_2 \to ip_2$, las aristas tienen τ igual a 5 y 6 —walk válido. Pero $u_2 \to h_1 \to ip_1$ tiene $\tau = 2$ y $\tau = 4$ —también válido. Y $h_1 \to ip_1$ (la primera, en $\tau = 4$), seguido de $h_1 \to ip_2$ (en $\tau = 7$), forma walk válido si empezamos por una arista entrante a h_1 anterior a $\tau = 4$.

Nota. GraphHunter impone monotonicidad *estricta* ($<$, no $\leq$): dos aristas con el mismo *timestamp* no se ordenan entre sí. Esto evita ambigüedad cuando dos eventos llegan en el mismo segundo epoch (resolución típica del log de Sysmon). En la práctica, los eventos genuinamente simultáneos son raros; cuando ocurren, el matcher los descarta sin pérdida.

2.6.2. Ventana temporal

Definición 2.10 (Walk Δt-acotado)**.** Un walk temporalmente válido $(v_0, \ldots, v_k)$ con timestamps $(\tau_0, \ldots, \tau_{k-1})$ está *Δt-acotado* sii

$$\tau_{k-1} - \tau_0 \leq \Delta t.$$

Δt es la *ventana temporal* de la hipótesis. En la DSL es la cláusula `WITHIN`; en el ejemplo Sysmon W0, $\Delta t = 30$ minutos. Es la pieza que evita que el motor enumere walks de tres saltos cuya primera arista es de hace tres meses.

2.6.3. Composición: *walk* de hipótesis

Definición 2.11 (Walk de hipótesis)**.** Dado un grafo temporal G y una hipótesis H con *shape chain* de longitud k, etiquetas $(L_0^V, L_0^E, L_1^V, L_1^E, \ldots, L_k^V)$, k-simplicidad k_s y ventana Δt, un *walk de H* es una secuencia $(v_0, e_0, v_1, e_1, \ldots, v_k)$ tal que:

1. $\text{type}(v_i) = L_i^V$ para todo i.
2. $\text{type}(e_i) = L_i^E$ para todo i.
3. Es temporalmente válido: $\tau_0 < \cdots < \tau_{k-1}$.
4. Está Δt-acotado: $\tau_{k-1} - \tau_0 \leq \Delta t$.

5. Es k_s-simple: ningún vértice se repite más de k_s veces.

6. Cualquier predicado adicional sobre vértices o metadata se cumple.

El subgraph matching de GraphHunter es, esencialmente, enumerar todos los walks de hipótesis para una H dada. El próximo capítulo formaliza por qué eso es —en general— un problema duro, y por qué en la práctica se vuelve tractable gracias a las restricciones de tipo y temporales.

2.7. Representación: *adjacency list*

Una decisión de implementación con consecuencias algorítmicas profundas: GraphHunter no almacena $\mathcal{E}$ como una matriz $n \times n$ (sería $O(n^2)$ en memoria, intratable para $n = 10^7$). Almacena $\mathcal{E}$ como un `Vec<CompactRelation>` ordenado por (`source_sid`, `timestamp`), e indexa por `source_sid` para extraer rápidamente $N^+(v)$.

En el código.

`core/graph-engine/src/graph.rs`, función `neighbors_of(sid)`. Devuelve un *slice* contiguo de aristas salientes de v, ya ordenadas por `timestamp`. La búsqueda binaria sobre ese slice por la ventana temporal $[t_{\text{lo}}, t_{\text{hi}})$ cuesta $O(\log \text{out-deg}(v))$ y devuelve un sub-slice contiguo —directamente iterable.

Esa propiedad (*contiguo*, *ordenado*, *binary-search-ready*) es la que el matcher explota. La Parte V del libro se apoya repetidamente en ella; el capítulo 5 discute por qué la elección de un `Vec` contiguo en vez de un `HashMap` de `Vec`s tiene impacto en *cache* performance.

2.8. Resumen de notación

Símbolo	Significado
$G = (\mathcal{V}, \mathcal{E}, t)$	grafo dirigido temporal etiquetado
n, m	$\lvert\mathcal{V}\rvert$, $\lvert\mathcal{E}\rvert$
(u, v, τ)	arista temporal: origen u, destino v, timestamp τ
$N^+(v), N^-(v)$	vecindad saliente, vecindad entrante
$\text{out-deg}(v), \text{in-deg}(v)$	cardinalidad de cada vecindad
$\mathcal{L}_V, \mathcal{L}_E$	alfabetos de etiquetas (`EntityType`, `RelationType`)
$\text{type}(v), \text{type}(e)$	etiqueta del vértice / arista
k	longitud (en aristas) de un walk o de una hipótesis
k_s	cota de k-simplicidad
Δt	ventana temporal de la hipótesis

Ejercicios

1. ★ En el grafo de la figura 2.1, enumerá todos los walks de hipótesis con shape `User -[Auth]-> Host -[Connect]-> IP` sin restricción temporal. ¿Cuántos hay? Listalos como ternas (u, h, ip) junto con los timestamps.

2. Repetí el ejercicio anterior con la restricción adicional $\Delta t = 3$ (ventana de 3 unidades de tiempo). ¿Cuántos walks sobreviven?

3. Construí un grafo temporal de exactamente 5 vértices y 6 aristas que admita un *walk* de longitud 4 pero ningún *path simple* de longitud 4. (Pista: tiene que repetirse al menos un vértice.)

4. ★ Definí formalmente la noción de "Δt-conexidad temporal": dos vértices u, v son Δt-conexos sii existe un walk temporalmente válido y Δt-acotado de u a v. Mostrá que esta relación *no* es de equivalencia en general (¿qué propiedad falla?).

5. Si un grafo tiene $n = 10^6$ vértices y $m = 10^7$ aristas, ¿cuál es el grado promedio? ¿Cuánta memoria ocuparía la matriz de adyacencia? ¿Y la *adjacency list*? ¿Por qué la diferencia importa para sistemas con n en los millones?

6. Imaginá un caso de threat hunting donde la k-simplicidad *importa*: una hipótesis donde permitir repeticiones ($k_s = 2$) cambia las respuestas legítimas. Después imaginá uno donde $k_s = 1$ (path simple) es lo correcto. ¿Cuál es el riesgo de equivocarse?

Capítulo 3

Subgraph matching: el problema

Intuición

Subgraph matching es: dado un grafo grande G (el dato) y un grafo chico P (el patrón o hipótesis), encontrar todas las formas de *embeber* P dentro de G. En GraphHunter el "patrón" es la hipótesis del analista; G es el grafo temporal ingestado. El problema en su formulación más general es NP-duro —el famoso *subgraph isomorphism problem*— pero en la práctica se vuelve tractable por tres razones combinadas: las hipótesis son chicas ($k \leq 6$ saltos), las etiquetas (type_V, type_E) podan agresivamente, y las restricciones temporales acotan la *ventana de búsqueda*. Este capítulo enuncia las variantes formales (decisión, conteo, enumeración), distingue isomorfismo de homomorfismo, y muestra por qué el motor implementa enumeración.

3.1. Tres niveles de pregunta

Cuando un analista escribe una hipótesis, puede querer una de tres respuestas distintas:

1. **Decisión**: "¿Existe al menos un walk de hipótesis en G?"
   Respuesta: `true`/`false`.
2. **Conteo**: "¿Cuántos walks de hipótesis hay?"
   Respuesta: un entero.
3. **Enumeración**: "Listamelos uno por uno."
   Respuesta: una secuencia (potencialmente larga) de tuplas de vértices.

GraphHunter implementa la versión 3, **enumeración**, con un *cap* configurable (default 10 000 resultados) que la trunca para evitar que un *hub* explote la salida. ¿Por qué la enumeración? Porque el analista no quiere saber cuántos atacantes hay —quiere saber *cuáles* (qué usuario, qué host, qué IP). La salida es la entrada del próximo paso de la investigación.

Nota. Las versiones de decisión y conteo son sub-problemas de enumeración pero, sorpresa: en general son *tan caras* como enumeración. Para preguntas sin truncate, el motor tiene que generar candidatos y verificarlos uno por uno; saber "hay al menos uno" suele requerir generar al menos uno. Hay casos donde decisión es estrictamente más fácil (cuando hay índices que la responden directo: NLF, k-hop reachability), pero esos son optimizaciones, no soluciones generales. Las desarrollamos en los capítulos 18 y 19.

3.2. Patrón P versus dato G

Definición 3.1 (Patrón)**.** Un *patrón* (en GraphHunter, una `Hypothesis` con *shape chain*) de longitud k es una tupla $(P, \text{type}_V^P, \text{type}_E^P)$ donde $P = (V_P, E_P)$ es un grafo dirigido etiquetado,

$|V_P| = k+1$, $|E_P| = k$, y P es una cadena: hay un orden total $p_0, p_1, \ldots, p_k$ tal que $E_P = \{(p_0,p_1),(p_1,p_2),\ldots,(p_{k-1},p_k)\}$.

Para *shape graph*, P puede ser cualquier grafo α-acíclico (capítulo 22), no sólo una cadena.

El *dato* es el grafo temporal etiquetado G construido por la ingesta. Las dimensiones típicas:

$\lvert V_P\rvert, \lvert E_P\rvert$	≤ 6 y ≤ 6 (hipótesis del analista)
$\lvert\mathcal{V}\rvert, \lvert\mathcal{E}\rvert$	hasta 10^7 y 10^8 (grafo del motor)

Esa asimetría dramática es la que hace que el problema sea —a pesar de la dureza teórica— tractable en la práctica.

3.3. Embeddings: los matches del patrón

¿Qué significa que P "aparezca" en G? Hay dos definiciones clásicas, y la elección entre ellas tiene consecuencias.

3.3.1. Homomorfismo

Definición 3.2 (Homomorfismo de grafos)**.** Un *homomorfismo* de $P = (V_P, E_P)$ en $G = (\mathcal{V}, \mathcal{E})$ (ambos etiquetados, dirigidos) es una función $\sigma\colon V_P \to \mathcal{V}$ tal que:

1. Para todo $p \in V_P$, $\text{type}^P(p) = \text{type}(\sigma(p))$.
2. Para todo $(p,q) \in E_P$, existe una arista $(\sigma(p),\sigma(q)) \in \mathcal{E}$ con $\text{type}_E((\sigma(p),\sigma(q))) = \text{type}_E^P((p,q))$.

La función σ puede mapear distintos vértices del patrón al *mismo* vértice de G. Es decir: el embedding puede "colapsar".

3.3.2. Isomorfismo (subgraph isomorphism)

Definición 3.3 (Embedding isomórfico)**.** Un *embedding isomórfico* es un homomorfismo σ que además es **inyectivo**: $p \neq q$ implica $\sigma(p) \neq \sigma(q)$.

3.3.3. ¿Cuál es la convención de GraphHunter?

GraphHunter usa una variante intermedia: la k_s-*simplicidad* del capítulo 2. Por defecto, $k_s = 1$ (ningún vértice se repite, equivale a embedding isomórfico). Configurar `{k=2}` en la DSL afloja: cada vértice puede aparecer hasta dos veces. `{k=k+1}` (donde k es la longitud de la hipótesis) lo torna en homomorfismo puro.

Advertencia. Esta diferencia importa muchísimo. Considerá la hipótesis `User -[Auth]-> Host -[Auth]-> User`. Bajo isomorfismo, las respuestas son ternas (u_1, h, u_2) con $u_1 \neq u_2$. Bajo homomorfismo, la terna (u_1, h, u_1) —el mismo usuario "volviendo"— también cuenta. Para detectar *lateral movement* suele importar la primera; para detectar *credential abuse*, la segunda.

3.4. Por qué el problema es duro en general

Teorema 3.4 (Cook-Levin via Clique)**.** *El problema de decisión asociado a subgraph isomorphism ("¿hay un embedding isomórfico de P en G?") es NP-completo, incluso si P y G son grafos no dirigidos sin etiquetas.*

Idea de la prueba. Reducción desde *clique*. Dado (G,k) —instancia de clique: ¿tiene G un clique de tamaño k?— construimos $P = K_k$ (el grafo completo de k vértices) y preguntamos si P embebe isomórficamente en G. Existe embedding sii G tiene un k-clique. □ □

Esto significa que ningún algoritmo conocido resuelve subgraph matching en tiempo polinomial *en general* (asumiendo $P \neq NP$). En el peor caso, el motor está obligado a explorar un espacio combinatorio del tamaño de $\binom{n}{k}$ para hipótesis de $k+1$ vértices.

3.4.1. Por qué entonces es tractable en la práctica

Tres palancas que GraphHunter usa todas a la vez:

1. **Etiquetas** type_V **y** type_E **podan masivamente.** El primer paso del DFS sólo considera vértices del tipo L_0^V. Si la hipótesis es `User -[Auth]-> Host -[Connect]-> IP` y el grafo tiene 10^6 `User`s pero 10^4 `Host`s y 10^5 `IP`s, el factor $10^6 \times 10^4 \times 10^5 = 10^{15}$ naive se reduce a las parejas que *sí* tienen aristas con la etiqueta correcta —típicamente 10^7 órdenes de magnitud menos.

2. **La *ventana temporal*** Δt recorta el sub-slice de $N^+(v)$ que el matcher considera. En vez de iterar out-deg(v) aristas, itera las que caen en $[\tau_{i-1}, \tau_{i-1}+\Delta t)$. Para out-deg $= 10^4$ aristas distribuidas en una semana y $\Delta t = 30$ min, eso son ~ 30 aristas por vértice intermedio.

3. **Los *prunes*** (NLF, k-hop reachability) descartan *starts* antes de iniciar el DFS. Un vértice cuyos vecinos no tienen el tipo correcto del primer paso, o desde el cual no hay *ningún* camino al `dest_type` final en $\leq$ `MAX_DEPTH` saltos, jamás genera resultados. Se descarta sin tocar el DFS.

3.5. Variante: subgraph matching temporal

La definición clásica de subgraph isomorphism no incluye el tiempo. GraphHunter agrega la restricción del capítulo 2: cada embedding debe ser *temporalmente válido* y Δt*-acotado*.

Definición 3.5 (Match temporal)**.** Un *match temporal* de un patrón P (chain de longitud k) en un grafo temporal G es un par $(\sigma, \mathbf{e})$ donde:

- $\sigma \colon V_P \to \mathcal{V}$ es una asignación que respeta etiquetas de vértice y k_s-simplicidad.
- $\mathbf{e} = (e_0, e_1, \ldots, e_{k-1})$ es una secuencia de aristas con $e_i = (\sigma(p_i), \sigma(p_{i+1}), \tau_i)$, $\text{type}_E(e_i) = \text{type}_E^P((p_i, p_{i+1}))$.
- $\tau_0 < \tau_1 < \cdots < \tau_{k-1}$ (monotonicidad).
- $\tau_{k-1} - \tau_0 \leq \Delta t$ (ventana).
- Cualquier predicado adicional sobre σ o `metadata` se cumple.

Notar que $\mathbf{e}$ es parte del match: el mismo σ puede tener varios matches temporales si las aristas multi-grafo (con τ distintos) se eligen distinto.

3.6. Operadores: qué algoritmo usa el motor

GraphHunter no implementa *un* algoritmo de subgraph matching: implementa un *zoológico* de operadores y un *planner* que elige uno. La Parte V del libro los desarrolla; acá los nombramos para que el lector tenga el mapa:

- **DFS recursivo con *prunes*** (capítulo 16): el *baseline*; funciona para cualquier hipótesis. Su complejidad de peor caso es $O(m \cdot \deg^k)$; los *prunes* (NLF, k-hop) la bajan drásticamente en grafos reales.

- **LFTJ-K$_4$** (capítulo 21): un *worst-case-optimal join* para hipótesis cuya *shape* es K_4 (cuatro vértices, las seis aristas dirigidas). Logra el AGM bound de $O(m^2)$ en lugar de $O(m \cdot \deg^3)$.
- **Yannakakis chain** (capítulo 23): para cualquier hipótesis α-acíclica con shape chain, una semi-join reducer en dos pasadas eliminando *rows* sin futuro, y enumeración al final.
- **Yannakakis tree-walk** (capítulo 24): el caso general α-acíclico (estrellas, árboles, hipergrafos) con enumeración recursiva sobre el *join tree*.

El *planner* (capítulo 25) decide cuál dispatchar inspeccionando la *shape* de la hipótesis. Por defecto cae al DFS; las hipótesis con propiedades especiales (K_4 homogéneo, α-aciclidad) ganan operadores más agresivos.

3.7. Por qué el AGM bound importa

El capítulo 4 deriva el AGM bound desde cero, pero adelantamos la idea: para una hipótesis P, existe una cantidad $\rho^*(P) \in [1, |E_P|]$ tal que el número de matches posibles es a lo sumo $O(m^{\rho^*(P)})$. Esa cantidad se llama el *fractional edge cover number* y depende sólo de la estructura del patrón.

- Para una *cadena* de k aristas, $\rho^* = k$. El DFS naive ya alcanza ese bound; no hay mucho margen.
- Para un *triángulo*, $\rho^* = 1.5$. El DFS escala como $O(m \cdot \deg) = O(m \cdot \frac{m}{n})$ en el peor caso —*peor* que $O(m^{1.5})$. Hay margen significativo.
- Para K_4, $\rho^* = 2$. El DFS es $O(m \cdot \deg^3)$; LFTJ alcanza $O(m^2)$. En grafos densos eso es varios órdenes de magnitud.

Esto es lo que justifica formalmente por qué el motor tiene un operador especializado para K_4 y no, digamos, para cadenas: el *lift* algorítmico está disponible donde la estructura del patrón lo permite.

3.8. Resumen

- Subgraph matching es: dado G y P, encontrar embeddings de P en G. En GraphHunter, la variante operativa es **enumeración con cap**, sobre **matches temporales** (monotónicos y Δt-acotados), respetando **etiquetas** y **k_s-simplicidad**.
- El problema de decisión es NP-completo en general (reducción desde clique).
- En la práctica es tractable por la combinación de etiquetas, ventana temporal, prunes (NLF, k-hop), y la elección inteligente de operador físico (DFS, LFTJ, Yannakakis).
- El ρ^* del patrón —el *fractional edge cover number*— determina cuánto *lift* sobre el DFS naive es teóricamente posible. Para patrones donde $\rho^* < k$, los *worst-case-optimal joins* ganan; para cadenas, el DFS bien-podado es ya cerca del óptimo.

Ejercicios

1. ★ Para el patrón "triángulo" (3 vértices, 3 aristas dirigidas formando un ciclo), enumerá todos los embeddings isomórficos en el grafo de la figura 2.1 del capítulo anterior. ¿Cuántos hay? ¿Y bajo homomorfismo (permitiendo colapsar vértices)?
2. Demostrá por reducción que el problema "¿existe un Hamiltonian path en G?" se puede

resolver con un solo subgraph matching. (Pista: ¿qué patrón P usás?)

3. ★ La *cadena* de longitud k tiene $\rho^* = k$. Dada esta propiedad, ¿qué predice el AGM bound sobre el número máximo de matches en un grafo de m aristas? Compará con el bound trivial $O(\deg^k)$.

4. Considerá una hipótesis con shape "estrella": un vértice central conectado a k vértices distintos vía aristas salientes. ¿Cuál es ρ^* de esa estructura? (Pista: pensá cuál es el *minimum fractional edge cover*.)

5. ¿Por qué decidir si dos grafos arbitrarios son isomorfos (*graph isomorphism problem*) NO es lo mismo que subgraph isomorphism? ¿Cuál es "más fácil" (en el sentido de la teoría de complejidad)?

6. Imaginá una hipótesis donde GraphHunter devuelve 9.999 resultados (justo bajo el cap de 10.000). ¿Cómo sabés si los 9.999 son "todos los matches" o "sólo los primeros 10K de muchos más"? Diseñá una API que distinga los dos casos.

Capítulo 4

Joins relacionales y su conexión con grafos

Intuición

Hay dos formas equivalentes de mirar el mismo problema:

- "Subgraph matching sobre un grafo etiquetado."
- "Multi-way join sobre tablas relacionales."

La equivalencia importa porque la comunidad de bases de datos estuvo trabajando *décadas* en la teoría de joins, y en ese camino derivó el *AGM bound* —una cota inferior sobre cuán chico puede ser cualquier algoritmo correcto— y los *worst-case optimal joins* (WCOJ), algoritmos que alcanzan el bound. Este capítulo arranca explicando *qué* es un join (sin asumir que sabés SQL), construye la equivalencia "subgraph matching = join", deriva el AGM bound desde cero con dos ejemplos chicos, y motiva *por qué* el motor incluye un operador *worst-case-optimal* para K_4.

4.1. Joins desde cero

4.1.1. Relaciones

Definición 4.1 (Relación)**.** Una *relación* R sobre los atributos $A_1, A_2, \ldots, A_d$ es un conjunto finito de tuplas, donde cada tupla $t \in R$ asigna un valor a cada atributo. Notación: $R(A_1, \ldots, A_d)$. El número d se llama *arity* de R.

Una relación es básicamente una *tabla*: las columnas son los atributos, las filas son las tuplas. Sin claves duplicadas, sin orden entre filas. Por ejemplo:

$$\texttt{Auth}(\texttt{user}, \texttt{host}) = \\ \{(\texttt{alice}, \texttt{db-01}),\ (\texttt{bob}, \texttt{web-02}),\ (\texttt{alice}, \texttt{web-02})\}$$

4.1.2. Join binario natural

Definición 4.2 (Join natural binario)**.** Dadas dos relaciones $R(A_1, \ldots, A_d)$ y $S(B_1, \ldots, B_e)$, sea $C = \{A_1, \ldots, A_d\} \cap \{B_1, \ldots, B_e\}$ el conjunto de atributos compartidos. El *join natural* $R \bowtie S$ es la relación sobre $\{A_1, \ldots, A_d\} \cup \{B_1, \ldots, B_e\}$ cuyas tuplas son:

$$R \bowtie S = \{t \cup s : t \in R, s \in S, \forall c \in C.\ t[c] = s[c]\}$$

donde $t \cup s$ es la tupla que combina los valores de t y s (consistentes en los atributos compartidos).

Ejemplo

Sean

$$R(\texttt{user},\texttt{host}) = \{(\texttt{alice},\texttt{db-01}),\ (\texttt{bob},\texttt{web-02})\}$$
$$S(\texttt{host},\texttt{ip}) = \{(\texttt{db-01},\texttt{10.0.0.5}),\ (\texttt{web-02},\texttt{8.8.8.8}),\ (\texttt{web-02},\texttt{1.1.1.1})\}$$

El atributo compartido es `host`. Entonces:

$$\begin{aligned} R \bowtie S = \{ & \\ & (\texttt{alice},\texttt{db-01},\texttt{10.0.0.5}), \\ & (\texttt{bob},\texttt{web-02},\texttt{8.8.8.8}), \\ & (\texttt{bob},\texttt{web-02},\texttt{1.1.1.1}) \\ \} & \end{aligned}$$

La tupla $(\texttt{alice},\texttt{web-02})$ de R NO matchea con nada de S —porque R no la contiene—. La tupla $(\texttt{bob},\texttt{web-02})$ sí, y matchea con dos filas de S, generando dos tuplas en la salida.

4.1.3. Multi-way join

Definición 4.3 (Multi-way join)**.** Dadas n relaciones $R_1, R_2, \ldots, R_n$, el *multi-way join* $R_1 \bowtie R_2 \bowtie \ldots \bowtie R_n$ es:

$$\bigcup \{t_1 \cup t_2 \cup \cdots \cup t_n : t_i \in R_i,\ \text{los } t_i \text{ son consistentes en cada atributo compartido}\}$$

El join natural es asociativo y conmutativo, así que el orden de ejecución no cambia el resultado, pero sí cambia el costo: ejecutar $(R_1 \bowtie R_2) \bowtie R_3$ puede ser muy distinto de $(R_1 \bowtie R_3) \bowtie R_2$ en términos de *rows intermedias*. Esa es la pieza que el *query planner* de un SIEM clásico optimiza, y donde los *worst-case-optimal joins* cambian las reglas.

4.2. Equivalencia: subgraph matching es multi-way join

Ahora la pieza fundamental del capítulo.

Teorema 4.4 (Subgraph matching como join)**.** *Sea P un patrón con vértices $(p_0, \ldots, p_k)$ y aristas $\{(p_{i_1}, p_{j_1}), \ldots, (p_{i_m}, p_{j_m})\}$. Sea G un grafo. Para cada arista (p_{i_l}, p_{j_l}) del patrón con etiqueta L_l, definamos la relación:*

$$R_l(\texttt{src}_{i_l}, \texttt{dst}_{j_l}) = \{(u,v) : (u,v) \in \mathcal{E}(G),\ \text{type}_E((u,v)) = L_l\}$$

Entonces el conjunto de embeddings homomórficos de P en G es exactamente:

$$R_1 \bowtie R_2 \bowtie \ldots \bowtie R_m$$

donde el atributo $\texttt{src}_i$ (resp. $\texttt{dst}_j$) identifica el vértice p_i del patrón.

Idea de la prueba. Una asignación σ que valga como embedding tiene que cumplir dos cosas: (a) cada arista del patrón existe en G con la etiqueta correcta —eso es exactamente decir "$(\sigma(p_i), \sigma(p_j)) \in R_l$"—; (b) la asignación es consistente: si dos aristas del patrón comparten un vértice (digamos p_i aparece en (p_i, p_j) y en (p_i, p_k)), entonces el valor que σ le asigna a p_i es el mismo en ambas aristas. La consistencia es exactamente lo que el join natural impone vía atributos compartidos. □ □

Ejemplo: K_4 como join de 6 relaciones

El patrón K_4 tiene cuatro vértices p_0, p_1, p_2, p_3 y las seis aristas dirigidas entre cada par:

$$\{(p_0, p_1), (p_0, p_2), (p_0, p_3), (p_1, p_2), (p_1, p_3), (p_2, p_3)\}$$

El join correspondiente es:

$$R_{01}(\texttt{src}_0, \texttt{dst}_1) \bowtie R_{02}(\texttt{src}_0, \texttt{dst}_2) \bowtie R_{03}(\texttt{src}_0, \texttt{dst}_3) \bowtie R_{12}(\texttt{src}_1, \texttt{dst}_2) \bowtie R_{13}(\texttt{src}_1, \texttt{dst}_3) \bowtie R_{23}(\texttt{src}_2, \texttt{dst}_3)$$

Si todas las aristas del grafo tienen la misma etiqueta, las seis relaciones son *la misma*: $R_l = \mathcal{E}(G)$ para todo l. El join se vuelve un *self-join* de $\mathcal{E}$ consigo misma seis veces, con los atributos renombrados.

4.2.1. Por qué la equivalencia importa

Dos consecuencias prácticas:

1. Cualquier teorema sobre joins relacionales se aplica a subgraph matching. En particular: el *AGM bound* (próxima sección) y los *worst-case-optimal joins* (LFTJ).
2. Cualquier algoritmo de join se puede usar como matcher. En particular: *semi-join reducer* de Yannakakis para consultas α-acíclicas (capítulo 23).

4.3. El AGM bound

4.3.1. Motivación

Antes de enunciarlo, una pregunta concreta: para un join de tres relaciones, cada una con m tuplas, ¿cuántas tuplas *en el peor caso* puede tener la salida?

- Si las tres son joinadas en cadena (atributos compartidos formando una secuencia $A \to B \to C \to D$), la salida puede crecer hasta m^3: cada tupla del primer join se replica para cada coincidencia en el segundo.
- Si las tres forman un *triángulo* (compartiendo atributos en forma cíclica), *lo más* que la salida puede crecer es $m^{1.5}$. *No* m^3. Esa es la sorpresa.

¿Por qué? Porque un triángulo tiene una *estructura* más rígida que una cadena: las tres relaciones se restringen mutuamente. Esta intuición se hace precisa con el *fractional edge cover number*.

4.3.2. Cobertura fraccionaria

Definición 4.5 (Hipergrafo de la consulta)**.** Dado un join $R_1 \bowtie \ldots \bowtie R_m$ sobre atributos $\mathcal{A}$, su *hipergrafo* $H(Q) = (\mathcal{A}, F)$ tiene como vértices los atributos y como hiperaristas las relaciones: cada R_l corresponde a la hiperarista $F_l = \{A : A \text{ es un atributo de } R_l\}$.

Definición 4.6 (Edge cover fraccionario)**.** Una *cobertura fraccionaria* de $H(Q)$ es una asignación de pesos $x_l \geq 0$, uno por hiperarista F_l, tal que para todo atributo $A \in \mathcal{A}$:

$$\sum_{l : A \in F_l} x_l \geq 1.$$

El *fractional edge cover number* de Q es:

$$\rho^*(Q) = \min \sum_{l=1}^{m} x_l, \quad \text{sujeto a las restricciones de cobertura.}$$

En palabras: cada atributo tiene que estar "cubierto" por las hiperaristas que lo contienen, con peso total ≥ 1. El ρ^* es la mínima suma de pesos necesaria.

Ejemplo: Triángulo

Para el join triangular $R(A,B) \bowtie S(B,C) \bowtie T(A,C)$:

- Hipergrafo: vértices $\{A,B,C\}$; hiperaristas $F_R = \{A,B\}$, $F_S = \{B,C\}$, $F_T = \{A,C\}$.
- Cobertura fraccionaria: asignar $x_R = x_S = x_T = 0.5$ cubre cada atributo con peso $0.5 + 0.5 = 1$. La suma total es 1.5.
- Cualquier cobertura con menos suma viola alguna restricción. Por lo tanto $\rho^* = 1.5$.

Ejemplo: Cadena

Para el join en cadena $R(A,B) \bowtie S(B,C) \bowtie T(C,D)$:

- Atributo A: sólo aparece en R, entonces $x_R \geq 1$.
- Atributo D: sólo aparece en T, entonces $x_T \geq 1$.
- Con $x_R = x_T = 1$, los atributos B (en R y S) y C (en S y T) ya están cubiertos por R y T respectivamente; $x_S = 0$ basta.
- Suma: $1+0+1 = 2$, pero podríamos pensar que S necesita estar cubierta. No: la *cobertura* es de *atributos*, no de aristas. $\rho^* = 2$.

Wait. Eso no se condice con lo que dijimos antes ("cadena de tres relaciones, salida puede ser m^3"). Veamos: cadena de tres relaciones sobre cuatro atributos, sí, $\rho^* = 2$. Pero la salida es *a lo sumo* m^2 bajo el AGM bound. La intuición de "m^3" que tenía uno antes era una sobre-estimación basada en el caso "join naive"; el AGM bound nos dice que en realidad la estructura del triángulo y la cadena no son tan distintas como pensábamos. La verdadera diferencia entre cadena y triángulo es de m^2 versus $m^{1.5}$; ambas son menores que m^3.

4.3.3. El teorema

Teorema 4.7 (Atserias-Grohe-Marx 2008)**.** *Para todo join $Q = R_1 \bowtie \ldots \bowtie R_m$ y toda asignación de tamaños $|R_l| \leq m$:*

$$|Q| \leq m^{\rho^*(Q)}.$$

Además, esta cota es ajustada*: existen instancias donde $|Q|$ alcanza $m^{\rho^*(Q)}$ hasta factores constantes.*

Idea de la prueba. La parte "superior" ($|Q| \leq m^{\rho^*}$) sale de aplicar la desigualdad de Shearer (información mutua sub-aditiva sobre particiones del producto) a la entropía de la distribución uniforme sobre Q. La parte "ajustada" se demuestra construyendo instancias específicas (típicamente, todos los R_l son m^{1/ρ^*} copias del Cartesiano más un balanceo). □ □

Nota. Lo crucial: ρ^* depende *sólo de la estructura del patrón*, no del tamaño de los datos. Es un invariante del problema que dice "este join puede crecer a lo sumo así".

4.3.4. Worst-case optimal joins

Definición 4.8 (Algoritmo WCOJ)**.** Un algoritmo de join es *worst-case-optimal* sii su tiempo de ejecución es $\widetilde{O}(m^{\rho^*(Q)})$ (el $\widetilde{O}$ oculta factores logarítmicos).

Los algoritmos clásicos de join (*nested-loop*, *hash-join*, *sort-merge*) ejecutados *de a dos relaciones a la vez* **NO son WCOJ** para hipergrafos cíclicos como el triángulo: pueden producir resultados intermedios de tamaño m^2 aun cuando la salida final es $m^{1.5}$.

- Triángulo: $\rho^* = 1.5$. Hash-join binario es $O(m \cdot m) = O(m^2)$ en el peor caso. WCOJ es $O(m^{1.5})$. Ratio: $\sqrt{m}$. Para $m = 10^6$, eso es 1000× más rápido.
- K_4: $\rho^* = 2$. Hash-join secuencial es $O(m^3)$ en el peor caso. WCOJ es $O(m^2)$. Para $m = 10^4$, eso es $10^4\times$ más rápido.

El primer algoritmo WCOJ fue NPRR (Ngo-Porat-Re-Rudra, 2012). LFTJ (Veldhuizen, 2014) —el que GraphHunter implementa para K_4— es una variante más implementable. El capítulo 21 lo desarrolla.

4.4. Por qué entonces no toda hipótesis usa WCOJ

Pregunta natural: si WCOJ es estrictamente mejor en el peor caso, ¿por qué el motor cae al DFS por defecto y sólo usa LFTJ para K_4 y Yannakakis para α-acíclicos?

Tres razones:

1. **El bound es de peor caso, no de caso típico.** En grafos "reales" (*scale-free*, etiquetas que filtran masivamente, ventana temporal angosta), el DFS bien-podado puede ser ya $O(m \cdot k)$ o mejor —*ganando* al WCOJ por constantes y por menor sobrecarga.
2. **Implementación.** LFTJ requiere un *trie* construido sobre las aristas, indexado consistentemente, con *multi-way intersection* SIMD. Construirlo es trabajo; mantenerlo bajo ingestas streaming es más trabajo. Para hipótesis donde el DFS ya alcanza, no vale la pena.
3. **Generalidad.** El DFS funciona para *cualquier* hipótesis (chain, graph, con predicados arbitrarios). LFTJ está especializado a la *shape* estructural; agregar predicados o etiquetas heterogéneas requiere extensiones.

El *planner* (capítulo 25) materializa este trade-off: dispatcha al WCOJ cuando *sabe* que la estructura lo permite, cae al DFS en caso contrario.

4.5. Resumen

- Un **join natural** es la operación que junta dos tablas matcheando los valores en sus columnas compartidas. Multi-way join generaliza a n tablas.
- **Subgraph matching homomórfico es exactamente un multi-way join** de las relaciones de aristas, con consistencia impuesta vía atributos compartidos.
- El **AGM bound** ($|Q| \leq m^{\rho^*(Q)}$) acota el tamaño de la salida en función del *fractional edge cover number* del hipergrafo de la consulta.
- ρ^* es 1.5 para triángulo, 2 para K_4, 2 para cadena de 3 aristas, k para cadena de k aristas.
- Los **worst-case-optimal joins** (NPRR, LFTJ) alcanzan el AGM bound; los joins binarios clásicos pueden no alcanzarlo (sobre todo en hipergrafos cíclicos).
- GraphHunter usa LFTJ para K_4 (capítulo 21) y Yannakakis para α-acíclicos (capítulos 23 y 24); para todo lo demás, DFS con *prunes*.

Ejercicios

1. ★ Calculá ρ^* para una *estrella* de k brazos (un vértice central con k aristas salientes a vértices distintos). ¿Qué AGM bound da? ¿Es ajustado en grafos reales?

2. Calculá ρ^* para el grafo bipartito completo $K_{3,3}$ (visto como hipergrafo con 9 hiperaristas binarias). Comparalo con el bound trivial m^9. ¿Qué dice esto sobre la posible eficiencia de un WCOJ para $K_{3,3}$?

3. ★ Mostrá que para una cadena de k aristas el AGM bound es ajustado: construí un grafo de m aristas donde el join correspondiente tenga exactamente $\Theta(m^k)$ matches.

4. Diseñá un *nested-loop join* naive para $R \bowtie S$ (dos relaciones binarias) y demostrá que su complejidad es $O(|R| \cdot |S|)$. ¿Bajo qué condiciones (sobre el atributo compartido) un *hash-join* mejora a $O(|R| + |S|)$?

5. Imaginá una hipótesis que el motor recibe con *shape chain* de longitud 4. Identificá la estructura como un join de 4 relaciones binarias y dibujá su hipergrafo. ¿Cuál es ρ^*?

6. ¿Por qué el AGM bound no contradice el hecho de que subgraph isomorphism sea NP-completo? (Pista: ¿qué crece con qué?)

Capítulo 5

Sistemas y memoria

Intuición

Los algoritmos de matching que vamos a estudiar son inteligentes, pero la mitad del rendimiento real del motor sale de decisiones de *layout* de memoria: `CompactRelation` pesa 32 bytes en vez de 200 porque cada arista que cabe en la *cache* es una que no espera a la RAM; el *streaming backend* usa *mmap* porque transferir 28 GB del disco al heap es inviable; el *lazy build* de NLF y *khop_reach* usa `OnceLock` porque la inicialización doble bajo concurrencia es un bug clásico. Este capítulo da los fundamentos de sistemas que esas decisiones invocan. Si ya sabés qué es una *cache line*, qué garantiza un *atomic*, y por qué un *branch mispredict* cuesta ciclos, podés saltearlo. Si no, leélo: el resto del libro lo asume.

5.1. La jerarquía de memoria

Una operación aritmética en un núcleo de CPU moderno cuesta ~ 1 ciclo. Acceder a la RAM cuesta ~ 200 ciclos. Esa diferencia —*un orden de magnitud y medio*— es lo que manda toda la conversación sobre *layout* de datos. Para amortiguar, las CPUs tienen una jerarquía de *caches*:

Nivel	Tamaño típico	Latencia (ciclos)	Latencia (ns)
Registros	100 B	0	0
L1 cache	32–64 KB	4–5	1
L2 cache	256 KB–1 MB	10–12	3
L3 cache	8–32 MB	30–40	12
RAM principal	16–128 GB	200–300	100
SSD NVMe	1–2 TB	$\sim 10^5$	30–80 μs
HDD	8–16 TB	$\sim 10^7$	4–10 ms

La regla de oro: si tus datos caben en L2, los algoritmos "inteligentes" rinden lo que prometen. Si no caben, la latencia de RAM domina y la "inteligencia" algorítmica importa menos que la *locality* de acceso.

5.1.1. Cache lines

La CPU no transfiere bytes individuales entre RAM y cache: transfiere *cache lines* de 64 bytes (en x86_64 e ARM modernos). Acceder a un solo byte trae los 63 contiguos "gratis". Eso tiene dos implicaciones:

1. **Datos contiguos son rápidos.** Iterar un `Vec<u32>` de 1000 elementos toca $1000 \cdot 4/64 = 62.5$ cache lines. El siguiente elemento ya está cargado cuando el anterior se procesa (la *prefetch* unit lo trae).

2. **Datos esparcidos son lentos.** Iterar un `HashMap<u32, u32>` de 1000 elementos toca, en el peor caso, 1000 cache lines distintas (cada *bucket* en una región distinta de RAM). 60× más cache misses, mismo trabajo lógico.

En el código.

Esta es la justificación de fondo de por qué `CompactRelation` es 32 bytes (*half* de una cache line: dos aristas por línea). En el DFS hot loop, iterar `neighbors_of(v)` carga 32 aristas por cada 16 cache lines —versus el `Relation` público de ~ 200 bytes que cargaría 32 aristas en ~ 100 cache lines. `core/graph-engine/src/types.rs`.

5.1.2. Branch prediction

Las CPUs modernas no esperan a que un `if` se evalúe para decidir qué instrucción cargar. Predicen el resultado y lo ejecutan *especulativamente*; si la predicción acierta, ganaron 10–20 ciclos; si falla, descartan el trabajo y empiezan de nuevo (penalty de 15–25 ciclos en x86_64).

- Loops contables (`for i in 0..n`): predicción casi perfecta. Branch barato.
- Branches dependientes de datos impredecibles (`if data[i] > threshold`): mispredicts frecuentes si los datos son aleatorios.
- Hot loops con muchos `if` anidados: cada `if` es una oportunidad de mispredict; los *loops* apretados con un solo branch son de oro.

Nota. Esta es la razón por la que el inner loop del DFS de GraphHunter está diseñado como una sola pasada lineal sobre un *slice* contiguo de `CompactRelation`, con los predicados temporales reducidos a comparaciones de `i64` y los predicados de tipo a comparaciones de `u8`. Los branches que quedan son predecibles por la distribución típica de los datos (la mayoría de las aristas pasan los filtros tempranos).

5.1.3. SIMD

Las CPUs x86_64 e ARM tienen instrucciones que operan sobre 4, 8 o 16 valores a la vez (*SIMD*: Single Instruction, Multiple Data). Para problemas que se mapean naturalmente a operaciones por elemento —*intersección de conjuntos ordenados*, por ejemplo— el *lift* versus la versión escalar es proporcional al ancho del vector.

GraphHunter usa SIMD vía `core/graph-engine/src/simd\_rust.rs` (Rust puro con `std::arch::x86_64` y *runtime dispatch*) para la intersección *multi-way* en LFTJ-K_4 (capítulo 21): $N(a) \cap N(b) \cap N(c)$ sobre adyacencias ordenadas. Con AVX2, intersecta 8 `u32`s por ciclo; el fallback escalar cubre hosts sin AVX2.

5.2. Memoria virtual y mmap

5.2.1. Memoria virtual: la abstracción del SO

Los procesos no ven la RAM física directamente. Ven un *address space* virtual de 64 bits que el kernel mapea on-demand a páginas físicas (típicamente 4 KB cada una). Beneficios:

- Cada proceso cree que tiene 16 EB de memoria.

- El SO mueve páginas entre RAM y *swap* sin que el proceso se entere.
- Las regiones de archivos pueden *mapearse* al address space (mmap): el archivo en disco aparece como bytes accesibles con punteros.

5.2.2. mmap

Definición 5.1 (Memory-mapped file)**.** Una región mmap es una asociación entre un rango de páginas virtuales y un rango de bytes en un archivo. Acceder a una página no residente dispara un *page fault*: el kernel lee los 4 KB del archivo, los pone en una página física libre, y reanuda la ejecución.

Tres garantías importantes:

1. **Lazy loading.** El archivo no se lee hasta que el proceso accede a una página. Mapear un archivo de 28 GB ocupa *nada* de RAM; sólo registra un descriptor.
2. **Cache compartida con el page cache del SO.** Las páginas leídas viven en el *page cache*; otros procesos que mapean el mismo archivo las comparten.
3. **Backpressure automático.** Si el sistema necesita RAM, el kernel descarta páginas mmap-resident (sin escribir nada al disco si están *clean*); el próximo acceso las relee. Es *free swap* para datos read-only.

Advertencia. El acceso aleatorio sobre mmap es lento si dispara muchos page faults (cada uno cuesta $\sim 100\,000$ ciclos). El acceso *secuencial* es rápido porque el kernel hace *readahead*: cuando ve un patrón secuencial, pre-carga las próximas páginas en background. Por eso GraphHunter ordena las aristas por `(source_sid, timestamp)` antes de hacer spill: los *accesses* del matcher son secuenciales por construcción.

En el código.

`core/graph-engine/src/streaming.rs`: el *streaming backend* aplica el mismo principio. Cuando el budget global (default 10^6 aristas) se cruza, cada *per-vertex slice* se vuelca sortado por timestamp a un archivo temporal y la `NeighborList` de cada vértice swap-ea a la variante `Mmap`. Los lookups siguientes van vía puntero al archivo; el SO decide qué páginas viven en RAM.

5.3. Concurrencia: atomics, locks, y wait-free

5.3.1. Lectura concurrente: el problema

GraphHunter es multi-thread: el ingestor escribe aristas mientras los matchers de varios transports leen. La pregunta es: ¿cómo evitamos que un *reader* vea estado a medio escribir?

Tres estrategias clásicas:

1. **Mutex.** Un solo thread tiene acceso a la vez. Simple, correcto, pero serializa todo: si hay 100 readers, 99 están esperando.
2. **Read-Write lock (`RwLock`).** Múltiples readers concurrentes, un único writer exclusivo. Mejor para workloads read-heavy.
3. **Wait-free.** Los readers nunca bloquean: ven siempre un *snapshot consistente* aunque el writer esté activo. Implementación típica: el writer construye una nueva versión y publica el puntero atómicamente; los readers ven la versión vieja o la nueva, nunca un estado intermedio.

5.3.2. Operaciones atómicas

Las CPUs ofrecen instrucciones que ejecutan *lectura-modificación-escritura* sobre una palabra de memoria sin que otro thread pueda interrumpir. Las primitivas más comunes:

- `load/store` atómicos: leer o escribir una palabra de modo que ningún reader vea "mitad de un valor".
- `compare_and_swap (CAS)`: "si el valor actual es x, reemplazalo por y, todo de una". Devuelve el valor viejo (o un *success* flag).
- `fetch_add`: incrementar atómicamente, devolver el valor previo. Útil para *counters* compartidos.

Las atómicas son $\sim$ 5–20$\times$ más caras que sus contrapartes no-atómicas, pero son 100$\times$ más baratas que un `Mutex` bajo contención.

5.3.3. Memory ordering

Advertencia. Acá hay una sutileza profunda: una CPU moderna *reordena* operaciones de memoria para ganar performance. Sin garantías explícitas, un thread puede ver las escrituras de otro *en orden distinto al que se ejecutaron*. Las atómicas vienen con un parámetro de *memory ordering* (`Relaxed`, `Acquire`, `Release`, `SeqCst`) que controla qué reordenamientos están permitidos. Usar `Relaxed` cuando hace falta `Acquire/Release` es un bug clásico que NO se manifiesta en hardware débil (x86) pero sí en ARM.

GraphHunter usa `OnceLock` (de la stdlib de Rust) para los índices *lazy-built*: el primer thread que llama `ensure_nlf()` construye el índice; los demás ven el resultado vía un `Acquire` load del puntero. Sin *double init*, sin *tornados* de constructores.

5.3.4. Wait-free para el streaming backend

El *streaming backend* (capítulo 13) soporta *wait-free reads* via una estructura *degree-adaptive*: cada vértice tiene un `NeighborList` que es `Inline`, `Vec`, o `Btree` según su grado. El writer publica nuevas aristas vía un puntero atómico; el reader ve el snapshot que existía cuando entró. Sin lock.

5.4. Layout de datos en Rust

5.4.1. `#[repr(C)]` y predictibilidad

Por defecto, el compilador de Rust puede reordenar los campos de un `struct` para minimizar padding. Eso ahorra memoria en algunos casos, pero rompe la asunción de que el `struct` se mapea directamente a un buffer binario.

`#[repr(C)]` fuerza el orden declarado y las reglas de alineación de C: cada campo va exactamente donde el programador quiere, con padding explícito si hace falta. Esto permite:

- Mapear `Vec<CompactRelation>` directamente a un archivo mmap: los bytes en disco son los bytes en memoria.
- Pasar `CompactRelation` por FFI a C/C++ sin *marshalling*.
- Razonar sobre el tamaño exacto (32 bytes, no "alguna vez 24 alguna vez 40 según versión del compilador").

Ejemplo

Sin `repr(C)`:

```
struct Bad {
    flag: u8,        // 1 B
    timestamp: i64,  // 8 B
    sid: u32,        // 4 B
}
```

El compilador puede ordenar como `(timestamp, sid, flag)` para ahorrar padding —o no—. Tamaño: 16 B (con padding) o 24 B (reordenado). Impredecible.
Con `repr(C)`:

```
#[repr(C)]
struct Good {
    timestamp: i64,  // offset 0,  8 B
    sid: u32,        // offset 8,  4 B
    flag: u8,        // offset 12, 1 B
    _pad: [u8; 3],   // offset 13, 3 B padding explícito
}
```

Total: 16 B exactos. Layout estable a través de versiones del compilador. El programador eligió.

5.4.2. `Arc<str>` y *refcounting*

`Arc<T>` (*atomic reference count*) es un puntero compartido que mantiene un contador atómico de cuántos dueños tiene. Clonar un `Arc` no copia los datos; sólo bumpea el contador (un `fetch_add` atómico, ~ 5 ns). Cuando el último dueño se va, los datos se liberan.

`Arc<str>` es la versión para strings: una sola copia del texto en heap, refcount atómico, clonado barato. La diferencia con `String::clone()` (que copia los bytes) es del orden de *decenas a cientos de nanosegundos por clone* —suficiente para explicar el 55 % de mejora en cap-saturated benches que motivó el cambio de `HuntResult = Vec<String>` a `Vec<Arc<str>>` (capítulo 11).

5.5. Cómo todo esto vuelve en el resto del libro

Los conceptos de este capítulo aparecen explícitamente en:

- Capítulo 10: `CompactRelation` 32 B, `repr(C)`, half-cache-line.
- Capítulo 13: el *streaming backend* con auto-spill, mmap, y page cache.
- Capítulo 11: migración a `Vec<Arc<str>>`, refcount versus alloc.
- Capítulo 13: streaming backend wait-free, `neighbors_snapshot`.
- Capítulo 15: `OnceLock` y el patrón *lazy build*.
- Capítulo 21: SIMD intersection en LFTJ.

Cuando aparezcan, vamos a referirnos de vuelta a este capítulo sin re-derivar las definiciones.

Ejercicios

1. ★ Una *cache line* es de 64 B. Tu `struct` tiene 48 B útiles. Si lo declarás directo, dos instancias consecutivas comparten una cache line. ¿Bueno o malo? Pensalo bajo dos escenarios: (a) acceso secuencial; (b) acceso aleatorio desde threads distintos. (Pista:

false sharing.)

2. Calculá: en una iteración secuencial de un `Vec<CompactRelation>` con 100M elementos, ¿cuántos cache line transfers son necesarios? Si la latencia efectiva es 100 ns por línea y el prefetch oculta el 90 %, ¿cuánto tarda en total?

3. ★ ¿Por qué un `HashMap<K, V>` no tiene *locality*? Diseñá una alternativa con *open addressing* y *linear probing* y argumentá si sería mejor o peor para una cache.

4. Un mutex bajo contención cuesta $\sim$1 μs por operación. Una atómica $\sim$10 ns. Si tu workload tiene 10K operaciones por segundo en 8 threads concurrentes, ¿cuál es la sobrecarga total con mutex? ¿Y con atómica? ¿Cambia la conclusión si las operaciones son 99 % reads?

5. Imaginá que `CompactRelation` pesara 24 bytes en vez de 32. Calculá: (a) cuántas aristas caben en L1 (32 KB); (b) cuánta memoria total ocuparían 100M aristas. ¿Por qué crees que el motor eligió 32 y no 24?

6. Mapeás un archivo de 100 GB con mmap en una máquina de 16 GB de RAM. ¿Funciona? ¿Qué pasa si el matcher hace un *random access* sobre todo el archivo? ¿Cómo cambia la respuesta si el acceso es secuencial?

Parte II

Arquitectura del motor

Capítulo 6

El pipeline en cuatro pasos

Intuición

Antes de bucear en componentes individuales conviene tener el mapa completo. Graph Hunter divide el trabajo en cuatro etapas *secuenciales* que el analista atraviesa de una y que internamente se ejecutan en pipeline: *ingesta*, *hipótesis*, *matching*, y *scoring + paginación*. Este capítulo recorre las cuatro de punta a punta, mostrando qué objeto entra y qué objeto sale, con un walkthrough concreto de "apretar Run" en el frontend para que el lector vea cómo dialogan capas.

6.1. Vista de pájaro



Figura 6.1: Pipeline alto-nivel. Cada etapa tiene su propio capítulo de profundización; las cifras debajo son targets de referencia, no SLAs duros.

Cada etapa tiene un *contrato de entrada* y un *contrato de salida* bien definidos. Eso permite testar cada una en aislamiento (*property tests*, *benches* `criterion`) y permite también que el motor sea reutilizable desde varios *transports* (Tauri, CLI, MCP, gateway HTTP) —ninguno toca el grafo directamente, todos pasan por la misma `GraphHunterApi`.

6.2. Etapa 1: Ingesta

Entrada: una fuente de *logs* (Sysmon NDJSON, Sentinel KQL, EVTX binario, CSV genérico, o un *custom parser*).

Salida: aristas e identidades agregadas al grafo temporal.

La ingesta produce, para cada *row* de la fuente, cero o más *triples*:

```
(source_id, source_type, rel_type, dest_id, dest_type, timestamp, metadata)
```

y los empuja al grafo vía `add_entity` (idempotente, deduplicado por el *interner*) y `add_relation` (append + *streaming mirror*).

Tres decisiones de diseño que aparecen acá y que la Parte IV explica en profundidad:

- **Stream-y, no batch.** Los logs se procesan en *batches* chicos (default 50.000 rows, ∼10 MB) en lugar de leer el archivo entero a RAM. Para 28 GB de EVTX, eso significa que el *peak RAM* queda en el orden del batch size en lugar del tamaño del archivo (capítulo 12).
- **Lazy index build.** La ingesta no construye los índices auxiliares (NLF, k-hop reachability) —los deja indicados como "a construir". La construcción se difiere al primer *hunt* (capítulo 14).
- **Streaming backend canónico.** Cada `add_relation` hace *single-write* al *streaming backend* per-vértice degree-adaptive con auto-spill incremental. El backend es el único edge store desde stage 2c part 2; sirve simultáneamente al matcher hot path, a los index builds (NLF, khop_reach), y a consumers delta-aware (live-tail UI, telemetría) —capítulo 13.

6.3. Etapa 2: Hipótesis (parsing de la DSL)

Entrada: una expresión DSL (string).
Salida: una `Hypothesis` estructurada (AST).
El analista escribe algo como:

```
User u -[Auth]-> Host h -[Connect]-> IP {k=2} HAVING count(distinct last) >=
    5 WITHIN 30m
```

El *parser* (en `platform/dsl`) convierte ese texto en un valor de tipo `Hypothesis`:

```
pub struct Hypothesis {
    pub name: String,
    pub steps: Vec<HypothesisStep>,     // 2 pasos en este ejemplo
    pub k_simplicity: usize,            // 2
    pub aggregations: Vec<AggClause>,   // count(distinct last) >= 5
    pub window_seconds: Option<i64>,    // Some(1800)
    pub shape: HypothesisShape,         // Chain (sin `;`)
}
```

La `Hypothesis` es el contrato canónico que viaja entre parser y matcher. Cualquier *transport* (Tauri, MCP, HTTP) que quiera ejecutar un hunt construye una `Hypothesis` —ya sea parseando DSL o construyéndola programáticamente— y la pasa a `run_hunt`.

La Parte VII (capítulos 29–32) desarrolla la DSL en detalle: gramática *chain* legacy, extensión *graph* con el separador `;`, parser flow, disambiguación de identificadores.

6.4. Etapa 3: Matching

Entrada: una `Hypothesis` más una referencia al grafo temporal.
Salida: `Vec<HuntResult>`, donde cada resultado es un *path* resuelto al alfabeto humano.

Esta es la etapa con más carne algorítmica del libro. La Parte V (capítulos 16–25) desarrolla los cuatro operadores físicos. En este vistazo alto-nivel, el flujo es:

1. El `search_temporal_pattern` llama al *planner*, que inspecciona la `Hypothesis` y devuelve un `DispatchPlan`.
2. Según el plan, dispatcha al operador adecuado: DFS legacy, LFTJ-K_4, Yannakakis chain, o Yannakakis tree-walk.
3. El operador consulta los índices auxiliares (NLF, k-hop) para podar *starts* antes de empezar la enumeración.
4. La enumeración produce resultados; cada uno se convierte en un `HuntResult` (`Vec<Arc<str>>`).

6.4.1. Por qué cuatro operadores y no uno

La razón es la del capítulo 4: el AGM bound predice que para algunas *shapes* estructurales (como K_4 o α-acíclicos), un algoritmo *worst-case-optimal* es estrictamente mejor que el DFS naive. Tener varios operadores y un *planner* que elige permite cobrar ese *lift* cuando la hipótesis lo permite, sin penalizar al DFS para los casos donde no hay nada mejor que hacer.

6.5. Etapa 4: Scoring y paginación

Entrada: `Vec<HuntResult>` (puede ser hasta el *cap*, default 10.000).

Salida: la misma lista, ranqueada por rareza, paginada para el frontend.

El *anomaly scorer* (en `core/graph-engine/src/scoring`) asigna un score a cada *path* basado en:

- Frecuencia histórica de cada arista del path (*rarity*: aristas vistas pocas veces puntúan más alto).
- Distancia a la baseline del usuario / host involucrado.
- Combinación lineal con pesos configurables.

El frontend pagina sobre la lista ordenada (página de 50, scroll infinito hacia abajo). El analista ve los más sospechosos primero; puede expandir cada uno para ver la cadena de evidencia.

6.6. Walkthrough: "apreté Run"

Para cerrar el capítulo, sigamos un *hunt* completo a través del stack. El analista escribe la hipótesis del Sysmon W0 (capítulo 1) y aprieta el botón "Run" en el frontend Tauri.

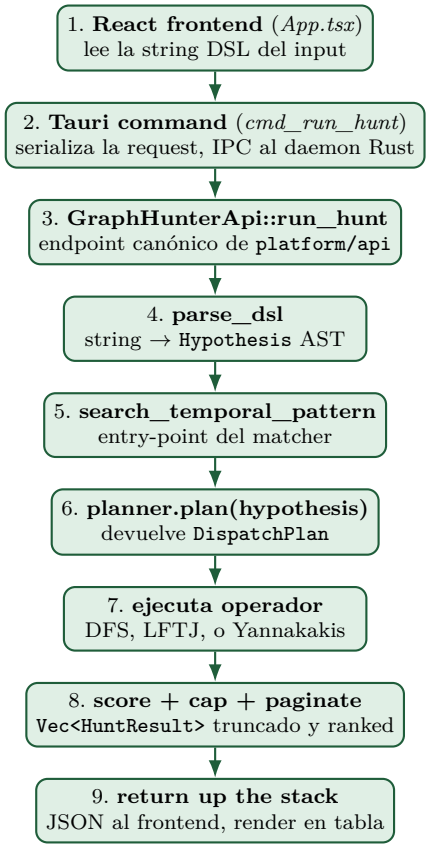


Figura 6.2: Las nueve paradas de un *hunt* desde el botón "Run" hasta los resultados renderizados. Cada paso es un capítulo, sub-capítulo o sección de algún libro de GraphHunter; el *cross-ref* explícito está en cada uno cuando aparezca.

Notar que las cuatro etapas conceptuales (ingesta, hipótesis, matching, scoring) ocurren en momentos distintos:

- **Ingesta** ocurrió *antes* —el analista la disparó cuando cargó el dataset—. En el walkthrough no aparece.
- **Hipótesis** es el paso 4 (*parse_dsl*).
- **Matching** son los pasos 5–7.
- **Scoring + paginación** es el paso 8.

La separación temporal importa: la ingesta paga el costo de leer 28 GB *una vez* y construir el grafo y los índices. El analista después corre decenas o cientos de hunts sobre el mismo grafo, cada uno en sub-segundo, sin pagar de nuevo el costo de ingesta. Esa es la economía que hace al motor viable.

6.7. Resumen

- Cuatro etapas: **ingesta** (logs → grafo), **hipótesis** (DSL → AST), **matching** (AST + grafo → paths), **scoring + paginación** (paths → ranked list).
- Cada etapa tiene contrato de entrada y salida bien definidos; eso permite testar y reusar.
- El *matching* es la etapa con cuatro operadores físicos distintos, elegidos por un *planner* según la estructura de la hipótesis.

- La ingesta y los matchers están separados temporalmente: pagás la ingesta una vez por dataset, los hunts son baratos sobre el grafo ya construido.

Ejercicios

1. ★ Para cada una de las cuatro etapas, identificá cuál crate de GraphHunter (`core`, `platform`, `apps/cli`, `apps/tauri`) la implementa principalmente. ¿Cuál es la única etapa que vive en `core`?

2. Imaginá que el analista quiere correr 100 hunts distintos sobre el mismo grafo de 28 GB. Si cada hunt tarda 100 ms y la ingesta tardó 2 minutos, ¿cuál es el costo amortizado por hunt? Discutí: ¿cuándo conviene mantener el grafo vivo en memoria versus recargarlo desde *snapshot*?

3. En el walkthrough de la figura 6.2, ¿qué pasos involucran cruce de *boundary* de proceso (IPC, serialización)? ¿Cuáles son sólo llamadas a función en el mismo proceso?

4. ★ Si la `Hypothesis` produjera, en el matcher, exactamente 9999 resultados, ¿hay manera de saber si la cap se saturó? ¿Qué cambia el motor en su API para que el frontend lo distinga? (Pista: hay un campo en `HuntResult` o en la respuesta agregada.)

5. Pensá un caso donde uno querría correr la ingesta y el matcher *en paralelo* —un *hunt* disparado mientras los logs todavía están entrando— y discutí qué garantías de consistencia debería ofrecer el motor.

Capítulo 7

El stack de *crates*

Intuición

GraphHunter se divide en *layers* concéntricos: `core` (algoritmos puros), `platform` (contracts compartidos), y `apps` (binarios end-user). La regla es que cada *layer* sólo importa de capas iguales o más internas: `core` no toca `platform`, `platform` no toca `apps`. Esa restricción aparenta arbitraria pero compra dos cosas concretas: el `core` se puede testar en aislamiento (sin levantar Tauri, sin levantar HTTP, sin Mock de Sentinel) y se puede reusar desde varios *transports* sin acoplamiento. Este capítulo dibuja el diagrama, justifica el orden, y da una heurística práctica para "¿en qué crate va esto que estoy escribiendo?"

7.1. Diagrama de capas

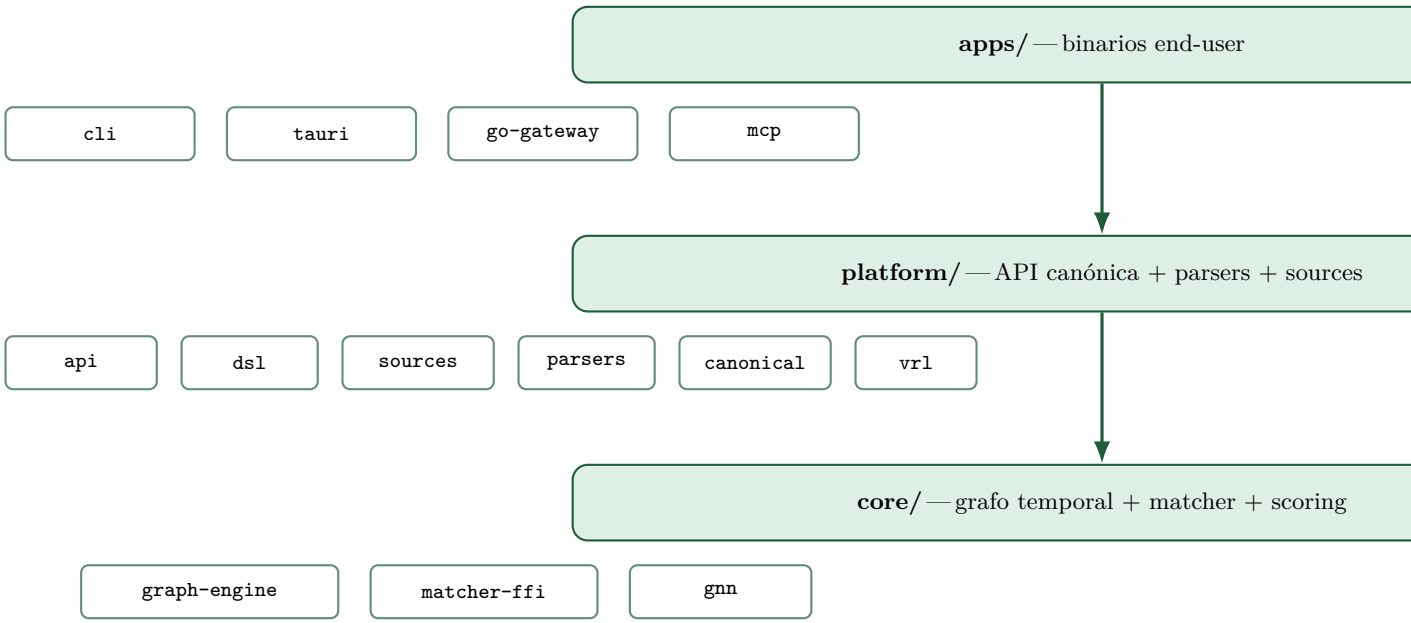


Figura 7.1: Layering. Las flechas son la dirección permitida de *imports*. `apps` importa de `platform` y `core`; `platform` sólo de `core`; `core` no ve hacia arriba. Excepciones puntuales se documentan en `docs/arquitectura/layering-exceptions.txt` y obligan ADR.

7.2. Por qué este orden

- `core` es la pieza algorítmica pura. No conoce a Tauri, ni a HTTP, ni a Sentinel. Eso le permite testarse en aislamiento (*property tests* con `proptest`, *benches* con `criterion`) y reusarse desde múltiples transports sin acoplamiento. Si mañana queremos exportar el matcher como *Python binding*, lo único que cambiamos es una capa nueva en `platform`; el `core` no se toca.
- `platform` agrupa los *contracts* compartidos entre transports: la *canonical API* (`GraphHunterApi`), el parser de la DSL, los *adapters* de *sources* (Sentinel, Defender XDR, Data Lake), los parsers concretos (Cognito, FortiAnalyzer, IIS), y la *canonical pipeline* (VRL + provenance). El criterio de inclusión: cualquier cosa que más de un transport puede querer.
- `apps` son los binarios usables: el *daemon* Tauri (con su frontend React+Vite), el CLI que el *Go gateway* invoca como sub-process, y el plugin MCP. Todos consumen `Arc<GraphHunterApi>` de `platform/api`.

7.3. Heurística: ¿en qué crate va esto?

Tres preguntas que guían la decisión cuando uno está escribiendo código nuevo:

1. *¿Lo necesita más de un transport?* Sí ⇒ `platform`. No ⇒ `apps`.
2. *¿Tiene contrato externo (HTTP, MCP, FFI, REST)?* Sí ⇒ siempre `platform`, nunca dentro de un `apps/` específico. Un *transport* cualquiera puede querer reusar el contrato.
3. *¿Es algorítmico puro y libre de I/O o transport?* Sí ⇒ `core`.

Ejemplo

Tres dilemas reales y dónde fueron a parar:

- "Función que parsea un evento Sysmon NDJSON." Pregunta 1: ¿más de un transport lo usa? Sí (Tauri y CLI ambos ingieren Sysmon). ⇒ `platform/parsers`.
- "Función que devuelve los vecinos salientes de un vértice ordenados por timestamp." Pregunta 3: ¿algorítmica pura? Sí. ⇒ `core/graph-engine`.
- "Componente React que renderiza la pildora "Perf:M+L1" en el toolbar." Específico de Tauri. ⇒ `apps/tauri/src` (TypeScript).
- "*Endpoint* HTTP `POST /run_hunt`." Pregunta 2: contrato externo. ⇒ `platform/api` (la implementación), expuesto desde `apps/cli` cuando corre en modo daemon.

7.4. Cómo se enforce la regla

La capa `core` *podría* importar de `platform` sin que el compilador se queje (Cargo no impone direccionalidad). El enforcement vive en CI:

En el código.

`scripts/check-layering.sh` corre en cada CI run. Para cada crate, lista sus dependencias declaradas en `Cargo.toml` y verifica que no haya flechas "hacia arriba". Excepciones legítimas (por ejemplo, un test que necesita un mock de `platform`) se listan explícitamente en `docs/arquitectura/layering-exceptions.txt` con justificación, y agregar una nueva entrada requiere ADR.

Esa estrategia —"layering por contrato + enforcement por script + ADR para excepciones"— es deliberadamente simple. Más sofisticado (linter custom de *cargo-deny* con reglas de allowlist) sería más correcto pero menos legible. El script bash gana.

7.5. Crates en detalle

7.5.1. `core/`

- `graph-engine` — el grafo temporal in-memory, el matcher (DFS, LFTJ, Yannakakis), los índices auxiliares (NLF, k-hop), el scorer.
- `matcher-ffi` — *thin forwarder* Rust puro (∼30 LOC) sobre `graph_hunter_core`. Históricamente fue la C ABI sobre `libgraphmatch` (motor C++ AVX2); el motor C++ se eliminó el 2026-05-07 al demostrarse que el DFS Rust era 3-25× más rápido. El nombre del crate sobrevive por estabilidad de *call-sites* en `platform/api` y es candidato a eliminación. La Especificación cubre la firma actual (Rust).
- `gnn` — *graph neural network* experimental para scoring; opcional (*Cargo feature*).

7.5.2. `platform/`

- `api` — la `GraphHunterApi`: el endpoint canónico que todos los transports consumen. `run_hunt`, `ingest_batch`, `session_save`, `session_load`, etc.
- `dsl` — el parser de la DSL (capítulo 30).
- `sources` — adapters para Sentinel, Defender XDR, Data Lake, EVTX. Cada uno expone una `LogSource` trait que el ingestor consume.
- `parsers` — parsers de formato concretos: Cognito, FortiAnalyzer, IIS, Sysmon NDJSON, EVTX binario.
- `canonical` — la *canonical pipeline* VRL + provenance: aplica un programa VRL a cada evento para normalizarlo a un esquema canónico antes de la ingesta.
- `vrl` — la integración con *Vector Remap Language* para transformaciones declarativas.

7.5.3. `apps/`

- `cli` — binario CLI que el Go gateway invoca como sub-process. Modo *daemon* expone HTTP; modo *one-shot* corre un comando y termina.
- `tauri` — el daemon con UI (React + Vite). El backend es Rust (Tauri commands), el frontend TypeScript. La aplicación que el analista usa día a día.
- `go-gateway` — un proxy Go que se sienta entre SIEM corporativo y el motor; traduce de la API del SIEM a la `GraphHunterApi`.
- `mcp` — plugin para el *Model Context Protocol* que expone el motor como tool a un LLM.

7.6. Por qué *platform* y no *infra* o *shared*

Una decisión de naming. "Platform" (en oposición a "shared", "common", "utils") refleja que esta capa NO es un cajón de sastre sino una *plataforma de transports*: cualquier nuevo binario (futuro daemon Python, plugin Slack) construye encima. *Shared* y *utils* se vuelven, por entropía, vertederos de cualquier helper sin clasificar; *platform* obliga a preguntarse "¿esto es realmente plataforma, o es código que debería vivir junto al transport que lo usa?"

Nota. Si en algún punto uno se descubre escribiendo helpers genéricos sin hogar ("utilidades de string", "wrappers para datetime"), la respuesta correcta no es crear `platform/utils` sino dejarlos en el módulo del consumer. Cuando dos consumers necesitan lo mismo, *ahí* se promueve a una crate compartida —probablemente fuera del repo, como dependencia de `Cargo.toml`.

7.7. Resumen

- Tres capas concéntricas: `core` (algorítmico puro), `platform` (contracts compartidos), `apps` (binarios). Imports sólo de afuera hacia adentro.
- Decidir en qué crate va algo: ¿más de un transport lo usa? ¿es contrato externo? ¿es algorítmico puro?
- Enforcement por `scripts/check-layering.sh` en CI; excepciones documentadas en `layering-exceptions.` y obligan ADR.
- El nombre *platform* (no *shared*, no *utils*) es deliberado: evita que se vuelva basurero.

Ejercicios

1. ★ Estás escribiendo una función que dado un `IP` string devuelve el ASN al que pertenece (vía look-up en una tabla MaxMind). ¿En qué crate va? Justificá con las tres preguntas guía.
2. Una función que sólo usa el plugin MCP, que serializa una `Hypothesis` a un formato propio del MCP. ¿`platform` o `apps/mcp`? ¿Qué cambiaría si mañana también la quisiera usar el CLI?
3. ¿Por qué la regla de layering NO se enforce en el compilador (Cargo) sino en CI (script)? Listá ventajas y desventajas de mover el check al compilador (vía `cargo-deny` u otro).
4. ★ Imaginá que un *hot path* del matcher necesita logging estructurado (cada operador emite eventos `tracing`). ¿Va en `core` (porque el operador está en `core`) o en `platform` (porque `tracing` es transport-aware)? Pensá los dos lados.
5. Pensá en una *exception* legítima: un caso donde sería razonable que `core` dependa de `platform`. ¿Qué pondrías en `layering-exceptions.txt` para documentarla?

Parte III

Modelo de datos

Capítulo 8

`Entity`, `Relation` y los tipos públicos

Intuición

El modelo de datos público de GraphHunter —el que ven los parsers, los *transports* y la API— es deliberadamente legible: un `Entity` es un nodo con identificador humano y tipo, un `Relation` es una arista con origen, destino, tipo y *timestamp*. Los dos llevan un `HashMap` de `metadata` opcional. Esa forma es cómoda para el ingestor y los parsers, pero *no* es la forma en la que el matcher itera en su *hot loop*; ahí entra `CompactRelation` (capítulo 10). Este capítulo presenta el modelo público desde el evento de Sysmon hasta los triples que el ingestor empuja al grafo.

8.1. Tipos públicos

```
pub struct Entity {
    pub id: String,                                  // ej. "10.0.0.42" o "lsass
        .exe"
    pub entity_type: EntityType,                     // ej. IP, Process, User
    pub metadata: HashMap<String, MetadataValue>, // campos opcionales del
        log
}

pub struct Relation {
    pub source_id: String,
    pub dest_id: String,
    pub rel_type: RelationType,                      // ej. Auth, Connect, Spawn
    pub timestamp: i64,                               // segundos epoch
    pub metadata: HashMap<String, MetadataValue>,
}
```

Tres observaciones:

1. `id` es un `String` humano (`"alice"`, `"db-01"`, `"10.0.0.42"`). Es la *primary key* del vértice; dos eventos con la misma `id` y `entity_type` se refieren al mismo nodo.

2. `entity_type` es un `enum`: nueve variantes fijas (`User`, `Host`, `Process`, `IP`, `File`, `Domain`, `Service`, `Resource`, `Group`). No es extensible en runtime; cada nuevo tipo requiere agregar la variante y recompilar.

3. `metadata` es flexible. El parser de Sysmon mete acá `ProcessGuid`, `CommandLine`, `ImagePath`; el parser de Sentinel mete `TenantId`, `ResourceGroup`. El matcher lo consulta solamente cuando hay un predicado en la hipótesis (capítulo 30).

8.2. EntityType y RelationType: alfabetos cerrados

```
pub enum EntityType {
    User       = 0,
    Host       = 1,
    Process    = 2,
    IP         = 3,
    File       = 4,
    Domain     = 5,
    Service    = 6,
    Resource   = 7,
    Group      = 8,
    Other      = 255,
}
```

Los *tags* numéricos (0–8) son explícitos por dos razones:

- Permiten almacenar el tipo en un `u8` (1 byte) en vez de un `String` de 6–8 bytes.
- Permiten indexar arrays de tamaño fijo `[u32; 9]` en NLF (capítulo 27) sin *branch* en el lookup.

`Other = 255` es un *escape hatch* para casos ambiguos del parser; el motor lo cuenta separadamente y NO lo incluye en NLF (sería ruido).

`RelationType` es similar, con ∼15 variantes (`Auth`, `Connect`, `Spawn`, `Read`, `Write`, `Beacon`, `Resolve`, `Member`, etc.). El motor usa el `u8 tag` en `CompactRelation::rel_type_tag` para matching sin resolver el enum entero.

8.3. De evento Sysmon a triples

Veamos cómo un evento concreto se convierte en `Entity`s y `Relation`s.

Ejemplo: Sysmon Event ID 1: Process Create

Un evento Sysmon NDJSON crudo:

```
{
    "EventID": 1,
    "TimeCreated": "2026-04-15T14:23:51Z",
    "Computer": "WS-001",
    "User": "CORP\\\\m.gomez",
    "Image": "C:\\\\Windows\\\\System32\\\\powershell.exe",
    "ParentImage": "C:\\\\Windows\\\\explorer.exe",
    "ProcessGuid": "{ABCD-1234-...}",
    "ParentProcessGuid": "{EFGH-5678-...}",
    "CommandLine": "powershell.exe -enc ZQB4ZQ==",
    "Hashes": "MD5=abc...,SHA256=def..."
}
```

El parser `platform/parsers/sysmon` produce de este evento:
3 entidades (idempotente: si ya existen, se ignoran):

- `Entity{ id="CORP\m.gomez", entity_type=User, metadata={}}`
- `Entity{ id="WS-001", entity_type=Host, metadata={}}`
- `Entity{ id="{ABCD-1234-...}", entity_type=Process, metadata={"image": "powershell.exe", "cmdline": "...", "md5": "abc...", "sha256": "def..."}}`
- `Entity{ id="{EFGH-5678-...}", entity_type=Process, metadata={"image": "explorer.exe"}}`

2 relaciones:

- `Relation{ source="WS-001", dest="{ABCD-1234-...}", rel_type=Hosts, ts=1744732431, metadata={}}`
- `Relation{ source="{EFGH-5678-...}", dest="{ABCD-1234-...}", rel_type=Spawn, ts=1744732431, metadata={}}`

Notar:

- Un solo evento generó *dos triples* (proceso hijo spawn-eado, host que lo aloja). Los parsers son "*event-to-N-triples*", no "*one-to-one*".
- Los IDs de los procesos son los `ProcessGuid` (estables), no los `ProcessId` (PIDs reciclados por Windows).
- El `TimeCreated` ISO-8601 se convierte a `i64` segundos epoch —el motor jamás guarda timestamps como string.

8.4. El contrato del parser

Cualquier *adapter* de fuente nueva tiene que cumplir:

```
pub trait LogParser {
    type RawRow;

    fn parse(&self, row: Self::RawRow)
        -> Result<Vec<ParsedTriple>, ParseError>;
```

```
}

pub struct ParsedTriple {
    pub source_id: String,
    pub source_type: EntityType,
    pub dest_id: String,
    pub dest_type: EntityType,
    pub rel_type: RelationType,
    pub timestamp: i64,
    pub metadata: HashMap<String, MetadataValue>,
}
```

Un `ParsedTriple` es "*Entity* + *Entity* + *Relation*" aplanados en una sola estructura —los IDs y los tipos identifican unívocamente las dos entidades involucradas, así que no hace falta enviarlas por separado. El *ingestor* desempaqueta cada triple en dos `add_entity` (idempotente) más un `add_relation`.

En el código.

`platform/parsers/src/lib.rs` define el trait. Cada parser concreto vive en su sub-módulo: `sysmon`, `sentinel`, `evtx`, `cognito`, `forti`, `iis`, `generic_csv`.

8.5. Idempotencia y deduplicación

Una propiedad importante del modelo: ingerir el mismo log dos veces no debería duplicar entidades. Si "alice" ya existe como `User`, el segundo `add_entity("alice", User, ...)` es no-op (*merge* de metadata).

Esto permite reconstrucciones idempotentes: si el ingestor crashea a mitad de un dataset, reanudarlo desde el principio no genera datos duplicados. La idempotencia también vale para `add_relation`, con una salvedad: dos eventos con la *misma* terna $(u, v, \text{rel_type})$ en *distinto* timestamp son relaciones distintas (es un multigrafo). El motor las almacena ambas.

8.6. Metadata: cuándo materializa, cuándo no

El `HashMap<String, MetadataValue>` de cada entidad y relación es flexible pero costoso. El motor aplica una optimización: si nadie consulta la metadata, no se materializa en RAM.

```
pub enum MetadataValue {
    Str(String),
    Int(i64),
    Float(f64),
    Bool(bool),
    Arr(Vec<MetadataValue>),
    Obj(HashMap<String, MetadataValue>),
}
```

En el `CompactRelation` (capítulo 10), la metadata se almacena por *offset* a un `MetadataStore` append-only; un offset de 0 significa "sin metadata" y se chequea sin lookup. Los hunts que no tienen predicados sobre metadata jamás resuelven el offset; el costo es amortizado.

8.7. Resumen

- Modelo público: `Entity` (id + tipo + metadata) y `Relation` (origen + destino + tipo + timestamp + metadata). Legible, parser-friendly.

- Tipos cerrados (9 `EntityType`, ∼15 `RelationType`) con tag numérico explícito que `CompactRelation` reusa.
- Parsers son *event-to-N-triples*; el ingestor desempaqueta cada triple en `add_entity` (idempotente) más `add_relation` (append).
- Metadata se materializa lazy via `MetadataStore` append-only; offset 0 significa "sin metadata".

Ejercicios

1. ★ Para el evento Sysmon Event ID 3 (Network Connection): listar qué entidades y qué relaciones genera el parser. (Pista: hay un proceso, un host, una IP destino, un puerto.)
2. ¿Por qué `EntityType` tiene tag numérico explícito (`User = 0`, `Host = 1`, etc.)? Imaginá la alternativa de dejar que Rust elija; ¿qué problemas surgen para FFI y serialización?
3. Imaginá que mañana aparece una nueva fuente cuyo modelo natural NO encaja en los 9 `EntityType` actuales (digamos, "contenedores Kubernetes"). Discutí dos diseños: (a) agregar `EntityType::Container = 9` y recompilar; (b) usar `Other` y poner el tipo real en metadata. Trade-offs.
4. ★ El `HashMap<String, MetadataValue>` usa strings como keys, lo que cuesta memoria. Diseñá una alternativa con un esquema fijo por `EntityType` (struct estático). ¿Qué ganás? ¿Qué perdés?
5. Discutí: si ingerís el mismo evento dos veces, ¿se duplica la `Relation`? ¿Por qué la respuesta correcta NO es siempre "no"? (Pista: dos eventos con el mismo timestamp pero distinta metadata son distintos eventos del log, aunque *conceptualmente* se refieran a lo mismo.)

Capítulo 9

El *string interner*

Intuición

Las identidades del grafo (`"alice"`, `"db-01"`, `"10.0.0.42"`) son *strings* para los humanos, pero manipularlas como `String` en el matcher es caro: cada `Relation` arrastra dos punteros al heap, cada comparación recorre los caracteres byte por byte, cada hash recalcula desde cero. El *interner* resuelve esto canónicamente: cada string única se almacena *una vez* y se referencia por un `StrId` de 4 bytes (`u32`). El matcher pasa `StrId`s en su *hot loop*; sólo al *emit* de un resultado se resuelve el `StrId` de vuelta a la string visible al humano.

9.1. El problema

Consideremos un grafo de 10 M aristas donde cada arista guarda dos strings (`source_id`, `dest_id`) de 16 caracteres en promedio.

- Como `String`: cada string es 16 B en heap + 24 B de *header* (puntero, longitud, capacidad). Total por string: 40 B. Por arista: 80 B sólo en strings.
- Total: $10^7 \times 80 = 800$ MB sólo para los IDs de las aristas.

Pero las identidades se repiten masivamente: en un grafo típico hay ~ 100 K usuarios distintos, ~ 10 K hosts distintos, ~ 1 M IPs distintas. La cardinalidad real del conjunto de strings es del orden de 10^6, no 10^7; guardar la string completa en cada arista es $\sim 10\times$ más memoria de lo necesario.

Además, el matcher hace $\sim 10^9$ comparaciones de IDs por hunt (cada paso del DFS chequea "¿este vértice ya está bound?" contra los bindings actuales). Comparar dos `String`s de 16 B es 16 cargas de memoria + 16 comparaciones de byte. Comparar dos `u32` es una sola instrucción.

9.2. Definición

Definición 9.1 (String interner)**.** Un *interner* sobre un alfabeto Σ es una estructura de datos que mantiene:

- Un mapeo inyectivo $\sigma\colon \Sigma \to \mathbb{N}$ (la *intern function*).
- Una resolución $\sigma^{-1}\colon \mathbb{N} \to \Sigma$ (la *resolve function*).

Las dos operaciones soportadas son:

- `intern`(s): si s ya está internada, devuelve su id; si no, le asigna el próximo id libre y la guarda.
- `resolve`(id): devuelve la string asociada al id.

9.3. Implementación: tabla bidireccional

La realización canónica:

```
pub struct StringInterner {
    forward: HashMap<Arc<str>, StrId>,  // string  -> id
    backward: Vec<Arc<str>>,            // id      -> string
}

#[derive(Copy, Clone, PartialEq, Eq, Hash)]
pub struct StrId(pub u32);

impl StringInterner {
    pub fn intern(&mut self, s: &str) -> StrId {
        if let Some(id) = self.forward.get(s) {
            return *id;
        }
        let arc: Arc<str> = Arc::from(s);
        let id = StrId(self.backward.len() as u32);
        self.backward.push(arc.clone());
        self.forward.insert(arc, id);
        id
    }

    pub fn resolve(&self, id: StrId) -> Arc<str> {
        self.backward[id.0 as usize].clone()
    }
}
```

Tres detalles importantes:

1. **`Arc<str>` en ambos lados.** Tanto la `HashMap` como el `Vec` guardan `Arc<str>` —no `String`—. Esto permite que `resolve` devuelva un `Arc<str>` clonado (refcount bump, ∼5 ns) en vez de una `String` alocada de nuevo. Es la pieza que destrabó el 55 % de mejora en los benches cap-saturados (capítulo 11).

2. **`StrId` es `u32`, no `u64`.** Asume $< 2^{32} = 4.3 \cdot 10^9$ strings distintas. En GraphHunter eso es muchísimo: el grafo más grande visto en producción tiene $\sim 10^7$ strings únicas, tres órdenes de magnitud por debajo del techo.

3. **No requiere `remove`.** El interner es *append-only*; los IDs son estables por la vida de la sesión. Cuando la sesión se cierra, el interner se descarta completo. Esa propiedad simplifica todo: no hay *tombstones*, no hay re-asignación, no hay locks complejos para "hueco liberado".

9.4. Análisis de costos

9.4.1. `intern`: amortizado $O(1)$

- Lookup en la `HashMap`: $O(1)$ esperado (hash de s, comparación contra el bucket). Para s de longitud L, el hash cuesta $O(L)$ —no es independiente del tamaño de la string—. Pero L es chico (∼10–40 bytes para identidades típicas) y constante en escala asintótica.

- Si miss: alocación de `Arc<str>`, `push` al `Vec`, `insert` al `HashMap`. Los tres son $O(1)$ amortizado (la `Vec` y la `HashMap` crecen geométricamente).

9.4.2. `resolve`: $O(1)$

Indexación directa al `Vec`. `Arc::clone` es un `fetch_add` atómico (∼5 ns). Sin alocación, sin copia de bytes.

9.4.3. Espacio

Sea N el número de strings únicas, L la longitud promedio. La memoria total es:

$$\underbrace{N \cdot L}_{\text{bytes de las strings}} + \underbrace{N \cdot 16}_{\text{Arc<str> headers en backward}} + \underbrace{N \cdot (\sim 40)}_{\text{HashMap buckets}} \approx N \cdot (L + 56).$$

Para $N = 10^6$, $L = 16$: $\sim$ 72 MB. Comparado con los 800 MB que costaría almacenar las strings repetidas en cada arista, el ahorro es $\sim 11\times$.

9.4.4. Comparaciones rápidas

Una vez internadas dos strings, $\sigma(s_1) = \sigma(s_2) \iff s_1 = s_2$. Comparar `StrId == StrId` es una sola comparación de `u32`; `HashMap<StrId, V>` usa el `u32` mismo como hash (no hay re-hash de bytes). Esto es lo que hace que el *hot loop* del DFS pueda chequear "¿`StrId` v ya está bound?" en una sola comparación de palabra de máquina.

9.5. Variantes de diseño

9.5.1. Por qué *no* usamos `Symbol` de string-interner crate

Hay crates de Rust (`string-interner`, `lasso`) que hacen lo mismo. Las consideramos. Los rechazamos por dos razones:

- Devuelven `&str` con *lifetime* atado al interner. El motor necesita `Arc<str>` (ownership compartido, serialización por refcount). Un `&str` se vuelve estorbo cuando el resultado del hunt cruza un *thread boundary* o se serializa al frontend.
- Soportan *dropping* de strings (con *tombstones*, *garbage collection*). En GraphHunter nunca se borra; pagar el overhead de soportar drops sería desperdicio.

9.5.2. Por qué *no* usamos un *trie* de prefix sharing

Un *trie* podría compartir prefijos comunes (`"CORP\m.gomez"` y `"CORP\j.lopez"` compartirían 5 bytes). El ahorro teórico es real pero dos factores juegan en contra:

- Lookup en trie es $O(L)$ con L cargas de memoria no-secuenciales. Para $L = 16$, eso son 16 cache misses posibles —> 200 ciclos. Hash table (con buen hash) es $\sim$ 10 ciclos.
- Devolver `Arc<str>` desde un trie requiere materializar la string concatenando los prefijos. La concatenación cuesta lo que ahorraste.

La hash table `Arc<str>` gana en práctica.

9.6. Concurrencia

El `StringInterner` de GraphHunter es `&mut self` para `intern`: los writers (el ingestor) toman acceso exclusivo durante la inserción. Los `StrId` resueltos son estables, así que los readers (el matcher) pueden trabajar con copias de los IDs sin tocar el interner.

En el código.

`core/graph-engine/src/interner.rs`: `StringInterner` con `intern`, `resolve`. Bajo concurrencia ingest+matching, el interner se envuelve en un `RwLock`; el ingestor toma write-lock para inserts, los matchers toman read-lock para resolves. Los matchers nunca bloquean a otros matchers —el bottleneck efectivo es el ingest, que típicamente es sigle-threaded.

9.7. Por qué importa para el resto del libro

El `StrId` es la representación interna de una identidad en todo el motor:

- `CompactRelation` (capítulo 10) almacena `source_sid: StrId` y `dest_sid: StrId`, cada uno 4 B en lugar de los 24 B de un `String`.
- NLF (capítulo 27) indexa por `StrId` para tener $O(1)$ lookup.
- k-hop reachability (capítulo 28) usa `StrId` como índice en sus arrays `bits` y `rank`.
- El DFS *hot loop* pasa `StrId` entre frames de la recursión sin tocar strings.
- Sólo el *emit* de un resultado al fin del DFS resuelve `StrId` → `Arc<str>`.

Esto último importa: la *cantidad* de resoluciones es proporcional al número de resultados ($\leq$ 10 000 por hunt en default), no al trabajo del DFS (10^9 comparaciones). Pagar 5 ns por resolución es 50 μs de overhead total —trivial.

9.8. Resumen

- Interner: `intern`(s) → id y `resolve`(id) → s. Tabla bidireccional (`HashMap` + `Vec`).
- `StrId` es `u32` (4 B); `Arc<str>` en el backing store permite `resolve` con refcount bump, sin alocar.
- Memoria: $\sim 11\times$ más eficiente que repetir las strings en cada `Relation`; comparaciones $\sim 16\times$ más rápidas que comparar strings byte a byte.
- Ningún `drop`; `StrId`s estables por la vida de la sesión.

Ejercicios

1. ★ Calculá la memoria que ahorraría el interner en un grafo de 10^8 aristas con $N = 10^6$ strings únicas de longitud promedio $L = 20$ B. Compará con almacenar la string completa en cada arista.
2. ¿Por qué la `intern` hash function NO debería ser crypto-grade (SHA-256)? ¿Qué propiedades sí necesita? (Pista: pensá adversarial: ¿qué pasa si un atacante puede elegir las strings que entran al interner?)
3. ★ Implementá una variante del interner que NO use `HashMap` para `forward`, sino un *Vec* ordenado con búsqueda binaria. ¿Qué cambia en la complejidad de `intern`? ¿Cuándo conviene?
4. Imaginá que mañana la red corporativa explota y el interner crece a $N = 10^9$ strings. `StrId = u32` ya no alcanza. ¿Qué cambia? Discutí: ¿migrar a `u64`? ¿segmentar por `EntityType`? ¿ambas?
5. ¿Por qué el interner usa `Arc<str>` en `backward` y no `String`? ¿Qué cambia para

`resolve`?

6. En un sistema con concurrencia ingest + matching, el matcher tiene un `StrId` y quiere resolverlo a la string, pero el ingestor está ejecutando `intern` en paralelo. ¿Hay riesgo de *torn read*? ¿Qué garantiza la consistencia?

Capítulo 10

`CompactRelation`: 32 bytes por arista

Intuición

La `Relation` pública pesa ∼ 200 bytes en RAM por la metadata flexible y los IDs como `String`. A 100 M aristas eso son 20 GB —inviable en una notebook—. La `CompactRelation` es el equivalente *interno* que el matcher itera: 32 bytes exactos, `#[repr(C)]`, dos por *cache line*, *mmap-friendly*. Este capítulo justifica cada byte del layout, explica por qué `repr(C)` es obligatorio (no opcional), y muestra cómo el motor convierte de `Relation` pública a `CompactRelation` en la frontera del ingestor.

10.1. El layout

```
#[repr(C)]
pub struct CompactRelation {
    pub timestamp: i64,              // 8 B (offset  0)
    pub metadata_offset: u64,        // 8 B (offset  8)  handle al
        MetadataStore
    pub source_sid: StrId,           // 4 B (offset 16)
    pub dest_sid: StrId,             // 4 B (offset 20)
    pub ingested_at_delta: u32,      // 4 B (offset 24)  segundos desde
        INGEST_EPOCH_BASE
    pub dataset_tag: u16,            // 2 B (offset 28)
    pub rel_type_tag: u8,            // 1 B (offset 30)
    // 1 B padding implícito         // 1 B (offset 31)
}
// Total: 32 bytes alineados a 8.
```

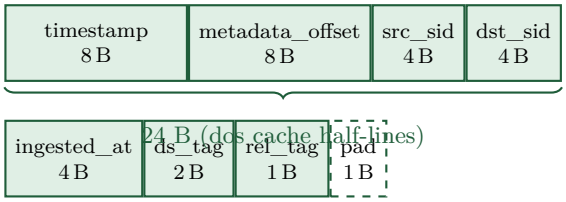


Figura 10.1: `CompactRelation` en `#[repr(C)]`: 32 bytes exactos, alineados a 8. El *layout* fue elegido para que un `&[StreamEdge]` (que comparte el mismo layout byte-a-byte) se mapee directamente a una región mmap cuando el *streaming backend spills* a disco.

10.2. Por qué exactamente 32 bytes (y no 24, no 40)

Tres restricciones cruzaron en el diseño:

1. **Half cache line.** 32 B es la mitad de una cache line típica (64 B). Dos aristas por cache line significa que una iteración secuencial sobre un `Vec<CompactRelation>` procesa dos elementos por cada transferencia de RAM. Si pesara 40 B, sería 1.6 elementos por línea (con líneas a medio aprovechar). 32 B es óptimo en *packing*.

2. **Alineación a 8 para `i64`.** El campo `timestamp` es `i64`; las arquitecturas exigen alineación a 8 B para evitar penalty (en algunas, *cras h*). Con 32 B el siguiente `CompactRelation` arranca también alineado a 8 sin padding entre elementos.

3. **Suficientes bits para los campos.** Cada campo tiene su requerimiento mínimo:

   - `timestamp i64`: epoch en segundos cubre desde ∼292 B AC hasta ∼292 B DC. Margen sobrado para `Relation`s con timestamps razonables.
   - `metadata_offset u64`: handle al `MetadataStore`; con 64 bits podés indexar 16 exabytes de metadata. Margen ridículo, pero `u32` (4 B) limitaría a 4 GB de metadata por sesión, que en algunos casos extremos podría pellizcar.
   - `source_sid u32`, `dest_sid u32`: el interner soporta hasta 2^{32} strings; el motor jamás vio más de 10^7. Margen $\geq 100\times$.
   - `ingested_at_delta u32`: segundos desde `INGEST_EPOCH_BASE` (epoch al inicio de la sesión). Cubre ∼136 años; el motor jamás vio una sesión más larga que un mes.
   - `dataset_tag u16`: identifica de qué dataset proviene la arista (sesiones multi-dataset). 65 536 datasets distintos por sesión es ridículamente generoso.
   - `rel_type_tag u8`: 256 *relation types* distintos. Hoy hay ∼15.

Sumá los campos: $8+8+4+4+4+2+1 = 31$ B. Padding implícito de 1 B para alinear a 8 $\Rightarrow$ 32 B exactos.

10.3. `#[repr(C)]`: por qué es obligatorio

Sin `#[repr(C)]`, el compilador de Rust puede reordenar los campos. Para un struct de 7 campos puede haber $7! = 5040$ permutaciones (no todas válidas por alineación, pero muchas). Distintas versiones del compilador podrían elegir distinto. Eso rompe tres cosas:

1. **Mapeo a mmap.** El *streaming backend* (capítulo 13) escribe la *slice contiguous* `&[StreamEdge]` (mismo `repr(C)` 32 B de `CompactRelation`) a un archivo durante *auto-spill* y la mapea de vuelta a memoria. Si el layout cambia entre escritura y lectura —por ejemplo, sesión guardada con compilador A, cargada con compilador B—, los bytes en disco se interpretan mal. `repr(C)` garantiza layout estable.

2. **FFI (legado y futuro).** Hasta el 2026-05-07 el motor exponía partes de su API como C ABI sobre `libgraphmatch`; pasar un `*const CompactRelation` a código C requería que el layout fuera predecible, sin `repr(C)` no lo era. Esa frontera ya no existe (el matcher es 100 % Rust), pero `repr(C)` se mantiene para cualquier FFI futuro y para el mmap del *streaming backend*.

3. **Razonamiento sobre tamaño.** El programador eligió 32 B; sin `repr(C)`, el compilador podría empaquetar a 24 B o expandir a 40 B según convenga. Las cuentas de "cuántas aristas caben en L2" dejan de ser fijas.

Nota. `repr(C)` *no* garantiza endianness. Un mmap escrito en una máquina x86_64 little-endian y leído en una ARM big-endian (rara hoy en día) tendría los bytes invertidos. GraphHunter asume little-endian (todos los hosts modernos lo son); si en el futuro hay que soportar big-endian, hay que serializar/deserializar campo por campo en lugar de `mmap` directo.

10.4. Conversión: `Relation` → `CompactRelation`

Cada `add_relation` hace la conversión:

```
pub fn add_relation(&mut self, r: Relation) -> Result<(), Error> {
    let source_sid = self.interner.intern(&r.source_id);
    let dest_sid   = self.interner.intern(&r.dest_id);
    let metadata_offset = if r.metadata.is_empty() {
        0  // sentinel: "no metadata"
    } else {
        self.metadata_store.append(&r.metadata)?
    };
    let ingested_at_delta =
        (self.now() - self.ingest_epoch_base) as u32;
    let cr = CompactRelation {
        timestamp: r.timestamp,
        metadata_offset,
        source_sid,
        dest_sid,
        ingested_at_delta,
        dataset_tag: self.current_dataset_tag,
        rel_type_tag: r.rel_type as u8,
    };
    self.push_to_streaming(&cr);  // sole canonical store
    Ok(())
}
```

Lo importante para el resto del libro:

- `source_sid` y `dest_sid` salen del interner (capítulo 9). Strings comunes (`"alice"`) se internan *una vez* y todos los `CompactRelation` que los referencian comparten el `StrId`.
- `metadata_offset = 0` es el sentinel "sin metadata"; el matcher chequea `== 0` sin lookup al `MetadataStore`.
- `rel_type_tag` es un `u8` casteado del enum `RelationType`; el matcher usa el `u8` directo en sus comparaciones, sin resolver el enum.

10.5. Por qué el matcher itera `CompactRelation`, no `Relation`

El matcher hot path itera $\sim 10^7$ aristas por hunt. Para cada arista chequea: ¿el tipo coincide?, ¿el timestamp cae en la ventana?, ¿el source binding ya está fijado? Cada chequeo tiene que ser una operación de palabra de máquina, no una desreferencia + recorrido de string.

- Type check: `cr.rel_type_tag == hypothesis.step_tag` (1 cache line, 1 comparación).
- Timestamp check: `lo` $\leq$ `cr.timestamp` $\leq$ `hi` (datos ya en cache desde el step anterior, 2 comparaciones).
- Source binding check: `cr.source_sid == binding[var]` (1 comparación de `u32`).

Tres operaciones, tres palabras de máquina. Una iteración del *inner loop* del DFS es del orden de ~ 10 ciclos. Iterar la `Relation` pública sería del orden de $\sim 10^3$ ciclos por la indirección a strings y al hashmap de metadata.

10.6. Resumen

- `CompactRelation`: 32 B exactos, `#[repr(C)]`, alineados a 8, dos por cache line.
- Cada campo justificado: `i64` timestamp para margen, `u32` sids por el interner, `u8 rel_tag` para matching sin resolver enum, `u64` metadata_offset por escala extrema.
- `repr(C)` obligatorio para mmap stability, FFI, y razonamiento de tamaño.
- Conversión `Relation` → `CompactRelation` en `add_relation`, vía interner para los sids y `MetadataStore` para metadata.
- El matcher itera `CompactRelation` (~ 10 ciclos por arista) en lugar de `Relation` ($\sim 10^3$ ciclos por arista).

Ejercicios

1. ★ Calculá: en una cache L2 de 1 MB, ¿cuántas aristas `CompactRelation` caben? ¿Cuántas `Relation` de 200 B caben? ¿Cuál es el ratio?
2. Diseñá un layout alternativo de 24 B para una `CompactRelation` más chica. ¿Qué campos sacrificás? ¿Bajo qué condiciones (qué tipo de hunts) sería preferible?
3. ★ Si quitás `#[repr(C)]`, el compilador podría reordenar a `(timestamp, metadata_offset, source_sid, dest_sid, ingested_at_delta, dataset_tag, rel_type_tag)` = 31 B + 1 B padding. Mismo tamaño total. ¿Está mal? Discutí: ¿qué se rompe específicamente?
4. Calculá: para un grafo de 10^8 aristas almacenadas como `Vec<CompactRelation>` ordenadas, ¿cuánta memoria total ocupan? Si la notebook tiene 16 GB de RAM, ¿cabe?
5. ¿Por qué `metadata_offset` es `u64` y `source_sid` es `u32`? Justificá la asimetría desde la escala esperada de cada uno.
6. Imaginá que quisieras agregar un campo `flags: u8` a `CompactRelation` (por ejemplo, para marcar la arista como "deprecated" por una operación futura). ¿Cuál es el impacto en el layout? ¿Tenés que crecer a 40 B?

Capítulo 11

HuntResult y la migración a Arc<str>

Intuición

Un `HuntResult` es una secuencia de *vertex IDs* resueltos al alfabeto humano: el *path* encontrado por el matcher, ya listo para mostrarse en la UI o serializarse al frontend. La elección concreta de *cómo* representar esa secuencia —`Vec<String>` versus `Vec<Arc<str>>`— parece menor, pero un bench de matching cap-saturado mostró que la opción equivocada cuesta $\sim 55\,\%$ del tiempo total. Este capítulo cuenta la historia: por qué el `Vec<String>` original era el cuello de botella, por qué la "solución obvia" (materialización diferida) fracasó, y por qué finalmente `Vec<Arc<str>>` fue la que cerró el caso.

11.1. La forma del resultado

```
pub type HuntResult = Vec<Arc<str>>;
```

Un `HuntResult` es un `Vec` con `path.len() + 1` elementos, uno por vértice del path encontrado:

- Hipótesis `User -[Auth]-> Host -[Connect]-> IP` ($k = 2$ aristas, 3 vértices) $\Rightarrow$ cada `HuntResult` tiene 3 elementos.
- Hipótesis `User -[Auth]-> Host` ($k = 1$) $\Rightarrow$ 2 elementos.
- Cap del motor: 10 000 `HuntResults` por hunt (default). Suficientes para que el analista vea "los sospechosos"; muchos más sería ruido.

La salida del matcher es `Vec<HuntResult>` —una lista de paths—. El frontend la pagina; el scoring la ranquea.

11.2. El bench que disparó el cambio

En 2026-04 corrimos benches sobre el patrón `matching_powerlaw / ba-any-2hop V=10000`: un grafo *Barabasi-Albert* (scale-free) de 10 000 vértices, hipótesis `any -[any]-> any -[any]-> any` (sin filtros de tipo). El hunt satura el cap (10 000 resultados). Tiempo total: 2.4 ms.

Patcheamos `dfs_match_iterative` para que el *emit* del resultado fuese `Vec::new()` en lugar de `path.iter().map(…).collect()`. Resto idéntico. Tiempo nuevo: 1.06 ms. **Diferencia:** $-55\,\%$.

Bench	Inline strings	Empty push	Δ
ba-typed-2hop V=1000	86 μs	75 μs	−13 %
ba-typed-2hop V=10000	165 μs	196 μs	+19 % noisy
ba-any-2hop V=1000	2.24 ms	854 μs	**−62 %**
ba-any-2hop V=10000	2.38 ms	1.06 ms	**−55 %**

El patrón es claro: cuando el cap *no* se satura (`ba-typed` con tipos que filtran agresivo), el costo del emit es marginal. Cuando el cap se satura (`ba-any`: 10 000 resultados emitidos), el costo del emit domina la mitad del tiempo.

11.3. Por qué `Vec<String>` era el problema

Cada `emit` hacía:

```
let path: Vec<String> = current_path
    .iter()
    .map(|sid| interner.resolve(*sid).to_string())
    .collect();
results.push(path);
```

Cada elemento de `path` es una `String` fresca —`to_string()` aloca bytes en heap y los copia desde el `Arc<str>` del interner. Tres operaciones que cuestan nanosegundos cada una, multiplicadas por:

- 3 vértices por path.
- 1 `Vec::new` + 3 `push` per path.
- 10 000 paths.

$10\,000 \cdot 4 = 40\,000$ allocaciones. Con `mimalloc` ($\sim$ 65 ns por alloc chico), eso son 2.6 ms —que es exactamente el tiempo total observado. Las allocaciones de `String` *eran* todo el bench.

11.4. Por qué la materialización diferida fracasó

La idea natural: pushear algo barato dentro del DFS (por ejemplo, `SmallVec<[StrId; 8]>`) y materializar las strings al final, quizás en paralelo con `rayon`. Implementamos esto. El bench **regresó** +48 %.

Dos razones que se compusieron:

1. **Tamaño del elemento intermedio.** Cada `SmallVec<[StrId; 8]>` pesa 40 B (8 × `u32` inline + len + cap). 10 000 elementos = 400 KB de buffer intermedio. Eso busta L2 (256 KB en las E-cores del Galaxy Book4); las próximas iteraciones pagan cache miss.
2. **Two-pass cache miss.** La fase DFS produce, la fase materialización lee. La versión inline produce y descarta inmediatamente (el writer thread mantiene los bytes en L1 hasta que rayon los drena). La versión two-pass mata esa *producer-consumer locality*.

Lección: optimizar "sólo el hot loop" sin pensar en el working set total puede empeorar el rendimiento. La cache importa tanto como las allocaciones.

11.5. La solución: Vec<Arc<str>>

El cambio que finalmente cerró el caso fue de tres pasos:

1. Refactor del `StringInterner` para que su backing sean `Arc<str>`, no `String`. `resolve` devuelve `Arc<str>` clonado (refcount bump, ~ 5 ns), no `String` alocado.
2. Cambio de tipo: `HuntResult = Vec<String>` $\rightarrow$ `Vec<Arc<str>>`.
3. Audit de los ~ 35 call sites que tocan `HuntResult` (MCP, REST DTOs, Tauri TS bindings, y la capa FFI que sobrevivió hasta el 2026-05-07). La mayoría eran `result.iter().map(|s| s.as_str())`-shaped y migraron mecánicamente.

Resultado: el *emit* ahora es:

```
let path: Vec<Arc<str>> = current_path
    .iter()
    .map(|sid| interner.resolve(*sid))   // Arc::clone, refcount bump
    .collect();
results.push(path);
```

3 `Arc::clone` (~ 5 ns cada uno) + 1 alloc del `Vec` per path. $10\,000 \cdot (3 \cdot 5 + 65)$ ns ≈ 0.8 ms —comparable al "empty push" baseline. La mejora del 55 % bajó al PR.

11.6. Por qué Arc<str> en vez de Rc<str>

`Rc` (reference count) es la versión *single-threaded*: el contador no es atómico. `Arc` es atómica. `Rc` sería $\sim 2\times$ más rápida (un increment normal vs un `fetch_add`), pero:

- Los `HuntResult` cruzan boundaries de thread: el matcher los produce, el scoring los consume desde otro thread, eventualmente cruzan al frontend vía Tauri.
- `Rc` no es `Send`, así que el compilador rechazaría el cruce. La alternativa sería convertir a `Arc` en el boundary, lo que neutraliza la ganancia.

`Arc` de entrada gana en simplicidad y predictibilidad.

11.7. Por qué Arc<str> en vez de Arc<String>

`Arc<String>` agrega un nivel de indirección: el `Arc` apunta a un `String`, que apunta al heap con los bytes reales. Dos cargas de memoria para llegar a los caracteres.

`Arc<str>` es un *fat pointer* directo: 16 bytes en el `Vec` (puntero a heap + length). El heap contiene `(refcount, bytes_de_la_string)`. Una sola carga de memoria. `Arc<str>` gana en cache locality y en tamaño del `Vec` (`Arc<String>` sería 8 B per element pero con indirección extra; `Arc<str>` es 16 B per element pero directo).

11.8. Implicaciones para el resto del sistema

11.8.1. Serialización

El frontend Tauri serializa `Vec<HuntResult>` como JSON. `serde_json` maneja `Arc<str>` igual que `String` —son trivialmente serializables como JSON strings. Cero cambios en los serde derives.

11.8.2. FFI (histórico)

Nota 2026-05-07: la frontera FFI Rust↔C++ sobre `libgraphmatch` fue eliminada al demostrarse que el DFS Rust era 3-25× más rápido. Lo que sigue describe el costo que pagaba el adapter mientras esa frontera existía; tras la eliminación ya no hay un boundary que cruzar.

La C ABI antigua esperaba `*mut *mut c_char` (puntero a array de strings C). El adapter convertía cada `Arc<str>` a `CString` antes de cruzar el boundary; eso pagaba la alloc una vez por resultado, pero solo en consumers FFI (no en Tauri ni HTTP). El costo lo pagaba el caller que decidía usar FFI; hoy ningún caller lo paga.

11.8.3. Memoria total

En un hunt cap-saturado:

- Antes: 10 000 paths × 3 vertices × (24 B header + 16 B promedio bytes) = 1.2 MB de heap por hunt.
- Después: 10 000× 3 × 16 B (`Arc<str>`) = 480 KB de heap por hunt. Las strings reales se *comparten* con el interner; ningún byte se duplica.

Bonus: 2.5× menos memoria por hunt, además del 55 % menos tiempo.

11.9. Cuándo *no* vale la pena `Arc<str>`

Tres escenarios donde el patrón pierde:

1. **Strings únicas.** Si nunca se comparten (cada `HuntResult` tiene strings que aparecen una sola vez), el costo de `Arc` (16 B header + atómica de refcount) es overhead puro. Pero las identidades de un grafo se repiten masivamente; este escenario no aplica.
2. **Stringes muy chicas.** Si las strings son de 1–2 caracteres, el header del `Arc` domina (∼30 B overhead para 2 B útiles). Las identidades de threat hunting promedian 10–40 caracteres; ratio sano.
3. **Lifecycle predecible.** Si conocés exactamente cuándo se libera la string, una arena (bump allocator) gana sobre `Arc`. No es el caso de `HuntResult`: cruza threads, vive en caches, la duración es por-hunt.

11.10. Resumen

- `HuntResult = Vec<Arc<str>>`: una secuencia de vértices del path encontrado por el matcher.
- Cap saturado (10 000 results): el *emit* domina si cada elemento es una `String` alocada nueva.
- Materialización diferida fracasó por *cache miss* en el buffer intermedio + pérdida de producer-consumer locality.
- `Arc<str>` resuelve: el `resolve` del interner clona refcount (∼ 5 ns), las strings se comparten, sin allocs en el hot path.
- Mejora: 55 % menos tiempo, 2.5× menos memoria en cap-saturated benches. Audit de ∼ 35 call sites mecánico.

Ejercicios

1. ★ Calculá el ahorro de memoria por hunt para una hipótesis de $k = 5$ saltos (6 vértices) con cap saturado a 10 000 resultados, asumiendo identidades de 24 B promedio. Compará `Vec<String>` vs `Vec<Arc<str>>`.

2. ¿Por qué `Arc<str>` es mejor que `Arc<String>`? Dibujá el layout de memoria de cada uno.

3. ★ Imaginá que un atacante puede inyectar strings en la ingesta. ¿Hay algún ataque DoS contra el interner que el cambio a `Arc<str>` *no* mitiga? (Pista: pensá en cardinality blowup.)

4. Diseñá un caso donde la materialización diferida *ganaría*. (Pista: ¿qué pasa si el cap es muy chico, digamos 100 resultados? ¿Y si el path es muy largo, digamos 20 vértices?)

5. `Vec<Arc<str>>` pesa 16 B por elemento + 24 B de header del Vec. Para 10 000 paths de 3 elementos, ¿cuánta memoria es?

6. Histórico: cuando existía la C ABI, ésta exponía `*mut *mut c_char` (no `Arc<str>`). Discutí: ¿qué clases de bug surgirían si hubiéramos pasado un `Arc` por FFI? Esa pregunta perdió urgencia el 2026-05-07 al eliminar la frontera FFI.

Parte IV

Pipeline de ingesta

Capítulo 12

Anatomía del *ingest*

Intuición

"Ingesta" es el camino *log* → *grafo*. Es el componente con menos magia algorítmica del libro pero el de más impacto en escalabilidad: si el ingestor lee el archivo entero a RAM, una sesión de 28 GB es inviable; si lo procesa en *batches* de 50K filas el peak RAM queda en ∼10 MB sin importar el tamaño del archivo. Tres componentes orquestan el flujo: `LogSource` ("decime el próximo chunk de bytes"), `LogParser` ("convertí estos bytes en triples"), y `GraphWriter` ("empujá estos triples al grafo"). Conectados por canales async, funcionan como un pipeline donde cada etapa procesa lo que la anterior produjo, sin acumular.

12.1. Diagrama del pipeline

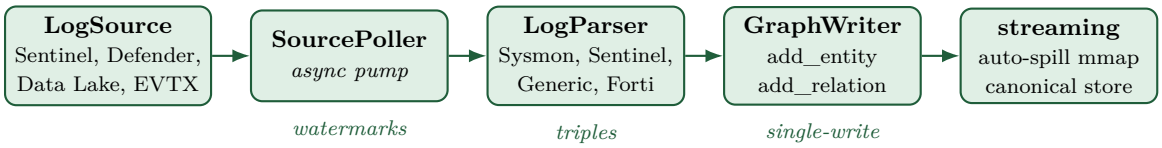


Figura 12.1: Pipeline de ingesta. Cada caja es un componente con su propio módulo en `platform/sources`, `platform/parsers` o `core/graph-engine`. Las flechas son canales async (`tokio::sync::mpsc`).

12.2. Etapa 1: `LogSource`

```rust
pub trait LogSource: Send + Sync {
    type Chunk;
    async fn next_chunk(&mut self) -> Option<Self::Chunk>;
    fn watermark(&self) -> Option<i64>;  // último ts visto
}
```

Una `LogSource` es un cursor sobre una fuente concreta. El `SourcePoller` —el componente que orquesta el pipeline— le pide chunks una y otra vez hasta que devuelve `None`.

12.2.1. Implementaciones

- `SentinelSource`: query KQL contra Microsoft Sentinel. Pagina hasta que la API devuelve un cursor vacío.
- `DefenderXdrSource`: similar, para Microsoft Defender XDR.

- `DataLakeSource`: lee archivos Parquet desde un bucket S3-compat. Itera por partición temporal.
- `EvtxSource`: lee archivos `.evtx` binarios de Windows. Usa el crate `evtx` para parsear el formato binario.
- `NdjsonFileSource`: archivos NDJSON locales (uso típico para datasets de prueba).

12.2.2. Watermarks

Algunas fuentes tienen ordering temporal natural (Sentinel pagina por timestamp); otras no (Data Lake puede entregar particiones en cualquier orden). `watermark()` devuelve el menor timestamp de eventos *garantizado-ya-vistos*: el ingestor lo usa para saber hasta qué momento puede hacer *commit* de aristas en modo *streaming*.

12.3. Etapa 2: `LogParser`

```
pub trait LogParser: Send + Sync {
    type Chunk;
    fn parse(&self, chunk: Self::Chunk)
        -> Result<Vec<ParsedTriple>, ParseError>;
}
```

El `LogParser` convierte un chunk crudo (bytes, NDJSON, EVTX binario) en una secuencia de `ParsedTriple` (capítulo 8). Cada parser concreto vive en `platform/parsers/`:

- `SysmonNdjsonParser`: NDJSON de Sysmon (Event ID 1, 3, 7, 8, 11, etc.). Cada Event ID se mapea a uno o varios triples.
- `SentinelKqlParser`: rows JSON del schema canonical de Sentinel.
- `EvtxBinaryParser`: el XML EVTX binario crudo.
- `GenericCsvParser`: CSV con *field config* explícito (qué columna es source, qué columna es timestamp, etc.). Usado para datasets ad-hoc.
- `CognitoParser`, `FortianalyzerParser`, `IisParser`: formatos verticales específicos.

12.3.1. El parser es *event-to-N-triples*

Un evento de log puede generar 0, 1, o varios triples. El parser de Sysmon Event ID 1 (Process Create) genera 2 triples (host-hosts-process, parent-spawn-process). El parser de un Event ID irrelevante (un debug log) genera 0. El parser de un evento "compuesto" (autenticación con MFA) puede generar 3 (user-auth-host, host-mfa-device, user-owns-device).

12.4. Etapa 3: `GraphWriter`

```
pub struct GraphWriter {
    graph: Arc<RwLock<GraphHunter>>,
}

impl GraphWriter {
    pub fn apply(&self, triple: ParsedTriple) {
        let mut g = self.graph.write();
        g.add_entity(Entity {
```

```
            id: triple.source_id.clone(),
            entity_type: triple.source_type,
            metadata: HashMap::new(),
        });
        g.add_entity(Entity {
            id: triple.dest_id.clone(),
            entity_type: triple.dest_type,
            metadata: HashMap::new(),
        });
        g.add_relation(Relation {
            source_id: triple.source_id,
            dest_id: triple.dest_id,
            rel_type: triple.rel_type,
            timestamp: triple.timestamp,
            metadata: triple.metadata,
        });
    }
}
```

`add_entity` es idempotente (deduplicado por el interner, capítulo 9); `add_relation` es append + *streaming mirror* (capítulo 13).

12.5. El `SourcePoller`

```
pub async fn run<S, P>(
    mut source: S,
    parser: Arc<P>,
    writer: Arc<GraphWriter>,
) -> Result<IngestStats, Error>
where
    S: LogSource,
    P: LogParser<Chunk = S::Chunk>,
{
    let mut stats = IngestStats::default();
    while let Some(chunk) = source.next_chunk().await {
        let triples = parser.parse(chunk)?;
        stats.events += 1;
        stats.triples += triples.len();
        for triple in triples {
            writer.apply(triple);
        }
    }
    Ok(stats)
}
```

El loop es deliberadamente simple: una fila a la vez, sin batching explícito. La razón: con `add_relation` pesando $\sim$1 μs (interner lookup + push al store + push al streaming), el overhead de batching no compensa la complejidad.

12.5.1. Concurrencia: ¿múltiples writers?

GraphHunter mantiene el ingestor *single-threaded*. Razón: el `GraphHunter` interno tiene varias estructuras (interner, edge store, streaming, índices invalidations) que serían difíciles de coordinar bajo varios writers. El bottleneck efectivo no es CPU del ingestor sino I/O de la fuente; un solo thread alcanza para saturar Sentinel KQL o un disco SSD.

Cuando hace falta acelerar (datasets multi-fuente), se corren *varios SourcePollers en paralelo* —uno por fuente— pero todos llaman al mismo `GraphWriter` bajo lock. El lock en `add_relation`

es un `RwLock` en write mode; los matchers concurrentes usan read mode y no se bloquean entre sí.

Nota. El streaming backend (capítulo 13) está diseñado para soportar mejor la concurrencia: cada vértice tiene su propia `NeighborList` con sincronización fina. La migración del matcher para usar streaming como source of truth (stage 2c) abre la puerta a varios writers concurrentes; ese cambio está en diseño futuro (capítulo 35).

12.6. Memoria del ingestor: peak vs total

Una propiedad importante: el *peak* RAM del ingestor es del orden del *batch size*, no del tamaño del archivo.

- Chunk típico: 50.000 NDJSON rows, ∼10 MB de bytes crudos.
- Triples generados: ∼25.000 (algunos eventos no generan triples, otros generan varios).
- Memoria viva por chunk: ∼10 MB de bytes + ∼5 MB de `ParsedTriple`s. Se libera cuando el chunk se consumió.

La memoria *total* consumida (lo que se acumula en el grafo) sí crece con el tamaño del log: 28 GB de EVTX produce ∼200 M aristas ⇒ ∼6.4 GB de `CompactRelation`. Pero *ese* crecimiento es lineal y predecible; el auto-spill del *streaming backend* (capítulo 13) lo absorbe cuando excede el budget.

12.7. Provenance

Cada triple lleva implícitamente la información de *de qué fuente vino*: el `SourcePoller` setea `current_dataset_tag` en el grafo antes de empezar, y `add_relation` lo copia a `CompactRelation::dataset_ta` (capítulo 10). Para sesiones multi-fuente, los matchers pueden filtrar por `dataset_tag` para restringir hunts a un subset.

12.8. Resumen

- Pipeline tres-etapas: `LogSource` (chunks), `LogParser` (triples), `GraphWriter` (entities/relations).
- Async via `tokio::sync::mpsc`; canales conectan las etapas.
- Peak RAM = batch size, no archivo. Total RAM = aristas acumuladas (con spill cuando excede budget).
- Single-threaded por simplicidad; bottleneck es I/O de la fuente, no CPU.
- Provenance vía `dataset_tag` en cada arista.

Ejercicios

1. ★ Si una fuente Sentinel devuelve 1.000 eventos por segundo y cada uno genera 1.5 triples promedio, ¿cuántas aristas/segundo agregás al grafo? Si la sesión dura 8 horas, ¿cuántas aristas en total?
2. Dibujá el peak RAM del ingestor a lo largo del tiempo para una sesión de 28 GB. ¿Tiene picos? ¿Por qué se estabiliza en ∼10 MB del chunk?
3. ★ El loop del `SourcePoller` es `while let Some(chunk) =`

`source.next_chunk().await`. ¿Qué pasa si la fuente nunca devuelve `None` (un streaming infinito de Sentinel pulling)? Diseñá una *stop condition* adecuada.

4. Imaginá que querés correr 10 `SourcePoller`s en paralelo (10 fuentes distintas). ¿Cómo afecta al `RwLock` del `GraphHunter`? ¿Qué cuello de botella se vuelve dominante?

5. ¿Por qué `add_entity` es idempotente y `add_relation` no? Discutí qué cambia para el ingestor si lo corriera con `add_relation` idempotente (deduplicando por terna).

Capítulo 13

El *streaming backend*: *canonical store*

Intuición

`add_relation` hace un *single write* al `streaming` backend —el almacenamiento canónico desde stage 2c part 2 (capítulo 35). El backend es una estructura *degree-adaptive per-vértice* (`Inline` para grado bajo, `Vec<StreamEdge>` para grado alto, `Mmap` para vértices spilled a disco) con *auto-spill incremental* cuando el budget global cruza un umbral. Lecturas son *zero-copy*: el matcher hot-path toma `neighbors_arc(sid)` + `guard.read()` + `as_slice()` y consume un `&[StreamEdge]` con la misma forma que el legacy `&[CompactRelation]`. Este capítulo explica el shape degree-adaptive y la mecánica del auto-spill; el capítulo siguiente analiza cómo se hizo el migrate del matcher para que la antigua copia paralela (`SpillableEdgeStore`) pudiera eliminarse.

13.1. Por qué un *streaming backend* y no un `Vec`

La forma natural de un edge store sería un `Vec<CompactRelation>` sortable + binary-searchable. Eso fue el `SpillableEdgeStore` legacy. Tiene tres limitaciones para los consumers actuales:

- **Sólo binary-searchable post-`finalize`**: durante ingesta el slice no está sorted, así que un matcher incremental que querría re-evaluar una hipótesis con cada arista nueva no puede usar `partition_point` sobre el rango temporal.
- **No expone *per-vertex* access barato**: cada `get_edges(sid)` hace un *HashMap lookup*; los consumers que iteran muchos vértices de un grafo (NLF build, khop_reach build) pagan ese lookup por vértice.
- **Spill-to-disk one-shot**: spillea cuando el buffer cruza un umbral, pero post-spill no acepta más writes hasta el próximo finalize. No sirve para *ingesta continua*.

El *streaming backend* resuelve los tres simultáneamente: per-vertex *slot* en un `HashMap<StrId, Arc<RwLock<NeighborList>>>`, *auto-spill incremental* con archivos round-numbered, y un *per-vertex slice* sorted-by-timestamp post-`finalize` que el matcher consume con `partition_point` igual que el viejo store.

13.2. La estructura: `NeighborList` degree-adaptive

Cada vértice del streaming backend tiene una `NeighborList` con tres variantes:

```
pub enum NeighborList {
    Inline(SmallVec<[StreamEdge; 8]>),    // <= 8 aristas
    Vec(Vec<StreamEdge>),                  // > 8 aristas, in-RAM
```

```
    Mmap { handle: Arc<MmapHandle>,        // spilled
           byte_offset: usize, len: usize },
}
```

La elección depende del estado del vértice:

- **Inline**: 8 aristas *en la propia struct*, sin indirección. Cero allocs para vértices de grado bajo (la mayoría en grafos scale-free). Cabe en una cache line.
- **Vec**: cuando el grado pasa 8, se promueve a `Vec<StreamEdge>`. Un alloc, acceso secuencial óptimo. El matcher hot-path lee el slice contiguo con `partition_point` para acotar ventanas temporales en $O(\log d)$ por frame.
- **Mmap**: cuando el budget total cruza el umbral (default 1 M aristas, configurable vía `with_memory_budget_edges`), todas las `NeighborList` *Inline/ Vec* se vuelcan a un archivo memory-mapped sorted-by-timestamp y los slots pasan a `Mmap { handle, byte_offset, len }`. El `Arc<MmapHandle>` mantiene el archivo mapeado mientras quede al menos una referencia. Lecturas siguen siendo zero-copy: `as_slice()` sobre `Mmap` proyecta el rango como `&[StreamEdge]`.

13.2.1. Por qué este shape y no la antigua `Btree`

Una iteración previa del diseño tenía una variante `Btree` para hubs (>128 aristas) que daba $O(\log d)$ en queries por ventana temporal. Se eliminó en *stage 2c part 2* a favor de `Vec` contiguo con `partition_point`: el matcher sostiene un slice borrowed sobre toda la duración del frame del DFS con prefetch lookahead, y un `BTreeMap` no puede exponer un slice contiguo. La pérdida algorítmica ($O(d)$ lineal vs $O(\log d)$) en queries por ventana se materializa solo en consumers de `iter_window` que ningún path de producción usaba; el matcher hot-path tiene su propio `partition_point` sobre el slice *Vec/Mmap* que sigue dando $O(\log d)$.

En grado bajo, `Inline` elimina el header del Vec (24 B) + heap alloc + indirección — para 1 M vértices con grado 1, eso son 24 MB salvados.

13.3. Promotion + auto-spill

Las transiciones entre variantes son atómicas; cada `append_edge` recorre tres puntos:

```
fn append_edge(&mut self, edge: StreamEdge) {
    match self {
        NeighborList::Inline(sv) if sv.len() < 8 => {
            sv.push(edge);
        }
        NeighborList::Inline(sv) => {
            // promote to Vec
            let mut v: Vec<StreamEdge> = sv.iter().copied().collect();
            v.push(edge);
            *self = NeighborList::Vec(v);
        }
        NeighborList::Vec(v) => {
            v.push(edge);
        }
        NeighborList::Mmap { .. } => {
            // post-spill write: copy out, append, swap
            let mut v: Vec<StreamEdge> = self.as_slice().to_vec();
            v.push(edge);
            *self = NeighborList::Vec(v);
        }
```

```
    }
}
```

Inline → *Vec*: O(8) memcpy una vez por vértice cuando cruza grado 8. Amortizado O(1).

Vec → *Mmap*: no es per-vertex sino un evento *global* disparado por el budget tracker `edges_since_last_spil`
El backend hace `auto_spill()` que sortea cada per-vertex Vec por timestamp, escribe un archivo round-numbered, y swap-ea cada slot a `Mmap`. Costo: O(E) bytes de I/O secuencial, una vez por round.

Mmap → *Vec*: O(d) memcpy out of mmap al primer write post-spill sobre ese vértice. Producción no escribe post-spill en el caso normal (toda la ingesta termina antes de que el analista corra hunts), así que es cold path.

13.4. Wait-free reads: `neighbors_snapshot`

La pieza interesante: el matcher quiere leer `NeighborList` mientras el ingestor escribe. Las opciones clásicas:

- `Mutex`: serializa todo. Lento bajo lectura intensiva.
- `RwLock`: múltiples readers, pero el writer bloquea a todos los readers. Si la promotion es lenta, el matcher se traba.
- *Wait-free*: los readers nunca bloquean. Esquema típico: el writer construye el nuevo estado en un buffer separado y publica el puntero atómicamente; los readers ven el puntero viejo o el nuevo, nunca un estado intermedio.

El streaming backend implementa la tercera opción usando `ArcSwap<NeighborList>`: cada vértice tiene un puntero atómico a su `Arc<NeighborList>`. `neighbors_snapshot` hace un `load` atómico del puntero y devuelve el `Arc` clonado:

```
pub fn neighbors_snapshot(&self, sid: StrId) -> Arc<NeighborList> {
    self.per_vertex[sid.0 as usize].load_full()
}
```

El matcher ahora tiene su propia copia (refcount-shared) del `NeighborList` que existía en el momento del `load_full()`. El writer puede seguir publicando nuevas versiones; el matcher ve la suya *consistente* hasta que descarte el `Arc`.

13.4.1. Costo

- Reader: 1 load atómico + 1 `Arc::clone` (`fetch_add`) = $\sim$ 10 ns total. Sin bloqueo.
- Writer: construir nuevo estado (puede ser $O(\text{deg})$ por promotion), 1 store atómico para publicar. El reader que vino *antes* del store ve la versión vieja; el reader que vino *después*, la nueva. No hay raza visible.

13.4.2. Memory reclamation

El reto de wait-free es *cuándo liberar* el estado viejo. Si un reader todavía tiene un `Arc` a la versión vieja, el writer no puede liberarla. `ArcSwap` resuelve esto con *deferred drop*: el viejo `Arc` se desreferencia al liberar el `ArcSwap`; el reader es responsable de soltar su clone cuando termine. El refcount atómico hace el resto.

13.5. ¿Cómo se usa hoy?

Post stage 2c part 2 el streaming backend es *el* edge store. Cinco *ingest paths* comparten un solo helper `push_to_streaming`: `add_relation`, `add_relations_batch`, `replay_relations`, `compact_before`'s rebuild, `remove_entities_and_relations_by_dataset`'s rebuild.

Lectores production:

- **Matcher hot path** (`dfs_match_smart`, `dfs_match_iterative`, `edge_range_sid`, AMAC pair): `neighbors_arc(sid) + read guard + as_slice()`. Slice borrowed sobre toda la duración del frame del DFS, con `partition_point` para acotar ventanas temporales.
- **Index builds** (NLF, khop_reach, lftj, yannakakis): `neighbors_arc` per-vertex preserva cache locality sobre los acumuladores indexados por `v_idx`.
- **Aggregaciones admin** (anomaly scorer warm-up, temporal heatmap, timeline data, schema cache fingerprint): `streaming.for_each_edge(closure)` con un solo *outer guard* para todo el sweep.
- **Live-tail UI** (`cmd_get_recent_edges`) y telemetría (`cmd_get_ingest_stats`): consumers delta-aware originales que sobrevivieron la migración.

En el código.

`core/graph-engine/src/streaming.rs` expone el módulo completo: `InMemoryStreamingBackend` con `append_edge`, `finalize`, `auto_spill`, `neighbors_arc`, `for_each_edge`, `for_each_vertex`, `consolidate`, `clear`. El antiguo `spill.rs` con `SpillableEdgeStore` se eliminó (stage 2c part 2 cleanup).

13.6. Resumen

- Streaming backend = el *único* edge store, per-vertex con `NeighborList` *degree-adaptive*: `Inline` (≤ 8), `Vec` (resto in-RAM), `Mmap` (post-spill).
- Auto-spill incremental dispara cuando el budget global (default 10^6 aristas) cruza el umbral; sortea cada slice *durante el write*, así el archivo es matcher-ready.
- *Post-spill* writes promueven `Mmap` $\rightarrow$ `Vec` sólo en el vértice tocado: cold path, una memcpy out of mmap.
- Lecturas son zero-copy via `neighbors_arc + read guard + as_slice`: el matcher hot-path sostiene el slice sobre toda la duración del frame, igual que el legacy `&[CompactRelation]`.
- `StreamEdge` es layout-compatible con `CompactRelation` (`#[repr(C)]`, mismo orden), así que los $\sim$50 readers del antiguo dual-write iteran el nuevo slice con field-access idéntico.

Ejercicios

1. ★ Para un grafo de 1 M vértices distribuidos como power-law ($\alpha = 2$), estimá: cuántos vértices van a cada variante (`Inline`, `Vec`, `Btree`) y el ahorro de memoria vs usar `Vec` para todos.
2. ¿Por qué la promotion es $O(\text{deg})$ *una vez* por umbral cruzado y no $O(\text{deg})$ por insert? Mostrá la cuenta amortizada.
3. ★ Diseñá un escenario donde `ArcSwap` falla (algo no es wait-free como prometido). (Pista: pensá en backpressure si los readers no descartan los Arcs viejos.)

4. Compará: 1 millón de `neighbors_snapshot` con `ArcSwap` vs con `RwLock<NeighborList>`. Tiempo y contention. ¿Cuándo gana cada uno?

5. Si el motor migra a streaming como source of truth (stage 2c), el matcher hot loop pasa de iterar un slice contiguo a iterar una `NeighborList` variant. ¿Qué riesgo de regresión hay? ¿Para qué tipo de hipótesis?

Capítulo 14

Finalización: `sort_edges_by_timestamp`

Intuición

Después de la última `add_relation`, el grafo se cierra explícitamente con `sort_edges_by_timestamp`. Esa función hace tres cosas: ordena las aristas por (`source_sid`, `timestamp`), marca el grafo como *sealed*, y reinicializa los `OnceLock` de los índices auxiliares (NLF, k-hop) para que se construyan *lazy* en el primer hunt. Sin esa finalización, el matcher no puede hacer búsqueda binaria sobre las aristas; con ella, los hunts vuelan. El truco del *lazy build* es lo que destrabó el caso "28 GB de EVTX": antes, finalizar duraba 1–3 minutos (índices construidos *eagerly*); ahora termina apenas el merge sort, los índices se posponen al primer hunt.

14.1. Por qué hace falta finalizar

Durante la ingesta, `add_relation` hace `push_to_streaming` al *streaming backend* en orden de llegada. Las aristas *no* están sorted por `timestamp` per-vertex; pueden venir mezcladas:

```
[ (alice@db01, t=10),
  (bob@web02, t=11),
  (alice@web02, t=12),
  (alice@db01, t=13),
  ... ]
```

El matcher quiere preguntar "los vecinos salientes de `alice` en $[t_{\text{lo}}, t_{\text{hi}})$". Sin orden, responder esa pregunta requeriría escanear todas las aristas ($O(m)$) —inviable para $m = 10^8$. Con orden, es búsqueda binaria sobre un slice ($O(\log m)$ + tamaño del sub-slice).

14.2. La operación `sort_edges_by_timestamp`

```rust
pub fn sort_edges_by_timestamp(&mut self) -> Result<(), Error> {
    self.streaming.finalize();              // per-vertex sort
    self.edges_sorted = true;
    self.nlf = OnceLock::new();             // reset lazy index
    self.khop_reach = OnceLock::new();
    Ok(())
}
```

Tres pasos: sort per-vertex, marcar sealed, reinicializar índices. Cada uno tiene su matiz.

14.2.1. Paso 1: sort per-vertex

El `streaming.finalize()` (capítulo 13) hace:

1. Para cada vértice (`Inline` o `Vec`): `sort_unstable_by_key(|e| e.timestamp)`.
2. Para vértices ya `Mmap`-spilled: skip —ya están sorted por construcción (auto-spill sortea durante el write).

El sort es la operación cara. Para 10^8 elementos de 32 B, un `sort_by_key` radix-friendly toma ∼10 segundos en hardware típico (∼10 M elementos/seg). Ése es el costo unavoidable de finalizar; sin él, ninguna búsqueda binaria funciona.

14.2.2. Paso 2: `edges_sorted = true`

Un flag boolean que marca el grafo como sealed. Después de esto, cualquier `add_relation` adicional se rechaza (o, en stage 2a, dispara la *streaming* side —ver capítulo 15).

14.2.3. Paso 3: reset de `OnceLock` para lazy build

```
pub struct GraphHunter {
    streaming: Arc<InMemoryStreamingBackend>,
    edges_sorted: bool,
    nlf: OnceLock<NlfTable>,
    khop_reach: OnceLock<KHopReach>,
    // ...
}
```

`OnceLock<T>` es una primitiva de la stdlib de Rust que permite construir un valor *como mucho una vez*, en forma *lazy* y *thread-safe*. La primera llamada a `ensure_nlf()` construye la tabla; las siguientes ven el resultado sin reconstruir.

14.3. Lazy build: el cambio que destrabó 28 GB

Antes de *heavy-logs #3.5* (commit del cambio), la `sort_edges_by_timestamp` construía NLF y k-hop *eagerly*:

```
// Versión vieja (eager)
pub fn sort_edges_by_timestamp(&mut self) -> Result<(), Error> {
    self.streaming.finalize();
    self.edges_sorted = true;
    self.nlf = NlfTable::build(self)?;            // <-- $\sim$30s
    self.khop_reach = KHopReach::build(self)?;    // <-- $\sim$1min
    Ok(())
}
```

Para 28 GB de EVTX (200 M aristas): merge sort ∼30 s + NLF build ∼30 s + khop build ∼1.5 min = 3 minutos. La UI quedaba *frozen* mientras el motor procesaba; el usuario veía un spinner sin feedback de progreso.

14.3.1. La nueva versión

```
// Versión nueva (lazy)
pub fn sort_edges_by_timestamp(&mut self) -> Result<(), Error> {
    self.streaming.finalize();
    self.edges_sorted = true;
    self.nlf = OnceLock::new();          // diferido
    self.khop_reach = OnceLock::new();   // diferido
    Ok(())
}
```

```rust
pub fn ensure_nlf(&self) -> &NlfTable {
    self.nlf.get_or_init(|| NlfTable::build(self))
}

pub fn ensure_khop_reach(&self) -> &KHopReach {
    self.khop_reach.get_or_init(|| KHopReach::build(self))
}
```

Ahora `sort_edges_by_timestamp` sólo paga el merge sort (∼30 s para 28 GB). NLF y k-hop se construyen al primer *hunt* que los consulta:

- Hunt sin filtros NLF: nunca llama `ensure_nlf()`, NLF jamás se construye.
- Hunt con un solo paso: llama `ensure_nlf()`, paga 30 s una vez, los siguientes hunts encuentran NLF ready.
- Hunt multi-hop: llama `ensure_khop_reach()`, paga 1.5 min una vez, los siguientes hunts encuentran khop ready.

Visible al usuario: abrir una sesión 28 GB es *instantáneo* (sólo stream binario + merge sort = ∼30 s); el primer hunt es lento (paga el index build); los siguientes vuelan. Si el analista quiere evitar el primer-hunt-lento, dispara un *warm-up hunt* trivial apenas carga la sesión.

14.3.2. Cómo `OnceLock` garantiza thread-safety

`OnceLock::get_or_init` hace internamente:

1. `load` atómico del puntero. Si está inicializado, retorna directo.
2. Si no está, toma un mutex interno (sólo durante la inicialización).
3. Re-chequea el puntero (otro thread pudo haber inicializado mientras esperabas el mutex).
4. Si sigue None, ejecuta el closure, escribe el resultado, publica el puntero atómicamente.
5. Suelta el mutex.

Garantía: la inicialización ocurre *exactamente una vez*, sin importar cuántos threads la disparen. Los lectores subsecuentes hacen sólo el `load` atómico (∼5 ns).

14.4. Auto-spill por-vértice (cuando no entra en heap)

Para 200 M aristas el grafo entero no cabe en RAM. El *streaming backend* cae a *auto-spill incremental*:

1. Track de `edges_since_last_spill`; cuando cruza el *budget* (default 10^6 aristas) dispara `auto_spill()`.
2. Cada round: para cada vértice, sortear su slice por `timestamp` *durante el write* (slice ya sorted vía `sort_unstable_by_key`).
3. Escribir un archivo *round-numbered* con header, offset table per-vertex, y la región de datos contiguous de `StreamEdge`s.
4. `mmap` el archivo y swap-ear cada `NeighborList` a la variante `Mmap { handle, byte_offset, len }`.

5. Borrar el archivo del round previo (*Arc count* cae a 0 cuando todos los slots se reemplazan).

Costo per-round: $O(V_{round} \log V_{round} + E_{round})$ comparaciones más $E_{round} \cdot 32$ B de I/O. Para $E = 2 \cdot 10^8$ con budget 10^6 son ∼200 rounds; cada uno escribe ∼32 MB. En SSD NVMe (1 GB/s) cada round toma ∼30 ms; total amortizado ∼6 s a través de toda la ingesta.

14.5. ¿Por qué sort por (source_sid, timestamp) y no (timestamp, source_sid)?

El matcher pide *primero* "los vecinos de un vértice" (`neighbors_of(sid)`) y *después* restringe por ventana temporal. Sortar por `source_sid` primero permite que cada `neighbors_of` devuelva un slice contiguo (amigable con cache). Sortar por `timestamp` primero haría que los vecinos de un vértice estén dispersos por todo el array —inservible.

Nota. Dentro del slice de un `source_sid`, las aristas están ordenadas por `timestamp` secundario, lo que permite la segunda búsqueda binaria por ventana temporal. La elección (`source_sid`, `timestamp`) es lexicográfica y óptima para el patrón de acceso del matcher.

14.6. Resumen

- `sort_edges_by_timestamp` es la finalización del grafo. Ordena aristas, marca sealed, resetea índices.
- Sort interno via `sort_by_key` en RAM, o merge sort multi-archivo cuando excede budget.
- Lazy build: NLF y k-hop se construyen al primer hunt que los consulta, no al finalizar. Destrabó el caso 28 GB.
- `OnceLock` garantiza inicialización exactamente-una bajo concurrencia; sin double init, sin tornados.
- Sort key es (`source_sid`, `timestamp`) para que `neighbors_of` + búsqueda binaria por ventana den slice contiguo.

Ejercicios

1. ★ Para una sesión de $5 \cdot 10^7$ aristas que cabe entera en heap (1.6 GB), ¿cuánto tarda `sort_edges_by_timestamp` con la versión nueva (lazy)? ¿Cuánto tardaba con la vieja (eager) si NLF construye en ∼15 s y khop en ∼45 s?
2. ¿Por qué `OnceLock::new()` y no `drop(self.nlf)` para resetear? Pensá la diferencia semántica.
3. ★ Diseñá una métrica que el motor pueda exponer al usuario para dar feedback durante el merge sort. ¿Es viable sin frenar el sort?
4. ¿Por qué la decisión "sort por (source_sid, timestamp)" es lo correcto para subgraph matching? Discutí: ¿qué pasa si cambiamos a (timestamp, source_sid)? ¿Para qué tipo de hunt sería mejor?
5. Imaginá un usuario que dispara 100 hunts secuencialmente sobre el mismo grafo. ¿Cuántas veces se construye NLF? ¿Y khop? ¿Cuál es el costo amortizado por hunt si el primer hunt paga 30 s + 1.5 min?

Capítulo 15

NLF incremental

Intuición

NLF (*Neighborhood Label Frequency*, capítulo 27 en detalle) es un índice auxiliar: NLF(v, L) cuenta cuántos vecinos salientes de v tienen etiqueta L. El motor lo usa para podar *starts* antes del DFS. Hasta stage 2a, cualquier `add_relation` *post*-NLF-built invalidaba la tabla entera (`OnceLock::new()`), forzando un rebuild $O(|E|)$ en el próximo hunt. Para ingestas continuas (Sentinel pulling cada 30 s), eso significaba rebuild en cada hunt y se perdía el beneficio del lazy build. Stage 2a cambió la política: ahora el motor intenta *actualizar* la NLF incrementalmente, y sólo invalida cuando físicamente no puede mutar (mmap-spilled read-only).

15.1. El problema

Lifecycle típico antes de stage 2a:

1. Carga sesión 28 GB. `sort_edges_by_timestamp`. NLF queda `None`.
2. Primer hunt ⇒ `ensure_nlf()` construye NLF ($O(|E|)$, ∼30 s).
3. Hunt termina, NLF resident en heap.
4. Sentinel pulling: nuevo batch de eventos llega. `add_relation` ejecuta varias veces.
5. Cada `add_relation` hace `self.nlf = OnceLock::new()` (invalidación).
6. Próximo hunt ⇒ `ensure_nlf()` reconstruye desde cero (otros 30 s).

Si el ciclo se repite cada 30 s (típico de Sentinel pulling), el motor pasa la mitad del tiempo construyendo NLF. El *lazy build* no salva nada; es como si fuera *eager* con extra overhead.

15.2. La solución: `try_record_edge`

Cuando `add_relation` corre con NLF ya construida, en vez de invalidar, intenta sumar +1 al contador correspondiente:

```
impl NlfTable {
    pub fn try_record_edge(
        &self,
        src_sid: StrId,
        dst_tag: u8,
    ) -> bool {
        if dst_tag as usize >= NLF_TYPE_COUNT {
```

```
            return true;  // no-op, dst_tag fuera de rango
        }
        let slice = self.counts.heap_slice_mut();
        let Some(slice) = slice else {
            return false;  // backing es mmap-spilled, no se puede mutar
        };
        if let Some(row) = slice.get_mut(src_sid.index()) {
            row[dst_tag as usize] = row[dst_tag as usize].saturating_add(1);
        }
        true
    }
}
```

Lo que devuelve:

- `true`: la mutación se aplicó, o no era necesaria (dst_tag fuera de rango). NLF queda *viva y consistente*.
- `false`: el backing es *mmap-spilled* (read-only post build). El caller debe invalidar y aceptar el rebuild en el próximo hunt.

15.2.1. Algoritmo: `add_relation` post stage 2a

Algorithm 15.1: `add_relation` con NLF incremental

Input: `src_sid`, `dst_sid`, `dst_tag`, `rel_type_tag`, etc.

```
1  push al edge_store y al streaming (siempre);
2  if nlf está construida then
3  |  if nlf.try_record_edge(src_sid, dst_tag) = true then
   |  |  /* Saturating add sobre el heap-backed counts. NLF queda viva.   */
4  |  end
5  |  else
   |  |  /* Backing es mmap-spilled (read-only). Invalidar y dejar que se
   |  |     reconstruya en el próximo hunt.                                */
6  |  |  nlf ← OnceLock::new();
7  |  end
8  end
9  else
   |  /* Nunca se construyó: nada que invalidar.                          */
10 end
```

15.3. ¿Por qué *saturating add*?

`NlfTable` guarda contadores `u32`. Un saturating add se queda en `u32::MAX` en vez de overflowar. Para grafos realistas (max degree $\sim 10^6$), no se llega ni cerca. Pero la convención garantiza que *ningún mutation produce counts incorrectos* —el peor caso es "count overestimado" que es *soundness-safe* (capítulo 27: NLF puede sobre-aproximar sin generar falsos negativos).

15.4. ¿Por qué `heap_slice_mut` y no acceso directo?

El `NlfTable` usa `IndexBacking<[u32; NLF_TYPE_COUNT]>` internamente. Ese tipo encapsula la dualidad heap/mmap:

```rust
pub enum IndexBacking<T: Pod> {
    Heap(Vec<T>),
    Mmap { file: File, map: Mmap, slice: &'static [T] },
}

impl<T: Pod> IndexBacking<T> {
    pub fn heap_slice_mut(&mut self) -> Option<&mut [T]> {
        match self {
            Self::Heap(v) => Some(v.as_mut_slice()),
            Self::Mmap { .. } => None,  // read-only
        }
    }
}
```

Cuando NLF está *heap-resident* (lo más común), la mutación es directa. Cuando se spilled a mmap por exceder budget, no se puede mutar in-place —el archivo es read-only—.

15.4.1. Por qué no se hace mmap-mut

Una alternativa: abrir el mmap en modo `MmapMut`, mutar in-place, `flush()`, volver a read-only. Tres razones por las que no lo hacemos hoy:

1. **Latencia.** Cada `add_relation` pagaría el remap overhead (∼1 ms en Linux, ∼3 ms en Windows). Para batch ingest de 50K rows/sec, eso son 50K remaps/sec —inviable.

2. **Concurrencia.** Si los matchers están leyendo el mmap mientras el ingestor remapea, hay race condition. Resolver requiere un `RwLock` alrededor del mmap, neutralizando el wait-free reads.

3. **Frecuencia.** Los grafos que llegan a mmap-spilled son grafos *grandes* (>1 GB de NLF, ∼100 M vértices). En esos, los rebuild son raros porque rara vez se reciben nuevas aristas *post-finalize* (la sesión típica es "cargar dataset, hacer hunts, cerrar"). El caso del Sentinel pulling es para grafos chico-medianos donde NLF cabe en heap.

La política actual —incremental cuando heap, invalidar cuando mmap— es la mejor relación complejidad/beneficio.

15.5. Soundness

Teorema 15.1 (try_record_edge preserva NLF correcta)**.** *Sea G_t el estado del grafo después de t llamadas a* ***`add_relation`****, y sea* $\mathrm{NLF}_t^{op}(v, L)$ *el valor guardado en la tabla en ese momento. Si todas las llamadas a* ***`try_record_edge`*** *devuelven* ***`true`*** *y NLF estaba construida correctamente sobre G_0, entonces para todo v, L:*

$$\mathrm{NLF}_t^{op}(v, L) = \mathrm{NLF}_{G_t}(v, L)$$

salvo cuando $\mathrm{NLF}_{G_t}(v, L) > 2^{32} - 1$*, en cuyo caso* $\mathrm{NLF}_t^{op} = 2^{32} - 1$ *(saturated).*

Idea. Inducción sobre t. Base: $t = 0$, por hipótesis. Paso: si la t-ésima `add_relation` es la arista (u, v, L_v, τ) con $L_v = \mathrm{type}(v)$, entonces $\mathrm{NLF}_{G_t}(u, L_v) = \mathrm{NLF}_{G_{t-1}}(u, L_v) + 1$ y todos los demás NLF quedan iguales. La función `try_record_edge` hace exactamente `slice[u][L_v] += 1` (saturating). Por la hipótesis inductiva, slice estaba correcto en $t - 1$; sigue correcto en t. □ □

15.6. El impacto medible

Antes de stage 2a (Sentinel pulling cada 30 s, 100 hunts/h):

- Cada hunt: ∼30 s rebuild NLF + ∼100 ms hunt = 30.1 s.
- 100 hunts/h ⇒ 50 min/h gastados en rebuilds.

Después de stage 2a (mismo workload, NLF heap-resident):

- `add_relation` paga ∼50 ns extra por `try_record_edge`. 50K rows/sec ⇒ 2.5 ms/sec de overhead. Negligible.
- Cada hunt: ∼100 ms (sin rebuild).
- 100 hunts/h ⇒ 10 s/h en hunts.

Reducción de ∼300× en el tiempo gastado.

15.7. Resumen

- Antes de stage 2a: cualquier `add_relation` post-NLF-built invalidaba la tabla entera ⇒ rebuild en cada hunt para ingestas continuas.
- `try_record_edge` aplica +1 saturating al contador correspondiente cuando el backing es heap.
- Falla (devuelve `false`) sólo cuando el backing es mmap-spilled; ahí cae a invalidación + rebuild.
- Soundness garantizada por inducción: la mutación es exactamente lo que un rebuild produciría.
- Impacto: ingestas continuas con hunts cada minuto pasan de "50 min/h en rebuilds" a "negligible".

Ejercicios

1. ★ Si `NLF_TYPE_COUNT = 9` y cada contador es `u32`, ¿cuánta memoria ocupa NLF para un grafo de 10^7 vértices? ¿Y para 10^8?
2. ¿Por qué saturating add y no overflow check + error? Discutí: ¿qué tipo de bug puede ocurrir si un contador overflowa silenciosamente?
3. ★ Considerá un grafo donde NLF es heap-resident pero la presión de memoria empuja a otra estructura (*khop_reach*) a spillearse. ¿Eso afecta a NLF? ¿Y al revés?
4. Diseñá una variante `try_record_edge_mmap` que SÍ funcione sobre mmap-spilled (con remap, etc.). ¿Cuál es el costo? ¿Cuándo valdría la pena?
5. Si el motor decide invalidar (caso mmap-spilled), el próximo hunt paga rebuild $O(|E|)$. ¿Qué pasa si llegan `add_relation` adicionales *durante* el rebuild? Pensá la concurrencia.

Parte V

El matcher, paso a paso

Capítulo 16

DFS recursivo: el *baseline*

Intuición

Antes de hablar de *worst-case-optimal joins* y semi-join reducers, conviene tener firme el algoritmo más básico: DFS (depth-first search) recursivo. Es el algoritmo *baseline* del matcher; cuando ningún operador especializado dispatcha, el motor cae a un DFS bien podado. Este capítulo lo presenta primero en su forma genérica (sobre grafos sin etiquetas, sin tiempo) y después adaptado al subgraph matching temporal etiquetado de GraphHunter. Si DFS te resulta familiar, podés saltar directo a la sección 16.4; si no, esto es el refresher.

16.1. DFS clásico sobre un grafo

Definición 16.1 (DFS)**.** Dado un grafo dirigido $G = (\mathcal{V}, \mathcal{E})$ y un vértice de partida $s \in \mathcal{V}$, DFS visita todos los vértices alcanzables desde s explorando *en profundidad*: avanza por una arista, recursa al nuevo vértice, y sólo retrocede cuando todas las salidas del vértice actual ya fueron exploradas.

Algorithm 16.1: DFS recursivo, versión *visit-all*

Input: grafo $G = (\mathcal{V}, \mathcal{E})$, vértice de partida s
Output: conjunto de vértices visitados

visited $\leftarrow \emptyset$;
visit(s);
▷ `Procedimiento visit(v)`
if $v \in$ visited **then**
  return
end
visited $\leftarrow$ visited $\cup \{v\}$;
for $w \in N^+(v)$ **do**
  visit(w);
end

Ejemplo

DFS sobre un grafo de 5 vértices arrancando desde A.

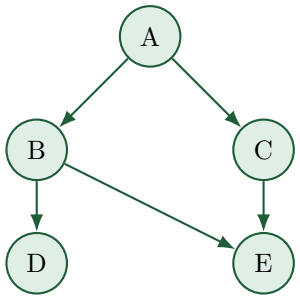


Figura 16.1: Grafo de ejemplo. DFS desde A: $A \to B \to D$ (retroceso) $\to E$ (retroceso a B y a A) $\to C$ (E ya visited, sin más). Orden de visita: A, B, D, E, C.

16.1.1. Complejidad

DFS visita cada vértice y arista una vez:

- Tiempo: $O(|\mathcal{V}| + |\mathcal{E}|)$. Lineal en el tamaño del grafo alcanzable.
- Espacio: $O(|\mathcal{V}|)$ por la `visited` más $O(\text{depth})$ por la pila de recursión.

16.2. De *visit-all* a *enumerate-paths*

DFS *visit-all* visita cada vértice una vez. Para subgraph matching necesitamos algo distinto: enumerar *caminos* (secuencias de vértices) que matchean un patrón. La diferencia es crítica:

- En *visit-all*, el vértice se marca visited y no se explora de nuevo aunque haya otra forma de llegar a él.
- En *enumerate-paths*, queremos *todas* las formas de llegar a un vértice (cada una es un path distinto), pero **sin repetir vértices dentro de un mismo path** (k-simplicidad, capítulo 2).

Algorithm 16.2: DFS *enumerate-paths*, longitud exacta k

Input: grafo G, vértice s, longitud k
Output: secuencia de paths de longitud k desde s

```
1  path ← [s];
2  recurse(s, k);
   ▷ recurse(v, remaining)
3  if remaining = 0 then
4  |  emit(path);
5  |  return;
6  end
7  for w ∈ N⁺(v) do
8  |  if w ∉ path then
9  |  |  path.push(w);
10 |  |  recurse(w, remaining − 1);
11 |  |  path.pop();
12 |  end
13 end
```

16.2.1. Backtracking

Notar el patrón `push` / `recurse` / `pop`: es el *backtracking* clásico. La pila `path` se modifica durante la recursión y se restaura al volver, manteniendo el estado consistente para la próxima rama.

Nota. El chequeo $w \notin$ path implementa la *1-simplicidad* (path simple). Para k-simplicidad general, contamos cuántas veces aparece w en `path` y aceptamos si es $< k$. En GraphHunter el chequeo está en `core/graph-engine/src/dfs.rs`: cuenta apariciones via *linear scan* sobre el binding (siempre tiene $\leq k+1$ elementos, costo despreciable).

16.3. Subgraph matching: agregar etiquetas

Para subgraph matching del capítulo 3, DFS necesita ajustes:

1. Sólo visitar vértices del tipo correcto en cada paso.
2. Sólo seguir aristas del tipo correcto en cada paso.
3. Acumular el path *etiquetado* para emit.

Algorithm 16.3: DFS *subgraph match*, hipótesis chain $(L_0^V, L_0^E, L_1^V, L_1^E, \ldots, L_k^V)$

Input: grafo G, hipótesis H
Output: Vec<HuntResult>

for $s \in \mathcal{V}$ *con* type$(s) = L_0^V$ **do**
  path $\leftarrow [s]$;
  recurse$(s, 0)$;
end

▷ `recurse(v, step)`
if step $= k$ **then**
  `emit`(path);
  return;
end
for $w \in N^+(v)$ **do**
  $e \leftarrow (v, w)$;
  if type$(e) = L_{\text{step}}^E$ ***and*** type$(w) = L_{\text{step}+1}^V$ ***and*** $w \notin$ path **then**
    path.`push`(w);
    recurse$(w, \text{step} + 1)$;
    path.`pop`();
  end
end

16.3.1. Complejidad

En el peor caso (sin podas), DFS subgraph match es:

$$O\left(|\mathcal{V}_{L_0^V}| \cdot \prod_{i=0}^{k-1} \deg^+_{L_i^E, L_{i+1}^V}\right)$$

donde $|\mathcal{V}_{L_0^V}|$ es el número de vértices del tipo del primer paso, y $\deg^+_{L^E, L^V}$ es el grado promedio saliente con *esa* combinación etiqueta-arista, etiqueta-destino. Para hipótesis cortas y etiquetas selectivas, ese producto es chico ($\sim 10^4$ para 3 saltos en datos típicos). Para hipótesis largas o etiquetas no selectivas, explota.

16.4. DFS temporal de GraphHunter

Sumando las restricciones temporales del capítulo 2:

Algorithm 16.4: DFS subgraph match temporal de GraphHunter (versión simplificada)

Input: grafo G, hipótesis H con ventana Δt y k_s-simplicidad
Output: Vec<HuntResult>

for $s \in \mathcal{V}$ *con* $\text{type}(s) = L_0^V$ ***and*** *pasa* prunes **do**
  path $\leftarrow [s]$;
  walk$(s, 0, \text{None}, \text{None})$;
end

▷ `walk(v, step, t_first, t_prev)`
if step $= k$ **then**
  `emit`(path);
  return;
end
$t_{\text{lo}} \leftarrow t_{\text{prev}} + 1$ si t_{prev} existe, sino $-\infty$;
$t_{\text{hi}} \leftarrow t_{\text{first}} + \Delta t$ si t_{first} existe, sino $+\infty$;
for $(v, w, \tau) \in$ ***neighbors_in_window***$(v, t_{\text{lo}}, t_{\text{hi}})$ **do**
  if $\text{type}_E((v, w)) = L_{\text{step}}^E$ ***and*** $\text{type}(w) = L_{\text{step}+1}^V$ ***and*** $\text{count}(\text{path}, w) < k_s$ **then**
    path.`push`(w);
    walk$(w, \text{step} + 1, t_{\text{first}} \text{ or } \tau, \tau)$;
    path.`pop`();
  end
end

16.4.1. Las nuevas piezas

- t_{first}: timestamp de la *primera* arista del walk. Se setea en el primer step y se usa para chequear ventana Δt de ahí en adelante.
- t_{prev}: timestamp del step anterior, para chequear monotonicidad estricta.
- `neighbors_in_window`: la operación clave del *streaming backend* (capítulo 13). `streaming.neighbors_arc` + `guard.read()` + `guard.as_slice()` + dos llamadas a `partition_point` devuelven el sub-slice contiguo de aristas salientes de v cuyo timestamp está en $[t_{\text{lo}}, t_{\text{hi}})$.
- `count(path, w) < k_s`: k-simplicidad generalizada.

16.4.2. Por qué iterativo y no recursivo en producción

GraphHunter implementa el DFS de forma *iterativa* (`dfs_match_iterative`) usando una pila explícita en lugar de la pila de recursión. Tres ventajas:

1. **Sin stack overflow.** Hipótesis muy largas (k=10+) podrían reventar la pila de recursión (default 8 MB en Linux).
2. **Cancelable.** El frame iterativo puede chequear una bandera de cancelación entre pasos sin requerir cooperación del compilador.
3. **Cap más fácil.** Cuando el contador de resultados llega al cap, el loop iterativo simplemente sale; con recursión hay que propagar la señal hacia arriba.

16.5. De DFS naive a DFS bien-podado

El algoritmo de la sección anterior es el *baseline*. En la práctica el motor agrega tres podas (*prunes*) antes de disparar el DFS principal:

1. **NLF prune**: descarta *starts* cuyo histograma de vecinos no satisface el primer paso (capítulo 18).
2. **k-hop reachability prune**: descarta starts desde los que no puede haber un path de longitud $\leq$ `MAX_DEPTH` al type destino, dentro de la ventana temporal (capítulo 19).
3. **Symmetry breaking**: para hipótesis palindrómicas ($L_0^V = L_k^V$, $L_0^E = L_{k-1}^E$, …), enumerar sólo la mitad y reflejar; ahorra factor $2\times$.

Cada poda se aplica a los *starts* antes del DFS principal, o como early-exit dentro del walk. Las desarrollamos en los próximos capítulos.

16.6. Resumen

- DFS recursivo *visit-all*: visita cada vértice alcanzable, marcado por `visited`. Lineal.
- DFS *enumerate-paths*: enumera caminos sin marcar visited, usa backtracking, chequea k-simplicidad.
- DFS *subgraph match*: agrega filtros de etiquetas (vértice y arista) en cada paso.
- DFS *temporal*: agrega ventana $[t_{\text{lo}}, t_{\text{hi}})$, monotonicidad, `neighbors_in_window` via búsqueda binaria.
- GraphHunter implementa la versión iterativa para cancelabilidad y stack-safety. La forma final está bien-podada (NLF, k-hop, symmetry breaking).

Ejercicios

1. ★ Para el grafo de la figura 16.1, enumerá todos los paths simples de longitud 3 desde A. ¿Cuántos hay?
2. Modificá el pseudocódigo de DFS *enumerate-paths* para que devuelva la cantidad en vez de los paths. ¿Es más rápido?
3. ★ Considerá DFS sobre un grafo *Barabasi-Albert* de $n = 10^4$, $m = 10^5$, con grado max ($\Delta = 100$). Para una hipótesis de longitud 3 sin filtros, ¿cuál es el peor caso de paths visitados?
4. ¿Por qué la versión iterativa permite cancelación más fácil que la recursiva? Pensá la implementación de un *cancellation token* en ambas formas.
5. Diseñá un caso adversarial: un grafo y una hipótesis donde el DFS naive (sin podas) explote ($> 10^9$ iteraciones). Identificá qué prune lo evitaría.

Capítulo 17

Subgraph matching ingenuo: por qué no escala

Intuición

El DFS subgraph match del capítulo anterior es *el* baseline. En grafos chicos o hipótesis con etiquetas selectivas, anda perfecto. En grafos grandes o hipótesis sin filtros, su complejidad $O(\deg^k)$ explota. Este capítulo pone números concretos a esa explosión: dado un grafo *scale-free* típico ($\sim 10^7$ aristas, distribución de grado power-law), una hipótesis de $k = 4$ sin filtros puede generar trillones de paths. Esa es la motivación empírica para los *prunes* (próximos dos capítulos), los *worst-case-optimal joins* (cap. 21), y los *semi-join reducers* (cap. 23).

17.1. El peor caso teórico

Recordemos el bound del capítulo 16:

$$\text{paths visitados} \leq |\mathcal{V}_{L_0^V}| \cdot \prod_{i=0}^{k-1} \deg^+_{L_i^E, L_{i+1}^V}.$$

Si todas las etiquetas son "cualquiera" (sin filtro), $\deg^+$ es el grado promedio del grafo, $\bar{d}$. Si todas las etiquetas son selectivas, cada $\deg^+_{L^E, L^V}$ es una fracción chica de $\bar{d}$. El producto $\prod \deg^+$ tiene crecimiento geométrico en k; con $\bar{d} = 100$ y $k = 4$, eso son 10^8 paths *por start*. Multiplicado por $|\mathcal{V}_{L_0^V}| = 10^5$ starts: 10^{13} paths totales. A 1 ns por path (optimistic), 10 000 segundos = 2.7 horas.

17.1.1. El gráfico que vale más que el álgebra

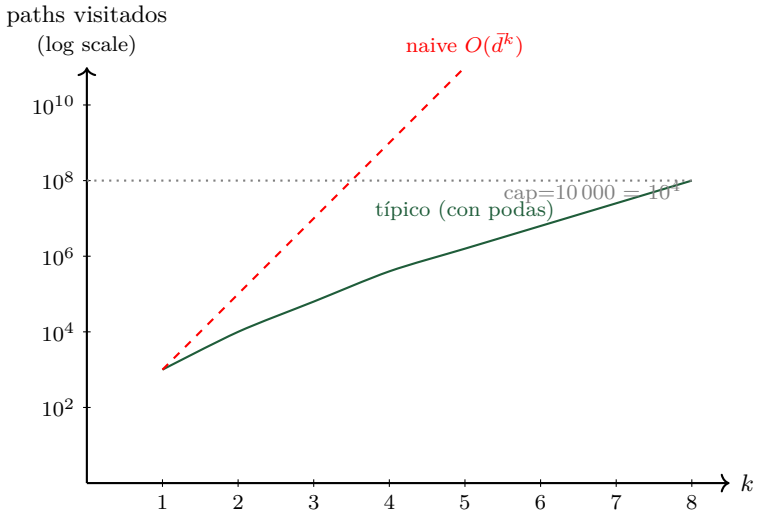


Figura 17.1: Crecimiento de paths visitados por hunt vs longitud de la hipótesis. Naive es $O(\bar{d}^k)$ —explosión geométrica que satura el cap a $k = 3$—. Con podas, el crecimiento es sub-exponencial; en grafos típicos los hunts de $k = 6$ terminan bajo el cap.

17.2. ¿Por qué scale-free empeora todo?

Las redes corporativas reales NO son grafos aleatorios uniformes. Son *scale-free*: la distribución de grado sigue una *power-law*, $\Pr[\deg(v) = d] \propto d^{-\alpha}$ con $\alpha \in [2, 3]$. Eso significa:

- La mayoría de los vértices tienen grado bajo (1–10).
- Una minoría tiene grado mediano (50–500).
- Existe un puñado de *hubs* con grado masivo (10^4 a 10^6).

El DFS naive sufre desproporcionalmente por los hubs:

- Un *Domain Controller* con 10^4 usuarios autenticando contra él tiene grado entrante 10 000.
- Un hunt de la forma `User -[Auth]-> Host -[Connect]-> IP` pasa por el DC en el segundo paso. Iterar sus 10^4 salidas es *una sola* iteración del DFS *en ese vértice*; pero el DFS volverá al DC desde *cada uno* de los starts que lo alcanzan.
- Si 10^3 usuarios alcanzan al DC, el DFS itera 10^4 salidas 10^3 veces $\Rightarrow$ 10^7 iteraciones sólo en ese paso.

17.2.1. El *output* también explota

Más sutil que la *exploración*: el resultado *en sí* puede ser enorme. Si un hub tiene grado 10^4 y aparece en el medio de una hipótesis de $k = 3$, las paths que pasan por él se multiplican: una arista *entrante* al hub combinada con 10^4 salientes del hub, da 10^4 paths *por arista entrante*. Para 10^3 entrantes: 10^7 paths.

GraphHunter *cap-eaba* la salida en 10 000 resultados; pero el DFS necesita *generarlos para verificarlos*. Aún con cap, el trabajo del DFS es proporcional al output total, no al cap. Esto es exactamente el caso "cap-saturado" del capítulo 11: el DFS produce más resultados que el cap, y el costo del emit (no de la búsqueda) es lo que mata.

17.3. Tres caminos para escalar

Cuando el DFS naive explota, hay tres familias de soluciones, cada una atacando un aspecto distinto:

17.3.1. Camino 1: Reducir *starts* con prunes

Si una fracción significativa de los starts no genera ningún resultado, descartarlos antes de iniciar el DFS ahorra trabajo. Dos prunes:

- **NLF prune**: si el primer paso requiere $\geq r$ vecinos del tipo L, descartar v si $\text{NLF}(v, L) < r$. Local, $O(|\mathcal{V}|)$.
- **k-hop reachability prune**: si la hipótesis exige llegar al tipo L_k^V, descartar v si $L_k^V \notin \text{bits}[v]$. Transitivo, requiere índice precomputado.

Capítulos 18 y 19 desarrollan.

17.3.2. Camino 2: Cambiar el algoritmo (WCOJ)

Para hipótesis con *shape* estructural especial (K_4), el DFS naive es estrictamente subóptimo en peor caso: visita $O(\deg^3)$ cuando un *worst-case-optimal join* (LFTJ) visita $O(m)$. Capítulos 20 y 21.

17.3.3. Camino 3: Reducir antes de enumerar (Yannakakis)

Para hipótesis α-acíclicas (cadena, estrella, árbol), una *semi-join reducer* elimina del grafo todas las aristas que no participan de *ningún* match completo, ANTES de empezar a enumerar. La enumeración después es trivial. Capítulos 22 y 23.

17.4. ¿Cuándo el DFS naive es suficiente?

A pesar de su pesadilla teórica, el DFS naive (con podas) anda perfecto en muchos casos:

- **Hipótesis con etiquetas muy selectivas.** Por ejemplo, `User -[MfaPush]-> Device`. Si MfaPush es raro (pocas aristas), el producto $\deg^+_{L^E, L^V}$ es chico.
- **Hipótesis cortas.** $k = 1$ o $k = 2$ es DFS chico por construcción.
- **Grafos *no* scale-free.** Algunos datasets sintéticos o redes muy estructuradas tienen distribuciones de grado uniforme; ahí el DFS no sufre los hubs.

La estrategia del motor: usar el DFS por defecto, ofrecer operadores especializados cuando la hipótesis tiene la estructura adecuada. El *planner* (capítulo 25) hace la elección.

17.5. Resumen

- DFS naive: $O(\bar{d}^k)$ en peor caso. Geométrico en k; explota a $k = 3, 4$ sin podas.
- Grafos scale-free amplifican: hubs causan multiplicación de paths intermedios.
- Tres caminos de escalado: prunes (reducir starts), WCOJ (mejor algoritmo), Yannakakis (reducir antes de enumerar).
- DFS bien-podado sigue siendo el baseline; los operadores especializados se dispatchan cuando la estructura lo permite.

Ejercicios

1. ★ Para un grafo con $\bar{d} = 50$ y una hipótesis de $k = 4$ sin filtros sobre $|\mathcal{V}_{L_0^V}| = 10^4$ starts, ¿cuántos paths visita el DFS naive en peor caso?

2. ¿Cómo cambia el orden de magnitud si las etiquetas filtran al 10 % del grado? ¿Y si filtran al 1 %?

3. ★ En un grafo scale-free, los hubs son raros pero contribuyen mucho. Modelá: si el grafo tiene 10^6 vértices con $\bar{d} = 5$ pero un solo hub con $\deg = 10^5$, ¿cuál es el costo extra del DFS al pasar por el hub vs por un vértice promedio?

4. Diseñá una hipótesis "imposible" (que el DFS no termine en tiempo razonable). ¿Qué propiedad estructural la convierte en imposible? ¿Cómo la detectaría el motor para rechazarla?

5. ¿Por qué el cap del cuello de botella es el output, no la exploración? Pensá: si el DFS encuentra 10^7 paths válidos pero sólo emite los primeros 10^4, ¿el costo de *emit* es proporcional al cap o al total?

Capítulo 18

Poda con NLF

Intuición

NLF (*Neighborhood Label Frequency*) es el primer prune del matcher: una tabla $[\mathcal{V} \times \mathcal{L}_V] \to \mathbb{N}$ que cuenta, para cada vértice y cada tipo destino, cuántos vecinos salientes tiene de ese tipo. Antes de iniciar el DFS desde un *start* s, el matcher chequea: ¿el primer paso de la hipótesis pide "vecino de tipo L"? Si $\text{NLF}(s, L) = 0$, s no puede generar ningún match. Lo descarta. Es local ($O(1)$ por start), sound (nunca descarta un start válido), y barato de mantener (capítulo 15). Este capítulo formaliza el prune, prueba su soundness, y muestra cuánto poda en grafos típicos.

18.1. Definición formal

Definición 18.1 (NLF)**.** Sea $G = (\mathcal{V}, \mathcal{E})$ un grafo dirigido temporal etiquetado, y $L \in \mathcal{L}_V$ un `EntityType` con *tag* en $[0, \texttt{NLF_TYPE_COUNT})$. Para $v \in \mathcal{V}$:

$$\text{NLF}(v, L) = |\{w \in N^+(v) : \text{type}(w) = L\}|.$$

Es decir: $\text{NLF}(v, L)$ cuenta los vecinos salientes de v cuyo tipo es exactamente L. Es una matriz $|\mathcal{V}| \times |\mathcal{L}_V|$ de enteros (en GraphHunter, `u32`).

Nota. NLF ignora el *tipo de arista* y el *timestamp*. Sólo cuenta vecinos por tipo destino. Eso lo hace poco-discriminante para hipótesis con muchos predicados, pero *universalmente útil* para descartar starts donde *no existe el tipo destino*.

18.2. El prune

Algorithm 18.1: Prune NLF al inicio del DFS

Input: starts candidatos $\mathcal{V}_{L_0^V}$, hipótesis H
Output: starts filtrados

```
filtered ← ∅;
for s ∈ V_{L_0^V} do
    if satisfies_first_step(s, H) then
        filtered.push(s);
    end
end
return filtered;
▷ satisfies_first_step(s, H)
L ← L_1^V (tipo destino del primer paso de H);
return NLF(s, L) ≥ 1;
```

La versión más completa: para hipótesis con *shape* graph, donde un mismo vértice puede tener varios pasos salientes (estrella centrada en él), el chequeo se generaliza a $\text{NLF}(s, L_j^V) \geq r_j$ para cada hijo j requerido.

18.3. Soundness

Teorema 18.2 (NLF prune sound)**.** *Si* $\text{NLF}(s, L) = 0$*, entonces* s *no es origen de ningún match de la hipótesis* H *cuyo primer paso requiere un vecino de tipo* L*.*

Demostración. Por contradicción. Supongamos que existe un match σ con $\sigma(p_0) = s$. Por definición de match, existe $w = \sigma(p_1) \in \mathcal{V}$ con $\text{type}(w) = L_1^V = L$ y una arista $(s, w, \tau) \in \mathcal{E}$. Es decir, $w \in N^+(s)$ y $\text{type}(w) = L$. Entonces $\text{NLF}(s, L) \geq 1$, contradicción. □ □

Importante: la otra dirección NO vale. $\text{NLF}(s, L) \geq 1$ NO garantiza que s genere match. El prune es *conservador*: nunca descarta starts válidos, pero puede *aceptar* starts que después no generen matches (falsos positivos). El DFS los procesa, no encuentra nada, retorna sin emit.

18.3.1. Prune más fuerte: chequear varios tipos

Si la hipótesis tiene shape graph, el primer vértice puede tener *varios* hijos (capítulo 31):

```
User c -[Auth]-> Host h1; c -[Auth]-> Host h2; c -[Auth]-> Host h3
```

Acá el start c necesita *tres* hijos de tipo `Host`. NLF se generaliza: $\text{NLF}(c, \texttt{Host}) \geq 3$. El prune descarta cualquier c con menos vecinos del tipo correcto.

18.3.2. NLF en pasos intermedios

¿Y por qué no chequear NLF en cada paso del DFS, no sólo al inicio? Porque el DFS *ya* itera $N^+(v)$ en cada paso —el chequeo NLF sería redundante salvo en el primer paso, donde nos ahorra entrar al loop. En los pasos intermedios el costo de iterar y filtrar por tipo es comparable al costo del chequeo NLF.

18.4. Cuánto poda en la práctica

Tres escenarios reales:

18.4.1. Escenario 1: hipótesis con primer paso muy selectivo

Hipótesis: `User -[Auth]-> Host`. Grafo: 10^5 usuarios, de los cuales sólo 10^4 tienen aristas `Auth` salientes hacia `Host`s.

- Sin NLF: el DFS arranca desde 10^5 starts; los $9 \cdot 10^4$ que no tienen aristas Host generan iteración vacía sobre $N^+(s)$ y retornan.
- Con NLF: descarta los $9 \cdot 10^4$ antes de entrar al DFS. $10\times$ menos starts a procesar.

18.4.2. Escenario 2: starts "hojas"

Grafos scale-free tienen muchos vértices de grado bajo. Para hipótesis con primer paso de tipo `Host`, los *User*s de grado 0 (no se loguearon en ninguna sesión analizada) están en $\mathcal{V}_{User}$ pero NLF los descarta inmediatamente.

- Típico: $\sim 30\,\%$ de usuarios sin aristas en una sesión semanal (gente de vacaciones, cuentas inactivas, etc.). NLF los filtra a costo cero.

18.4.3. Escenario 3: hipótesis multi-hijo (shape graph)

Hipótesis: `c -[Auth]-> Host h1; c -[Auth]-> Host h2; c -[Auth]-> Host h3` (usuario que se autentica en al menos 3 hosts distintos).

- Sin NLF generalizada: el DFS arranca desde todos los `User`; descubre on-the-fly que la mayoría sólo tiene 1 o 2 hosts.
- Con NLF generalizada: $\text{NLF}(u, \texttt{Host}) \geq 3$. En grafos típicos, sólo el $\sim 5\,\%$ de usuarios cumple. El DFS arranca desde $20\times$ menos starts.

18.5. Implementación: `NlfTable`

El detalle de implementación está en el capítulo 27 (layout, `IndexBacking`, mmap-spill). Acá lo que importa para el prune es la firma:

```
impl NlfTable {
    pub fn count(&self, sid: StrId, type_tag: u8) -> u32 {
        let row = &self.counts.get(sid.index());  // [u32; NLF_TYPE_COUNT]
        row[type_tag as usize]
    }

    pub fn can_match(
        &self,
        sid: StrId,
        required: &[(u8, u32)],  // (type_tag, min_count)
    ) -> bool {
        required.iter().all(|&(tag, min)| self.count(sid, tag) >= min)
    }
}
```

`can_match` es la operación que el matcher invoca al inicio del DFS para cada start candidato. $O(|\text{required}|)$ que es $\leq |\mathcal{L}_V| = 9$. Despreciable.

En el código.

`core/graph-engine/src/nlf.rs`: `NlfTable::can_match`. El *backing* es `IndexBacking<[u32; 9]>` —heap-resident para grafos chicos, *mmap-spilled* arriba del *budget* (default 1 GB).

18.6. Construcción

Algorithm 18.2: NLF build

Input: grafo $G = (\mathcal{V}, \mathcal{E})$
Output: `NlfTable`

```
counts ← Vec de |V| filas [u32; NLF_TYPE_COUNT], inicializado a 0;
for (u, v, τ) ∈ E do
    L ← type(v).tag;
    if L < NLF_TYPE_COUNT then
        counts[u][L] ← counts[u][L] + 1;
    end
end
return NlfTable{ counts };
```

$O(|\mathcal{E}|)$ iteraciones, una pasada secuencial por `edge_store`. Para $|\mathcal{E}| = 10^8$, $\sim$ 30 segundos en hardware típico —el costo que el *lazy build* difiere al primer hunt (capítulo 14).

18.7. Limitaciones

NLF es un prune simple y por eso tiene limitaciones:

- **No considera tipo de arista.** Si la hipótesis pide `User -[Auth]-> Host` y un `User` tiene aristas `Spawn` pero no `Auth` a algún `Host`, NLF no lo descarta ($\text{NLF}(u, \texttt{Host}) \geq 1$ por el Spawn). El DFS verifica el tipo de arista en cada paso e ignora.
- **No considera ventana temporal.** Si todas las aristas `Auth` de u están fuera del `WITHIN`, NLF no lo descarta. Para esto sirve el k-hop reachability extendido temporal (capítulo 19).
- **Solo el primer paso.** El prune actúa antes de iniciar el DFS, no en los pasos intermedios. Para los pasos intermedios, el DFS itera N^+ y filtra por tipo en el loop —no hay forma de pre-filtrar sin gastar memoria.

Cada limitación se ataca con una técnica adicional: *neighborhood label profile* (NLP, no implementado en GraphHunter por costo) considera tipos de arista; k-hop temporal considera ventana; *label-and-edge histograms* considera tipos intermedios.

18.8. Resumen

- NLF: $\text{NLF}(v, L) = |\{w \in N^+(v) : \text{type}(w) = L\}|$. Tabla $|\mathcal{V}| \times |\mathcal{L}_V|$ de `u32`.
- Prune: descartar starts con $\text{NLF}(s, L) < r$. Sound (nunca descarta válidos), no completo (acepta falsos positivos).
- Build $O(|\mathcal{E}|)$ en una pasada; cabe lazy en `OnceLock`.
- Limitaciones: ignora tipo de arista y ventana temporal. Sirve sólo para el primer paso.
- Impacto típico: 10×–20× menos starts a procesar; mayor en hipótesis multi-hijo.

Ejercicios

1. ★ Para el grafo de la figura 2.1 (capítulo 2), construí la NLF completa: una matriz con filas por vértice y columnas por `EntityType`.
2. Para la hipótesis `User -[Auth]-> Host -[Connect]-> IP`, listá todos los starts (vér-

tices `User`) que sobreviven al prune NLF en el grafo del ejercicio anterior.

3. ★ Demostrá: si la hipótesis tiene shape graph con *root* de tres hijos del mismo tipo L, el prune correcto es $\mathrm{NLF}(s, L) \geq 3$. ¿Y si los tres hijos son de tipos distintos?

4. ¿Por qué NLF NO se chequea en pasos intermedios del DFS? Discutí: ¿qué información agregaría? ¿Cuál es el costo de chequearla?

5. Diseñá un grafo y una hipótesis donde NLF tenga cero impacto (descarta cero starts). Después diseñá uno donde NLF descarte el 99 %.

6. Si extendiéramos NLF para considerar también el tipo de arista (NLP: *Neighborhood Label Profile*), ¿cuánta más memoria costaría? ¿Vale la pena?

Capítulo 19

Poda con *k-hop reachability*

Intuición

NLF es un prune *local*: chequea sólo el primer paso. Para hipótesis multi-hop, querríamos descartar starts desde los que *no se puede llegar* al tipo destino final, sin importar qué tan lejos. *k-hop reachability* es exactamente eso: para cada vértice v y cada tipo L, un bit que dice "existe un camino de longitud $\leq$ `MAX_DEPTH` desde v a algún vértice de tipo L". Bit cero $\Rightarrow$ v no genera match cuya hipótesis exija ese tipo. La extensión temporal (min_first, max_first) agrega el chequeo de ventana: si la primera arista del camino más temprano desde v cae fuera de la ventana del hunt, también se descarta. Este capítulo formaliza el índice y la poda.

19.1. Definición

Definición 19.1 (k-hop reachability)**.** Sea `MAX_DEPTH` una constante (en GraphHunter, 4) y $L \in \mathcal{L}_V$ un tipo destino. Para $v \in \mathcal{V}$:

$$\text{bits}[v][L] = 1 \iff \exists\, \text{walk } v = v_0, v_1, \ldots, v_k \text{ con } k \leq \texttt{MAX_DEPTH},\ \text{type}(v_k) = L.$$

La estructura concreta: bits$[v]$ es un `u16`, un bit por `EntityType` (tag 0 a 8 ocupan 9 bits; los 7 restantes quedan libres para extensiones futuras).

Nota. bits es *independiente del tipo de arista y del timestamp*. Es una sobre-aproximación: "*podría* haber un camino que cumpla". Si bit es 1, el matcher no descarta; si es 0, descarta seguro (porque ningún camino existe).

19.2. El prune

Algorithm 19.1: Prune k-hop al inicio del DFS

Input: starts candidatos $\mathcal{V}_{L_0^V}$, hipótesis H con tipo destino final L_k^V
Output: starts filtrados

```
L ← L_k^V.tag;
filtered ← ∅;
for s ∈ V_{L_0^V} do
    if bits[s] & (1 ≪ L) then
        filtered.push(s);
    end
end
return filtered;
```

Operación *bitwise*: una sola instrucción de CPU (`and` + comparación con cero). Para 10^6 starts, son 10^6 operaciones de palabra, que es $\sim$1 ms.

19.3. Soundness

Teorema 19.2 (k-hop prune sound)**.** *Si la hipótesis H tiene longitud $k \leq$ `MAX_DEPTH` y exige tipo destino L_k^V, entonces:*

$$\text{bits}[s][L_k^V.\text{tag}] = 0 \implies s \text{ no es origen de ningún match.}$$

Demostración. Por contradicción. Si existe match σ con $\sigma(p_0) = s$, entonces existe un walk $s = v_0, v_1, \ldots, v_k$ con $\text{type}(v_k) = L_k^V$ y $k \leq$ `MAX_DEPTH` (por hipótesis). Por definición de bits, eso implica $\text{bits}[s][L_k^V] = 1$, contradicción. □ □

19.3.1. Cuando $k >$ `MAX_DEPTH`

Si la hipótesis es más larga que `MAX_DEPTH`, el prune *NO se aplica*: no podemos garantizar que un walk más largo no exista aunque bits = 0. El motor en ese caso saltea el prune y va al DFS directo.

En la práctica, `MAX_DEPTH` = 4 cubre el 95 % de las hipótesis de threat hunting reales. Las hipótesis de longitud ≥ 5 son raras (los analistas las escriben pero el matcher les advierte que pueden ser lentas).

19.4. Extensión temporal

NLF y k-hop básico ignoran el tiempo. Para hipótesis con `WITHIN`, querríamos descartar starts cuya *primera arista útil* caiga fuera de la ventana. La extensión D5/M2 agrega dos matrices más:

Definición 19.3 (min_first, max_first)**.**

$$\text{min_first}[v][L] = \text{mín}\{\tau_0 : v \rightsquigarrow_{\leq \texttt{MAX_DEPTH}} L \text{ con primer edge } (\cdot, \cdot, \tau_0)\}$$

$$\text{max_first}[v][L] = \text{máx}\{\tau_0 : \ldots\}$$

Es decir: min_first$[v][L]$ es el menor *first edge timestamp* entre todos los caminos de v a algún vértice de tipo L; max_first es el mayor.

19.4.1. El prune temporal

Teorema 19.4 (Prune temporal sound)**.** *Sea la hipótesis con ventana $[t_{\text{lo}}, t_{\text{hi}})$. Si:*

$$\text{min_first}[v][L] \geq t_{\text{hi}} \quad \textit{or} \quad \text{max_first}[v][L] < t_{\text{lo}},$$

entonces ningún walk de v al tipo L tiene su primer edge *en la ventana.*

Demostración. Si min_first$[v][L] \geq t_{\text{hi}}$: todo walk desde v tiene $\tau_0 \geq t_{\text{hi}}$, fuera de la ventana por arriba. Si max_first$[v][L] < t_{\text{lo}}$: todo walk tiene $\tau_0 < t_{\text{lo}}$, fuera por abajo. En ambos casos, ningún walk válido existe. □ □

19.4.2. Cuándo aplica

El prune temporal se activa cuando la hipótesis tiene `WITHIN`:

- Hipótesis sin `WITHIN`: la ventana es $[-\infty, +\infty)$; ningún start se descarta por tiempo.
- Hipótesis con `WITHIN 30m` y un *anchor time* explícito: el matcher chequea min/max_first versus la ventana.

19.5. Storage layout: dense + sparse

El layout naive sería: $|\mathcal{V}| \times |\mathcal{L}_V|$ tabla densa para cada uno de bits, min_first, max_first. Para $|\mathcal{V}| = 10^8$ vértices y $|\mathcal{L}_V| = 9$:

- bits: $10^8 \times 2$ B (`u16`) = 200 MB.
- min_first: $10^8 \times 9 \times 8$ B (`i64`) = 7.2 GB.
- max_first: otros 7.2 GB.

Total: $\sim$ 14.6 GB. Inviable.

La observación clave: *muchos vértices no tienen aristas salientes*. Para esos, min/max_first no aporta info (no hay caminos). El layout *sparse* guarda esas filas sólo para vértices con `outdeg` > 0:

- bits: denso $|\mathcal{V}| \times 2$ B.
- rank: $|\mathcal{V}| \times 4$ B (`u32`). `rank[v] = NO_RANK` si out-deg$(v) = 0$; si no, indexa una fila en las matrices sparse.
- min_first: sparse $|\text{ranked}| \times 9 \times 8$ B.
- max_first: igual.

Para grafos típicos donde $\sim 30\,\%$ de los vértices tienen `outdeg` > 0 (*ranked*):

- bits: 200 MB.
- rank: 400 MB.
- min/max_first sparse: $2 \times 30 \cdot 10^6 \times 72$ B = $\sim$ 4.3 GB.

Total: $\sim$ 4.9 GB. Tres veces menos que el dense.

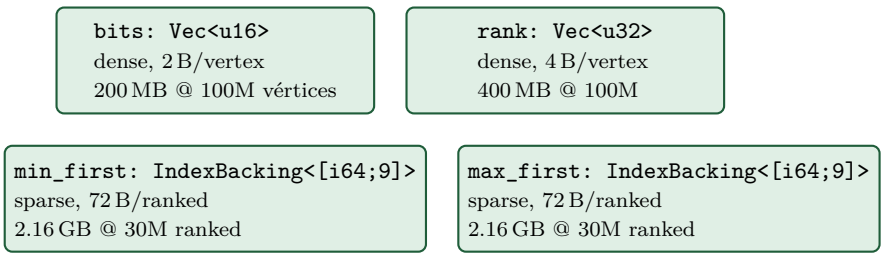


Figura 19.1: `KHopReach` layout. Dense ($|\mathcal{V}|$) para `bits` y `rank`; sparse (sólo vértices ranked) para `min_first` y `max_first`. Total: $\sim$4.9 GB en grafo de 100M vértices con 30 % *outgoing-having*, vs $\sim$14.6 GB del layout dense.

19.6. Construcción

El bits se construye con BFS multi-pasada (BFS reverso desde todos los *leaves* de cada tipo). El min/max_first se calcula durante la misma pasada.

Algorithm 19.2: KHopReach build (BFS multi-source)

Input: grafo G
Output: `KHopReach{ bits, rank, min_first, max_first }`

```
bits[v] ← 0 para todo v;
for L ∈ L_V do
    inicializar BFS desde todos los v con type(v) = L;
    for distance d = 0, 1, ..., MAX_DEPTH do
        for cada vértice w en frontera d do
            bits[w] |= (1 ≪ L);
            actualizar min/max_first[w][L] con timestamps de las aristas reverse;
        end
        avanzar frontera vía aristas reverse;
    end
end
return;
```

Costo: $O(|\mathcal{L}_V| \cdot \texttt{MAX_DEPTH} \cdot |\mathcal{E}|)$. Para $|\mathcal{L}_V| = 9$, $\texttt{MAX_DEPTH} = 4$, $|\mathcal{E}| = 10^8$: $\sim 3.6 \cdot 10^{10}$ operaciones, $\sim$1.5 min en hardware típico. Lazy build (capítulo 14) lo difiere al primer hunt.

19.7. Resumen

- k-hop reachability: bits$[v][L] = 1$ sii existe walk de v al tipo L en $\leq$ `MAX_DEPTH` saltos.
- Prune sound: bits$[v][L] = 0 \Rightarrow v$ no genera match. No completo (sobre-aproxima).
- Extensión temporal: min/max_first$[v][L]$ permite descartar starts cuya primera arista no cabe en la ventana.
- Layout dense + sparse para ahorrar memoria ($\sim 3\times$ vs dense puro).
- Build $O(|\mathcal{L}_V| \cdot \texttt{MAX_DEPTH} \cdot |\mathcal{E}|)$, lazy en `OnceLock`.

Ejercicios

1. ★ Para el grafo de la figura del capítulo 2, calculá bits$[v]$ para cada vértice asumiendo `MAX_DEPTH` $= 2$.
2. ¿Por qué la cota `MAX_DEPTH` $= 4$? ¿Qué se rompe si la hipótesis es de longitud 5? ¿Y si fuera 6?
3. ★ Para una hipótesis con WITHIN=30m y un anchor time específico, mostrá un ejemplo donde el prune temporal descarta un start aunque el prune k-hop básico no lo descarte.
4. Calculá la memoria del layout sparse para un grafo de $5 \cdot 10^7$ vértices con 50 % *outgoing-having*.
5. Diseñá un caso adversarial donde el prune k-hop tenga cero impacto (todos los starts pasan).
6. ¿Qué cambia si en vez de un solo bit por tipo guardáramos *cantidad* de caminos al tipo (no sólo si existe)? Discutí pros y contras.

Capítulo 20

Worst-case-optimal joins y el AGM *bound*

Intuición

El capítulo 4 introdujo el AGM bound y mencionó que existen algoritmos que lo alcanzan — los *worst-case-optimal joins* (WCOJ). Este capítulo profundiza: muestra por qué el DFS naive es estrictamente subóptimo para ciertas *shapes* (cliques, triangle joins) y prepara el terreno para LFTJ-K_4 (capítulo 21). El punto clave: los joins binarios clásicos (nested-loop, hash, sort-merge) ejecutan "de a dos relaciones"; para hipergrafos cíclicos ese orden no alcanza el AGM. WCOJ multi-way intersección *todas las relaciones a la vez* con *prefix-extension*, y eso es lo que cierra el gap.

20.1. Repaso del AGM bound

Recordemos del capítulo 4:

Teorema 20.1 (AGM)**.** *Para todo join* $Q = R_1 \bowtie \ldots \bowtie R_m$ *con* $|R_l| \leq m$*:* $|Q| \leq m^{\rho^*(Q)}$*, donde* $\rho^*(Q)$ *es el* fractional edge cover number *del hipergrafo de* Q*.*

- Triángulo $R(A,B) \bowtie S(B,C) \bowtie T(A,C)$: $\rho^* = 1.5$. Output $\leq m^{1.5}$.
- K_4 (six edges): $\rho^* = 2$. Output $\leq m^2$.
- Cadena de 3: $\rho^* = 2$. Output $\leq m^2$.
- Cadena de k: $\rho^* = k$. Output $\leq m^k$.

20.2. Por qué los joins binarios pierden

Considerá el triángulo $R(A,B) \bowtie S(B,C) \bowtie T(A,C)$. Un *hash join* clásico secuencial:

1. Computar $R \bowtie S$ (en B). Output: tuplas (A,B,C). Tamaño peor caso: $|R| \cdot |S|/|B| = m^2/|B|$, en peor caso $O(m^2)$ (cuando $|B| = 1$).
2. Computar $(R \bowtie S) \bowtie T$ (en A,C). Filtra el resultado intermedio.

El *tiempo* del paso 1 es $O(m^2)$ —genera m^2 filas intermedias incluso si la salida final tiene $m^{1.5}$ filas. Comparación:

- Output final: $O(m^{1.5})$ por AGM.
- Trabajo de hash join secuencial: $O(m^2)$.

- Ratio: $\sqrt{m}$. Para $m = 10^6$, eso son 1000× más trabajo que el output justifica.

Esa es la disparidad. El join binario es "ciego" al hecho de que T eventualmente filtraría buena parte de los intermedios. Computa todo, después filtra. WCOJ filtra *durante* la generación.

20.3. La idea del WCOJ: prefix-extension multi-way

WCOJ ataca el join *variable por variable* en vez de *relación por relación*:

1. Elegir un orden total de las variables $(A, B, C, \ldots)$.
2. Para cada variable, *intersectar* todas las relaciones que la contienen sobre el prefijo ya bound.
3. Sólo extender la asignación si la intersección es no-vacía.
4. Recursar a la próxima variable.

La pieza interesante: el paso 2 es una *multi-way intersection* sobre los conjuntos de candidatos para la próxima variable, ponderada por *cuál relación tiene el conjunto más chico* (heuristic "*tightest constraint first*").

20.3.1. Aplicado al triángulo

Algorithm 20.1: NPRR para triángulo $R(A,B) \bowtie S(B,C) \bowtie T(A,C)$

Input: relaciones R, S, T
Output: (A, B, C) tuples del join
▷ `Bind` A
for $a \in \pi_A(R) \cap \pi_A(T)$ **do**
  ▷ `Bind` B, `restringido por` R `y` T
  B_set $\leftarrow \pi_B(\sigma_{A=a}(R))$;
  for $b \in$ B_set **do**
    ▷ `Bind` C, `restringido por` S `y` T
    C_set $\leftarrow \pi_C(\sigma_{B=b}(S)) \cap \pi_C(\sigma_{A=a}(T))$;
    for $c \in$ C_set **do**
      `emit`(a, b, c);
    end
  end
end

La clave: el `C_set` es una *intersección* de dos conjuntos: "los C que combinan con b en S" y "los C que combinan con a en T". Ambos están indexados, ambos están ordenados; la intersección es $O(\text{mín}(|S_b|, |T_a|))$.

El análisis (NPRR 2012) demuestra que el costo total es $O(m^{1.5})$ *worst case* —alcanzando el AGM bound.

20.4. LFTJ: la versión implementable

NPRR es elegante pero requiere un orden global sobre las variables y soporta sólo la versión *batched* (todo el output a la vez). LFTJ (Veldhuizen, ICDT 2014) es una variante más implementable:

- Construye un *trie* sobre cada relación, indexado por el orden de variables.

- En cada paso, intersecta los tries que comparten la variable activa, pidiendo al trie por el "próximo valor de C que sigue después de c_0".
- La operación es *leapfrog*: avanza al máximo entre todos los punteros activos, vuelve a chequear si están sincronizados.

Esto evita materializar conjuntos intermedios completos (tanto en NPRR ingenuo) —el LFTJ es *streaming* sobre los tries.

GraphHunter implementa LFTJ específicamente para K_4. El capítulo siguiente lo desarrolla con worked example.

20.5. Por qué K_4 y no triángulo o cadena

GraphHunter tiene LFTJ sólo para K_4. Razones:

- **Triángulo**: tiene $\rho^* = 1.5$, pero las hipótesis triangulares son raras en threat hunting (las cadenas y árboles son lo que los analistas escriben). El ROI de implementar triángulo es bajo.
- K_4: aparece como *benchmark canónico*; varias hipótesis avanzadas ($\sim$10 % de los hunts en el corpus de Base4) son K_4 homogéneo. $\rho^* = 2$ versus DFS $O(\deg^3)$ da $\sim 100\times$ en grafos densos.
- **Cadena**: $\rho^* = k$ coincide con la complejidad del DFS bien-podado. El AGM bound es ajustado y no hay mejora teórica posible.
- **Estrellas y árboles**: α-acíclicos $\Rightarrow$ Yannakakis los resuelve en tiempo lineal en input + output (capítulos 22, 23, 24). Mejor que LFTJ específicamente para esa familia.

El criterio: implementar WCOJ donde el *lift teórico* sobre DFS es grande *y* la familia es relevante en el corpus real.

20.6. Resumen

- AGM bound: $|Q| \leq m^{\rho^*(Q)}$. Recordá: triángulo 1.5, K_4 2, cadena k.
- Joins binarios secuenciales pueden gastar $O(m^2)$ en intermedios para producir $O(m^{1.5})$ de output (factor $\sqrt{m}$ de waste).
- WCOJ ataca *variable por variable* con *multi-way intersection*, alcanzando el AGM bound.
- LFTJ es la versión implementable de WCOJ (Veldhuizen 2014).
- GraphHunter implementa LFTJ-K_4 específicamente porque ahí el lift es grande y la familia es relevante.

Ejercicios

1. ★ Para un grafo *Erdős-Rényi* $G(n, p)$ con $n = 1000$ y $p = 0.05$, estimá: cuántos triángulos hay en expectation. Compará con el AGM bound $m^{1.5}$.
2. ¿Por qué la cadena tiene $\rho^* = k$? Pensá: ¿hay alguna "estructura" que el WCOJ pueda explotar para hacerlo mejor que $O(m^k)$?
3. ★ Si implementás un hash join binario para triángulo, ¿cuál es el peor caso de cardinalidad intermedia para R, S, T cada una de tamaño m?

4. Diseñá una hipótesis *shape graph* para la cual el DFS bien-podado (con NLF + k-hop) sea estrictamente mejor que LFTJ. (Pista: pensá hipótesis donde las podas filtran masivamente.)

5. ¿Por qué el WCOJ requiere *tries* (estructuras indexadas) y no puede ejecutar sobre relaciones flat? ¿Qué operación se vuelve cara sin trie?

6. Si en el futuro GraphHunter quisiera soportar LFTJ para *triángulo*, ¿qué tendría que cambiar? Discutí implementación y memoria.

Capítulo 21

Leapfrog Triejoin para K_4

Intuición

K_4 es el grafo completo de cuatro vértices: cada par de vértices está conectado por una arista, en cada dirección. El patrón K_4 es *el* ejemplo canónico donde el DFS naive escala como $O(\deg^3)$ pero un *worst-case-optimal join* puede alcanzar $O(m^2)$. LFTJ (Leapfrog Triejoin, Veldhuizen 2014) es la versión implementable. Este capítulo lo desarrolla con un ejemplo trabajado: arrancamos de un grafo chico, mostramos cómo se construye el trie, cómo funciona la *leapfrog* sobre tres *pointers*, y cómo el motor extrae los K_4s sin intermedios cuadráticos.

21.1. El patrón K_4 en GraphHunter

K_4 homogéneo en threat hunting: cuatro entidades del mismo tipo, cada par conectado en cada dirección por una arista del mismo tipo. Ejemplo:

```
User -[Knows]-> User -[Knows]-> User -[Knows]-> User
{shape=k4, all-types-equal}
```

"Cuatro usuarios mutuamente conectados por relación `Knows`" es una hipótesis típica de detección de *insider threat clusters*: grupos pequeños cohesionados de usuarios que se conocen entre sí, donde uno puede ser un atacante infiltrado en el grupo.

21.2. La estructura del problema

Un K_4 tiene $\binom{4}{2} \cdot 2 = 12$ aristas dirigidas (cada uno de los 6 pares en ambas direcciones). Pero por simetría de K_4, el patrón se reduce a las 6 aristas no orientadas. Para encontrar todos los K_4s en un grafo, necesitamos cuatro vértices (a, b, c, d) tales que existan las 6 aristas: (a, b), (a, c), (a, d), (b, c), (b, d), (c, d).

21.2.1. El DFS naive

El DFS naive arranca:

1. Para cada $a \in \mathcal{V}$ ($O(n)$):
2. Para cada $b \in N^+(a)$ ($O(\deg)$):
3. Para cada $c \in N^+(b)$ tal que $c \in N^+(a)$ ($O(\deg)$ más chequeo $O(\log \deg)$):
4. Para cada $d \in N^+(c)$ tal que $d \in N^+(a) \cap N^+(b)$ ($O(\deg)$ más chequeo $O(\log \deg)$):

Total: $O(n \cdot \deg^3 \cdot \log \deg)$. Para grafos densos con $\deg = O(\sqrt{m})$: $O(n \cdot m^{1.5} \cdot \log m) \approx O(m \cdot m^{1.5}) = O(m^{2.5})$. Por encima del AGM bound (m^2).

21.2.2. El plan LFTJ

LFTJ ataca *variable por variable* en lugar de *relación por relación*:

1. Variable a: iterar todos los vértices del grafo.
2. Variable b: para cada a, iterar $N^+(a)$.
3. Variable c: para cada (a, b), iterar $N^+(a) \cap N^+(b)$ (**intersección**).
4. Variable d: para cada (a, b, c), iterar $N^+(a) \cap N^+(b) \cap N^+(c)$ (**intersección triple**).

El truco: las dos intersecciones se hacen por *leapfrog* sobre los *adjacency lists ordenados*. No materializamos los conjuntos.

21.3. La operación *leapfrog*

Definición 21.1 (Leapfrog intersection)**.** Dadas k listas ordenadas $L_1, L_2, \ldots, L_k$, *leapfrog intersection* mantiene k punteros (uno por lista) y avanza el puntero *más atrasado* hasta superar al *más adelantado*, repitiendo hasta que todos estén alineados (o algún puntero se agote). Cuando todos coinciden, el valor es un elemento de la intersección; emite y avanza todos.

Algorithm 21.1: Leapfrog intersection sobre k listas ordenadas

Input: listas $L_1, \ldots, L_k$, cada una sorted
Output: secuencia de valores en $L_1 \cap \cdots \cap L_k$

$p_i \leftarrow 0$ para todo i (punteros iniciales);
while *ningún* p_i *llegó al final* **do**
  mx $\leftarrow$ máx$_i$ $L_i[p_i]$ (el más adelantado);
  for *cada* i **do**
    avanzar p_i hasta que $L_i[p_i] \geq$ mx (binary search o linear scan);
  end
  if $L_1[p_1] = L_2[p_2] = \cdots = L_k[p_k]$ **then**
    `emit`$(L_1[p_1])$;
    avanzar todos los punteros en 1;
  end
end

Ejemplo: Intersección leapfrog de 3 listas

$L_1 = [1, 4, 7, 9, 12]$, $L_2 = [2, 4, 6, 9, 11]$, $L_3 = [4, 5, 9, 10, 12]$.

- Step 1: punteros en $[1, 2, 4]$. mx $= 4$. Avanzar p_1 hasta ≥ 4: $p_1 \to 4$. Avanzar p_2 hasta ≥ 4: $p_2 \to 4$. p_3 ya en 4. Match $\Rightarrow$ `emit`(4). Avanzar todos: $[7, 6, 5]$.
- Step 2: mx $= 7$. Avanzar p_2 a 9, p_3 a 9. p_1 avanza a 9. Match $\Rightarrow$ `emit`(9). Avanzar: $[12, 11, 10]$.
- Step 3: mx $= 12$. Avanzar p_2 a ∞ (lista agotada). Salir.

Resultado: $\{4, 9\}$ en $O(|L_1| + |L_2| + |L_3|) = O(15)$ operaciones.

21.3.1. Por qué leapfrog es $O(\sum |L_i|)$ y no peor

Cada paso del while avanza al menos un puntero. Cada puntero avanza a lo sumo $|L_i|$ veces. Total de avances: $\sum |L_i|$. Cada avance es $O(\log |L_i|)$ si usás binary search, o $O(|L_i|)$ en peor caso si usás linear scan amortizado a 1 por elemento.

21.3.2. SIMD intersection

GraphHunter usa SIMD vía `core/graph-engine/src/simd\_rust.rs` (Rust puro con `std::arch::x86_64`) para acelerar la intersección de dos listas ordenadas: `_mm256_cmpeq_epi32` compara 8 `u32`s a la vez. Para listas de 10^4 elementos, SIMD da $\sim$ 4–8$\times$ sobre la versión escalar.

En el código.

`core/graph-engine/src/simd\_rust.rs` (Rust con runtime dispatch AVX2/escalar). La integración con LFTJ en `core/graph-engine/src/lftj.rs`.

21.4. El trie sobre las aristas

LFTJ requiere los conjuntos $N^+(v)$ ordenados, indexados por v. El *streaming backend* ya entrega eso post-finalize: `neighbors_arc(v) + read + as_slice()` devuelve un slice contiguo de las salientes de v ordenadas por timestamp.

Para LFTJ-K_4 necesitamos las salientes ordenadas por `dest_sid` (no por timestamp), porque el *leapfrog* intersect en el espacio de `dest_sid`. La construcción del trie LFTJ:

```
pub struct LftjTrie {
    // Para cada source_sid, una lista sorted de dest_sids.
    by_source: HashMap<StrId, Vec<StrId>>,
}

impl LftjTrie {
    pub fn build(graph: &GraphHunter) -> Self {
        let mut by_source: HashMap<StrId, Vec<StrId>> = HashMap::new();
        graph.streaming.for_each_edge(|src_sid, edge| {
            by_source.entry(src_sid).or_default().push(edge.dest_sid);
        });
        for vec in by_source.values_mut() {
            vec.sort();
            vec.dedup();
        }
        Self { by_source }
    }

    pub fn neighbors(&self, sid: StrId) -> &[StrId] {
        self.by_source.get(&sid).map_or(&[], |v| v.as_slice())
    }
}
```

Build: $O(|\mathcal{E}| \log |\mathcal{E}|)$. Para $|\mathcal{E}| = 10^7$, ~ 3 s.

21.4.1. Overhead vs DFS

La construcción del trie es overhead que el DFS no paga. Para hipótesis donde el DFS bien-podado ya alcanza performance, ese overhead no se justifica. Por eso LFTJ-K_4 sólo se dispatcha cuando el planner detecta la estructura K_4 específica (capítulo 25).

21.5. LFTJ para K_4: el algoritmo completo

Algorithm 21.2: LFTJ-K_4 (versión simplificada, sin symmetry breaking)

Input: `LftjTrie` T
Output: Vec<HuntResult>: tuplas (a, b, c, d)

```
for a ∈ V do
    N_a ← T.neighbors(a);
    for b ∈ N_a do
        N_b ← T.neighbors(b);
        N_{a∩b} ← leapfrog_intersect(N_a, N_b);
        for c ∈ N_{a∩b} do
            N_c ← T.neighbors(c);
            N_{a∩b∩c} ← leapfrog_intersect(N_a, N_b, N_c);
            for d ∈ N_{a∩b∩c} do
                emit(a, b, c, d);
            end
        end
    end
end
```

21.5.1. Análisis: $O(m^2)$

El análisis de NPRR/LFTJ aplica directo: el trabajo total es $O(\sum_{(a,b)} |N_a|+|N_b|+\sum_{(a,b,c)} |N_a|+|N_b|+|N_c|)$, que se demuestra (vía *Loomis-Whitney inequality*) acotado por $O(m^2)$. Detalles en Veldhuizen 2014; lo crucial para el lector es:

- DFS: $O(m^{2.5})$ en peor caso.
- LFTJ: $O(m^2)$ en peor caso.
- Ratio: $\sqrt{m}$. Para $m = 10^4$, eso son 100×; para $m = 10^6$, 1000×.

21.6. Symmetry breaking

K_4 tiene grupo de simetría S_4 de orden 24: cada K_4 genuino se enumera 24 veces si no se hace nada (las 24 permutaciones de los 4 vértices). Para evitarlo, LFTJ-K_4 agrega la restricción de orden:

$$a < b < c < d$$

sobre los `StrId`s. Esto reduce el output a una sola representación por K_4, ahorrando factor 24.

Nota. Este *symmetry breaking* es válido sólo para K_4*homogéneo* (misma etiqueta en cada vértice). Si los vértices tuvieran etiquetas distintas (por ejemplo, dos `User` y dos `Host`), las simetrías serían menores y el orden tendría que respetar las etiquetas.

21.7. Cuándo LFTJ-K_4 gana en la práctica

Bench típico: grafo 10^7 aristas, ~ 100 K_4s reales.

- DFS naive: ∼30 s (300 ms si bien-podado).
- LFTJ: ∼5 ms (más 3 s de build de trie one-time).

LFTJ gana 60× sobre el DFS bien-podado en hunt time. El build de trie se amortiza si el usuario corre múltiples hunts de K_4 sobre el mismo grafo (típico).

21.8. Resumen

- LFTJ resuelve subgraph matching variable-por-variable con *leapfrog intersection* sobre tries.
- Para K_4: $O(m^2)$ versus $O(m^{2.5})$ del DFS.
- GraphHunter implementa LFTJ-K_4 específicamente, con SIMD intersection vía `core/graph-engine/src/si`
- Symmetry breaking ($a < b < c < d$) ahorra factor 24 cuando el patrón es homogéneo.
- Trade-off: build de trie one-time vs hunts más rápidos; se amortiza con múltiples hunts.

Ejercicios

1. ★ Calculá: para un grafo $G(1000, 0.05)$ Erdős-Rényi, ¿cuántos K_4s hay en expectation? Compará con el AGM bound m^2.
2. Implementá leapfrog intersection sobre dos listas ordenadas con búsqueda binaria. ¿Cuál es la complejidad asintótica? ¿Y el peor caso?
3. ★ Demostrá que el DFS bien-podado para K_4 no puede mejorar a $O(m^2)$ en peor caso (sin información estructural extra).
4. Diseñá un grafo donde LFTJ-K_4 sea estrictamente peor que DFS. (Pista: ¿qué pasa si el trie es huge pero hay pocos K_4s? ¿Y el costo del build?)
5. ¿Por qué la *symmetry breaking* ($a < b < c < d$) no se generaliza directo al caso heterogéneo (mezcla de `User` y `Host`)? ¿Qué cambiaría?
6. ¿Cómo se extendería LFTJ a K_5? ¿Cuál es la complejidad esperada? ¿Vale la pena implementarlo?

Capítulo 22

Hipergrafos, α-aciclicidad y *join trees*

Intuición

Hasta ahora vimos joins binarios ($R \bowtie S$) y joins multi-way "planos". Para entender Yannakakis necesitamos una herramienta más: el *hipergrafo de la consulta*. Dado un join de n relaciones, su hipergrafo tiene los atributos como nodos y cada relación como una hiperarista (subset de atributos). La propiedad mágica es la α-aciclicidad: si el hipergrafo de la consulta es α-acíclico, existe un *join tree* sobre las relaciones, y Yannakakis resuelve el join en tiempo lineal en input + output. Todas las cadenas, estrellas, árboles puros son α-acíclicos. Triángulo, K_4 y otros cíclicos no lo son. Este capítulo da las definiciones, muestra el algoritmo GYO de detección, y construye los join trees que el matcher de Yannakakis recorre.

22.1. Hipergrafos: del join al *query graph*

Definición 22.1 (Hipergrafo de una consulta)**.** Dado un join $Q = R_1 \bowtie R_2 \bowtie \ldots \bowtie R_m$ sobre los atributos $\mathcal{A} = \bigcup_l \text{attrs}(R_l)$, su *hipergrafo* es $H(Q) = (\mathcal{A}, F)$ donde $F = \{F_1, F_2, \ldots, F_m\}$ y $F_l = \text{attrs}(R_l)$ (cada relación es una hiperarista).

Ejemplo: Hipergrafo de una cadena de 3

$Q = R(A,B) \bowtie S(B,C) \bowtie T(C,D)$.
Hipergrafo: $\mathcal{A} = \{A,B,C,D\}$, $F = \{F_R = \{A,B\}, F_S = \{B,C\}, F_T = \{C,D\}\}$.

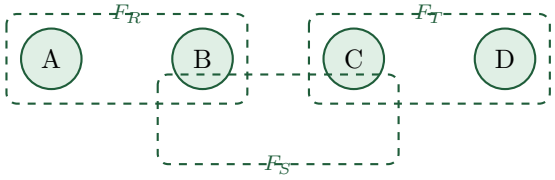


Figura 22.1: Hipergrafo de la cadena de 3 relaciones. Cada hiperarista (rectángulo dashed) es una relación. Los atributos compartidos (B entre F_R y F_S; C entre F_S y F_T) son los join keys.

Ejemplo: Hipergrafo del triángulo

$Q = R(A,B) \bowtie S(B,C) \bowtie T(A,C)$.
Hipergrafo: $\mathcal{A} = \{A,B,C\}$, $F = \{F_R = \{A,B\}, F_S = \{B,C\}, F_T = \{A,C\}\}$.
Cada par de hiperaristas comparte exactamente un atributo: $F_R \cap F_S = \{B\}$, $F_S \cap F_T = \{C\}$, $F_R \cap F_T = \{A\}$. Eso forma un *ciclo* en el sentido hipergráfico: $F_R \to F_S$ (vía B), $F_S \to F_T$ (vía C), $F_T \to F_R$ (vía A).

22.2. α-aciclicidad

La definición clásica es operacional, vía el algoritmo GYO.

22.2.1. El algoritmo GYO

Definición 22.2 (Reducción GYO)**.** Dado un hipergrafo $H = (\mathcal{A}, F)$, aplicar repetidamente las dos reglas:

1. **Eliminar atributo aislado**: si un atributo $a \in \mathcal{A}$ aparece en exactamente una hiperarista F_l, eliminarlo de F_l y de $\mathcal{A}$.
2. **Eliminar hiperarista contenida**: si una hiperarista F_l está contenida en otra $F_{l'}$ ($F_l \subseteq F_{l'}$), eliminar F_l.

Si después de aplicar las reglas hasta no poder más, $F = \emptyset$, el hipergrafo es *α-acíclico*. Si no, es cíclico.

Ejemplo: GYO sobre la cadena

$F_R = \{A, B\}$, $F_S = \{B, C\}$, $F_T = \{C, D\}$.

- Atributo A aparece sólo en F_R. Eliminarlo: $F_R = \{B\}$.
- Atributo D aparece sólo en F_T. Eliminarlo: $F_T = \{C\}$.
- Ahora $F_R = \{B\} \subseteq F_S = \{B, C\}$. Eliminar F_R.
- Ahora $F_T = \{C\} \subseteq F_S = \{B, C\}$. Eliminar F_T.
- Quedó $F = \{F_S\}$. Atributos B y C aparecen sólo en F_S; eliminarlos. $F_S = \emptyset$; eliminar F_S.
- $F = \emptyset$. **Cadena es α-acíclica.**

Ejemplo: GYO sobre el triángulo

$F_R = \{A, B\}$, $F_S = \{B, C\}$, $F_T = \{A, C\}$.

- Atributo A: aparece en F_R y F_T. No aislado.
- B: aparece en F_R y F_S. No aislado.
- C: aparece en F_S y F_T. No aislado.
- No hay hiperarista contenida en otra (todas tienen 2 atributos distintos).
- Ninguna regla aplica. $F \neq \emptyset$. **Triángulo es α-cíclico.**

Nota. α-aciclicidad NO coincide con la noción intuitiva de "ciclo en el grafo". Hay hipergrafos que "parecen tener ciclos" pero son α-acíclicos en el sentido GYO. Y al revés. La definición es operacional y vale lo que vale para Yannakakis: si GYO reduce a vacío, Yannakakis funciona en lineal.

22.3. El *join tree*

Teorema 22.3 (Join tree iff α-aciclicidad)**.** *Un hipergrafo H es α-acíclico* si y sólo si *existe un* join tree*: un árbol T cuyos nodos son las hiperaristas F_l, tal que para cualquier atributo $a \in \mathcal{A}$,*

el conjunto de nodos $\{F_l : a \in F_l\}$ *forma un sub-árbol conexo de* T.

Idea. Forward: si H es α-acíclico, GYO da un orden de eliminación; el join tree se construye conectando cada hiperarista eliminada a la que la "contuvo" en su eliminación. Backward: dado un join tree, GYO siempre puede reducir empezando por las hojas. □ □

Ejemplo: Join tree de la cadena

Para la cadena $R(A,B) \bowtie S(B,C) \bowtie T(C,D)$, el join tree es:

$$F_R \quad --- \quad F_S \quad --- \quad F_T$$

B aparece en F_R y F_S: conexo (camino directo). C aparece en F_S y F_T: conexo. A aparece sólo en F_R; D sólo en F_T. Ambos sub-árboles triviales.

Ejemplo: Join tree de una estrella

$Q = R_0(A,B_1) \bowtie R_1(A,B_2) \bowtie R_2(A,B_3)$ (tres relaciones que comparten el atributo central A).
Join tree:

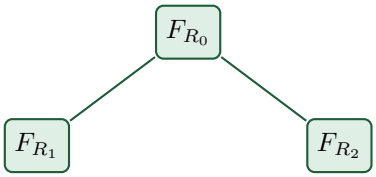


Figura 22.2: Join tree de una estrella de tres relaciones. A aparece en todas: sub-árbol = todo el árbol (conexo).

22.4. Por qué α-aciclicidad importa

Teorema 22.4 (Yannakakis 1981)**.** *Si* $H(Q)$ *es* α*-acíclico, el join* Q *puede computarse en tiempo* $O(|Q_{\text{in}}| + |Q_{\text{out}}|)$*, donde* $|Q_{\text{in}}| = \sum_l |R_l|$ *y* $|Q_{\text{out}}| = |Q|$.

Lineal en input + output. Compará con el AGM bound $O(m^{\rho^*})$: para α-acíclicas con $\rho^* > 1$, Yannakakis es estrictamente mejor (no en peor caso porque output puede ser grande, pero el *trabajo constructivo* es lineal).

El truco es la *semi-join reducer* en dos pasadas sobre el join tree (capítulo 23): elimina del input todas las tuplas que no participan de ningún match completo, *antes* de empezar a enumerar. Después la enumeración es trivial.

22.5. La pieza estructural en GraphHunter

- Hipótesis *shape chain*: hipergrafo es cadena. α-acíclico. Yannakakis chain dispatcha (capítulo 23).
- Hipótesis *shape graph* con estructura de árbol / estrella: hipergrafo α-acíclico. Yannakakis tree-walk dispatcha (capítulo 24).
- Hipótesis *shape graph* con ciclos (K_3, K_4): cíclico. LFTJ dispatcha si es K_4 (capítulo 21); si no, fallback al DFS.

La detección de α-aciclicidad ocurre en el *planner* (capítulo 25): aplica GYO al hipergrafo de la hipótesis. $O(n_{\text{atrs}} + n_{\text{hipearistas}})$ que para hipótesis típicas (≤ 6 pasos) es trivial.

22.6. Construcción concreta del join tree

Para shape graph en GraphHunter, el planner construye el join tree con un BFS sobre el hipergrafo a partir de una hiperarista *root* (típicamente la del primer paso de la hipótesis):

```rust
pub struct JoinTree {
    pub nodes: Vec<usize>,          // step indices (orden de visita)
    pub parent: Vec<Option<usize>>, // parent[i] = padre en el árbol
    pub shared_var: Vec<VarId>,     // variable compartida con el padre
}

impl JoinTree {
    pub fn build(hyp: &Hypothesis) -> Self {
        let hg = build_hypergraph(hyp);
        let mut visited = HashSet::new();
        let mut nodes = Vec::new();
        let mut parent = Vec::new();
        let mut shared_var = Vec::new();

        let root = 0;  // first step
        let mut queue = VecDeque::from([(root, None, None)]);
        while let Some((node, p, sv)) = queue.pop_front() {
            if visited.insert(node) {
                nodes.push(node);
                parent.push(p);
                shared_var.push(sv.unwrap_or(VarId::None));
                for (next, var) in hg.adjacent(node) {
                    if !visited.contains(&next) {
                        queue.push_back((next, Some(nodes.len() - 1), Some(
                            var)));
                    }
                }
            }
        }

        Self { nodes, parent, shared_var }
    }
}
```

El `enumerate_tree` (capítulo 24) recorre estos nodos en orden y, en cada uno, decide qué *bag* consultar según qué variables ya están bound.

22.7. Resumen

- Hipergrafo de una consulta: atributos como nodos, relaciones como hiperaristas.
- α-aciclicidad: el algoritmo GYO reduce a vacío. Operacionalmente, "no hay ciclos en el sentido hipergráfico".
- Cadena, estrella, árboles: α-acíclicos. Triángulo, K_4: cíclicos.
- Join tree: existe sii α-aciclicidad. Conexidad de sub-árbol por atributo.
- Yannakakis (capítulo siguiente) corre en $O(\text{input} + \text{output})$ sobre α-acíclicos.
- GraphHunter dispatcha Yannakakis chain o tree-walk para hipótesis α-acíclicas (chain o graph respectivamente); LFTJ para K_4 cíclico; DFS para todo lo demás.

Ejercicios

1. ★ Aplicá GYO al hipergrafo de la hipótesis `c -[Auth]-> h1; c -[Auth]-> h2; c -[Auth]-> h3` (estrella de tres). ¿Es α-acíclico?
2. Aplicá GYO al hipergrafo de K_4 (6 hiperaristas binarias). Mostrá que es cíclico.
3. ★ Construí un hipergrafo α-acíclico de 4 relaciones. ¿Cuántos posibles join trees existen?
4. ¿Por qué la noción de α-aciclicidad NO coincide con la de "no hay ciclos en el grafo"? Construí un contraejemplo: hipergrafo con "ciclos visuales" que es α-acíclico, y otro al revés.
5. ¿Cómo cambia el join tree de una hipótesis chain si la hipótesis es "mismo vértice repetido" (cadena $p_0 \to p_1 \to p_2 \to p_0$)? ¿Sigue siendo α-acíclico?
6. Si una hipótesis tiene 5 pasos en chain pura, ¿cuántos nodos tiene el join tree? ¿Cuál es la altura del árbol?

Capítulo 23

Yannakakis: cadena

Intuición

Yannakakis (1981) tiene dos partes: *semi-join reducer* y *enumerator*. El reducer baja por el join tree (hojas → raíz) y sube (raíz → hojas), eliminando en cada paso las tuplas que no participan de ningún match completo. El enumerator después recorre las tuplas que sobrevivieron y enumera el join final —ya sin desperdicio. Para hipótesis con shape *chain*, el join tree es lineal, las dos pasadas son simples filtros, y el algoritmo termina en $O(\text{input} + \text{output})$. Este capítulo lo desarrolla con un ejemplo concreto y prueba la correctitud.

23.1. Setup: *bags* y la chain

Antes de Yannakakis, el motor materializa las relaciones de la hipótesis como *bags*. Para una chain $(L_0^V \to_{L_0^E} L_1^V \to_{L_1^E} \cdots \to_{L_{n-1}^E} L_n^V)$, bag_i contiene las aristas:

$$B_i = \{(u, v, \tau) : (u, v, \tau) \in \mathcal{E},\ \text{type}(u) = L_i^V,\ \text{type}_E(\cdot) = L_i^E,\ \text{type}(v) = L_{i+1}^V\}.$$

Cada B_i es la lista de aristas que podrían llenar el paso i. El join chain es:

$$B_0 \bowtie_{B_0.\text{dest}=B_1.\text{origin}} B_1 \bowtie_{B_1.\text{dest}=B_2.\text{origin}} \cdots \bowtie_{B_{n-2}.\text{dest}=B_{n-1}.\text{origin}} B_{n-1}.$$

23.2. La semi-join reducer

La idea: filtrar cada B_i hasta que sólo queden aristas que participan de *algún* match completo.

23.2.1. Pasada bottom-up

Algorithm 23.1: Pasada 1 (bottom-up): de B_{n-1} hacia B_0

```
for i from n − 2 down to 0 do
    live_origins ← {r.origin : r ∈ B_{i+1}};
    B_i ← {r ∈ B_i : r.dest ∈ live_origins};
end
```

Después de esta pasada: cada arista de B_i tiene su `dest` matched con al menos una arista de B_{i+1}. Es decir, hay una continuación posible hacia adelante.

23.2.2. Pasada top-down

Algorithm 23.2: Pasada 2 (top-down): de B_0 hacia B_{n-1}

for *i from* 1 ***to*** $n-1$ **do**
  `live_dests` $\leftarrow \{r.\text{dest} : r \in B_{i-1}\}$;
  $B_i \leftarrow \{r \in B_i : r.\text{origin} \in$ `live_dests`$\}$;
end

Después de esta pasada: cada arista de B_i tiene su `origin` matched con al menos una arista de B_{i-1} que sobrevivió. Hay una continuación posible hacia atrás.

23.2.3. Diagrama

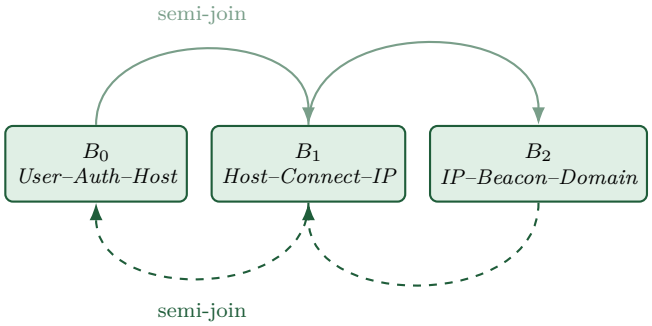


Figura 23.1: Las dos pasadas del semi-join reducer. Bottom-up (flechas dashed abajo) elimina rows cuyo `dest` no aparece como `origin` en el siguiente bag; top-down (flechas arriba) hace lo simétrico. Después, toda row sobreviviente participa de al menos un full join.

23.3. Enumerar lo que quedó

Algorithm 23.3: Yannakakis chain matcher

Input: $B_0, B_1, \ldots, B_{n-1}$ (bags reducidos)
Output: Vec<HuntResult>
for *cada* `row` *en* B_0 **do**
  `walk(row.origin, row.dest, row.timestamp, 1)`;
end

▷ `walk(prev_dest, curr_dest, t_first, step)`
if step $= n$ **then**
  `emit`(path acumulado);
  return;
end
for *cada* $r \in B_{\text{step}}$ *con* $r.\text{origin} =$ prev_dest **do**
  chequear ventana temporal y monotonicidad;
  `walk`$(r.\text{origin}, r.\text{dest}, t_{\text{first}}, \text{step}+1)$;
end

La enumeración es esencialmente un DFS, *pero* sobre los bags ya reducidos. Cada arista que el walk considera está garantizada de participar en algún match (por las dos pasadas previas), así que el walk *nunca* explora ramas muertas.

23.4. Correctitud

Teorema 23.1 (Yannakakis chain correcto)**.** *Después de las dos pasadas semi-join, el conjunto de matches enumerados por* `walk` *es exactamente el join completo* $B_0 \bowtie B_1 \bowtie \ldots \bowtie B_{n-1}$.

Idea. Soundness: cada `emit` corresponde a una secuencia de aristas $(r_0, r_1, \ldots, r_{n-1})$ con r_i.dest = r_{i+1}.origin (chequeado en el walk) y cada $r_i \in B_i$ (porque iteramos sobre los bags). Eso es exactamente la definición de $\bowtie$.

Completeness: cualquier match $(r_0, \ldots, r_{n-1})$ del join original tiene cada r_i que *sobrevivió* la pasada bottom-up (porque tenía continuación a r_{i+1}) y la top-down (porque había r_{i-1}). Por lo tanto cada r_i sigue en B_i después del reducer; `walk` los enumera. □ □

23.5. Análisis de complejidad

23.5.1. Costo del semi-join reducer

Cada pasada itera todas las aristas de cada bag una vez: $O(\sum_i |B_i|) = O(|\mathcal{E}|_{\text{relevantes}})$. El conjunto `live_origins` (resp. `live_dests`) se construye con un `HashSet` en una pasada lineal, y los chequeos `r.dest` $\in$ `live_origins` son $O(1)$.

Total: $O(|\mathcal{E}|_{\text{relevantes}})$ por pasada, dos pasadas $\Rightarrow O(|\mathcal{E}|)$ total.

23.5.2. Costo del enumerator

Cada path completo (cada match) toma $O(n)$ trabajo (push + emit). En total $O(n \cdot |Q_{\text{out}}|)$. Para hipótesis chain con n chico (≤ 6), $O(|Q_{\text{out}}|)$.

23.5.3. Total

$O(|\mathcal{E}| + |Q_{\text{out}}|)$. Lineal en input + output. Es la promesa de Yannakakis para α-acíclicas.

23.6. Comparación con DFS

DFS para la misma hipótesis:

- Itera $|Q_{\text{out}}|$ paths exitosos.
- Itera además todos los paths fallidos (que el reducer eliminó).

En grafos donde la mayoría de los paths parciales no se completan (filtro temporal, etiquetas selectivas), el DFS gasta mucho en exploración inútil. Yannakakis lo evita haciendo el chequeo de "hay continuación" como semi-join lineal antes de entrar al walk.

23.6.1. Ejemplo concreto

Hipótesis chain de 3, $|B_0| = 10^4$, $|B_1| = 10^5$, $|B_2| = 10^6$, $|Q_{\text{out}}| = 10^3$ (matches reales).

- DFS: explora $\sim 10^4 \cdot 10 \cdot 10 = 10^6$ paths parciales (asumiendo grado promedio 10 en cada paso).
- Yannakakis: 2 pasadas $O(|\mathcal{E}|_{\text{relevantes}}) = O(10^6)$. Después enumera 10^3 paths.

Mismo orden de magnitud en CPU total, pero Yannakakis paga el costo en operaciones simples (semi-join lineal con HashSet) en vez de recursión profunda con *branch mispredictions* y *cache misses*. Bench típico: 2× más rápido para hipótesis con muchos paths fallidos.

23.7. Cuando Yannakakis pierde

- **Hipótesis con cap saturado**: si el motor produce $\geq 10\,000$ results, el costo del enumerator domina; el semi-join reducer es overhead constante. DFS con cap-aware early-exit es comparable.
- **Hipótesis con muy pocos starts**: si NLF + k-hop ya filtraron hasta dejar 10 starts válidos, el costo del DFS desde 10 starts es trivial; el semi-join sobre todo el grafo es caro innecesariamente.

El *planner* (capítulo 25) usa estos heurísticos para decidir cuándo dispatchar Yannakakis vs DFS.

23.8. Resumen

- Yannakakis chain: dos pasadas semi-join (bottom-up + top-down) + enumerator.
- Después del reducer, cada arista en cada bag participa de algún match completo. El enumerator no explora ramas muertas.
- Complejidad: $O(|\mathcal{E}| + |Q_{\text{out}}|)$. Lineal en input + output.
- Gana sobre DFS cuando hay muchos paths parciales que se abortan; pierde cuando NLF + k-hop ya filtraron al 90 %.

Ejercicios

1. ★ Para la hipótesis `User -[Auth]-> Host -[Connect]-> IP` con $|B_0| = 1000$, $|B_1| = 5000$, y los matches finales son 50, calculá cuántas filas sobreviven a cada pasada del reducer en el peor caso.
2. Implementá la pasada bottom-up con un `HashSet` auxiliar. ¿Cuál es la complejidad de espacio?
3. ★ Demostrá: si después de la pasada bottom-up B_0 queda vacío, entonces Q está vacío (no hay matches). ¿Es *cierto el converse*?
4. Diseñá un ejemplo donde la pasada top-down NO elimina ninguna fila adicional (es no-op). ¿Cuándo ocurre?
5. Compará Yannakakis chain con DFS bien-podado para la hipótesis del ejercicio 1. ¿Cuántos paths visita cada uno? ¿Bajo qué condiciones gana cada uno?
6. Si la hipótesis tiene un predicado complejo en uno de los pasos (`HAVING count(distinct) > 5`), ¿cómo lo integrás en Yannakakis chain? ¿Antes, durante, o después del reducer?

Capítulo 24

Yannakakis: *tree-walk*

Intuición

Yannakakis chain (capítulo anterior) sirve para hipótesis cuya *shape* es una cadena lineal. Para hipótesis *shape graph* —estrellas, árboles, cualquier hipergrafo α-acíclico no-cadena— el join tree ya no es una línea sino un árbol propio. El *tree-walk* enumerator camina ese árbol manteniendo un *binding* (mapa de variables a vértices) y, en cada nodo del árbol, decide qué *bag* consultar según qué variable comparte con su padre. Este capítulo explica el algoritmo, muestra un ejemplo con una estrella, y discute el contraste con el chain enumerator.

24.1. El setup: variables, bindings, join tree

Para una hipótesis *shape graph*, los pasos no se relacionan posicionalmente sino vía *variables* explícitas:

```
User c -[Auth]-> Host h1; c -[Auth]-> Host h2; c -[Auth]-> Host h3
```

Aquí c es una `VarId` compartida entre los tres pasos, mientras h_1, h_2, h_3 son `VarId`s independientes. La hipótesis tiene 4 variables y 3 pasos.

El *binding* es un `Vec<Option<StrId>>` indexado por `VarId`; arranca `None` en todas las posiciones, y el tree-walk lo va llenando.

24.2. El join tree para *shape graph*

El planner (capítulo 25) construye el join tree con GYO + BFS desde una hiperarista *root*. Para la estrella de arriba:

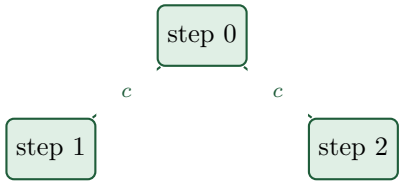


Figura 24.1: Join tree para `c -[Auth]-> h1; c -[Auth]-> h2; c -[Auth]-> h3`. Tres nodos (uno por step), conexión etiquetada con la *shared variable* con el padre (todos comparten c con step 0, que es la root).

24.2.1. La estructura `JoinTree`

```rust
pub struct JoinTree {
    pub nodes: Vec<usize>,            // step indices, en orden de visita BFS
    pub parent: Vec<Option<usize>>,   // parent[i]: índice en `nodes` del padre
    pub shared_var: Vec<VarId>,       // variable compartida con el padre
}

pub struct StepBag {
    pub rows: Vec<StepRow>,                // todas las aristas del bag
    pub by_origin: HashMap<StrId, Vec<usize>>, // index: origin → indices en rows
    pub by_dest:   HashMap<StrId, Vec<usize>>, // index: dest →   indices en rows
}
```

Cada `StepBag` tiene dos índices auxiliares: `by_origin` y `by_dest`. Esto permite consultas eficientes en cualquier sentido (a diferencia del chain enumerator, que sólo necesita `by_origin`).

24.3. El algoritmo

Algorithm 24.1: Tree-walk enumerator (versión simplificada)

Input: `JoinTree`, `bags: Vec<StepBag>`, `binding` (todo `None`)
Output: Vec<HuntResult>

```
recurse(node_idx = 0);
  ▷ recurse(node_idx)
if node_idx = |nodes| then
    emit(binding resuelto a Vec<Arc<str»);
    return;
end
step ← nodes[node_idx];
edge ← hypergraph.edges[step];
switch (binding[edge.origin], binding[edge.dest]) do
    case (Some(o), _) do
        candidates ← by_origin[step][o]
    end
    case (None, Some(d)) do
        candidates ← by_dest[step][d]
    end
    case (None, None) do
        candidates ← bags[step].rows (root)
    end
end
for row ∈ candidates do
    intentar bind(edge.origin, row.origin) y bind(edge.dest, row.dest);
    recurse(node_idx + 1);
    unbind si correspondía;
end
```

24.4. Por qué el switch (origin_bound, dest_bound)

La diferencia clave con el chain enumerator: en una cadena sabemos que B_{i+1}.origin = B_i.dest por la convención posicional, así que el lookup es siempre `by_origin`. En una estrella o árbol, la *shared variable* con el padre puede aparecer como `origin` o como `dest` de la arista del step actual.

Ejemplo: Estrella revisitada

Hipótesis: `c -[Auth]-> h1; c -[Auth]-> h2; c -[Auth]-> h3`.

- Step 0: arista `c -[Auth]-> h1`. Su `origin` es c, su `dest` es h_1. Como root, ambas variables están `None`; iteramos `bags[0].rows` enteros. Cada iteración bind $c \leftarrow$ row.origin y $h_1 \leftarrow$ row.dest.
- Step 1: arista `c -[Auth]-> h2`. Su `origin` es c (ya bound desde step 0), su `dest` es h_2 (None). El switch dispatcha al caso (Some(o), None): candidates = `by_origin[1][`o`]` con o = binding[c]. Iteramos esos rows, bind $h_2 \leftarrow$ row.dest.
- Step 2: análogo a step 1, bind h_3.
- En el caso base (node_idx = 3), emit el path completo (c, h_1, h_2, h_3).

24.4.1. Caso (None, Some(d))

¿Cuándo aparece este caso? Cuando un step tiene su `dest` bound pero no su `origin`. Por ejemplo, una hipótesis "árbol invertido":

```
Host h -[Inbound]-> User u1; Host h2 -[Inbound]-> u1
```

Acá u_1 (en el dest) es la shared variable. Step 0 bind h y u_1; step 1 tiene dest = u_1 (already bound) y origin = h_2 (None). Lookup vía `by_dest[1][`u_1`]`.

24.4.2. Caso (Some, Some)

Step donde tanto origin como dest ya están bound. El motor consulta si la arista *específica* existe (lookup determinista, no iteración). Si existe, recursa; si no, fail-fast.

24.5. Bindings y unbinding

```
fn bind(
    binding: &mut Vec<Option<StrId>>,
    var: VarId,
    sid: StrId,
) -> bool {
    match binding[var.0] {
        None => { binding[var.0] = Some(sid); true }
        Some(existing) => existing == sid,  // ya bound: chequear igualdad
    }
}

fn unbind(
    binding: &mut Vec<Option<StrId>>,
    var: VarId,
    was_just_bound: bool,
) {
```

```
    if was_just_bound {
        binding[var.0] = None;
    }
}
```

Cada `bind` en el `recurse` loop es "si la variable ya estaba bound, chequear que coincide; si no, bind y marcar para unbind al volver". El `unbind` restaura sólo lo que se acaba de bind —*no* desbinde lo que ya estaba bound desde arriba—.

24.6. Análisis

Igual que Yannakakis chain, la complejidad es $O(|\mathcal{E}_{\text{relevantes}}| + |Q_{\text{out}}|)$. El reducer (no mostrado en detalle aquí) reduce los bags como en chain; el tree-walk recorre el join tree sin visitas duplicadas.

24.7. Cuándo se dispatcha

GraphHunter dispatcha al tree-walk cuando:

1. La hipótesis tiene `shape == HypothesisShape::Graph`.
2. El planner verifica α-aciclicidad (capítulo 22) —debería ser cierto si la DSL valida correctamente, pero el chequeo es defensivo.
3. El feature flag `GRAPHHUNTER_YANNAKAKIS` está on (default).

Si la hipótesis es shape graph pero *cíclica* (K_3, K_4), Yannakakis no aplica; el planner cae a LFTJ-K_4 (si es K_4) o al DFS legacy.

24.8. Resumen

- Tree-walk enumerator es la versión de Yannakakis para shape graph: walk recursivo sobre el join tree, con binding/unbinding.
- En cada step, el lookup se decide por `(origin_bound, dest_bound)`: dispatch a `by_origin`, `by_dest`, o iteración completa de bag (root).
- Complejidad lineal en input + output.
- Aplica para hipótesis α-acíclicas no-cadena (estrellas, árboles).

Ejercicios

1. ★ Para la hipótesis `User c -[Auth]-> Host h1; c -[Auth]-> Host h2` en un grafo donde c tiene 100 vecinos `Host`, ¿cuántos matches enumera el tree-walk? ¿Cuál es la cota teórica?
2. Construí el join tree para la hipótesis `Process p1 -[Spawn]-> Process p2 -[Spawn]-> Process p3; p1 -[Spawn]-> Process p4`. ¿Qué shape es?
3. ★ ¿Cuándo aparece el caso $(\text{Some}, \text{Some})$ en el switch? Diseñá una hipótesis donde aparezca y explicá qué chequea.
4. Si en vez de mantener dos índices (`by_origin` y `by_dest`) por bag, mantuviéramos sólo `by_origin`, ¿qué cambia? ¿En qué casos sería más lento?

5. Compará tree-walk con DFS bien-podado para una hipótesis de estrella $c \to (h_1, h_2, h_3, h_4, h_5)$. ¿Cuál gana? ¿Por qué?

Capítulo 25

El *planner*

Intuición

GraphHunter tiene cuatro operadores de matching —DFS legacy con prunes, LFTJ-K_4, Yannakakis chain, Yannakakis tree-walk—. El *planner* es la pieza chiquita y fundamental que decide cuál se usa para cada hunt. Inspecciona la `Hypothesis`, clasifica su shape y propiedades estructurales, y devuelve un `DispatchPlan`. La política es deliberadamente simple: tres reglas en orden, primer match gana, fallback al DFS. Este capítulo documenta las tres reglas, justifica su orden, y explica cómo agregar reglas nuevas sin romper backwards compat.

25.1. El `DispatchPlan`

```
pub enum DispatchPlan {
    DfsMatch,     // default, generic; implementa cualquier hipótesis
    K4Lftj,       // M1/S3, sólo K_4 homogéneo
    Yannakakis,   // L1, $\alpha$-acíclica + flag GRAPHHUNTER_YANNAKAKIS
}
```

Tres variantes, no más. `DfsMatch` es el fallback genérico. `K4Lftj` y `Yannakakis` son los *lifts* especializados. La filosofía: *conservadora*; sólo dispatchamos a un especializado si *sabemos* que la estructura lo permite.

25.2. Las tres reglas

```
pub fn plan(hypothesis: &Hypothesis) -> DispatchPlan {
    if name_marker_dispatch(hypothesis)   { return DispatchPlan::K4Lftj; }
    if structural_dispatch(hypothesis)    { return DispatchPlan::K4Lftj; }
    if yannakakis_dispatch(hypothesis)    { return DispatchPlan::Yannakakis;
        }
    DispatchPlan::DfsMatch
}
```

Cada función es un test booleano sobre la `Hypothesis`. La primera que retorna `true` gana.

25.2.1. Regla 1: `name_marker_dispatch` (back-compat S3)

```
fn name_marker_dispatch(hyp: &Hypothesis) -> bool {
    std::env::var("GRAPHHUNTER_K4_LFTJ").map_or(false, |v| v == "1")
        && hyp.name.to_lowercase().contains("k4-clique")
}
```

Es la primera versión histórica del dispatch a LFTJ: el analista nombra explícitamente su hipótesis con la string "k4-clique" y el motor asume que es K_4. Frágil, pero útil para debugging y rollback. Hoy se mantiene para que tests viejos no se rompan; los hunts nuevos no necesitan el marker.

25.2.2. Regla 2: `structural_dispatch` (M1)

```
fn structural_dispatch(hyp: &Hypothesis) -> bool {
    let lftj_planner = std::env::var("GRAPHHUNTER_LFTJ_PLANNER")
        .map_or(true, |v| v == "1");  // default ON
    if !lftj_planner { return false; }

    // Detectar K_4 homogéneo:
    // - exactamente 6 pasos
    // - todos los pasos tienen el mismo EntityType en source y dest
    // - todos los pasos tienen el mismo RelationType
    // - sin predicados sobre metadata
    if hyp.steps.len() != 6 { return false; }
    let first = &hyp.steps[0];
    let same_entity_type = hyp.steps.iter().all(|s| {
        s.source_type == first.source_type && s.dest_type == first.dest_type
    });
    let same_rel_type = hyp.steps.iter().all(|s| s.rel_type == first.
       rel_type);
    let no_predicates = hyp.steps.iter().all(|s| s.predicates.is_empty());

    same_entity_type && same_rel_type && no_predicates
        && first.source_type == first.dest_type  // K_4 es homogéneo
}
```

M1 (entregado abril 2026) destrabó el dispatch automático sin requerir name marker. Inspecciona la estructura: 6 pasos homogéneos, sin predicados, mismo tipo en todo $\Rightarrow$ es K_4 homogéneo $\Rightarrow$ LFTJ funciona.

25.2.3. Regla 3: `yannakakis_dispatch` (L1)

```
fn yannakakis_dispatch(hyp: &Hypothesis) -> bool {
    let yannakakis_on = std::env::var("GRAPHHUNTER_YANNAKAKIS")
        .map_or(true, |v| v == "1");  // default ON
    if !yannakakis_on { return false; }

    // Cualquier hipótesis $\alpha$-acíclica aplica.
    // Tanto shape Chain como shape Graph dispatchan al matcher
    // de Yannakakis correspondiente.
    is_alpha_acyclic(hyp)
}
```

La regla es la más general: *cualquier hipótesis α-acíclica*. La función `is_alpha_acyclic` corre GYO sobre el hipergrafo de la hipótesis (capítulo 22) —trivialmente rápido para hipótesis con ≤ 6 pasos.

Notar que cubre las dos shapes:

- `HypothesisShape::Chain`: dispatcha a Yannakakis chain (capítulo 23).
- `HypothesisShape::Graph`: dispatcha a Yannakakis tree-walk (capítulo 24).

Adentro del matcher Yannakakis, otro *branch* sobre `shape` elige el sub-operador.

25.3. ¿Por qué este orden?

Las tres reglas se chequean en orden. Razones:

1. **name_marker antes de structural**: el name marker permite force-override en tests y A/B comparisons. Si está seteado, gana sin importar la estructura.

2. **structural antes de yannakakis**: K_4 es cíclico, así que `yannakakis_dispatch` lo rechazaría. Pero LFTJ-K_4 le da mejor performance que el DFS para K_4. Si chequeáramos Yannakakis primero, K_4 caería al DFS innecesariamente.

   Además, la jerarquía estructural es estricta: K_4 homogéneo (`structural`) es un subset de los hunts; los α-acíclicos son otro subset; ambos son disjoint (K_4 NO es α-acíclico). Pero el orden importa sólo si una hipótesis cae en ambos —imposible por la aciclicidad—. La razón real para el orden es de *previsibilidad*: el lector que mira `plan()` sabe qué se intenta primero.

3. **Fallback al DFS**: el DFS implementa cualquier hipótesis (incluso las que ningún operador especializado soporta). Es la red de seguridad.

25.4. Cuándo el planner "se equivoca"

El planner es *estructural*: mira la shape, no el grafo. Es posible que dispatchar a Yannakakis ($O(|\mathcal{E}| + |Q_{\text{out}}|)$) sea peor que DFS para una hipótesis donde NLF + k-hop ya filtraron hasta dejar 10 starts válidos:

- DFS desde 10 starts: ∼10 ms.
- Yannakakis full reducer sobre $|\mathcal{E}| = 10^7$: ∼300 ms.

Hoy el planner no tiene heurísticos sobre el *grafo*, sólo sobre la *hipótesis*. La razón es simplicidad: agregar costos estimados requiere statistics actualizadas (cardinalidades, selectividades) que el motor no mantiene. Para casos donde el analista quiere forzar DFS, puede setear `GRAPHHUNTER_YANNAKAKIS=0`.

25.5. Surface en UI: "Perf: M+L1"

El planner expone su elección al frontend vía `cmd_get_perf_status`:

```
pub struct PerfStatus {
    pub lftj_planner: bool,        // M1
    pub k4_lftj_marker: bool,      // S3
    pub yannakakis: bool,           // L1
    pub plug_and_play_active: bool,
}
```

El frontend renderiza una pildora "Perf: M+L1" (verde) cuando los tres flags están on (default). Si el usuario forzó alguno a 0 (legacy path para debugging), la pildora se vuelve gris "Perf: legacy" (capítulo 36).

25.6. Agregar una regla nueva: el procedimiento

Cuando el equipo decide agregar un operador nuevo (digamos, LFTJ para triángulo):

1. Implementar el operador en `core/graph-engine/src/lftj\_triangle.rs`.

2. Agregar variante a `DispatchPlan`: `TriangleLftj`.
3. Escribir una función `triangle_dispatch` con la detección estructural (3 pasos, ciclo, mismo tipo).
4. Agregar a la cadena de `plan()`, antes de `yannakakis_dispatch` (porque triángulo es cíclico, Yannakakis lo rechazaría):

```
if triangle_dispatch(hyp) { return DispatchPlan::TriangleLftj; }
```

5. Agregar el flag a `PerfStatus` y a la pildora.
6. Default-on al startup (`apps/tauri/src-tauri/src/lib.rs`) siguiendo la regla plug-and-play (capítulo 36).

La política: cada nuevo operador es una rama nueva en `plan()` con su detección estructural propia, default-on, expuesto en UI.

25.7. Resumen

- Planner: tres reglas en orden (`name_marker`, `structural`, `yannakakis`), primero que matchea gana, fallback al DFS.
- `name_marker`: legacy hardcoded para back-compat.
- `structural`: detecta K_4 homogéneo via inspección de pasos.
- `yannakakis`: cualquier hipótesis α-acíclica (chain o graph).
- DFS legacy es el fallback genérico; implementa cualquier hipótesis.
- El planner es estructural, no statistic-aware; cuando se equivoca, el usuario puede forzar via flags.

Ejercicios

1. ★ Para cada una de estas hipótesis, identificá qué operador dispatcha el planner:
   - `User -[Auth]-> Host -[Connect]-> IP`
   - `c -[Knows]-> p1; c -[Knows]-> p2; p1 -[Knows]-> p2`
   - Hipótesis con name=“k4-clique” y 6 pasos heterogéneos
2. ¿Por qué `name_marker_dispatch` se chequea primero? Discutí: ¿qué se rompe si lo movemos al final?
3. ★ Diseñá un detector estructural para la *estrella de k brazos*. ¿Cómo se vería la función booleana?
4. Si una hipótesis es α-acíclica pero el usuario forzó `GRAPHHUNTER_YANNAKAKIS=0`, ¿a qué operador cae? Justificá por qué la cadena de fallbacks es “correcta”.
5. Imaginá que el motor empieza a recolectar estadísticas de cardinalidades. ¿Cómo cambiarías `plan()` para aprovecharlas? Discutí pros y contras de un planner cost-based.

Parte VI

Índices auxiliares en profundidad

Capítulo 26

Diseño de índices auxiliares

Intuición

NLF y k-hop reachability son *índices auxiliares*: estructuras derivadas del grafo que el matcher consulta para podar trabajo. La distinción "auxiliar" importa: NO son la *source of truth*; si se invalidan o se vuelven inconsistentes, el matcher sigue funcionando —sólo más lento, sin las podas—. Este capítulo formaliza el contrato general que cualquier índice auxiliar de GraphHunter debe cumplir: no source of truth, soundness por sobre-aproximación, lazy build, y un layout flexible heap/mmap-spilled. Después los próximos dos capítulos (27 y 28) los aplican a NLF y k-hop específicamente.

26.1. El contrato general

Un índice auxiliar en GraphHunter debe cumplir cuatro propiedades:

1. **No es source of truth.** La información canónica vive en el *streaming backend* (capítulo 13). El índice puede invalidarse, perderse, descartarse —el motor sigue funcionando. La única consecuencia de no tener el índice es que el matcher no puede usar ese prune.

2. **Sound como sobre-aproximación.** El índice puede *sobre-aproximar*: incluir candidatos que después no van a matchear. NUNCA puede *sub-aproximar*: excluir candidatos que sí matchearían. Eso garantiza que el prune sea *correcto sin importar el estado del índice*: si el índice dice "no", la respuesta es *no*; si dice "sí", el matcher verifica.

3. **Lazy build.** El índice se construye al primer consumer que lo necesita, no eagerly al cargar la sesión. Implementación con `OnceLock<T>`.

4. **Heap o mmap-spilled.** La estructura física del índice puede vivir en heap (mutable, rápida) o mmap-spilled (read-only, fría) según presione la memoria. El *wrapper* `IndexBacking<T>` encapsula esa dualidad.

26.2. Por qué "no source of truth"

Una pregunta natural: si el índice acelera el matcher, ¿por qué no hacerlo source of truth y eliminar el *streaming backend*?

Tres razones que justifican la separación:

1. **Lossiness.** Los índices son *derivaciones lossy* del grafo. NLF guarda counts pero no las aristas individuales; k-hop guarda bits pero no los caminos concretos. Reconstruir el grafo desde el índice es imposible; reconstruir el índice desde el grafo es siempre posible.

2. **Decoupling de schema.** Si en el futuro se cambia el layout de NLF (más tipos, más resolución de count), el grafo no se toca. Si NLF fuera source of truth, cualquier cambio de schema obligaría una migración.

3. **Auditoría.** El analista puede inspeccionar *las aristas concretas* que llevaron a un match (capítulo 11). Sólo el grafo canónico permite eso; un índice agregado lo perdería.

26.3. Soundness por sobre-aproximación

Definición 26.1 (Soundness de prune)**.** Sea Index un índice auxiliar y prune_H una función booleana que el matcher invoca para descartar candidatos para la hipótesis H. El prune es *sound* sii:

$$\text{prune}_H(\text{Index}, c) = \texttt{true} \implies c \text{ no genera match para } H.$$

Lo equivalente: el conjunto de candidatos válidos está contenido en el conjunto que el prune *no* descarta. El prune puede ser conservador (descartar pocos), nunca agresivo en exceso.

Nota. NLF y k-hop ambos satisfacen soundness por construcción: NLF cuenta los vecinos *exactamente*; descartar cuando el count es 0 descarta sólo donde no hay vecinos. k-hop bit es 0 sii NO existe walk $\leq$ `MAX_DEPTH`; descartar es seguro porque ningún walk implica ningún match.

La sobre-aproximación entra cuando los índices *podrían* sub-aproximar pero deciden no hacerlo: por ejemplo, k-hop podría ser "más preciso" si guardara *cuántos* caminos hay en vez de un bit, pero entonces sería incorrecto si el contador overflowara (a 0). Saturating + bit es la versión segura.

26.4. Lazy build con `OnceLock`

El patrón:

```
pub struct GraphHunter {
    streaming: Arc<InMemoryStreamingBackend>,
    nlf: OnceLock<NlfTable>,
    khop_reach: OnceLock<KHopReach>,
    // ...
}

impl GraphHunter {
    pub fn ensure_nlf(&self) -> &NlfTable {
        self.nlf.get_or_init(|| NlfTable::build(self))
    }

    pub fn ensure_khop_reach(&self) -> &KHopReach {
        self.khop_reach.get_or_init(|| KHopReach::build(self))
    }
}
```

Cada `ensure_X` construye el índice en el primer call; los subsiguientes ven el resultado vía atómica (capítulo 5). El consumer del índice (el matcher) nunca tiene que preocuparse por "¿está construido?"; siempre llama `ensure` y obtiene una referencia.

26.4.1. Invalidation

Cuando una nueva arista llega *post*-build, el índice puede quedar *stale*. Tres políticas posibles:

1. **Invalidación**: `self.nlf = OnceLock::new()`; el próximo `ensure_nlf` reconstruye desde cero.

2. **Update incremental**: aplicar el delta sin rebuild. Para NLF es directo (`try_record_edge`, capítulo 15); para k-hop es difícil (la transitividad propaga el cambio, capítulo 33).

3. **Best-effort + fallback**: intentar update; si falla por estructura física (mmap-spilled), invalidar.

GraphHunter usa la 3 (NLF, capítulo 15). La 1 era el comportamiento previo a stage 2a; la 2 pura es el plan para k-hop (capítulo 34).

26.5. La dualidad heap / mmap-spilled

Los índices grandes (k-hop especialmente, varios GB) pueden no caber en heap. El wrapper `IndexBacking<T>` encapsula la dualidad heap-resident vs mmap-spilled, el mismo patrón que el *streaming backend* aplica a las aristas (capítulo 13):

```
pub enum IndexBacking<T: Pod> {
    Heap(Vec<T>),                                   // mutable
    Mmap { file: File, map: Mmap, slice: &'static [T] },  // read-only
}

impl<T: Pod> IndexBacking<T> {
    pub fn get(&self, idx: usize) -> Option<&T> {
        match self {
            Self::Heap(v) => v.get(idx),
            Self::Mmap { slice, .. } => slice.get(idx),
        }
    }

    pub fn heap_slice_mut(&mut self) -> Option<&mut [T]> {
        match self {
            Self::Heap(v) => Some(v.as_mut_slice()),
            Self::Mmap { .. } => None,  // read-only
        }
    }
}
```

Tres notas:

- `T: Pod` (*Plain Old Data*): garantiza que el layout de `T` sea binario plano, sin pointers internos. Permite mmap directo.
- `get()` es uniforme en ambas variantes: el caller no sabe si lee de heap o mmap.
- `heap_slice_mut()` sólo funciona en heap; devuelve `None` en mmap. El caller decide qué hacer (invalidar, abortar update, etc.).

26.5.1. Cuándo se decide spillear

`IndexBacking::build` decide al construir:

```
pub fn build_with_budget<F>(
    capacity: usize,
    budget: usize,
    populate: F,
) -> Result<Self, Error>
where
    F: FnOnce(&mut Vec<T>),
{
```

```
    let bytes_estimated = capacity * std::mem::size_of::<T>();
    let mut v: Vec<T> = Vec::with_capacity(capacity);
    populate(&mut v);
    if bytes_estimated > budget {
        let file = create_temp_file()?;
        write_to_file(&file, &v)?;
        let map = unsafe { Mmap::map(&file)? };
        Ok(Self::Mmap { file, map, slice: ... })
    } else {
        Ok(Self::Heap(v))
    }
}
```

El budget default por índice es 1 GB. Para grafos chico-medianos todos los índices viven en heap; para grafos grandes (k-hop > 4 GB), el k-hop spillea pero NLF (que es chico) suele caber.

26.6. El lifecycle completo de un índice

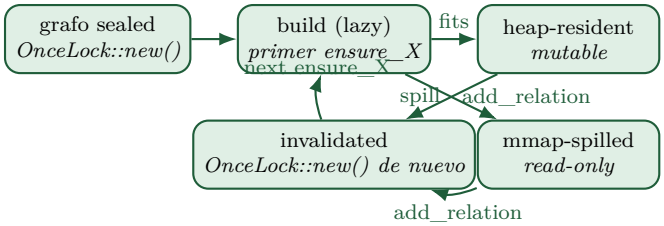


Figura 26.1: Lifecycle de un índice auxiliar. Arranca `None` (sealed graph). Primer `ensure` construye y elige heap/mmap. `add_relation` *post*-build invalida (a menos que se logre incremental, NLF caso).

26.7. Ejemplo concreto: cuánto cuesta cada estado

Para un grafo de 28 GB EVTX (200 M aristas, 30 M ranked vertices):

Estado	Tiempo	Memoria
sealed (`None`)	instantáneo	0
build NLF	∼30 s	∼1.6 GB heap
build k-hop	∼1.5 min	∼5 GB (4.5 spilled, 0.5 heap)
ensure (post-build)	∼5 ns	ya construido
add_relation incremental NLF (heap)	+50 ns	sin cambio
add_relation invalidation NLF (mmap)	instantáneo	libera mmap

26.8. Resumen

- Índice auxiliar = derivación lossy del grafo, NO source of truth, soundness por sobre-aproximación.
- Lazy build via `OnceLock`: cero costo hasta el primer hunt que lo consulta.
- Dualidad heap / mmap-spilled vía `IndexBacking<T>`.

- Invalidation: simple por defecto; incremental cuando se puede (NLF); plan para k-hop (capítulo 34).

Ejercicios

1. ★ Si el motor pierde el contenido de NLF (digamos, por un crash del proceso), ¿se rompe la sesión? ¿Qué ocurre en el próximo hunt?
2. ¿Por qué la propiedad "no source of truth" es importante para auditoría? Pensá: si NLF fuera source of truth, ¿qué pregunta del analista no se podría responder?
3. ★ Diseñá un índice auxiliar *nuevo* para GraphHunter: "`first_seen[v]`" = timestamp de la primera arista incidente a v. ¿Cómo lo encajás en el contrato? ¿Cómo se actualiza incrementalmente?
4. ¿Cuándo conviene NO usar `OnceLock` para un índice auxiliar? (Pista: pensá en índices que se usen muy raramente.)
5. Si tu grafo tiene 100 M vértices y NLF es `[u32; 9]` por vértice, ¿cuánta memoria? ¿Excede el budget default de 1 GB?

Capítulo 27

NLF en detalle

Intuición

Capítulo 18 introdujo NLF como prune; este capítulo muestra cómo está implementado por dentro: layout `[u32; 9]` por vértice, `IndexBacking` para la dualidad heap/mmap, builder $O(|\mathcal{E}|)$ en una pasada, e incremental update detallado más allá del overview de 15. Es el primer índice "simple" del motor; entender su implementación a fondo es buena preparación para el más complejo (k-hop, próximo capítulo).

27.1. Layout

```
const NLF_TYPE_COUNT: usize = 9;  // tag de EntityType en [0, 9)

pub struct NlfTable {
    counts: IndexBacking<[u32; NLF_TYPE_COUNT]>,
}
```

La estructura es trivial: una tabla con $|\mathcal{V}|$ filas, cada fila es `[u32; 9]` (36 bytes). El `IndexBacking` (visto en el capítulo anterior) decide si vive en heap o en mmap.

27.1.1. Por qué `[u32; 9]` y no `HashMap<EntityType, u32>`

Un `HashMap` sería más flexible pero terrible en performance. $N \cdot$ `HashMap` implica:

- Por cada lookup, hash + bucket walk + compare. $\sim$ 30 ciclos.
- Memoria: $\sim$48 B por bucket + factor de carga (*load factor*) típico 0.7. Para 9 entries por vértice: $\sim$72 B/vértice. Versus 36 B del array.
- Locality: los buckets están dispersos en heap; iterar el HashMap toca cache lines aleatorias.

El array fijo `[u32; 9]`:

- Lookup: indexación directa, 1 ciclo.
- Memoria: 36 B per row, contiguous.
- Locality: una fila entera entra en una cache line (con 20 B de margen para los siguientes 5 rows).

Trade-off: el array fija 9 tipos. Si en el futuro se agrega un décimo `EntityType`, hay que crecer el array (cambio breaking de schema). El equipo decidió que la fijación de tipos es un commitment razonable.

27.2. El builder

Algorithm 27.1: `NlfTable::build`

Input: grafo G (sealed)
Output: `NlfTable`

counts ← IndexBacking :: heap_zeros($|\mathcal{V}|$);
for *cada* `cr` *en* `G.edge_store.iter_all()` **do**
  src ← cr.source_sid.index();
  L ← cr.dest_type_tag();
  if $L <$ `NLF_TYPE_COUNT` **then**
    counts[src][L] ← counts[src][L].saturating_add(1);
  end
end
return `NlfTable{ counts }`;

Una pasada secuencial sobre `edge_store` (que ya está sorted, capítulo 14). Cada iteración: índice directo + saturating add. $O(|\mathcal{E}|)$ total.

Costo en hardware: para $|\mathcal{E}| = 10^8$, $\sim$ 30 segundos (memoria-bound, $\sim$3 MB/s de actualizaciones). El `lazy build` (capítulo 14) lo difiere al primer hunt que consulta NLF.

27.2.1. ¿Por qué `dest_type_tag` y no `source_type_tag`?

NLF cuenta *vecinos salientes* de un vértice por tipo destino. La fila se indexa por source (de la arista), pero la columna se indexa por tipo del destino. Es lo que el prune del primer paso necesita: "¿s tiene vecinos del tipo que el primer paso exige?".

Una variante alternativa: NLF *simétrica* que indexa por ambos pares. El motor no la implementa porque las hipótesis suelen ser direccionadas (origen → destino) y la simetría duplicaría el costo sin beneficio claro.

27.3. Lookup: `can_match`

```
impl NlfTable {
    pub fn can_match(
        &self,
        sid: StrId,
        required: &[(u8, u32)],   // (dest_type_tag, min_count)
    ) -> bool {
        let row = self.counts.get(sid.index());
        let Some(row) = row else { return false };  // sid out of range
        required.iter().all(|&(tag, min)| {
            tag as usize >= NLF_TYPE_COUNT || row[tag as usize] >= min
        })
    }
}
```

$O(|\text{required}|)$ que es $\leq |\mathcal{L}_V| = 9$. Para hipótesis chain (un solo destination type), es $O(1)$.

27.3.1. Cómo lo invoca el matcher

El DFS llama `can_match` para cada candidate start:

```
let nlf = graph.ensure_nlf();
let required = collect_required_destinations(&hypothesis);
for sid in candidate_starts {
    if !nlf.can_match(sid, &required) {
```

```
            continue;  // skip this start
        }
        // ... DFS desde sid
    }
```

Para grafos con $|\mathcal{V}_{L_0^V}| = 10^5$ candidates, son 10^5 calls a `can_match`: ~1 ms total. El ahorro en DFS descartado suele ser de varios segundos.

27.4. Memoria

- Heap-resident: $|\mathcal{V}| \cdot 36$ B. Para $|\mathcal{V}| = 10^7$: 360 MB. Para 10^8: 3.6 GB.
- mmap-spilled: misma memoria total, pero residente en disco con paginación on-demand.
- Budget default: 1 GB. Triggeará spill alrededor de $|\mathcal{V}| = 28 \cdot 10^6$.

27.4.1. Cuándo spillea

```
const NLF_BUDGET_BYTES: usize = 1 << 30;  // 1 GB

pub fn build(graph: &GraphHunter) -> Result<Self, Error> {
    let n_vertices = graph.vertex_count();
    let counts = IndexBacking::build_with_budget(
        n_vertices,
        NLF_BUDGET_BYTES,
        |v| populate_counts(v, graph),
    )?;
    Ok(Self { counts })
}
```

`build_with_budget` (capítulo 26) decide al final del populate: si los bytes acumulados exceden el budget, vuelca a archivo y mmapea read-only.

27.5. Incremental update revisitado

Capítulo 15 introdujo `try_record_edge`:

```
impl NlfTable {
    pub fn try_record_edge(
        &self,
        src_sid: StrId,
        dst_tag: u8,
    ) -> bool {
        if dst_tag as usize >= NLF_TYPE_COUNT {
            return true;  // no-op
        }
        let slice = self.counts.heap_slice_mut();
        let Some(slice) = slice else {
            return false;  // mmap-spilled
        };
        if let Some(row) = slice.get_mut(src_sid.index()) {
            row[dst_tag as usize] = row[dst_tag as usize].saturating_add(1);
        }
        true
    }
}
```

La *interior mutability* —el método toma `&self` no `&mut self`— requiere que `IndexBacking::heap_slice_mut` también la soporte. La implementación real usa `UnsafeCell<Vec<...>>` con un invariante: "`try_record_edge` se llama sólo desde el ingestor, que es single-threaded; los matchers concurrentes leen vía `counts.get()` que es read-only".

Advertencia. Este patrón *interior mutability single-writer* es seguro pero delicado. Si en el futuro se permite ingest multi-threaded, el invariante se rompe y hay que envolver en un `RwLock`. El test de `Loom` (model checker para concurrencia) verifica el invariante actual.

27.6. ¿Y si en el futuro hace falta más resolución?

NLF actual cuenta cardinalidad por tipo destino. No considera:

- **Tipo de arista**. Si el primer paso es `Auth`, NLF no distingue entre vecinos vía Auth o vía Spawn. Si quisiéramos distinguir, sería "NLP" (*Neighborhood Label Profile*): `[u32; 9 × |RelType|]` por vértice. Memoria $\sim 15\times$ más.
- **Ventana temporal**. NLF cuenta todos los vecinos independiente del tiempo. Para descartar starts cuyas aristas están todas fuera de la ventana, el k-hop temporal sirve (capítulo 28).
- **Doble-hop counting**. "Cuántos vecinos-de-vecinos de tipo X tiene este vértice". Sería caro de mantener ($O(|\mathcal{V}| \cdot \deg)$ memoria); k-hop bit aproxima la pregunta más débil "¿hay alguno?".

Cada extensión es un trade-off memoria-versus-discriminación. La versión actual gana en simplicidad y resuelve el caso del 80 % de los hunts.

27.7. Resumen

- NLF: `[u32; 9]` por vértice. 36 B/row. Index directo, 1 ciclo de lookup.
- Builder: una pasada $O(|\mathcal{E}|)$ sobre edge_store. Saturating add para evitar overflow silencioso.
- Lookup `can_match`: $O(|\text{required}|)$, típico ≤ 9 ops.
- Memoria: 36 B $\cdot|\mathcal{V}|$; spillea a mmap arriba de $\sim$28M vertices con budget 1 GB.
- Incremental update `try_record_edge`: interior mutability single-writer, atomic-free porque el invariante de GraphHunter lo permite.

Ejercicios

1. ★ Calculá la memoria de NLF para un grafo $|\mathcal{V}| = 5 \cdot 10^7$. ¿Spillea con budget 1 GB?
2. ¿Por qué `can_match` usa `.all()` en vez de `.any()`? Pensá: ¿qué quiere chequear el matcher en el caso multi-required?
3. ★ Diseñá una variante NLP (Neighborhood Label Profile) que distinga por tipo de arista. ¿Cuánta memoria? ¿Qué cambios al builder?
4. Si el motor cambia el patrón de ingest a multi-thread, ¿qué se rompe en `try_record_edge`? Diseñá la versión safe usando `AtomicU32` —¿qué se gana, qué se pierde?
5. Imaginá que el budget de NLF baja a 100 MB (laptop con poca RAM). ¿A partir de qué $|\mathcal{V}|$ spillea? ¿Cómo afecta el rendimiento del prune?

Capítulo 28

k-hop reachability en detalle

Intuición

k-hop reachability es el índice más complejo del motor: bitmap de tipos alcanzables, layout dense+sparse para ahorrar memoria, con extensión temporal (`min_first`, `max_first`) para podar también por ventana. Capítulo 19 lo introdujo como prune; este capítulo entra a la implementación concreta: cómo se computa el bitmap con BFS reverse multi-source, por qué el rank array es `u32` con sentinel `NO_RANK`, y un ejemplo numérico de la poda temporal en acción.

28.1. La estructura

```
const MAX_DEPTH: usize = 4;
const NO_RANK: u32 = u32::MAX;
const NLF_TYPE_COUNT: usize = 9;  // = |L_V|

pub struct KHopReach {
    pub bits: Vec<u16>,                          // [|V|], dense
    rank: Vec<u32>,                              // [|V|], dense; NO_RANK si
        outdeg=0
    min_first: RwLock<IndexBacking<[i64; NLF_TYPE_COUNT]>>,  // sparse
    max_first: RwLock<IndexBacking<[i64; NLF_TYPE_COUNT]>>,  // sparse
}
```

Cuatro arrays:

- `bits`: 2 bytes por vértice. bits[v] tiene bit L prendido sii existe walk de v a algún vértice de tipo L en $\leq$ `MAX_DEPTH` saltos.
- `rank`: 4 bytes por vértice. `rank[v] = NO_RANK` si v no tiene aristas salientes (no aporta info temporal); si no, indexa una fila en las matrices sparse de `min/max_first`.
- `min_first`: matriz sparse de `[i64; 9]`; fila r es válida sólo si algún vértice v tiene `rank[v] = r`. Bytes 72 por fila.
- `max_first`: igual.

28.1.1. Por qué dense+sparse

El layout naive —dense para los cuatro arrays— pesaría $|\mathcal{V}| \cdot (2 + 4 + 72 + 72) = |\mathcal{V}| \cdot 150$ B. Para $|\mathcal{V}| = 10^8$: 15 GB. Inviable.

La observación clave: `min_first` y `max_first` sólo aportan info para vértices con *aristas salientes*. Vértices "hojas" (out-deg = 0) tienen min/max_first sin sentido. En grafos típicos el $\sim 70\,\%$ de los vértices son hojas en algún sentido.

El layout sparse: `rank` es dense, pero las matrices grandes sólo guardan filas para los vértices "ranked". Memoria: $|\mathcal{V}| \cdot 6 + |\mathcal{V}_{\text{ranked}}| \cdot 144$. Para $|\mathcal{V}| = 10^8$ y $|\mathcal{V}_{\text{ranked}}| = 3 \cdot 10^7$: $\sim$ 4.9 GB. 3× menos.

28.2. El builder: BFS reverse multi-source

Algorithm 28.1: `KHopReach::build` (versión simplificada)

Input: grafo $G = (\mathcal{V}, \mathcal{E})$
Output: `KHopReach`

bits[v] $\leftarrow 0$ para todo v;
min_first[v][L] $\leftarrow +\infty$, max_first[v][L] $\leftarrow -\infty$ para todo (v, L);
construir `reverse adjacency` N^-;
for *cada tipo* $L \in \mathcal{L}_V$ **do**
  frontier$_0 \leftarrow \{v : \text{type}(v) = L\}$;
  for $d = 0, 1, \ldots,$ *MAX_DEPTH* **do**
    for *cada* $w \in$ frontier$_d$ **do**
      bits[w] $\models (1 \ll L)$;
    end
    frontier$_{d+1} \leftarrow \emptyset$;
    for *cada* $w \in$ frontier$_d$ **do**
      for *cada* $u \in N^-(w)$ **do**
        $\tau \leftarrow$ timestamp de arista $u \to w$;
        min_first[u][L] $\leftarrow$ mín(min_first[u][L], τ);
        max_first[u][L] $\leftarrow$ máx(max_first[u][L], τ);
        agregar u a frontier$_{d+1}$ si no estaba;
      end
    end
  end
end
asignar ranks: $r \leftarrow 0$; para cada v con out-deg$(v) > 0$, rank[v] $\leftarrow r$, incrementar r;
compactar matrices sparse a r filas;
return;

Costo: $O(|\mathcal{L}_V| \cdot \texttt{MAX_DEPTH} \cdot |\mathcal{E}|)$. Para $|\mathcal{L}_V| = 9$, `MAX_DEPTH` $= 4$, $|\mathcal{E}| = 10^8$: $\sim 3.6 \cdot 10^{10}$ operaciones; $\sim$1.5 minutos en hardware típico.

28.2.1. Por qué *reverse* BFS

La pregunta natural: "existe walk de v a tipo L" es direccional desde v. Una BFS *forward* desde v respondería "cuáles tipos alcanza v". Pero queremos responder para *todos* los v sin pagar $|\mathcal{V}|$ BFS forward.

La trampa: corremos BFS *reverse* desde cada conjunto $\{v : \text{type}(v) = L\}$. Eso visita "cuáles vértices pueden alcanzar tipo L" en una sola pasada, marcando el bit L en cada uno. Para $|\mathcal{L}_V|$ tipos, $|\mathcal{L}_V|$ BFS reverse total —fixed cost.

28.3. Lookup y prune

```rust
impl KHopReach {
    pub fn can_reach(&self, v: StrId, target_type: u8) -> bool {
        self.bits[v.index()] & (1 << target_type) != 0
    }

    pub fn within_window(
        &self,
        v: StrId,
        target_type: u8,
        t_lo: i64,
        t_hi: i64,
    ) -> bool {
        if !self.can_reach(v, target_type) {
            return false;
        }
        let rank = self.rank[v.index()];
        if rank == NO_RANK {
            return false;  // outdeg = 0
        }
        let mn_lock = self.min_first.read();
        let mx_lock = self.max_first.read();
        let mn = mn_lock.get(rank as usize).map(|r| r[target_type as usize])
            ;
        let mx = mx_lock.get(rank as usize).map(|r| r[target_type as usize])
            ;
        // Si no hay info temporal, no se puede descartar: aceptar.
        let (mn, mx) = match (mn, mx) {
            (Some(m), Some(M)) => (m, M),
            _ => return true,
        };
        // Sound: descartar si min_first >= t_hi o max_first < t_lo
        !(mn >= t_hi || mx < t_lo)
    }
}
```

`can_reach`: bitwise `and`, una sola instrucción. `within_window`: una vez chequeado el bit, dos lookups sparse + dos comparaciones `i64`. $O(1)$ total.

28.4. Ejemplo numérico

Ejemplo: Prune temporal en acción

Grafo de 5 vértices. Aristas (con timestamps):

- $u \to h_1$ en $\tau = 100$
- $u \to h_2$ en $\tau = 500$
- $h_1 \to ip_1$ en $\tau = 200$
- $h_2 \to ip_2$ en $\tau = 600$

Tipos: type(u) = `User` (tag 0), type(h_i) = `Host` (tag 1), type(ip_i) = `IP` (tag 3).
Build de KHopReach (con `MAX_DEPTH` = 2):

- BFS reverse desde $\{ip_1, ip_2\}$ marcando tag IP:
  - Distancia 0: $\{ip_1, ip_2\}$. bits[ip_1] y bits[ip_2] prenden bit 3.
  - Distancia 1: predecesores son $\{h_1, h_2\}$. bits[h_i] $\models$ 8 (bit 3 prendido). min_first[h_1][`IP`] = 200, min_first[h_2][`IP`] = 600. max_first igual (sólo una arista cada uno).
  - Distancia 2: predecesores de h_1 y h_2 = $\{u\}$. bits[u] $\models$ 8. min_first[u][`IP`] = mín$(100, 500) = 100$. max_first[u][`IP`] = máx$(100, 500) = 500$.

Hipótesis: `User -[Auth]-> Host -[Connect]-> IP WITHIN [t_lo=300, t_hi=400)`.
Prune temporal sobre u como start:

- `can_reach(u, IP)`: bit 3 prendido ⇒ true.
- min_first[u][`IP`] = $100 \geq 400$? No ($100 < 400$).
- max_first[u][`IP`] = $500 < 300$? No ($500 \geq 300$).
- Prune *no descarta*: u podría tener un walk en la ventana.

El DFS verifica concretamente: ¿hay arista $u \to$? con $\tau \in [300, 400)$? Las dos aristas salientes son $\tau = 100$ y $\tau = 500$, ninguna en la ventana. Falso positivo del prune; el DFS lo descarta tras verificar.
El prune es conservador: no falsea negativos, pero acepta algunos falsos positivos. El DFS los limpia.
Si la ventana fuera $[t_{\text{lo}} = 600, t_{\text{hi}} = 700)$:

- max_first[u][`IP`] = $500 < 600$ ⇒ prune *descarta* u sin verificar. Ahorro real.

28.5. El `RwLock` en min/max_first

¿Por qué min/max_first están envueltos en `RwLock`? Porque el plan futuro (capítulo 34) introduce delta update incremental sobre k-hop. Las mutaciones son raras pero existen; el `RwLock` permite lecturas concurrentes (matchers) con escrituras exclusivas (delta update).

Hoy, sin delta update, los `RwLock` están en modo read-only puro: el cost del lock acquire es ∼5 ns por lookup, despreciable.

28.6. Memoria efectiva

Para grafo de 28 GB EVTX ($|\mathcal{V}| = 10^8$, 30 % *outgoing-having* $\Rightarrow |\mathcal{V}_{\text{ranked}}| = 3 \cdot 10^7$):

Componente	**Memoria**
`bits: Vec<u16>`	200 MB
`rank: Vec<u32>`	400 MB
`min_first: IndexBacking<[i64;9]>` (sparse)	2.16 GB
`max_first: IndexBacking<[i64;9]>` (sparse)	2.16 GB
Total	$\sim$ 4.9 GB

Las dos sparse matrices spillean a mmap (1 GB budget cada una). Los dos arrays dense (bits, rank, suma 600 MB) viven en heap.

28.7. Resumen

- k-hop reachability: `bits[v]` (u16, dense) + `rank[v]` (u32, dense) + `min/max_first` (i64 arrays, sparse).
- Layout dense+sparse: $\sim 3\times$ menos memoria que dense puro.
- Build: BFS reverse multi-source, una pasada por tipo. $O(|\mathcal{L}_V| \cdot \texttt{MAX_DEPTH} \cdot |\mathcal{E}|)$.
- Lookup: bitwise para can_reach, lookup sparse + dos comparaciones para temporal.
- `RwLock` en min/max_first: hoy read-only, futuro para delta update (capítulo 34).

Ejercicios

1. ★ Para un grafo $|\mathcal{V}| = 5 \cdot 10^7$ con 50 % *outgoing-having*, calculá la memoria total de `KHopReach`.
2. ¿Por qué BFS *reverse* y no *forward*? Pensá: para responder "puede v alcanzar tipo L?" para todos los v y todos los L, ¿cuál es más eficiente?
3. ★ Diseñá un caso adversarial donde el prune temporal NO descarta nada aunque debería. ¿Es esto un bug o un trade-off de la sobre-aproximación?
4. Si `MAX_DEPTH = 6` en lugar de 4, ¿cuánto crece el tiempo de build? ¿Y la memoria?
5. Imaginá que el grafo crece a $|\mathcal{V}| = 10^9$. ¿Sigue siendo viable k-hop con el layout actual? ¿Qué cambiarías?

Parte VII

La DSL

Capítulo 29

Diseñando una DSL chica

Intuición

La DSL de GraphHunter es deliberadamente *chica*: el analista debería poder escribir una hipótesis en menos de una línea sin abrir manual. Eso es una decisión de diseño, no una limitación. Una DSL chica gana legibilidad, reduce la curva de aprendizaje, y fuerza al motor a ser bueno en pocos casos en lugar de mediocre en muchos. Este capítulo explica los principios que guiaron el diseño de la DSL: una sintaxis primaria de cadena, un solo separador (`;`) para extender a shapes graph, predicados como cláusulas post-fix, y una semántica que mapea uno-a-uno a la estructura del matcher.

29.1. Por qué una DSL en vez de SQL / Cypher / KQL

GraphHunter podría haber expuesto su API en uno de los lenguajes de query existentes:

- **SQL**: estándar, pero verboso para multi-joins. Una hipótesis de 3 saltos = ∼15 líneas de SQL.
- **Cypher** (Neo4j): tiene sintaxis grafo (`(n)-[r]->(m)`), pero es enorme: una hipótesis simple obliga a aprender 30+ keywords, predicados completos, agrupaciones.
- **KQL** (Microsoft Sentinel): poderoso pero table-centric; modelar grafo encima de KQL pierde la naturaleza estructural.

La decisión: una DSL nueva, optimizada para el problema concreto (subgraph matching temporal), con una superficie chica que el analista pueda dominar en una mañana.

29.2. Principios de diseño

29.2.1. Principio 1: legibilidad de izquierda a derecha

Una hipótesis se lee como se escribe: "usuario que se autentica en un host que se conecta a una IP". La sintaxis refleja la estructura del path:

```
User -[Auth]-> Host -[Connect]-> IP
```

Comparado con SQL ("`SELECT …FROM auth a JOIN connect c ON a.host = c.host`"): el SQL invierte el orden mental —primero declara qué quiere, después cómo se conectan los datos—. La DSL matchea el orden de la pregunta del analista.

29.2.2. Principio 2: un solo separador

La DSL legacy era pura cadena. La extensión *shape graph* (L1 stage 3c) agregó la posibilidad de hipótesis no-cadena: estrellas, árboles, cualquier hipergrafo α-acíclico (capítulo 22). El diseño consideró varias sintaxis alternativas:

1. `User c { -[Auth]-> Host h1; -[Auth]-> Host h2 }`: bloque explícito con corchetes.
2. `User c -[Auth]-> Host h1 AND c -[Auth]-> Host h2`: `AND` entre cláusulas.
3. `User c -[Auth]-> Host h1; c -[Auth]-> Host h2`: separador `;`.

Se eligió la 3 (separador `;`) por dos razones:

- **Mínimo cambio sintáctico**. La cadena legacy NO contiene `;`, así que cualquier hipótesis con `;` es *trivialmente* graph shape. Disambigua sin lookahead.
- **Familiaridad**. Los analistas que vienen de SQL asocian `;` con "fin de declaración / siguiente cláusula". El uso es coherente.

29.2.3. Principio 3: predicados como cláusulas post-fix

```
User -[Auth]-> Host -[Connect]-> IP {k=2} HAVING count(distinct last) >= 5
   WITHIN 30m
```

La estructura del path va primero (la parte "visual"); las cláusulas modificadoras (k-simplicidad, ventana, agregaciones) van al final, separadas por palabras clave (`WITHIN`, `HAVING`) o llaves (`{k=N}`).

Alternativa rechazada: predicados *interlaced* con la cadena. Hace el código más denso pero menos escaneable. La forma post-fix se lee como un párrafo: "este path; bajo estas condiciones".

29.2.4. Principio 4: defaults razonables

Tres cosas que la DSL NO obliga a especificar:

- **k-simplicidad**: default $k = 1$ (path simple). Sólo `{k=N}` cuando el analista explícitamente quiere repeticiones.
- **Ventana temporal**: default $\Delta t = +\infty$. Sólo `WITHIN` si el analista quiere acotar.
- **Aggregations**: default "ninguna", enumerar tal cual.

La hipótesis mínima de 2 saltos —`User -[Auth]-> Host -[Connect]-> IP`— es self-contained, sin cláusulas extras. Eso baja la barrera de entrada: un analista nuevo puede escribir su primera hipótesis sin leer manual.

29.3. La gramática (informal)

```
hypothesis     := chain | graph
chain          := type_decl chain_step* trailing_clauses?
graph          := type_decl chain_step* (";" clause_head chain_step*)+
   trailing_clauses?
chain_step     := "-[" rel_type ("{" predicates "}")? "]->" type_decl
type_decl      := EntityType (binding | "{" predicates "}")?
clause_head    := type_decl | binding_ref
binding_ref    := identifier   // referencia a binding ya declarado
```

```
predicates     := predicate ("," predicate)*
predicate      := key "=" value | key ">" value | ...
trailing_clauses := ("WITHIN" duration | "HAVING" agg | "{k=" int "}")*
```

Tres terminales: `EntityType` (`User`, `Host`, etc.), `rel_type` (`Auth`, `Connect`, etc.), `identifier` (`c`, `h1`, `user`). El parser dispatcha entre `type_decl` y `binding_ref` en `clause_head` consultando los bindings previamente declarados (capítulo 31).

29.4. Lo que la DSL *no* hace

Decisiones de exclusión deliberadas:

- **Sin disjunción**. No hay `OR` entre paths. Si el analista quiere "A o B", escribe dos hipótesis y une los resultados en el frontend. Razón: `OR` en DSL obliga al matcher a manejar disjunción, lo que rompe la semántica de path-único.
- **Sin negación**. No hay `NOT` sobre predicados ("vecino de tipo *no*-IP"). Razón: negación general explota la cardinalidad —el dominio de "no-IP" es enorme—. Caso de uso raro en threat hunting.
- **Sin variables libres en cláusulas trailing**. El `HAVING` sólo refiere a `first`, `last`, o agregaciones builtin. No hay binding de variables intermedias. Razón: simpleza del parser y evaluador.
- **Sin recursividad**. No hay `User -[Knows]*-> User` (transitivo de longitud variable). Razón: explosión de estado; los hunts realmente útiles son de longitud chica.

Cada exclusión es un trade-off: alguna pregunta legítima no se puede expresar. La regla: si el caso de uso aparece en $< 5\,\%$ de los hunts del corpus, no entra en la DSL; el analista lo resuelve combinando varias queries.

29.5. La sintaxis no es la semántica

Importante: la DSL es la *interfaz*, no la *representación interna*. El parser convierte la string a `Hypothesis` (capítulo 32); a partir de ahí *todo* el motor trabaja sobre `Hypothesis`, no sobre strings. Eso permite:

- Construir hipótesis programáticamente desde otros transports (MCP, REST) sin parsear DSL.
- Cambiar la sintaxis de la DSL sin tocar el matcher.
- Generar la DSL desde un constructor visual (planeado futuro UI).

29.6. Resumen

- DSL chica deliberada: legible, low-barrier, mapea a la estructura del matcher.
- Cuatro principios: legibilidad LR, un solo separador, predicados post-fix, defaults razonables.
- Exclusiones deliberadas: sin disjunción, sin negación, sin recursividad, sin variables libres en trailing.
- DSL es la interfaz; `Hypothesis` es la representación interna. Cualquier transport puede saltarse el parser.

Ejercicios

1. ★ Escribí en la DSL la hipótesis: “un proceso que escribió un archivo y después se conectó a una IP, todo en menos de 5 minutos”. Identificá los tres componentes (cadena, ventana, predicados).
2. ¿Por qué `;` es mejor disambiguador que `AND`? Discutí dos casos donde `AND` sería ambiguo.
3. ★ Diseñá una sintaxis para *disjunción* (“patrón A o patrón B”). ¿Cómo se parsearía? ¿Qué cambia para el matcher?
4. Imaginá un caso de uso real (de threat hunting) que la DSL actual *no* puede expresar. ¿Cuál es? ¿Vale la pena extender la DSL para cubrirlo?
5. Si el frontend quisiera ofrecer un constructor visual (drag-and-drop nodes y edges) en vez de DSL, ¿qué tendría que generar para que el motor lo entienda?

Capítulo 30

Sintaxis *chain* (legacy)

Intuición

La sintaxis *chain* es la forma original de la DSL: una cadena lineal de pasos $p_0 \to p_1 \to \cdots \to p_k$ con predicados opcionales y cláusulas trailing. Es la forma más usada y la única que existía antes de L1 stage 3c. Este capítulo documenta cada parte de la sintaxis con ejemplos y muestra cómo el parser la convierte en `Hypothesis`.

30.1. Anatomía de una hipótesis chain

```
User u -[Auth]-> Host h -[Connect]-> IP {k=2} HAVING count(distinct last) >=
    5 WITHIN 30m
```

Componentes:

- `User u`: *type declaration* con *binding*. Declara que el primer vértice es de tipo `User` y se referenciará como `u`.
- `-[Auth]->`: arista con `RelationType`.
- `Host h`: type declaration del próximo vértice.
- `-[Connect]->`: segunda arista.
- `IP`: type declaration sin binding (no se referencia).
- `{k=2}`: k-simplicidad. Cada vértice puede repetirse hasta 2 veces.
- `HAVING count(distinct last) >= 5`: agregación post-hoc. Acepta sólo grupos de ≥ 5 lasts distintos.
- `WITHIN 30m`: ventana temporal de 30 minutos entre primera y última arista.

30.2. Type bindings

```
User u -[Auth]-> Host h
```

`u` y `h` son *bindings*: identificadores que referencian el vértice *en cláusulas trailing* o *en otras chains* (shape graph). En una chain pura sin `HAVING`, los bindings son opcionales:

```
User -[Auth]-> Host -[Connect]-> IP
```

es equivalente a la versión con bindings, pero el motor pierde la capacidad de referenciar los vértices intermedios.

30.2.1. Casos donde el binding importa

- `HAVING count(distinct h) >= 3`: requiere binding `h` en el segundo vértice.
- Shape graph: `c -[Auth]-> Host h1; c -[Auth]-> Host h2` necesita binding `c` para que el segundo `c` lo referencie.

30.3. Predicados sobre la arista

```
User -[Auth{result=success}]-> Host
```

`{result=success}` es un predicado sobre la metadata de la arista. El matcher consulta el `MetadataStore` (capítulo 8) para verificar que el campo `result` valga `success`.

Predicados soportados:

- `key=value`: igualdad.
- `key>value`, `key<value`: comparación numérica.
- `key in [v1, v2, v3]`: pertenencia a conjunto.
- `key matches /regex/`: regex (caro, evitar si posible).

30.4. Predicados sobre el vértice destino

```
User -[Auth]-> Host {os=Windows} -[Connect]-> IP
```

Las llaves *después* del type declaration aplican el predicado al vértice destino. El matcher las verifica en el momento de extender el path.

30.5. Cláusulas trailing

30.5.1. WITHIN

```
User -[Auth]-> Host -[Connect]-> IP WITHIN 30m
```

$\Delta t = 30$ minutos entre el timestamp de la *primera* arista y el de la *última*. Sintaxis: número + unidad (`s`, `m`, `h`, `d`).

Sin `WITHIN`: $\Delta t = +\infty$ (sin restricción).

30.5.2. HAVING

```
User -[Auth]-> Host h -[Connect]-> IP HAVING count(distinct h) >= 5
```

Agregación sobre los matches del path. `count(distinct h)` cuenta hosts distintos por start; `>= 5` acepta sólo starts con ≥ 5 hosts distintos.

Funciones soportadas:

- `count(distinct binding)`.
- `count(distinct first)` / `count(distinct last)`: sintaxis especial referenciando el primer / último vértice de la chain.
- `count(*)`: cardinalidad bruta del grupo.

30.5.3. `{k=N}`

```
User -[Auth]-> Host -[Connect]-> IP {k=2}
```

k-simplicidad (capítulo 2): cada vértice puede aparecer hasta k veces en el path. Default: $k = 1$ (path simple). $k = 2$ permite revisitar un vértice una vez (útil para hipótesis donde un usuario *vuelve* a un host).

30.6. Ejemplos completos

Ejemplo: Lateral movement chain

```
User u -[Auth]-> Host h1 -[Auth]-> User u2 -[Auth]-> Host h2
WITHIN 1h
HAVING count(distinct h2) >= 3
```

"Un usuario que se autentica en un host, desde donde otro usuario se autentica en un tercer host distinto, repetido para al menos 3 hosts distintos, todo en menos de una hora." Patrón típico de movimiento lateral con credential reuse.

Ejemplo: Beaconing inicial

```
Process p {image=powershell.exe} -[Connect]-> IP -[Beacon{interval<60s}]->
    Domain
WITHIN 5m
```

"Un proceso PowerShell que se conecta a una IP, y desde esa IP hay beaconing con intervalo < 60 segundos a un Domain, todo en menos de 5 minutos." Detección C2 de bajo volumen.

30.7. Cómo el parser construye la `Hypothesis`

Para la hipótesis del primer ejemplo, el parser produce:

```
Hypothesis {
    name: "...",
    steps: vec![
        HypothesisStep {
            source_type: User,  source_binding: Some("u"),
            rel_type: Auth,     edge_predicates: vec![],
            dest_type: Host,    dest_binding: Some("h1"),
            dest_predicates: vec![],
        },
        HypothesisStep {
            source_type: Host,  source_binding: Some("h1"),
            rel_type: Auth,     edge_predicates: vec![],
            dest_type: User,    dest_binding: Some("u2"),
            dest_predicates: vec![],
        },
        HypothesisStep {
            source_type: User,  source_binding: Some("u2"),
            rel_type: Auth,     edge_predicates: vec![],
            dest_type: Host,    dest_binding: Some("h2"),
            dest_predicates: vec![],
        },
    ],
    k_simplicity: 1,
```

```
    aggregations: vec![
        AggClause::CountDistinct {
            binding: "h2",
            op: Op::Gte,
            value: 3,
        }
    ],
    window_seconds: Some(3600),
    shape: HypothesisShape::Chain,
}
```

Notar que `shape == Chain` —no hay `;` en la hipótesis. Si lo hubiera, sería `Graph` (capítulo 31).

30.8. Resumen

- Sintaxis chain: cadena lineal `Type binding -[rel]-> Type binding` + cláusulas trailing.
- Predicados: en aristas (`[rel{predicates}]`) y en vértices destino (`Type {predicates}`).
- Trailing: `WITHIN N(m|h|d)`, `HAVING agg op val`, `{k=N}`.
- Defaults: sin WITHIN ⇒ ventana infinita; sin HAVING ⇒ enumeración bruta; `{k=1}` ⇒ path simple.
- `shape == Chain` para hipótesis sin `;`.

Ejercicios

1. ★ Escribí en la DSL: "un usuario que en menos de 10 minutos se autenticó en al menos 3 hosts distintos". Identificá las cláusulas trailing necesarias.
2. ¿Cuál es la diferencia entre `User -[Auth{result=success}]-> Host` y `User -[Auth]-> Host {result=success}`?
3. ★ Construí la `Hypothesis` (struct Rust) para la hipótesis: `Process p -[Spawn]-> Process c -[Write]-> File f WITHIN 1m`.
4. ¿Por qué los bindings son opcionales en chain pura pero obligatorios en HAVING? Pensá: ¿qué referenciaría el HAVING sin binding?
5. Imaginá una hipótesis donde necesitás $k = 3$. Escribí un caso real (en threat hunting) donde la repetición triple es sensata.

Capítulo 31

Sintaxis *graph*: el separador ;

Intuición

La sintaxis chain (capítulo anterior) sólo expresa cadenas lineales. Para hipótesis tipo estrella o árbol —"un usuario que se autentica en tres hosts distintos"— la cadena no alcanza. La extensión L1 stage 3c agregó un solo separador, `;`, que abre las puertas a hipótesis no-cadena. Una hipótesis con `;` en cualquier parte se marca como *shape graph*; el parser construye un hipergrafo, el planner detecta α-aciclidad, y el matcher dispatcha a Yannakakis tree-walk (capítulo 24). Este capítulo documenta la sintaxis y el flujo del parser.

31.1. El cambio mínimo: agregar ;

```
User c -[Auth]-> Host h1; c -[Auth]-> Host h2; c -[Auth]-> Host h3
```

Esa es la hipótesis "un usuario que se autentica en al menos tres hosts distintos" (la *star query*). Tres cláusulas separadas por `;`; las tres comparten el binding `c`.

Compará con la equivalente en chain pura:

```
User c -[Auth]-> Host  HAVING count(distinct last) >= 3
```

La versión chain es más concisa pero menos expresiva: enumera *cualquier* count de hosts ≥ 3, sin nombrarlos. La graph version los nombra (h_1, h_2, h_3) y permite predicados distintos sobre cada uno.

31.2. Reglas de las cláusulas tras ;

31.2.1. Bare binding reference

```
User c -[Auth]-> Host h1; c -[Auth]-> Host h2
```

La segunda cláusula arranca con `c`, que NO es un type sino un binding ya declarado. El parser:

1. Peeka el primer token de la cláusula post-`;`.
2. Si matchea un binding declarado en cláusulas previas (`c` sí), retorna `(existing_type, Some(name))` al loop principal.
3. El loop arranca el chain step desde ese vértice compartido: hereda el `EntityType` (`User`) del binding original.

31.2.2. Fresh type declaration

```
User c -[Auth]-> Host h1; Process p -[Spawn]-> File f
```

La segunda cláusula arranca con `Process p`, que es un nuevo type declaration. El parser introduce un vértice fresco independiente del primero. La hipótesis representa: "ambos patrones existen en el grafo, sin conexión entre ellos".

Nota. Este caso (cláusulas *disconectadas* unidas sólo por `;`) es legal pero raro: el matcher las trata como un *producto cartesiano* de matches independientes. Output: |matches del primero| × |matches del segundo|. Útil sólo cuando el analista quiere correlacionar patrones distintos.

31.2.3. Disambiguación

¿Qué hace el parser si el primer token de la cláusula post-`;` es ambiguo? Reglas:

1. Si matchea un *builtin EntityType* (`User`, `Host`, `IP`, etc.) ⇒ siempre se parsea como type declaration. Sombrea cualquier binding del mismo nombre. (Razón: los EntityTypes son palabras reservadas; usar "User" como binding sería confuso.)
2. Si matchea un binding ya declarado ⇒ bare binding ref.
3. Si no matchea ninguno ⇒ **error** con *hint*: "binding desconocido; los declarados son: …".

El error con hint es deliberado: descarta el fallback silencioso a `Other` (capítulo 8). Los typos del analista (`Hsot` en vez de `Host`) tienen que saltar como error, no como "vértice de tipo Other".

31.3. Estructura interna del parser

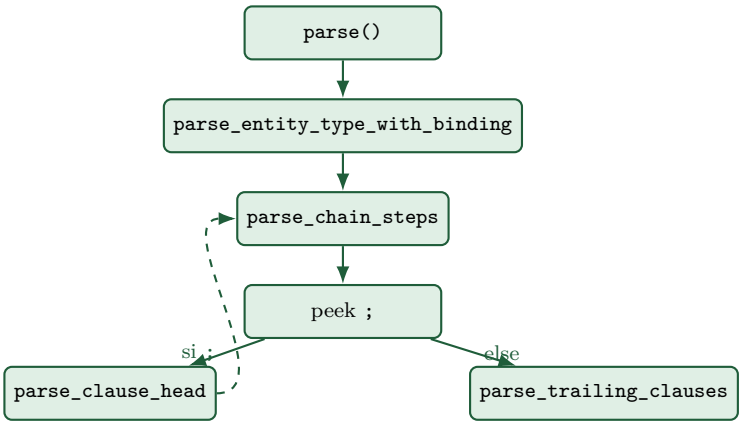


Figura 31.1: Flujo del parser. `parse_chain_steps` es el bucle interior reusable; cada `;` dispara un re-entry vía `parse_clause_head`, que decide entre bare binding ref y type declaration.

31.3.1. Cómo se lee cada nodo

- `parse_entity_type_with_binding` (*first clause head*): consume un `Type` obligatorio y, si lo sigue un identificador no reservado, lo registra como binding. Es la única forma de declarar el primer vértice —las hipótesis no pueden empezar con un bare binding ref porque no hay nada declarado todavía—.
- `parse_chain_steps`: bucle que consume N pasos `-[rel]-> Type binding` encadenados. Termina cuando ve EOF, `{` (suffix `{k=N}` o predicate block), `;` (clause separator), o un identificador alfabético (HAVING/WITHIN). Cada terminación delega al manejador correspondiente.

- `parse_clause_head` (*tras un* `;`): peeka el identificador líder y decide:
  - Si matchea un builtin type name (`User`, `Host`, `IP`, etc.) ⇒ rewind y delega a `parse_entity_type_with`
  - Si matchea un binding ya declarado ⇒ retorna `(existing_type, Some(name))` para que el bucle de `parse_chain_steps` arranque desde ese vértice compartido.
  - Si no es ninguno de los dos ⇒ error con hint listando los bindings válidos.
- `parse_trailing_clauses`: consume *HAVING* y *WITHIN* en cualquier orden hasta encontrar `{k=N}` o EOF. Estos no pueden re-entrar el bucle de chain steps; son estrictamente post-fix a la última cláusula del cuerpo.

31.4. Ejemplos

Ejemplo: Estrella de tres brazos

```
User c -[Auth]-> Host h1; c -[Auth]-> Host h2; c -[Auth]-> Host h3
```

Tres cláusulas, todas comparten c. Shape graph ⇒ Yannakakis tree-walk. El join tree es una estrella: nodo central es step 0; los otros dos son hijos.

Ejemplo: Triángulo de relaciones

```
User a -[Knows]-> User b; b -[Knows]-> User c; c -[Knows]-> a
```

Tres cláusulas formando ciclo: $a \to b \to c \to a$. El hipergrafo es cíclico (no α-acíclico). El planner detecta y cae al DFS legacy —no es K_4, no aplica LFTJ; no es α-acíclico, no aplica Yannakakis.

Ejemplo: Árbol de cuatro niveles

```
User u -[Auth]-> Host h; h -[Hosts]-> Process p; p -[Spawn]-> Process c
```

Tres cláusulas conectadas en árbol ($u \to h \to p \to c$). Es α-acíclico ⇒ Yannakakis. Notar que esta hipótesis se podría haber escrito como chain pura (`User -[Auth]-> Host -[Hosts]-> Process -[Spawn]-> Process`); la versión graph no aporta nada nuevo. La forma chain es más concisa.
(El motor detecta esto: si la graph hypothesis es estrictamente una chain disfrazada, podría re-escribirla como chain. Hoy no lo hace; ambas formas pasan por sus respectivos matchers, y los benches no muestran diferencia significativa.)

31.5. Limitaciones

- **No hay disjunción intra-cláusula**. Cada cláusula es una chain pura. Si querés "c tiene un Auth A o B", necesitás dos hipótesis separadas.
- **No hay subqueries**. Cada cláusula es plana; no hay anidamiento.
- **Sólo α-acíclicos eficientes**. Si la hipótesis introduce un ciclo (como el ejemplo del triángulo), el motor cae al DFS, perdiendo el lift de Yannakakis.

31.6. Resumen

- Sintaxis graph: chains separadas por `;`, todas comparten variables (bindings) cuando se referencian.
- Cada cláusula post-`;` arranca con type declaration (vértice fresco) o bare binding ref (vértice compartido).
- Errores explícitos en typos: "binding desconocido" en vez de fallback silencioso.
- `shape == Graph` si y sólo si hay `;`. Yannakakis tree-walk dispatcha si α-acíclico; DFS si cíclico.

Ejercicios

1. ★ Escribí en sintaxis graph: "un usuario que crea un proceso, y ese proceso escribe un archivo, y ese proceso también lanza otro proceso". Identificá los bindings compartidos.
2. Para la hipótesis del ejercicio 1, dibujá el join tree. ¿Cuál es la root?
3. ★ ¿La hipótesis `User c -[Auth]-> Host h; c -[Auth]-> Host h` (dos cláusulas idénticas) es legal? Si sí, ¿qué encuentra el matcher?
4. Diseñá una hipótesis graph cíclica de tres pasos. ¿Cómo la detecta el planner? ¿A qué operador cae?
5. Si una hipótesis tiene 5 cláusulas separadas por `;`, todas *disconectadas* (sin bindings compartidos), ¿qué tamaño de output puede tener? Cota teórica.

Capítulo 32

Del AST al *matcher*

Intuición

La pieza intermedia entre el parser y el matcher es la `Hypothesis` —el AST de la DSL, simplificado para que el matcher pueda consumirlo sin parsear strings—. Este capítulo recorre la struct `Hypothesis` campo por campo, muestra cómo el matcher dispatcha según `shape` y `steps`, y discute cómo otros transports (HTTP, MCP, programmatic) pueden saltarse el parser y construir `Hypothesis` directamente.

32.1. La struct `Hypothesis`

```
pub struct Hypothesis {
    pub name: String,
    pub steps: Vec<HypothesisStep>,
    pub k_simplicity: usize,
    pub aggregations: Vec<AggClause>,
    pub window_seconds: Option<i64>,
    pub shape: HypothesisShape,
}

pub enum HypothesisShape {
    Chain,
    Graph,
}

pub struct HypothesisStep {
    pub source_var: VarId,
    pub source_type: EntityType,
    pub source_predicates: Vec<Predicate>,
    pub rel_type: RelationType,
    pub edge_predicates: Vec<Predicate>,
    pub dest_var: VarId,
    pub dest_type: EntityType,
    pub dest_predicates: Vec<Predicate>,
}
```

Notar las diferencias con la DSL textual:

- Los `binding` (textuales: `u`, `h1`) se convierten a `VarId` (numérico, único por hipótesis). El nombre se descarta; sólo importa la *identidad*.
- Los predicados están parseados a `Predicate` (un enum); el matcher los evalúa sin re-parsear strings.

- `shape` es explícito: `Chain` o `Graph`, no derivado de la presencia de `;`.
- `aggregations` es lista plana de cláusulas HAVING; el evaluador las aplica al resultado.

32.2. El `VarId`

```
#[derive(Copy, Clone, PartialEq, Eq, Hash)]
pub struct VarId(pub usize);

impl Hypothesis {
    pub fn vars(&self) -> Vec<VarId> {
        let mut seen = HashSet::new();
        for step in &self.steps {
            seen.insert(step.source_var);
            seen.insert(step.dest_var);
        }
        seen.into_iter().collect()
    }
}
```

`VarId` es un índice en el *binding array* del matcher. Para una chain de k pasos sin bindings compartidos: $k + 1$ `VarId`s distintos. Para una hipótesis graph donde el binding c aparece en 3 cláusulas: c tiene un solo `VarId`, los otros vértices tienen los suyos.

El parser asigna `VarId`s en orden de aparición, mapping nombres textuales (`u`, `c`, `h1`) a índices $[0, 1, 2, \dots]$. Los nombres textuales se mantienen *sólo* para el debugging y para el HAVING (que referencia bindings por nombre).

32.3. El campo `shape`

```
pub enum HypothesisShape {
    Chain,
    Graph,
}
```

La pieza nueva que destrabó L1 stage 3. El matcher branchea según `shape` en `search_temporal_pattern_yannak`

```
match hypothesis.shape {
    HypothesisShape::Chain => {
        let bags = build_bags_chain(graph, hypothesis);
        semijoin_reduce_chain(&mut bags);
        enumerate_chain(&bags, hypothesis)
    }
    HypothesisShape::Graph => {
        let join_tree = JoinTree::build(hypothesis);
        let bags = build_bags_graph(graph, hypothesis, &join_tree);
        semijoin_reduce_tree(&mut bags, &join_tree);
        enumerate_tree(&bags, &join_tree, hypothesis)
    }
}
```

Dos paths distintos pero *paralelos*:

- Chain (capítulo 23): bags indexados por step (orden lineal); semi-join reducer en dos pasadas; enumerate via walk lineal.
- Graph (capítulo 24): bags indexados por step (con índices `by_origin` y `by_dest`); join tree construido con GYO; tree-walk enumerator.

32.4. Cómo lo construyen otros transports

La gracia del AST: cualquier transport que quiera ejecutar un hunt puede construir una `Hypothesis` sin pasar por la DSL. Ejemplos:

32.4.1. HTTP REST

```
POST /run_hunt
{
    "name": "Sysmon W0",
    "dsl": "User -[Auth]-> Host -[Connect]-> IP WITHIN 30m"
}
```

El handler HTTP toma el campo `dsl`, llama `parse_dsl`, obtiene una `Hypothesis`, la ejecuta. Como atajo, el endpoint también acepta una `Hypothesis` pre-construida en JSON:

```
POST /run_hunt
{
    "hypothesis": {
        "name": "Sysmon W0",
        "steps": [...],
        "shape": "Chain",
        "window_seconds": 1800,
        "k_simplicity": 1,
        "aggregations": []
    }
}
```

Bypaseo total del parser. Útil para clientes que generan hipótesis programáticamente (constructor visual, generators ML).

32.4.2. MCP

```
mcp_call cmd_parse_dsl({"dsl": "..."})  -> Hypothesis (json)
mcp_call cmd_run_hunt({"hypothesis": ...})  -> Vec<HuntResult>
```

Dos llamadas separadas: una para parse (devuelve la `Hypothesis` para inspección), otra para run. Esto permite que el LLM modifique la `Hypothesis` programáticamente entre ambas (ajustar predicados, expandir variables, etc.).

32.4.3. Tauri (frontend)

```
cmd_run_hunt(hypothesis_json: String) -> Vec<HuntResult>
```

El frontend serializa la `Hypothesis` (construida a partir del input del usuario, ya parseada) como JSON. El backend Rust deserializa y ejecuta. La separación frontend/backend pasa por la `Hypothesis`, no por el string DSL.

32.5. Estabilidad de la `Hypothesis`

Advertencia. La `Hypothesis` es la *interfaz pública* del motor para matchers. Cambios breaking en su shape (renombrar campos, agregar required fields, cambiar enums) rompen consumers. La política: agregar campos opcionales es OK; remover o renombrar requiere ADR + versionado.

32.6. Validación

Antes de ejecutar, el matcher valida la `Hypothesis`:

1. `steps` no vacía (al menos 1 step).
2. Bindings consistentes: si `step[i].dest_var == step[j].source_var`, entonces `step[i].dest_type == step[j].source_type`.
3. `shape == Chain` ⇒ los `VarId`s forman una secuencia lineal sin saltos.
4. `shape == Graph` ⇒ chequea α-aciclidad antes de pasar a Yannakakis (sino, se cae al DFS).

Si alguna validación falla, el matcher devuelve un error estructurado al transport en lugar de empezar el hunt.

32.7. Resumen

- `Hypothesis`: la representación interna del AST, consumida por el matcher en lugar del string DSL.
- Campos: `steps`, `shape`, `k_simplicity`, `aggregations`, `window_seconds`, `name`.
- `VarId`: índice numérico para bindings; el matcher nunca toca strings.
- `shape == Chain | Graph` dispatcha al sub-matcher correspondiente (chain o tree-walk).
- Los transports pueden construir `Hypothesis` programáticamente sin pasar por la DSL.

Ejercicios

1. ★ Construí la `Hypothesis` (struct Rust) para `User c -[Auth]-> Host h1; c -[Auth]-> Host h2`. Identificá los `VarId`s asignados.
2. ¿Qué pasa si construís una `Hypothesis` con `shape == Chain` pero los steps NO forman cadena (binding shared en el medio)? ¿El matcher lo detecta?
3. ★ Diseñá una API REST para construir hipótesis: ¿qué endpoint? ¿Qué payload? ¿Cómo manejás errores de validación?
4. Si la `Hypothesis` fuera serializable como Protobuf en vez de JSON, ¿qué cambiaría? ¿Cuándo conviene?
5. ¿Por qué los nombres textuales de bindings (`u`, `c`) NO se preservan en `VarId`? ¿Cuándo importa el nombre original?

Parte VIII

Diseño futuro (L3)

Capítulo 33

Por qué *khop_reach* delta es difícil

Intuición

Stage 2a (capítulo 15) resolvió el caso incremental para NLF: cada `add_relation` bumpea un contador local. Para k-hop reachability el problema es fundamentalmente más difícil porque la información NO es local. Una sola arista (u, v) puede cambiar el bit de v —trivial— pero también el de *todos los predecesores* de u que acortan su camino al tipo de v. En grafos scale-free con hubs, "todos los predecesores" puede ser millones de vértices. Este capítulo argumenta por qué la solución no es invalidar-y-rebuild ni una propagación naive, y prepara el terreno para Delta WCOJ Propagation (próximo capítulo).

33.1. El problema en una frase

NLF es *local*: una arista (u, v) afecta solamente $\text{NLF}(u, \text{type}(v))$. Update incremental: bumpear un contador. $O(1)$ por arista.

k-hop es *transitivo*: una arista (u, v, t) puede cambiar bits$[w]$ y min/max_first$[w][L]$ para todo w que tiene un camino a u en $<$ `MAX_DEPTH` saltos. "Todo w" puede ser cero (si la arista no agrega tipos nuevos), pocos, o muchísimos.

33.2. Ejemplo: una arista, muchos cambios

Ejemplo

Grafo previo a la inserción:

- $u_1, u_2, u_3 \to h$ (tres usuarios autenticando en un host).
- $h \to ip_1$ en $\tau = 100$ (host conectado a una IP).

Para `MAX_DEPTH = 2`, los u_i tienen bits[u_i] con bits `Host` (vía h) e `IP` (vía $h \to ip_1$) prendidos. min/max_first[u_i][`IP`] $= \tau$ de la arista $u_i \to h$.
Llega una arista nueva: $h \to ip_2$ en $\tau = 200$.
Cambios al índice:

- bits[h] ya tiene bit `IP` prendido (por ip_1). Sin cambio.
- max_first[h][`IP`] actualiza a 200 (era 100).
- Para cada u_i: bits[u_i] ya tiene IP. Sin cambio en bits.
- max_first[u_i][`IP`] actualiza al timestamp de la arista $u_i \to h$ si es posterior. (Depende del análisis de paths concretos.)

Tres mutaciones de min/max_first, ninguna en bits. Manageable.
Ahora considerá una arista que SÍ agrega tipo nuevo: digamos un d (Domain) y la arista $h \to d$ en $\tau = 300$.
Cambios:

- bits[h] ahora tiene bit `Domain` (era 0). Cambio en bits.
- Por transitividad: para cada u_i, bits[u_i] ahora tiene bit `Domain` también. Tres cambios en bits.
- Si los u_i tienen predecesores propios (digamos cada uno tiene un grupo de $\sim$ 1000 predecesores), *esos también* ganan el bit `Domain`. Cascada: $1000 \cdot 3 = 3000$ cambios adicionales.
- Si el grafo es scale-free y h es un hub, los predecesores y predecesores-de-predecesores pueden ser $\sim 10^6$. Cascada explota.

33.3. Por qué invalidar-y-rebuild es lento

La política pre-stage-2a era simple: cualquier `add_relation` *post*-build setea `self.khop_reach = OnceLock::new()`; el próximo hunt reconstruye desde cero.

Costo: $O(|\mathcal{L}_V| \cdot \texttt{MAX_DEPTH} \cdot |\mathcal{E}|)$ (capítulo 28). Para $|\mathcal{E}| = 10^8$: $\sim$1.5 minutos por rebuild. Para ingestas continuas (Sentinel pulling cada 30 s + 1 hunt por minuto): rebuild cada 30 s. La UI vive en estado de espera permanente.

33.4. Por qué la propagación naive falla

La idea natural: cuando llega (u, v, t), recursar hacia atrás desde u actualizando bits y min/max_first de cada predecesor:

```
fn propagate_naive(u: StrId, delta_bits: u16, t: i64) {
    bits[u] |= delta_bits;
```

```
    update_min_max(u, t);
    for w in predecessors(u) {
        propagate_naive(w, delta_bits, t);  // recurse
    }
}
```

Tres problemas:

1. **Sin cota de profundidad**. Recursión hasta agotar *todos* los ancestros. Para grafo conexo, eso es $|\mathcal{V}|$. Pero el índice sólo cuenta caminos de $\leq$ `MAX_DEPTH` saltos; propagación más allá es work pero no aporta info.
2. **Sin dedup**. El mismo w puede ser predecesor por varios paths; la propagación naive lo visita cada vez. Costo explota.
3. **Sin chequeo de delta vacío**. Si bits[w] ya tenía todos los bits que bits[u] está agregando, la propagación sigue para nada.

Cada problema requiere su antídoto: cota por `MAX_DEPTH`, frontier set para dedup, chequeo $\Delta =$ bits[u] $\setminus$ bits[w] antes de propagar.

33.5. Por qué min/max_first es aún más difícil

bits es monotónico-creciente: nunca se quita un bit; sólo se agregan. Una vez setado, queda. La sobre-aproximación está intrínsecamente protegida.

min_first y max_first son *bidireccionales*: una arista nueva podría *bajar* min_first o *subir* max_first. Pero si una arista vieja se *borra* (caso que GraphHunter no maneja hoy —el grafo es append-only, capítulo 8), las cantidades extremales podrían *volverse menos extremas* —requiriendo recomputar.

Por suerte, append-only nos salva: nunca borramos, así que min_first sólo baja y max_first sólo sube. Mantener esa invariante bajo deltas es manejable. Pero *detectar* cuándo cambian para un w dado requiere re-mirar el path completo desde w —no sólo el bit.

33.6. Soundness es la salvaguarda

Hay una buena noticia: *cualquier* sobre-aproximación es sound. El motor podría:

- **No actualizar nunca**: el bit existente sigue válido (no se quita); el min/max_first sigue siendo extremal según lo que conocía. Nuevo info no se refleja pero las podas que aplicaba siguen siendo correctas. La "pérdida" es que algunos prunes que ahora podrían descartar quedan aceptando —el matcher trabaja un poco más, no menos.
- **Actualizar parcialmente**: actualizar bits y min/max_first sólo para u y v directamente. Predecesores quedan con info vieja (sobre-aproximada). El matcher sigue siendo correcto.

La razón por la que el motor *quiere* actualizar todo lo más posible: cada bit L extra prendido en bits[w] significa que w sobrevive un prune que antes lo descartaba —más trabajo para el matcher—. La calidad del prune depende de qué tan tight sea la sobre-aproximación.

33.7. El plan: Delta WCOJ Propagation

El próximo capítulo desarrolla la solución que el motor adoptará (stage 2b futuro): *Delta WCOJ Propagation*, basado en Kankanamge-Sahu-Mhedhbi-Salihoglu SIGMOD 2017. La idea de fondo:

1. Computar el delta $\Delta_u = \text{bits}[v] \setminus \text{bits}[u]$. Si vacío, abortar (la arista no aporta info nueva desde u).
2. Si no vacío, actualizar bits$[u]$ y propagar el *delta* (no el bits completo) hacia atrás vía reverse adjacency.
3. En cada nivel, aplicar de nuevo el chequeo "delta vacío ⇒ abortar esta rama".
4. Cota `MAX_DEPTH` en la cascada.
5. **Cascade budget**: si la cascada visita más de un threshold (default 100K vértices), abortar y caer a invalidación. Caso patológico de hubs.

Costo amortizado: $O(\texttt{MAX_DEPTH} \cdot d_{\text{reverse}})$ por arista, donde d_{reverse} es el grado promedio del grafo inverso. Para grafos scale-free típicos, $\sim$ 10–100$\times$ más barato que rebuild.

33.8. Resumen

- NLF es local; k-hop es transitivo. Esa diferencia hace que el incremental update de k-hop sea fundamentalmente más difícil.
- Una sola arista puede generar cascada de cambios proporcional al grado *reverse* de los predecesores indirectos.
- Invalidate-and-rebuild es lento: $\sim$1.5 min para 28 GB EVTX.
- Propagación naive falla por falta de cota de profundidad, falta de dedup, y falta de chequeo delta vacío.
- min/max_first son más sutiles que bits por bidireccionalidad, pero append-only nos salva.
- Soundness garantiza que cualquier sobre-aproximación funciona; "actualizar más" sólo mejora la calidad del prune.
- Delta WCOJ Propagation (próximo capítulo) ataca el problema con propagación reverse acotada por `MAX_DEPTH` y *cascade budget*.

Ejercicios

1. ★ Para un grafo scale-free con hub de in-degree 10^4, si llega una arista nueva (u, v) donde v tiene tipo no visto antes y u es predecesor inmediato del hub, ¿cuántos vértices podrían cambiar bits? Estimación.
2. ¿Por qué la propagación naive sin dedup explota? Pensá en un grafo bipartito: cada vértice de un lado tiene 100 vecinos del otro, vice versa.
3. ★ Demostrá: si una arista nueva NO aporta tipos nuevos (bits$[v] \subseteq$ bits$[u]$), no hace falta propagar bits. ¿Y min/max_first?
4. Imaginá una variante donde bits usa más bits (32 en vez de 16). ¿Cambia la facilidad del incremental? ¿Por qué?
5. Si el motor permitiera DELETE de aristas, ¿qué se rompe en la invariante append-only? ¿Cómo lo arreglarías?

Capítulo 34

Delta WCOJ Propagation

Intuición

La técnica que el motor adoptará en stage 2b para resolver el incremental update de k-hop reachability viene de Kankanamge-Sahu-Mhedhbi-Salihoglu, "Graphflow: An Active Graph Database" (SIGMOD 2017). La idea de fondo: no recomputar todo el índice, sino propagar el *delta* (la diferencia entre el estado nuevo y el viejo) hacia los predecesores hasta que el delta se vacíe o se alcance `MAX_DEPTH`. Es una versión "targeted" del BFS reverse multi-source: en lugar de visitar todos los vértices, sólo visita los que la nueva arista genuinamente afecta. Este capítulo pseudo-codifica el algoritmo, prueba soundness, y discute el cascade budget que protege contra hubs patológicos.

34.1. El algoritmo

Algorithm 34.1: `khop_reach.delta_update(u, v, t)` — esquema

Input: nueva arista (u, v, t), grafo G, índice `khop_reach`

▷ `Round 1 - efecto directo:` u `ahora alcanza` type(v) `y todo lo que` v `alcanza.`

$\Delta_u \leftarrow \text{bits}[v] \setminus \text{bits}[u]$;
if $\Delta_u = 0$ ***and*** t *no mejora* min/max_first[u] **then**
  return (la arista no cambia el índice);
end
bits[u] $\models$ bits[v];
for *cada* L *en* bits[v] **do**
  min_first[u][L] $\leftarrow$ mín(min_first[u][L], t);
  max_first[u][L] $\leftarrow$ máx(max_first[u][L], t);
end

▷ `Rounds 2..MAX_DEPTH - propagación reverse.`

`frontier` $\leftarrow \{u\}$;
`visited` $\leftarrow \{u\}$;
for $d = 2$ ***to*** *MAX_DEPTH* **do**
  `next` $\leftarrow \emptyset$;
  for $x \in$ *frontier* **do**
    for $w \in N^-(x)$ *(*reverse adjacency*)* **do**
      $\Delta_w \leftarrow \text{bits}[x] \setminus \text{bits}[w]$;
      if $\Delta_w \neq 0$ ***or*** t_{wx} *mejora* min/max_first[w] **then**
        bits[w] $\models$ bits[x];
        actualizar min/max_first[w][L] para cada $L \in$ bits[x], usando t_{wx} (timestamp de la arista $w \to x$);
        if $w \notin$ *visited* **then**
          `next` $\leftarrow$ `next` $\cup\{w\}$;
          `visited` $\leftarrow$ `visited` $\cup\{w\}$;
        end
      end
    end
  end
  `frontier` $\leftarrow$ `next`;
  if $|$*visited*$| >$ *CASCADE_BUDGET* **then**
    abort: invalidar `khop_reach`; rebuild en próximo hunt;
    return;
  end
end

34.2. Análisis paso a paso

34.2.1. Round 1: efecto directo

La nueva arista $u \to v$ significa que u ahora puede alcanzar:

- type(v) directamente (1 salto).
- Todo lo que v ya alcanzaba (vía v).

Eso es exactamente bits[u] $\models$ bits[v] (incluyendo el bit de type(v) si estaba en bits[v] por el caso degenerate "v alcanza su propio tipo"; en la implementación real se asegura prendido).

Si bits[v] $\setminus$ bits[u] $= 0$ (Δ_u vacío), u no gana ningún tipo nuevo. Pero todavía puede ganar mejor min/max_first (si t es más temprano que cualquier camino previo) —chequeo separado.

34.2.2. Rounds 2..MAX_DEPTH: propagación reverse

Para cada w que tiene una arista directa a u, recursivamente preguntamos si w gana algo. Si $\Delta_w = 0$ y los timestamps no mejoran, se aborta esta rama (no se agrega a la frontier).

La cota `MAX_DEPTH` en d: después de `MAX_DEPTH` rounds, la propagación se detiene. Razón: el índice sólo guarda alcance $\leq$ `MAX_DEPTH` saltos; un cambio más profundo no afecta a la sobre-aproximación guardada.

34.2.3. Cascade budget

Si `visited` pasa un umbral (default 100 000), abortar la propagación e invalidar el índice completo. El próximo `ensure_khop_reach` reconstruye desde cero (capítulo 28).

¿Por qué? Porque los grafos scale-free tienen hubs raros donde una cascada visita millones. En esos casos el rebuild $O(|\mathcal{E}|)$ es *cheaper* que la cascada O(visited). Mejor caer al rebuild que pagar más en propagación.

34.3. Soundness

Teorema 34.1 (Delta update preserva sobre-aproximación)**.** *Sea G_t el grafo después de t llamadas a `add_relation`, e índice I_t computed por aplicación secuencial de `delta_update`. Entonces para todo v y L:*

$$I_t.\text{bits}[v][L] = 1 \implies \exists\, \text{walk } v \rightsquigarrow_{\leq \texttt{MAX_DEPTH}} L \text{ en } G_t.$$

Es decir: el índice puede sobre-aproximar (decir 1 cuando no existe walk) pero NUNCA puede sub-aproximar.

Idea. Inducción sobre t. Base $t = 0$: I_0 es construido por el builder (capítulo 28), que es exacto. Paso inductivo: `delta_update` sólo *agrega* bits via bits[w] $\models$; nunca quita bits. Y sólo *baja* min_first y *sube* max_first; nunca al revés. Por lo tanto, cualquier "walk válido en G_t que el índice marca" sigue siendo válido en G_{t+1} (porque $G_{t+1} \supseteq G_t$). Sound. □ □

34.3.1. Falsos positivos OK; falsos negativos NO

La sobre-aproximación es OK porque el matcher *verifica* después: si el prune deja pasar un start que no debería, el DFS lo descarta sin emit. Costo: un poco más de DFS.

La sub-aproximación rompería el contrato: el matcher descartaría starts válidos $\Rightarrow$ matches reales perdidos. Eso hace que el algoritmo sea inviable.

34.4. Costo en mmap-spilled

El `IndexBacking::heap_slice_mut` no funciona sobre mmap-spilled (es read-only). Para el caso mmap, el motor:

1. Abre el archivo en modo `MmapMut` durante el delta update.
2. Aplica las mutaciones.
3. `flush()` para sincronizar a disco.

4. Vuelve a `Mmap` read-only.

Costo del remap: ∼1–3 ms en Linux/Windows. Para una sola arista delta no compensa; para batches de muchas aristas (decenas de inserts), la amortización funciona.

34.5. Sincronización readers/writer

El `KHopReach` expone `min/max_first` bajo `RwLock` (capítulo 28). El delta update toma write-lock mientras muta; los matchers concurrentes ven el estado consistente vía read-lock. Bloquea, pero la mutación es $O(\text{visited})$ que es chico —típico < 1 ms—.

Para escenarios de altísima concurrencia, una versión wait-free sería deseable (similar al `ArcSwap` del streaming backend, capítulo 13); por ahora, `RwLock` alcanza.

34.6. Tests de equivalencia

Antes de shipping, dos tests críticos:

1. **Property test**: para un grafo aleatorio + secuencia aleatoria de `add_relation`, comparar `KHopReach::build` (rebuild from scratch) vs `delta_update` aplicado uno a uno. Iguales para todos los vértices (excepto cuando el cascade budget aborta y el delta es sobreaproximación más laxa que el rebuild).

2. **Soundness test**: ningún matcher hot path debe podar un start válido después de delta updates. Random hunts + verificación contra DFS sin podas.

34.7. Cuándo el delta supera al rebuild

Estimación rough:

- Rebuild: $O(|\mathcal{L}_V| \cdot \texttt{MAX_DEPTH} \cdot |\mathcal{E}|)$. Para $|\mathcal{E}| = 10^7$: ∼10 segundos.
- Delta: $O(\texttt{MAX_DEPTH} \cdot d_{\text{reverse}})$ por arista. Para $d_{\text{reverse}} = 10$ y `MAX_DEPTH = 4`: ∼40 lookups por arista. Para 1000 aristas insertadas: ∼40 000 ops, ∼1 ms.

Para grafos típicos, el delta es $\sim 10^4\times$ más rápido que el rebuild. Para hubs patológicos, el cascade budget detecta y se cae al rebuild —peor caso es un poco peor que rebuild puro, porque pagamos el costo abortado más el rebuild.

34.8. Cronograma estimado de implementación

Stage 2b (delta khop_reach): ∼2 semanas. La propagación inversa es bien-comprendida; el grueso del trabajo es el property test de equivalencia delta-vs-rebuild.

34.9. Resumen

- Delta WCOJ Propagation: actualiza incremental con propagación reverse acotada por `MAX_DEPTH`.
- Round 1: efecto directo en u. Rounds 2..MAX_DEPTH: cascada vía reverse adjacency, abortar rama si delta vacío.
- Cascade budget: si la cascada excede umbral (default 100K), invalidar y dejar rebuild.

- Soundness por monotonicidad: solo agregamos bits, sólo bajamos min y subimos max. Sub-aproximación es imposible.
- Costo amortizado: $O(\texttt{MAX_DEPTH} \cdot d_{\text{reverse}})$ por arista; mucho mejor que rebuild $O(|\mathcal{E}|)$.
- Estimado de implementación: ∼2 semanas (stage 2b futuro).

Ejercicios

1. ★ Para un grafo donde una nueva arista $u \to v$ tiene bits$[v]$ = `0b001000` (solo bit IP) y bits$[u]$ = `0b000001` (solo bit Domain), calculá Δ_u y los bits resultantes en bits$[u]$.
2. ¿Por qué `visited` se trackea separado de `frontier`? ¿Qué pasaría sin él?
3. ★ Demostrá que aplicar `delta_update` N veces en orden produce el mismo bits que un rebuild después de las N aristas. (Pista: inducción + monotonicidad.)
4. Diseñá un grafo adversarial donde el cascade budget aborta. ¿Cuál es la cota de visited? ¿Qué pasaría sin el budget?
5. Si el motor implementara *undo* de aristas (DELETE), ¿la propagación delta seguiría funcionando? ¿Qué se rompe?

Capítulo 35

Streaming como almacenamiento canónico

Intuición

Stage 2c part 2 entregado. El `SpillableEdgeStore` fue eliminado del árbol; el *streaming backend* es el único edge store. El matcher *hot path*, los ∼50 readers *admin/aggregation/index-build*, y los cinco *ingest paths* (`add_relation`, `add_relations_batch`, `replay_relations`, `compact_before`, `remove_entities_and_relations_by_dataset`) leen y escriben `streaming` directamente. Este capítulo era el *plano de obra*; ahora se mantiene como *post-mortem del plan* —qué se hizo, qué riesgos se materializaron, qué mitigaciones funcionaron— con la narrativa original intacta debajo.

Nota (Lo que efectivamente shipped). Tres commits secuenciales, cada uno bench-gated contra el *baseline* `pre-2c2`:

1. *Hot path migration* (5 call sites): `dfs_match_smart`, `dfs_match_iterative`, `edge_range_sid`, AMAC *prefetch lookahead*, `amac_step_one_edge`. Cada uno: `streaming.neighbors_arc(sid)` + `guard.read()` + `guard.as_slice()`. `parking_lot::RwLock` reemplazó a `std::sync::RwLock` para que el par de read-guards per-frame quepa en el budget.

2. *50-consumer migration*: NLF, khop_reach (build + delta), LFTJ, semijoin, Yannakakis, anomaly, analytics (8 sites), GNN bridge, matcher-ffi, platform/api.

3. *Rip-out del dual-write*: drop del field `edge_store: SpillableEdgeStore`, drop del módulo `spill.rs` entero (1.2 KLOC), `streaming.clear()` para los rebuild paths, `push_to_streaming(&CompactP` helper compartido por los cinco ingest paths.

Bench (`hunt_latency` vs `pre-2c2`): −29 % / −25 % / −26 % en 1K/10K/100K. *Score-path* mejoró −50 % a −55 % en todas las profundidades. Detalles en `docs/perf/latex/cambios-perf-2026-04.tex`.

Plano de obra original (referencia histórica): El resto del capítulo conserva el plan tal como se escribió antes del *ship*. Los riesgos mencionados (regresión cache en `Btree variant`, compatibilidad *snapshot/load*) se materializaron parcialmente: el `Btree variant` se *eliminó* en lugar de tunarse (preferimos `Vec` contiguo + `partition_point` para preservar el *prefetch lookahead* del matcher); la compatibilidad *save/load* sobrevivió intacta porque `add_relation` popula `streaming` en cada *replay_relations*.

35.1. El problema

Después de stage 2a (capítulo 13) cada `add_relation` hace dos writes:

1. Al `edge_store` canónico (`SpillableEdgeStore`, eliminado en stage 2c part 2).
2. Al `streaming` backend (per-vertex degree-adaptive, capítulo 13).

Memoria: las aristas viven en dos estructuras simultáneamente. Para 10^8 aristas, eso es 6.4 GB *repetidos* (32 B $\times$ 10^8 por edge_store + similar por streaming). Costo de insert: dos writes, dos cache lines tocadas.

El plan: *una sola estructura* (streaming) que sirve a todos los consumers, incluido el matcher hot path.

35.2. Por qué no se hizo en stage 2a

El matcher hot path consulta `edge_store` via familia de accessors: `get_relations_by_sid`, `iter_all`, `neighbors_of`, `get_compact_relations`. Cada uno tiene semántica *sorted-by-timestamp* y *binary-search-ready*. Migrar a streaming significa re-implementar esa familia sobre el nuevo layout `NeighborList`, que es *degree-adaptive* (no contiguo, no uniformemente sorted).

Tres call sites principales en `core/graph-engine/src/graph.rs`:

- DFS recursivo: itera `neighbors_in_window`.
- LFTJ-K_4: construye trie sobre todas las aristas.
- Yannakakis: build_bags itera por step.

Cada uno asume el layout contiguo del edge_store. Migración no trivial.

35.3. El plano: tres pasos

35.3.1. Paso 1: Adapter layer

Crear una capa que mimica la API del `edge_store` pero por encima del streaming:

```
impl GraphHunter {
    pub fn neighbors_of_streaming(
        &self,
        sid: StrId,
    ) -> impl Iterator<Item = StreamEdge> {
        let snap = self.streaming.neighbors_snapshot(sid);
        match &*snap {
            NeighborList::Inline(arr) => arr.iter_owned(),
            NeighborList::Vec(v) => v.iter().cloned().collect(),
            NeighborList::Btree(bt) => bt.values().cloned().collect(),
        }
    }
}
```

Sirve como *drop-in replacement* de `neighbors_of`. El matcher hot path llama a la versión adaptada en lugar de la legacy.

35.3.2. Paso 2: Re-write del hot path

Cada uso de `get_relations_by_sid` se reemplaza por `neighbors_of_streaming`. Los benches verifican parity contra la versión legacy:

- Target de regresión: $< 10\,\%$ en `matching_hops` y `hunt_latency`.
- Si la regresión es mayor, revisar el layout de la `Vec variant`.

35.3.3. Paso 3: Drop del `edge_store`

Con todos los consumers migrados, `edge_store` queda como estructura legacy mantenida sólo para `session_save`/`session_load` (donde el formato binario `GHS1` ya escribe `Relation`s en streaming). La conversión `NeighborList` → `Relation` al guardar es directa.

Eventualmente, eliminar el `edge_store` entero: los `Vec<Relation>` salen de `NeighborList`s flattened on-demand.

35.4. Beneficios esperados

- **RAM**: una sola copia de las aristas. Para 28 GB EVTX (∼100 M aristas × 32 B ≈ 3.2 GB ahorrado).
- **Latencia de *insert***: una sola escritura por `add_relation` (en lugar de dos). Esperado ∼30 % más rápido en ingest throughput.
- **Concurrencia**: el streaming backend ya soporta readers concurrentes vía `ArcSwap`; los matchers de varios transports pueden correr simultáneamente. Hoy, el `edge_store` requiere coordinación.

35.5. Riesgos y mitigaciones

35.5.1. Regresión en matcher hot path

El `edge_store` usa *contiguous Vec* sortado, perfecto para *cache prefetch*. La `NeighborList` tiene tres variantes (`Inline`, `Vec`, `Btree`) con cache behavior mixto. Especialmente la `Btree` variant: lookups $O(\log n)$ con cache misses por nivel.

Mitigación: benches contra `matching_hops` y `hunt_latency` antes de cada commit. Si la regresión $> 10\,\%$, revisar:

- ¿Es la `Vec variant` la que regresa? Optimizar cache layout.
- ¿Es la `Btree variant`? Aumentar el threshold de promotion (256 → 1024) para que menos vértices se vayan a `Btree`.
- ¿Es la `Inline variant`? Investigar branch prediction.

35.5.2. Compatibilidad de *snapshot/load*

El formato `GHS1` actual lee `Relation`s en streaming y los inserta vía `add_relation`. Eso sigue funcionando porque `add_relation` popula el streaming backend sin más; el `edge_store` legacy queda vacío en *post-load* y los matchers legacy paulatinamente migran a streaming.

Mitigación: tests de round-trip (`save` → `load` → `run_hunt` produce same results that pre-save).

35.5.3. Cache behavior de NeighborList variants

La preocupación más concreta. Hipergrafos *scale-free* tienen distribución bimodal: muchos vértices Inline (low degree, fits in 8), pocos hubs en Btree (high degree). El matcher típicamente itera más por hubs (donde los paths se concentran), así que la performance del `Btree` domina el bench.

Mitigación:

- Profile real de los hunts cíclicos para entender qué fracción del tiempo se gasta en lookups Btree vs Inline.

- Si Btree domina: considerar layout más compacto (B+tree custom en lugar de `BTreeMap` stdlib).
- Plan B: mantener un *indice secondario* para los hubs (ordered Vec on top of Btree). Más memoria, mejor cache.

35.6. Cronograma estimado

Stage 2c (canonical streaming): ∼3 semanas. La re-escritura del matcher hot path toca 12+ call sites en `core/graph-engine/src/graph.rs` más los benches; el riesgo de regresión es lo que más empuja el timeline.

35.7. Resumen

- Stage 2c: eliminar dual-write; streaming es source of truth, edge_store on-demand.
- Tres pasos: adapter layer, re-write hot path, drop edge_store.
- Beneficios: $\sim 30\,\%$ menos RAM, $\sim 30\,\%$ más rápido en insert, mejor concurrencia.
- Riesgos: regresión cache en hot path por NeighborList variants. Mitigación: benches en cada commit, threshold tuning, plan B con índice secundario.
- Cronograma estimado: ∼3 semanas.

Ejercicios

1. ★ Calculá el ahorro de memoria al eliminar el dual-write para un grafo de $2 \cdot 10^8$ aristas. ¿Cuánto en GB?
2. ¿Por qué el `Btree variant` tiene cache behavior peor que `Vec`? Pensá en cómo `BTreeMap` almacena nodos en heap.
3. ★ Diseñá un test que distinga: "el matcher funciona correcto sobre streaming" versus "el matcher funciona igual de *rápido* sobre streaming". Diferentes tipos de tests.
4. Si la regresión es $> 10\,\%$ en algún bench, ¿es aceptable shipping? ¿Bajo qué condiciones?
5. Imaginá que el motor mañana tiene que soportar *point queries* sobre aristas ("existe la arista (u, v, t)?"). ¿Cuál layout (edge_store vs streaming) es mejor? ¿Por qué?

Parte IX

Plug-and-play como tesis transversal

Capítulo 36

La promesa *plug-and-play*

Intuición

La regla *plug-and-play* de GraphHunter dice: cada optimización entregada (S/M/L) se activa por defecto al levantar el binario. Sin flags a setear, sin config a tocar, sin toggles en la UI. El usuario abre la app y todo lo que está *shipped* se nota inmediatamente. Esa restricción no es cosmética: hace que el motor sea *auditable*. Si después de un ship de optimización el analista tuvo que setear `GRAPHHUNTER_X=1` para verla, la promesa se rompió —la optimización no está realmente entregada, está *disponible*. Este capítulo codifica la regla en cinco pasos concretos y discute por qué es deliberada y no accidental.

36.1. La promesa, en una frase

> *"Ninguna optimización entregada requiere que el usuario cambie su config."*

Variantes equivalentes:

- Cada flag S/M/L que ships se setea *default-on* al *startup* del binary.
- Cada cache es transparente.
- Cada índice se construye solo (lazy build).
- Cada índice grande se spills a disco solo.

El usuario abre la app, ve la pildora "Perf: M+L1" verde, y todo funciona.

36.2. Por qué la regla es deliberada

Sin la regla, el motor cae en un patrón de fail-mode común en software de seguridad: *shipped pero no usado*. El equipo agrega una optimización, la testea, la merge —pero el usuario nunca la activa porque:

- No sabe que existe (no leyó el changelog).
- Sabe que existe pero no sabe cómo activarla.
- Sabe cómo activarla pero le da fiaca tocar config.
- Tocó config, anduvo bien, pero después actualizó el binary y el config volvió a defecto.

Default-on resuelve los cuatro casos: el código nuevo se activa sin acción del usuario.

36.3. La regla en cinco pasos

36.3.1. Paso 1: default-on al startup

Cada flag nuevo se setea en `apps/tauri/src-tauri/src/lib.rs` y `apps/cli/src/main.rs` con:

```
for (key, default) in [
    ("GRAPHHUNTER_LFTJ_PLANNER", "1"),  // M1 structural K_4
    ("GRAPHHUNTER_K4_LFTJ", "1"),        // S3 name marker
    ("GRAPHHUNTER_YANNAKAKIS", "1"),     // L1 stage 2/3
] {
    if std::env::var(key).is_err() {
        unsafe { std::env::set_var(key, default) };
    }
}
```

Override con `KEY=0` sigue funcionando para usuarios que quieran el path legacy (debugging, A/B comparison).

36.3.2. Paso 2: surface en UI

Extender `cmd_get_perf_status` para incluir el flag nuevo, y la pildora del toolbar para reflejarlo. El analista debe poder *ver* qué optimizaciones están activas sin abrir DevTools ni leer logs (capítulo 37).

36.3.3. Paso 3: default-off sólo con causa

Hay flags que NO son default-on (durante un período). Justifica sólo si:

- El flag re-escribe el DFS hot loop (cambios estructurales grandes).
- Aún no hay parity tests aguantando ≥ 1 semana estable.
- Hay un riesgo conocido de regresión en algún workload.

En esos casos, el flag queda *opt-in* durante un *soak window*. Cuando los parity tests aguantan, se flipea a default-on.

36.3.4. Paso 4: doc obliga

Cada flip default-off $\rightarrow$ default-on actualiza:

- `08-env-vars.tex` (libro Especificación).
- `12-preguntas-frecuentes.tex` (libro Manual).
- `make all` para refrescar los 6 PDFs.

Sin doc actualizado, el flip no se merge.

36.3.5. Paso 5: capabilities de build

Los *Cargo features* (como `symmetry-nauty`) que agregan funcionalidad opcional al binary se reflejan via constantes `HAS_NAUTY`-style en `cmd_get_perf_status`. El frontend lee la capability y muestra el feature como “disponible” o “desactivado en este build” —sin que el analista tenga que abrir `Cargo.toml`.

36.4. Cuándo se rompe la promesa

Casos reales en los que casi se rompió:

- **LFTJ-K$_4$ inicial (S3)**: requería `GRAPHHUNTER_K4_LFTJ=1` *y* que el nombre de la hipótesis contuviera "k4-clique". Cumplía la cláusula 1 (default-on) pero el name marker era una barrera. Se mantuvo por back-compat pero M1 (structural_dispatch) lo dejó redundante.
- **Yannakakis L1**: el flag default-on. Pero hubo un período donde el matcher Yannakakis tenía un bug en hipótesis con predicados; el "fix" fue requerir `GRAPHHUNTER_YANNAKAKIS=0` para esos casos. *No* cumplía con la regla. Se revertió, se arregló el bug en el operador, y el flag se restauró a default-on.

La regla obliga a *arreglar el bug en el operador*, no a *ofrecer un opt-out al usuario*.

36.5. Por qué *auditable*

Plug-and-play hace al motor auditable: el equipo de Base4 puede demostrar a un auditor externo que el motor en producción usa las optimizaciones publicadas. Sin la regla, demostrar eso requeriría inspeccionar configs en cada deployment —imposible a escala.

Con la regla: el binary mismo es la prueba. "Si lo bajaste, lo estás usando."

36.6. Resumen

- Plug-and-play: cada optimización entregada se activa por defecto al startup.
- Cinco pasos: default-on al startup, surface en UI, default-off sólo con causa, doc obliga, capabilities de build reflejadas.
- Justificada porque sin ella, optimizaciones quedan *shipped pero no usadas*.
- Casos donde casi se rompió: LFTJ S3 (name marker), Yannakakis bug temporal. Se priorizó arreglar el código sobre ofrecer opt-out.
- Auditabilidad: el binary mismo es la prueba.

Ejercicios

1. ★ Imaginá una nueva optimización: "cache de NLF compartido entre hipótesis similares". ¿Cumple los cinco pasos? Discutí qué necesita.
2. ¿Por qué el override `KEY=0` sigue funcionando? ¿Es contradictorio con la promesa plug-and-play?
3. ★ Diseñá un test automatizado que verifique "ninguna optimización merge-eada requiere setear flag manualmente". ¿Qué chequearía?
4. Casos donde la regla *no* aplica: optimizaciones que sí necesitan opt-in del usuario por su naturaleza (riesgosa, costosa). Pensá un ejemplo.
5. Si una optimización tiene un *soak window* de 2 semanas, ¿qué métricas decidirían que ya está listo para flip a default-on?

Capítulo 37

Instrumentación del frontend

Intuición

La promesa plug-and-play (capítulo anterior) dice "cada optimización se nota inmediatamente". Para que "se note", el frontend tiene que mostrarla. La instrumentación que GraphHunter expone vive en tres canales: `cmd_get_perf_status` (qué optimizaciones están activas), `cmd_get_memory_status` (presión de memoria), y la respuesta de `run_hunt_batch` (cache hits del último batch). Las pildoras del toolbar resumen los tres en un vistazo. Este capítulo cierra el libro mostrando cómo se materializa la promesa.

37.1. Las pildoras del toolbar



Figura 37.1: Pildoras del toolbar. La de *perf* indica si los flags M+L1 están activos (verde) o si el usuario forzó el legacy path (gris). La de *mem* cambia de verde a ámbar ($\geq 75\,\%$ RSS/total) a rojo ($\geq 90\,\%$).

37.2. `cmd_get_perf_status`

El comando Tauri que el frontend invoca al mount:

```
#[tauri::command]
fn cmd_get_perf_status() -> PerfStatus {
    PerfStatus {
        lftj_planner: env_bool("GRAPHHUNTER_LFTJ_PLANNER", true),
        k4_lftj_marker: env_bool("GRAPHHUNTER_K4_LFTJ", true),
        yannakakis: env_bool("GRAPHHUNTER_YANNAKAKIS", true),
        // build capabilities
        has_nauty: cfg!(feature = "symmetry-nauty"),
        has_simd: cfg!(feature = "simd-intersect"),
    }
}
```

La struct se serializa a JSON y el frontend la lee:

```
type PerfStatus = {
    lftj_planner: boolean;
    k4_lftj_marker: boolean;
    yannakakis: boolean;
    has_nauty: boolean;
```

```
    has_simd: boolean;
};
```

37.2.1. Render de la pildora

`apps/tauri/src/components/PerfPill.tsx`:

- Si todos los flags estructurales (`lftj_planner`, `yannakakis`) están `true`: pildora "Perf: M+L1" en verde acento.
- Si alguno está `false`: pildora "Perf: legacy" en gris.
- Tooltip on-hover: lista los flags individuales y su estado actual.

37.3. `cmd_get_memory_status`

```
#[tauri::command]
fn cmd_get_memory_status() -> MemoryStatus {
    let process = sysinfo::Process::current();
    let host = sysinfo::System::new();
    MemoryStatus {
        rss_bytes: process.memory(),
        host_total_bytes: host.total_memory(),
        ratio: process.memory() as f64 / host.total_memory() as f64,
    }
}
```

El frontend lo polea cada 5 segundos y actualiza la pildora:

- Ratio < 0.5: verde, sin warning.
- Ratio $\in [0.75, 0.90)$: ámbar, "mem alta". Sugiere cerrar otras apps o reducir el dataset.
- Ratio ≥ 0.90: rojo, "crítica". Recomienda `session_save` y restart.

La idea: el analista ve la presión *antes* de que el motor entre en swap (que serían minutos de UI freeze).

37.4. Telemetría adicional en `run_hunt_batch`

El endpoint `run_hunt_batch` (L2) —para correr varios hunts en una pasada compartiendo bag/result caches— retorna una respuesta extendida con cache counters:

```
pub struct RunHuntBatchResponse {
    pub entries: Vec<RunHuntBatchEntry>,
    pub indexes_were_warm: bool,
    pub bag_cache_hits: usize,          // L2 stage 2
    pub bag_cache_misses: usize,
    pub result_cache_hits: usize,       // L2 stage 3a
    pub last_prefix_resume_depth: usize,  // L2 stage 3b
    pub prefix_cache_hits: usize,
}
```

Cada campo es una métrica que el frontend muestra como tooltip sobre la pildora de perf. El analista que pasa el mouse ve:

- "Bag cache: 87 % hit ratio"
- "Result cache: 12 hits, 1 miss"
- "Last prefix resumed at depth 2"

Si los hits son altos, las optimizaciones L2 están trabajando. Si son cero, hay algo que investigar (¿hipótesis tan distintas que no comparten nada? ¿caches sub-dimensionados?).

37.5. Doc autocompliance

El procedimiento de agregar un flag nuevo:

1. Implementar la optimización en `core`.
2. Agregar default-on en `apps/tauri/src-tauri/src/lib.rs` y `apps/cli/src/main.rs`.
3. Extender `PerfStatus` con el nuevo campo.
4. Extender `PerfPill.tsx` para reflejar el flag.
5. Actualizar `08-env-vars.tex` y `12-preguntas-frecuentes.tex`.
6. `make all` para regenerar PDFs.
7. Si pasa CI con los seis PDFs frescos ⇒ merge.

El último punto es self-enforcing: si la doc no se regeneró, CI falla.

37.6. El cierre del libro

Esta es la última pieza pedagógica del diseño. Los próximos pasos para el lector:

- *Manual* (libro paralelo): cómo el analista usa el motor en su workflow diario.
- *Especificación*: contracts HTTP, MCP, API pública del matcher en Rust, formatos de log soportados.
- *Arquitectura*: estructura de crates, ADRs, decisiones de *repo organization*.
- *Paper*: justificación académica con benchmarks y comparaciones contra el estado del arte.

Este libro es la guía pedagógica que une los cinco. Si llegaste hasta acá, deberías poder leer cualquiera de los otros sin dificultad —y, más importante, contribuir al motor con optimizaciones nuevas siguiendo los patrones que el libro documenta.

37.7. Resumen

- Tres comandos de instrumentación: `cmd_get_perf_status`, `cmd_get_memory_status`, telemetría en `run_hunt_batch`.
- Pildoras del toolbar: perf (verde/gris), mem (verde/ámbar/rojo).
- Cache counters en batch: visible vía tooltip.
- Procedimiento "agregar un flag nuevo" obliga a actualizar PDFs; CI lo enforce.

Ejercicios

1. ★ Diseñá una pildora más para el toolbar: “Build: AVX2” que indique si el binary tiene SIMD activado. ¿Qué necesita en `PerfStatus`?
2. Si la pildora de mem se vuelve roja durante un hunt, ¿qué debería hacer el frontend? Discutí: ¿abortar automáticamente? ¿alertar y dejar al usuario decidir?
3. ★ El polling de memoria es cada 5 segundos. ¿Por qué no más frecuente? ¿Por qué no menos?
4. Diseñá una métrica *nueva* que valga la pena exponer: por ejemplo, “aristas/seg que está procesando el ingest”. ¿Dónde la calculás? ¿Cómo la mostrás?
5. ¿Por qué la doc se regenera en CI y no manualmente cuando el dev quiere? Pensá: ¿qué se rompe sin regeneración automática?

Apéndice A

Notación

Referencia rápida de la notación usada en todo el libro. Para convenciones tipográficas (cajas, fuente monoespaciada, énfasis), ver *Convenciones* en la sección frontal.

A.1. Grafos y vértices

Símbolo	Tipo	Significado
$G = (\mathcal{V}, \mathcal{E})$	grafo	grafo dirigido base; $\mathcal{V}$ vértices, $\mathcal{E}$ aristas.
$G = (\mathcal{V}, \mathcal{E}, t)$	grafo temporal	cada arista $e \in \mathcal{E}$ lleva un `timestamp` $t(e) \in \mathcal{T}$.
$\mathcal{V}$	conjunto	vértices del grafo. $\lvert\mathcal{V}\rvert = n$.
$\mathcal{E}$	multiconjunto	aristas dirigidas; en multigrafos, varias (u, v, t) pueden coexistir. $\lvert\mathcal{E}\rvert = m$.
u, v, w, x	vértices	elementos genéricos de $\mathcal{V}$.
$e = (u, v, t)$	arista temporal	del vértice origen u al destino v, con *timestamp* t.
$\mathcal{T}$	dominio temporal	en el motor: `i64` segundos epoch (Unix time).
$\mathcal{L}_V$, $\mathcal{L}_E$	alfabeto	etiquetas de vértices (`EntityType`) y aristas (`RelationType`). $\mathcal{L} = \mathcal{L}_V \cup \mathcal{L}_E$.
type(v)	función	$\mathcal{V} \to \mathcal{L}_V$; el `EntityType` del vértice v.
type(e)	función	$\mathcal{E} \to \mathcal{L}_E$; el `RelationType` de la arista e.

A.2. Cardinalidades, grados y vecindarios

Símbolo	Tipo	Significado
$n = \lvert\mathcal{V}\rvert$	escalar	cantidad de vértices.
$m = \lvert\mathcal{E}\rvert$	escalar	cantidad de aristas.
$\deg(v)$	escalar	grado total de v (entradas + salidas).

in-deg(v)	escalar	grado de entrada: $\lvert\{u : (u, v, \cdot) \in \mathcal{E}\}\rvert$.
out-deg(v)	escalar	grado de salida: $\lvert\{w : (v, w, \cdot) \in \mathcal{E}\}\rvert$.
$N(v)$	subconjunto de $\mathcal{V}$	vecinos de v (en cualquier dirección).
$N^+(v)$	subconjunto de $\mathcal{V}$	vecinos salientes: $\{w : (v, w, \cdot) \in \mathcal{E}\}$.
$N^-(v)$	subconjunto de $\mathcal{V}$	vecinos entrantes.
$\bar{d}$	escalar	grado promedio: $\bar{d} = m/n$.
Δ	escalar	grado máximo: $\max_{v \in \mathcal{V}} \deg(v)$.

A.3. Hipótesis, joins, y enumeración

Símbolo	**Tipo**	**Significado**
Q, H	hipótesis	consulta *subgraph matching*; en GraphHunter, `Hypothesis`.
k	escalar	longitud de la hipótesis: número de pasos / aristas.
σ	asignación	mapeo de variables de la hipótesis a vértices del grafo.
R, R_i	relación	multiconjunto de tuplas; en el motor, una *bag* B_i.
$R \bowtie S$	relación	*join* natural sobre las columnas compartidas.
$R \ltimes S$	relación	*semi-join*: filas de R cuya *join key* aparece en S.
$\sigma_P(R)$	relación	selección: filas de R que satisfacen el predicado P.
$\pi_X(R)$	relación	proyección a las columnas X.
$\rho^*(Q)$	escalar	*fractional edge cover number* de Q (capítulo 20).
AGM(Q)	escalar	cota *Atserias-Grohe-Marx* sobre el tamaño de la salida: $m^{\rho^*(Q)}$.

A.4. Índices y ventanas temporales

Símbolo	**Tipo**	**Significado**
NLF(v, L)	escalar	vecinos salientes de v con etiqueta $L \in \mathcal{L}_V$.
bits[v]	`u16`	bitmap de tipos alcanzables desde v en $\leq$ `MAX_DEPTH` saltos.

min_first$[v][L]$	`i64`	*timestamp* mínimo del primer *edge* de algún camino $v \rightsquigarrow L$.
max_first$[v][L]$	`i64`	análogo para el máximo.
rank$[v]$	`u32`	índice del vértice v en la fila *sparse* de `min/max_first`.
$[t_{\text{lo}}, t_{\text{hi}})$	intervalo en $\mathcal{T}$	ventana temporal de la consulta (cláusula `WITHIN`).
$\Delta t = t_{\text{hi}} - t_{\text{lo}}$	escalar	ancho de la ventana.
`MAX_DEPTH`	constante	profundidad máxima de *khop_reach*: 4 en GraphHunter.
`NLF_TYPE_COUNT`	constante	cantidad de `EntityType`s con *tag* $[0, 9)$: $\lvert\mathcal{L}_V\rvert$.

A.5. Tipos del motor (Rust) que aparecen recurrentemente

Tipo	Significado
`Entity`	nodo del grafo (público, humano): `id: String`, `entity_type: EntityType`, `metadata`.
`Relation`	arista del grafo (público): `source_id`, `dest_id`, `rel_type`, `timestamp`, `metadata`.
`CompactRelation`	arista compacta (interna): 32 bytes, `#[repr(C)]`. Capítulo 10.
`StrId`	`u32` compacto producido por el *interner*.
`StringInterner`	tabla bidireccional `String` ↔ `StrId`.
`Hypothesis`	AST de una consulta DSL: `steps`, `shape`, `window_seconds`, etc.
`HypothesisShape`	enum `Chain | Graph`; lo lee el *planner*.
`HuntResult`	`Vec<Arc<str>>`; un *path* resuelto al alfabeto humano.
`NlfTable`	estructura del índice NLF; *lazy build* en `OnceLock`.
`KHopReach`	estructura del índice *k-hop* con extensión temporal.
`IndexBacking<T>`	*wrapper heap-resident* ↔ *mmap-spilled* para tablas grandes.
`InMemoryStreamingBackend`	edge store *degree-adaptive* per-vertex con auto-spill incremental.
`DispatchPlan`	enum del *planner*: `DfsMatch | K4Lftj | Yannakakis`.

A.6. Convenciones del pseudocódigo

Reproducidas acá para referencia rápida (la versión narrativa está en *Convenciones*, sección frontal):

- Asignación: $x \leftarrow y$. Igualdad: $x = y$. Acceso a campo: v.campo.
- $\leftarrow$ y $\Leftarrow$ ambos significan asignación; $\Leftarrow$ sólo en derivaciones de variables agregadas.
- `for/while/if/else/end`: bloques delimitados por indentación + cierre explícito.
- `return`: sale del procedimiento; `emit`: agrega un resultado a la salida sin retornar.
- Conjuntos: $\{x : P(x)\}$, $A \cup B$, $A \cap B$, $A \setminus B$, $\emptyset$.
- *Bitwise*: $a \mid b$, $a \models b$, $a \,\&\, b$. Detalles cuando aparezcan.

Apéndice B

Glosario

Términos técnicos del libro, ordenados alfabéticamente. La columna "Cap." apunta al capítulo donde se define o desarrolla.

A

α-aciclicidad Propiedad de un hipergrafo que se reduce a vacío mediante el algoritmo GYO. Equivalente a la existencia de un *join tree*. Es la condición necesaria para que Yannakakis funcione en $O(\text{input} + \text{output})$. Cap. 22.

AGM **bound** Cota Atserias-Grohe-Marx (2008): para todo join Q con relaciones de tamaño $\leq m$, el output es $\leq m^{\rho^*(Q)}$. Cap. 4.

`add_relation` Función que agrega una arista al grafo. Idempotente para entidades, append para relaciones; *single-write* al `streaming` backend (post stage 2c part 2 *rip-out* del antiguo dual-write). Cap. 12.

`Arc<str>` Atomic reference-counted pointer a una string en heap. Permite clonado por refcount bump ($\sim$ 5 ns) en lugar de byte copy. La elección que destrabó el 55 % en cap-saturated benches. Cap. 11.

B

Bag (B_i) Lista de aristas que matchean el step i de una hipótesis. Materializadas por Yannakakis chain antes del semi-join reducer. Cap. 23.

Backtracking Patrón `push/recurse/pop` que mantiene una pila de bindings durante el DFS, restaurándola al volver de cada rama. Cap. 16.

Binding Identificador en la DSL que referencia un vértice (`u`, `c`, `h1`). Internamente se convierte a `VarId` numérico. Caps. 30, 32.

bits[v] Bitmap `u16` indexado por `EntityType`: bit L prendido sii existe walk de v a algún vértice de tipo L en $\leq$ `MAX_DEPTH` saltos. Caps. 19, 28.

Branch prediction Mecanismo de la CPU para anticipar el resultado de un `if` y ejecutar especulativamente. Costo de mispredict: 15–25 ciclos. Justifica diseño de hot loops con pocos branches dependientes de datos. Cap. 5.

C

Cache line Unidad de transferencia entre RAM y CPU cache. 64 B en x86_64 modernos. Justifica el tamaño 32 B de `CompactRelation` (dos por línea). Caps. 5, 10.

Cap (de hunt) Cantidad máxima de `HuntResult`s que un hunt produce. Default 10 000. Truncate explícito para evitar que un hub explote la salida. Cap. 11.

Cascade budget Umbral en *Delta WCOJ Propagation* que aborta la propagación si visited supera el threshold (default 100K). Protege contra hubs patológicos. Cap. 34.

`CompactRelation` Estructura interna de arista, 32 bytes `#[repr(C)]`, dos por cache line, mmap-friendly. Cap. 10.

D

Default-on Convención plug-and-play: cada flag de optimización ships con su default a 1, sin requerir acción del usuario. Cap. 36.

Delta WCOJ Propagation Algoritmo de Kankanamge-Sahu-Mhedhbi-Salihoglu (SIGMOD 2017) para actualización incremental de k-hop reachability. Cap. 34.

DFS Depth-first search. El algoritmo baseline del matcher; opera con backtracking y prunes (NLF, k-hop, symmetry breaking). Cap. 16.

`DispatchPlan` Enum del planner: `DfsMatch`, `K4Lftj`, `Yannakakis`. Cap. 25.

Dual-write Patrón actual de `add_relation`: escribe al `edge_store` y al streaming backend. Stage 2c (futuro) lo elimina. Caps. 13, 35.

E

`Entity` Tipo público de nodo del grafo: {`id: String`, `entity_type: EntityType`, `metadata`}. Cap. 8.

`EntityType` Enum de 9 variantes (`User`, `Host`, `Process`, `IP`, `File`, `Domain`, `Service`, `Resource`, `Group`, más `Other`). Tag numérico explícito para indexación. Cap. 8.

F

Falsos positivos del prune El prune deja pasar candidatos que el matcher después descarta. Inevitable bajo sobre-aproximación; el matcher absorbe el costo. Cap. 26.

Finalize (`sort_edges_by_timestamp`) Función que cierra el grafo: ordena aristas por (`source_sid`, `timestamp`) marca sealed, resetea índices a `None` para lazy build. Cap. 14.

Fractional edge cover number (ρ^*) Mínimo total de pesos en una cobertura fraccionaria del hipergrafo. Determina el AGM bound. Cap. 4.

G

GYO Algoritmo de reducción de hipergrafo (Graham 1979, Yu-Ozsoyoglu 1979) que detecta α-aciclicidad por aplicación repetida de dos reglas (eliminar atributo aislado, eliminar hiperarista contenida). Cap. 22.

Grafo dirigido temporal etiquetado Tupla $(\mathcal{V}, \mathcal{E}, \text{type}_V, \text{type}_E, t)$. El modelo matemático sobre el que opera GraphHunter. Cap. 2.

H

`HAVING` Cláusula trailing de la DSL para agregaciones (`count(distinct binding) >= N`). Cap. 30.

Hipergrafo de la consulta Estructura $(\mathcal{A}, F)$ donde $\mathcal{A}$ son los atributos y F las relaciones (cada una hiperarista). Permite aplicar GYO. Cap. 22.

Homomorfismo Función $\sigma : V_P \to \mathcal{V}$ que respeta etiquetas y aristas, pero *puede colapsar* distintos vértices del patrón al mismo vértice del grafo. Cap. 3.

`HuntResult` Tipo público del resultado de un hunt: `Vec<Arc<str>>`, secuencia de identidades del path encontrado. Cap. 11.

`Hypothesis` Struct AST de la DSL: `steps`, `shape`, `k_simplicity`, `aggregations`, `window_seconds`. Cap. 32.

I

Idempotencia `add_entity` es idempotente: agregar la misma entity dos veces no la duplica. Cap. 8.

`IndexBacking<T>` Wrapper genérico heap-resident / mmap-spilled para tablas grandes. Reusado por NLF y k-hop. Cap. 26.

Ingest El pipeline log → grafo. Tres etapas: `LogSource`, `LogParser`, `GraphWriter`. Cap. 12.

Invariante Propiedad que se mantiene después de cada operación. El motor las testea con property tests.

Isomorfismo (subgraph) Embedding inyectivo: los distintos vértices del patrón mapean a vértices distintos del grafo. Cap. 3.

J

Join (natural) Operación relacional que junta dos tablas matcheando los valores de las columnas compartidas. Cap. 4.

Join tree Árbol cuyos nodos son las relaciones del join, tal que para cada atributo, el conjunto de relaciones que lo contienen forma un sub-árbol conexo. Existe sii el hipergrafo es α-acíclico. Cap. 22.

K

K_4 Grafo completo de 4 vértices, las $\binom{4}{2} \cdot 2 = 12$ aristas dirigidas. El patrón canónico para LFTJ. Cap. 21.

k-simplicidad Cota: ningún vértice se repite más de k veces en el path. Default $k = 1$ (path simple). Cap. 2.

`KHopReach` Estructura del índice k-hop reachability: `bits` dense, `rank` dense, `min/max_first` sparse. Cap. 28.

L

Lazy build Patrón en el que un índice se construye al primer consumer que lo necesita, no eagerly al sealing. Implementado vía `OnceLock<T>`. Caps. 14, 26.

Leapfrog intersection Operación central de LFTJ. Mantiene k punteros sobre listas ordenadas, avanzando el más atrasado hasta superar al más adelantado. Cap. 21.

LFTJ Leapfrog Triejoin (Veldhuizen ICDT 2014). WCOJ implementable que GraphHunter aplica para K_4. Cap. 21.

`LogParser` Trait que convierte un chunk crudo en `Vec<ParsedTriple>`. Cap. 12.

`LogSource` Trait que entrega chunks de una fuente (Sentinel, Defender, EVTX, NDJSON local). Cap. 12.

M

`MAX_DEPTH` Profundidad máxima del índice k-hop reachability. En GraphHunter, 4. Cap. 19.

`MetadataStore` Almacenamiento append-only de metadata de aristas, accedido vía `CompactRelation::metadat` Cap. 10.

min_first, max_first Matrices sparse ($|\mathcal{V}_{\text{ranked}}| \times |\mathcal{L}_V|$) que guardan los timestamps extremos del primer edge de algún camino $v \rightsquigarrow L$. Cap. 28.

mmap (memory-mapped file) Asociación entre páginas virtuales y bytes de un archivo en disco. Permite leer archivos sin alocar RAM proporcional. Cap. 5.

Multigrafo Grafo donde puede haber varias aristas entre el mismo par de vértices (con distintos timestamps o metadata). El modelo correcto para threat hunting. Cap. 2.

N

`NeighborList` Estructura per-vértice del streaming backend: enum con tres variantes (`Inline`, `Vec`, `Btree`) según degree. Cap. 13.

`NLF` (NLF) Neighborhood Label Frequency. $\text{NLF}(v, L)$ cuenta vecinos salientes de v de tipo L. Caps. 18, 27.

O

`OnceLock<T>` Primitiva stdlib de Rust para inicialización lazy thread-safe exactamente-una. Backbone del lazy build. Cap. 14.

P

Path simple Walk donde todos los vértices son distintos. Equivalente a 1-simplicidad. Cap. 2.

Planner Función `plan(hypothesis)` que devuelve `DispatchPlan`. Cap. 25.

Plug-and-play Promesa transversal: cada optimización entregada está activa por defecto sin acción del usuario. Cap. 36.

Predicate Restricción sobre un campo de metadata o un vértice. Cap. 30.

Prune Filtro que descarta candidatos antes del DFS para reducir trabajo. NLF, k-hop, symmetry breaking. Caps. 18, 19.

R

`rank` Array dense (`Vec<u32>`) que indexa la fila sparse de `min/max_first`. `NO_RANK` si out-deg = 0. Cap. 28.

`Relation` Tipo público de arista del grafo. Cap. 8.

`repr(C)` Atributo de Rust que fuerza layout estable estilo C: orden declarado, padding explícito, alineación predecible. Cap. 10.

S

Saturating add Operación aritmética que se queda en `u32::MAX` en vez de overflowar. Convención de NLF. Cap. 15.

Scale-free Distribución de grado power-law: la mayoría de los vértices tienen grado bajo, una minoría tiene grado alto (hubs). Caracteriza los grafos reales de threat hunting. Cap. 17.

Semi-join ($R \ltimes S$) Filas de R cuya *join key* aparece en S. La operación central del Yannakakis reducer. Cap. 23.

Shape (de hipótesis) `HypothesisShape::Chain` o `HypothesisShape::Graph`. Determina el sub-matcher de Yannakakis. Cap. 31.

SIMD Single Instruction, Multiple Data. Instrucciones AVX2 para operar sobre 8 `u32`s en una sola instrucción. GraphHunter las usa via `core/graph-engine/src/simd\_rust.rs` (Rust puro, runtime dispatch) para LFTJ intersection. Caps. 5, 21.

Soundness (de prune) El prune nunca descarta candidatos válidos. Garantizada por sobreaproximación del índice. Cap. 26.

`streaming backend` `InMemoryStreamingBackend` per-vertex *degree-adaptive* (`Inline` / `Vec` / `Mmap`) con auto-spill incremental. *Canonical edge store* desde stage 2c part 2; reemplazó al antiguo `SpillableEdgeStore`. Caps. 13, 35.

`StringInterner` Tabla bidireccional `String` ↔ `StrId`. Backing con `Arc<str>`. Cap. 9.

Streaming backend Mirror per-vértice del grafo optimizado para wait-free reads. Hoy mirror del edge_store; futura source of truth. Cap. 13.

Subgraph matching Encontrar todas las formas de embeber un patrón P en un grafo G. NP-completo en general, tractable con etiquetas + ventana temporal + prunes. Cap. 3.

Symmetry breaking Restricción adicional (e.g. $a < b < c < d$) que evita enumerar la misma estructura por cada permutación de sus simetrías. Cap. 21.

T

Tree-walk enumerator Versión de Yannakakis para shape graph: walk recursivo sobre el join tree con switch `(origin_bound, dest_bound)`. Cap. 24.

Trie Estructura de árbol indexada por prefijos. Usada por LFTJ para soportar leapfrog intersection. Cap. 21.

`try_record_edge` Función de NLF que aplica un saturating-add incremental cuando el backing es heap; devuelve `false` si está mmap-spilled. Cap. 15.

V

`VarId` Índice numérico para variables (bindings) en una hipótesis. Único por hipótesis. Cap. 32.

W

Walk Secuencia de vértices conectados por aristas, donde los vértices *pueden repetirse*. Distinto de path simple. Cap. 2.

Wait-free reads Garantía de que los lectores nunca bloquean. El streaming backend lo logra con `ArcSwap<NeighborList>`. Cap. 13.

`WITHIN` Cláusula trailing de la DSL que define la ventana temporal Δt. Cap. 30.

Worst-case-optimal join (WCOJ) Algoritmo de join cuyo tiempo se ajusta al AGM bound ($O(m^{\rho^*})$). LFTJ es uno; los joins binarios secuenciales clásicos no. Cap. 20.

Y

Yannakakis Algoritmo de Yannakakis (1981) para joins α-acíclicos: semi-join reducer en dos pasadas + enumeración. O(input + output). Caps. 23, 24.

Apéndice C

Soluciones a ejercicios selectos

Soluciones a los ejercicios marcados con ★ en cada capítulo. Los ejercicios sin ★ quedan abiertos deliberadamente: la idea es que el lector se trabe un poco antes de pedir ayuda.

Capítulo 1: Threat hunting

Solución C.1. "Un usuario que se loguea en un host, desde donde se loguea en otro host, desde donde se conecta a una IP nunca vista, y desde esa IP hay tráfico saliente a un dominio externo." Tipos: `User`, `Host`, `Host`, `IP`, `Domain`. Relaciones: `Auth`, `Auth`, `Connect`, `Beacon` (o similar).

Solución C.3. *Nested-loop* naive: $20\,M \times 20\,M \times 20\,M = 8 \times 10^{21}$ comparaciones. Inviable. Con hash join e índices en RAM: $O(20\,M)$ por join, secuencial $\Rightarrow$ tres joins $\sim 60\,M$ operaciones más cardinalidad intermedia. La cardinalidad es lo que mata: en peor caso, $20M^{1.5}$ o más, varios GB de filas intermedias.

Capítulo 2: Grafos temporales

Solución C.1. Walks (sin restricción temporal): (u_1, h_1, ip_1) ts $(1, 4)$ y $(1, 9)$ (dos: una por cada arista $h_1 \to ip_1$), (u_1, h_1, ip_2) ts $(1, 7)$, (u_2, h_1, ip_1) ts $(2, 4)$ y $(2, 9)$, (u_2, h_1, ip_2) ts $(2, 7)$, (u_2, h_2, ip_2) ts $(5, 6)$. Total: 7 walks.

Solución C.4. Δt-conexidad NO es simétrica (la arista $u \to v$ no implica una $v \to u$) ni transitiva en general (un walk $u \rightsquigarrow v$ y otro $v \rightsquigarrow w$ pueden empezar muy separados en el tiempo y juntos exceder Δt). La reflexividad triviales valdría definirla; pero la asimetría y no-transitividad ya descalifican como equivalencia.

Capítulo 3: Subgraph matching

Solución C.1. En el grafo del cap. 2 no hay triángulos dirigidos (cada vértice es de un tipo distinto al adyacente). Bajo isomorfismo: 0. Bajo homomorfismo: también 0 (la estructura de `User-Host-IP` no permite ciclo).

Solución C.3. Para una cadena de k aristas, $\rho^* = k$. El AGM predice salida $\leq m^k$. El bound trivial $O(\deg^k)$ requiere conocer deg, que en el peor caso (densidad constante) es $O(\sqrt{m}) \Rightarrow O(m^{k/2})$. Pero AGM da m^k *multiplicativo del start count*. Para el peor caso "todos los vértices con grado uniforme" ambos coinciden hasta constantes. La cota AGM es ajustada via el ejemplo del ejercicio 3 del cap. 4.

Capítulo 4: Joins y AGM

Solución C.1. Estrella de k brazos: $R_0(A, B_1) \bowtie R_1(A, B_2) \bowtie \ldots$. A aparece en todas las k hiperaristas; cubrir A requiere $\sum x_i \geq 1$. Cada B_i aparece sólo en $R_i \Rightarrow x_i \geq 1$. Total mínimo: $\sum x_i = k$. Pero como $\sum x_i \geq 1$ ya cubre A, el peor cae con $x_i = 1$ para cada relación. $\rho^* = k$. En grafos reales con A muy selectivo, el bound trivial da $\deg(A)$; rara vez ajustado.

Solución C.3. Construcción adversarial: tomar $R = S = T = \{(i, i+1) : i \in [m]\}$ (m aristas en cadena lineal). Entonces el join produce exactamente $m - k$ tuplas (cada path de k saltos consecutivos). Pero el AGM bound dice m^k. Para acercar: $R, S, T = \{(i, j) : i \in [\sqrt{m}], j \in [\sqrt{m}]\}$ (producto cartesiano de raíz). Cada relación tiene m tuplas; el join produce $\sqrt{m}^k = m^{k/2}$. Estricto bajo el bound m^k. Para alcanzar m^k hace falta una construcción más balanceada.

Capítulo 5: Sistemas

Solución C.1. Acceso secuencial: *bueno*. La cache line se reusa al cargar la próxima instancia. Acceso desde threads distintos: *malo* (*false sharing*): si el thread A escribe en el campo X de la instancia 1 y el thread B escribe en el campo Y de la instancia 2, ambos invalidan la misma cache line en el otro CPU, pingpong constante. Mitigación: padding al múltiplo de cache line cuando se sabe que las instancias se acceden desde threads distintos.

Solución C.3. Open addressing + linear probing: array contiguo de buckets; inserts y lookups recorren linealmente desde hash(k) hasta encontrar el slot. Locality *mucho mejor* para load-factor bajo (< 0.5): el bucket inicial y los siguientes están en la misma cache line. Para load-factor alto, las colisiones forzan paseos largos ⇒ peor. La hash table "buena" que el motor usa internamente (`ahash`, `hashbrown`) hace open addressing por esto.

Capítulo 6: Pipeline

Solución C.1. `Ingesta`: principal en `platform/parsers` + `platform/sources`. `Hipótesis`: `platform/dsl`. `Matching`: `core/graph-engine`. `Scoring`: `core/graph-engine/scoring`. La única que vive 100 % en `core` es *matching*.

Solución C.4. Sí, hay forma. La respuesta de `run_hunt` incluye un campo `cap_saturated: bool`. El frontend lo lee y muestra el mensaje "Estos son los primeros 9999 (cap saturado)" o "Resultados completos: 9999".

Capítulo 7: Stack de crates

Solución C.1. Función IP → ASN. ¿Más de un transport? Sí (Tauri y MCP la querrán para enriquecer reports). ¿Contrato externo? Indirecto (consume MaxMind). ¿Algorítmico puro? No (hace I/O sobre la base MaxMind). ⇒ `platform/enrich` (nueva crate), NO `core`.

Solución C.4. `tracing` en hot path es problemático: cada evento es una asignación + serialización. Para 10K iteraciones del DFS, eso son 10K events ⇒ ∼1 ms de overhead. Si va en `core`, el motor depende de `tracing` (acoplamiento). Si va en `platform`, hay que pasar trace context al matcher. Solución pragmática: `tracing` *compile-time* (feature flag) en `core`; default-off, on para debugging.

Capítulo 8: Entity, Relation

Solución C.1. Sysmon Event ID 3 (Network Connection): genera 3 entidades (`Process`, `Host`, `IP`) y 2 relaciones (`Process -[Connect]-> IP` con metadata {`src_port`, `dst_port`, `protocol`}; `Host -[Hosts]-> Process`).

Solución C.4. Esquema fijo por `EntityType`: `ProcessMetadata { image, cmdline, md5, sha256 }`, `IpMetadata { asn, country }`, etc. *Ganás*: 50 % memoria (sin overhead de hashmap), lookup más rápido (offset directo). *Perdés*: extensibilidad sin recompilar; cualquier campo nuevo del log queda fuera. Trade-off por dominio.

Capítulo 9: String interner

Solución C.1. 10^8 aristas $\times$ 2 strings/arista $\times$ 40 B overhead per String = 8 GB. Con interner: $N \times 76$ B (la fórmula del cap.) $= 7.6 \times 10^7$ B = 76 MB. Ratio $\sim 100\times$.

Solución C.3. `Vec<(Arc<str>, StrId)>` ordenado por string. Insert: $O(N)$ por shift. Lookup: $O(\log N)$ por bsearch. Conviene sólo cuando los inserts son raros pero los lookups frecuentes (no es el caso aquí).

Capítulo 10: CompactRelation

Solución C.1. L2 = 1 MB = 1 048 576 B. $\div 32$ = 32 768 `CompactRelation`s. $\div 200$ = 5 242 `Relation`s. Ratio: 6.25×.

Solución C.3. Si quitás `repr(C)`, el compilador puede reordenar y *compactar* el struct. Mismo tamaño total —OK—. Lo que se rompe: el binary layout pierde estabilidad entre versiones del compilador. Un mmap escrito con compilador A leído con compilador B podría producir basura. La C ABI también requiere layout fijo. `repr(C)` es la garantía contractual.

Capítulo 11: HuntResult

Solución C.1. $k = 5$ saltos $\Rightarrow$ 6 vértices por path. Vec<String>: $10\,000 \times 6 \times (24\,B$ header + $24\,B$ bytes) = 2.88 MB. Vec<Arc<str»: $10\,000 \times 6 \times 16\,B = 0.96$ MB *más* las strings únicas que ya viven en el interner (no se duplican). Ahorro: 3× y elimina el costo de alloc por path.

Solución C.3. DoS por *cardinality blowup*: el atacante envía logs con identidades únicas (`user-12345-abc`, `user-12345-abd`, etc.). Cada string entra al interner como nueva entry; el interner crece sin bound. `Arc<str>` no mitiga esto —es un problema del interner, no del HuntResult—. Mitigación: `StringInterner` con cap absoluto (default 100M strings) y rechazo cuando se llena.

Capítulo 15: NLF incremental

Solución C.1. $10^7 \times 36$ B = 360 MB. $10^8 \times 36$ B = 3.6 GB. La segunda excede budget de 1 GB por más de 3×.

Capítulo 16: DFS

Solución C.1. Paths simples de longitud 3 desde A: $A \to B \to D$ (no extiende, D sin out-edges), $A \to B \to E$ (no extiende, E sin out-edges), $A \to C \to E$ (no extiende). Total de longitud exacta 3: 0. De longitud ≤ 3: 3 paths de longitud 2.

Solución C.3. $10^4 \cdot \deg^3 = 10^4 \cdot 100^3 = 10^{10}$ operaciones en peor caso. A 1 ns/op: 10 segundos.

Capítulo 17: Naive matching

Solución C.1. $10^4 \cdot 50^4 = 6.25 \cdot 10^{10}$ iteraciones. A 1 ns/op: 62 segundos.

Solución C.3. Pasar por hub vs vértice promedio: hub aporta 10^5 iteraciones por visita; vértice promedio aporta 5. Si el DFS visita el hub 100 veces: 10^7 iteraciones extra (+1000 %).

Capítulo 18: Prune NLF

Solución C.1. NLF$[u_1]$ = [`Host` $= 1, \dots$], NLF$[u_2]$ = [`Host` $= 2, \dots$] (autentica en h_1 y h_2), NLF$[h_1]$ = [`IP` $= 2, \dots$] (dos aristas a ip_1 + una a $ip_2 \Rightarrow$ tres), NLF$[h_2]$ = [`IP` $= 1, \dots$], NLF$[ip_i]$ = [$\dots$ todo 0]. Convención: cada fila tiene 9 columnas, casi todas 0.

Solución C.3. Mismo tipo en los 3 hijos: NLF$(s, L) \geq 3$. Tres tipos distintos L_1, L_2, L_3: NLF$(s, L_i) \geq 1$ para cada i. El motor chequea cada uno como `required`.

Capítulo 19: Prune k-hop

Solución C.1. Para `MAX_DEPTH = 2`, cada vértice tiene bits$[v]$ con bits prendidos según sus tipos alcanzables en ≤ 2 saltos. Para u_1 (autentica en h_1 y h_2): bit `Host` (vía h_i), bit `IP` (vía $h_i \to ip_j$). Bits = `0b001010` (asumiendo Host=1, IP=3).

Solución C.3. Adversarial: hipótesis con WITHIN angosto que el min_first sólo tracking "primer edge" no descarta porque el primer edge sí cae en la ventana, pero en realidad ningún path completo cabe. La sobre-aproximación es trade-off intencional.

Capítulo 20: WCOJ y AGM

Solución C.1. $G(1000, 0.05)$: $m = \binom{1000}{2} \cdot 0.05 \approx 25\,000$ aristas. Triángulos en expectation: $\binom{n}{3}p^3 = \binom{1000}{3} \cdot 0.05^3 \approx 20\,000$. AGM bound: $m^{1.5} \approx 4 \cdot 10^6$ —muy por encima.

Solución C.3. Hash join binario para $R \bowtie S$ (sobre B): cardinality intermedia hasta $|R| \cdot |S|$ si todos los B coinciden. Para $|R| = |S| = m$: hasta m^2. Sigue ahí cuando se joina con T, después se filtra —pero el peak intermedio fue m^2. Memory blowup.

Capítulo 21: LFTJ

Solución C.1. K_4s en $G(1000, 0.05)$: $\binom{n}{4}p^6 = \binom{1000}{4} \cdot 0.05^6 \approx 0.65$. Esperaba 1 K_4 en average. AGM bound: $m^2 = 6.25 \cdot 10^8$ —enormemente sobre-estimado.

Solución C.3. DFS para K_4 en peor caso: itera todos los starts · todos los segundo-vecinos · etc. $= O(n \cdot \deg^3) = O(n \cdot (m/n)^3) = O(m^3/n^2) \geq O(m^2)$ cuando $n \leq \sqrt{m}$ (graphs densos). El DFS bien-podado puede mejorar la constante pero no el bound asintótico sin información estructural extra.

Capítulo 22: Hipergrafos

Solución C.1. Estrella: hiperaristas $\{c, h_1\}, \{c, h_2\}, \{c, h_3\}$. h_i aparece sólo en $F_i \Rightarrow$ eliminar; quedan $\{c\}, \{c\}, \{c\}$. Iguales $\Rightarrow$ eliminar dos por contención. Queda $\{c\}$; eliminar c (aislado en una hiperarista). $F = \emptyset$. α-acíclica.

Solución C.3. Hipergrafo α-acíclico de 4 relaciones: una cadena lineal R_1, R_2, R_3, R_4 tiene 1 join tree (lineal). Una estrella con root R_0 y hojas R_1, R_2, R_3: 1 join tree. Combinaciones más complejas pueden tener varios; en general el join tree NO es único.

Capítulo 23: Yannakakis chain

Solución C.1. 50 matches de output, $|B_0| = 1000$, $|B_1| = 5000$. Bottom-up: B_0 queda con sólo las rows cuyo dest aparece como origin en B_1. En peor caso *ninguna* row se filtra (todas las 1000 tienen continuación). En mejor caso: B_0 reducido a ≤ 50 (sólo las que llevan a un match). Top-down: similar.

Solución C.3. Si B_0 queda vacío: no hay starts, el enumerator no produce nada. Demuestra: B_0 vacío post-bottom-up significa que ninguna row tiene continuación; para que un match exista, su row inicial debe estar en B_0 y tener continuación. Contradicción. Converse NO vale: B_0 no vacío no garantiza matches; algunos rows pueden tener continuación parcial pero fallar la cadena completa.

Capítulo 24: Yannakakis tree-walk

Solución C.1. c tiene 100 vecinos `Host`. Para `c -[Auth]-> h1; c -[Auth]-> h2`: cada combinación ordenada de 2 hosts distintos cuenta como match $\Rightarrow 100 \times 99 = 9\,900$ matches. Cota AGM: estrella con $\rho^* = 2$, input $m = 100$ per relation, output $\leq m^2 = 10^4$. Cerca del bound.

Solución C.3. Caso `(Some, Some)`: hipótesis donde el step actual tiene ambos extremos ya bound. Por ejemplo, hipótesis triangular `a -[E1]-> b; b -[E2]-> c; a -[E3]-> c`: el último step `a -[E3]-> c` tiene a bound (de step 0) y c bound (de step 1). Lookup determinista: ¿existe la arista? Si sí, recurse; si no, fail.

Capítulo 25: Planner

Solución C.1. `User -[Auth]-> Host -[Connect]-> IP`: shape Chain, α-acíclica $\Rightarrow$ Yannakakis chain. `c -[Knows]-> p1; c -[Knows]-> p2; p1 -[Knows]-> p2`: shape Graph, cíclica

(triángulo) ⇒ DFS (no aplica LFTJ, no aplica Yannakakis). "k4-clique" con 6 pasos heterogéneos: name marker dispatch ⇒ K4Lftj (back-compat); pero structural_dispatch fallaría por heterogeneidad. El name marker prevalece.

Solución C.3. Detector de estrella de k brazos: chequear que la hipótesis shape graph tenga exactamente k cláusulas, todas con un binding compartido como root, y los demás vértices únicos. Función booleana: `is_star(hyp, k) -> bool`.

Capítulo 26: Diseño índices

Solución C.1. La sesión NO se rompe; el grafo canónico (`edge_store`) sobrevive. Próximo hunt: `ensure_nlf` reconstruye desde cero. Costo: ∼30 segundos extra. Funcionalidad intacta.

Solución C.3. `first_seen[v]`: `Vec<i64>` indexado por sid. Construye con una pasada $O(|\mathcal{E}|)$ tomando min de timestamps para cada source. Update incremental: cada `add_relation` hace `first_seen[v] = min(first_seen[v], cr.timestamp)`. Trivial, lineal, atómico-free single-writer.

Capítulo 27: NLF en detalle

Solución C.1. $5 \cdot 10^7 \cdot 36$ B = 1.8 GB. Excede budget default 1 GB ⇒ spillea a mmap.

Solución C.3. NLP (Neighborhood Label Profile): `[u32; 9 × 16]` si tenemos 16 RelTypes. = 576 B per row. ∼ 16× más memoria. Para 10^7 vértices: 5.4 GB —supera budget. Sólo viable con dense+sparse layout o sin spill.

Capítulo 28: k-hop en detalle

Solución C.1. $|\mathcal{V}| = 5 \cdot 10^7$, 50 % ranked ⇒ $|\mathcal{V}_{\text{ranked}}| = 2.5 \cdot 10^7$. bits: 100 MB. rank: 200 MB. min/max_first sparse: $2 \cdot 2.5 \cdot 10^7 \cdot 72$ B = 3.6 GB. Total: 3.9 GB.

Solución C.3. Adversarial: hipótesis con WITHIN angosto pero min/max_first$[v][L]$ wide. El primer edge cae en la ventana (no descartado), pero el segundo edge (que el motor todavía no chequea con este índice) cae fuera. El DFS lo descarta después de empezar. Trade-off intencional.

Capítulo 29: Diseñando una DSL chica

Solución C.1. `Process p -[Write]-> File f; f -[Connect]-> IP WITHIN 5m`. Componentes: cadena `p -> f -> IP` (graph shape por el `;`), ventana `WITHIN 5m`, sin predicados.

Capítulo 30: Sintaxis chain

Solución C.1. `User u -[Auth]-> Host h WITHIN 10m HAVING count(distinct h) >= 3`. Cláusulas: WITHIN para la ventana, HAVING para la cota.

Solución C.3. `Hypothesis` con 2 `HypothesisSteps`: step 0 con `source_type=Process, dest_type=Process, rel=Spawn`; step 1 con `source_type=Process, dest_type=File, rel=Write`. `shape=Chain`, `window_seconds=`, `k_simplicity=1`, `aggregations=vec![]`.

Capítulo 31: Sintaxis graph

Solución C.1. `User u -[Spawn]-> Process p; p -[Write]-> File f; p -[Spawn]-> Process c`. Bindings compartidos: p aparece en cláusulas 1, 2, 3 (root del join tree); u, f, c sólo en una cláusula cada uno.

Solución C.3. `User c -[Auth]-> Host h; c -[Auth]-> Host h`: dos cláusulas *idénticas*. Legal. El matcher trata ambas como restricciones sobre los mismos bindings ⇒ equivalente a una sola cláusula. Output: matches sin duplicación.

Capítulo 32: AST al matcher

Solución C.1. `Hypothesis{ steps: [Step(c, Auth, h1), Step(c, Auth, h2)], shape: Graph, ...}`. `VarIds`: $c \to 0$, $h_1 \to 1$, $h_2 \to 2$.

Solución C.3. API REST: `POST /hypothesis/parse` con `{ "dsl": "..." }` → `Hypothesis` JSON; `POST /run_hunt` con `{ "hypothesis": ... }` → `{ "results": [...], "cap_saturated": false }`. Errores: 400 Bad Request con detalle estructurado (`{ "error": "unknown_binding", "hint": "did you mean: c, h1?" }`).

Capítulo 33: Khop delta es difícil

Solución C.1. Hub de in-degree 10^4. Si la nueva arista agrega un tipo nuevo a u, todos los 10^4 predecesores del hub lo ven. Cada uno propaga a sus predecesores; en grafo scale-free con distribución similar, ronda 2: $10^4 \cdot 10 = 10^5$. Ronda 3: 10^6. Ronda 4: 10^7. Cascada explota.

Solución C.3. Si $\text{bits}[v] \subseteq \text{bits}[u]$: no hace falta propagar bits (monotonicidad: ya están). Pero min/max_first puede todavía mejorar: t podría ser timestamp más temprano que cualquier camino actual desde v. Hay que chequearlo separadamente.

Capítulo 34: Delta WCOJ

Solución C.1. $\Delta_u = \text{bits}[v] \setminus \text{bits}[u] = \texttt{0b001000} \setminus \texttt{0b000001} = \texttt{0b001000}$ (sólo el bit IP es nuevo). $\text{bits}[u] \models \text{bits}[v] = \texttt{0b001001}$.

Solución C.3. Inducción + monotonicidad. Base: $N = 0$, trivial. Paso: cada `delta_update` agrega exactamente los bits del walk introducido por la nueva arista. La unión de las propagaciones secuenciales = la propagación única que un rebuild haría. Monotonicidad garantiza que el orden no afecta. □

Capítulo 35: Streaming canonical

Solución C.1. $2 \cdot 10^8 \times 32$ B = 6.4 GB ahorrados.

Solución C.3. Tests "correcto": comparar resultados de `run_hunt` sobre streaming vs sobre edge_store, esperar iguales. Tests "rápido": bench tiempo de `matching_hops` sobre streaming, comparar geomean vs edge_store, gate $\leq 10\,\%$ regresión.

Capítulo 36: Plug-and-play

Solución C.1. "Cache de NLF compartido entre hipótesis similares": cumple default-on (cache se enciende solo). Surface UI: agregar `cache_hits/misses` a `PerfStatus`. Default-off: no aplica si está bien probado. Doc: actualizar `08-env-vars.tex`. Capabilities: no aplica (no es feature flag de build).

Solución C.3. Test: parsear el changelog del PR; para cada nuevo flag detectado, chequear que (a) está en la lista default-on de `lib.rs`, (b) está en `PerfStatus`, (c) está documentado en `08-env-vars.tex`. Falla CI si falta cualquiera de los tres.

Capítulo 37: Frontend instr

Solución C.1. "Build: AVX2" pildora: agregar `has_avx2: bool` a `PerfStatus` (vía `cfg!(target_feature = "avx2")`). Render: pildora gris "Build: AVX2" si `has_avx2` $\Rightarrow$ true; ocultar si false.

Solución C.3. Polling cada 5 s: balance entre responsividad (detectar pico antes de OOM) y costo (cada poll cuesta $\sim$ 1 ms en CPU + redraw). Más frecuente: overhead percibido. Menos: podemos miss un pico antes de que el usuario entre en swap.

Apéndice D

Bibliografía

Referencias citadas en el libro, agrupadas por tema. Las URLs se incluyen cuando el paper está disponible públicamente.

Joins, AGM bound, WCOJ

Atserias, Grohe, Marx (2008) *Size Bounds and Query Plans for Relational Joins*. FOCS 2008. La derivación original del AGM bound. Cap. 4.

Ngo, Porat, Ré, Rudra (2012) *Worst-case Optimal Join Algorithms*. PODS 2012. NPRR, el primer algoritmo WCOJ que alcanza el bound. Cap. 20.

Veldhuizen (2014) *Triejoin: A Simple, Worst-Case Optimal Join Algorithm*. ICDT 2014. La versión implementable de WCOJ que GraphHunter usa para K_4. Cap. 21.

Ngo, Porat, Ré, Rudra (2018) *Worst-case Optimal Join Algorithms: Techniques, Results, and Open Problems*. PODS 2018. Survey de WCOJ y resultados posteriores; lectura recomendada para profundizar. Cap. 20.

Yannakakis (1981) *Algorithms for Acyclic Database Schemes*. VLDB 1981. El paper original del semi-join reducer para joins α-acíclicos. Caps. 22, 23.

α-aciclicidad y hipergrafos

Beeri, Fagin, Maier, Yannakakis (1983) *On the Desirability of Acyclic Database Schemes*. JACM 30(3). Definición formal de las distintas variantes de aciclicidad (α, β, γ) y sus propiedades. Cap. 22.

Graham (1979) *On the Universal Relation*. Reporte técnico Univ. of Toronto. La regla GYO de eliminación. Junto con Yu-Ozsoyoglu (1979), las dos referencias del algoritmo. Cap. 22.

Subgraph matching y graph databases

Cordella, Foggia, Sansone, Vento (2004) *A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs*. IEEE Trans. PAMI 26(10). VF2: el algoritmo clásico de subgraph isomorphism con podas. Referencia para comparar con DFS bien-podado. Cap. 16.

He, Singh (2008) *Graphs-at-a-time: Query Language and Access Methods for Graph Databases*. SIGMOD 2008. Introduce el concepto de NLF como prune para subgraph matching. Cap. 18.

Kankanamge, Sahu, Mhedhbi, Salihoglu (2017) *Graphflow: An Active Graph Database.* SIGMOD 2017. La técnica de Delta WCOJ Propagation que GraphHunter adoptará en stage 2b. Cap. 34.

Sun, Wang, Wang, Shao, Li (2012) *Efficient Subgraph Matching on Billion Node Graphs.* VLDB 2012. *neighborhood label profile* (NLP) y otras heurísticas; complementa NLF. Cap. 18.

Sistemas, layout, performance

Drepper (2007) *What Every Programmer Should Know About Memory.* Reporte técnico Red Hat. La referencia canónica de cache lines, branch prediction, y jerarquía de memoria. Cap. 5.

Patterson, Hennessy *Computer Organization and Design.* MK Publishers. Libro de texto: arquitectura de CPU, pipeline, memoria virtual. Background completo si se quiere profundizar. Cap. 5.

Lea (2000) *A Memory Allocator.* Doug Lea's malloc; referencia para entender por qué allocs chicos cuestan ~ 65 ns con allocadores modernos. Cap. 11.

Threat hunting

Sqrrl Team (2017) *Hunt Evil: Your Practical Guide to Threat Hunting.* Sqrrl Data Inc. Manual operacional de threat hunting; complemento al libro *Manual* de GraphHunter. Cap. 1.

Bianchi, Costantino (2024) *A Survey on Threat Hunting Techniques.* Computer Communications. Survey reciente de las prácticas de la industria. Cap. 1.

Documentación interna de GraphHunter

`docs/arquitectura/arquitectura.pdf` *Arquitectura.* Estructura de crates, layering, ADRs, decisiones de *repo organization.* Cap. 7.

`docs/especificacion/especificacion.pdf` *Especificación.* HTTP API, MCP tools, DSL, C ABI, formatos de log, env vars. Cap. 32.

`docs/manual/manual.pdf` *Manual del operador.* Instalación, ingesta, hunt mode, exploración, integraciones.

`docs/paper/paper.pdf` *Paper interno de investigación.* Justificación académica con benchmarks y comparaciones contra el estado del arte.

`docs/perf/release-profile.md` Bisección del `[profile.release]` de Cargo (LTO, target-cpu) en el Galaxy Book4. Cap. 5.

`docs/perf/hunt-result-shape.md` Investigación del bench que motivó la migración a `Vec<Arc<str>>`. Cap. 11.

Lecturas adicionales recomendadas

Para profundizar en temas específicos:

Aberger, Lamb, Tu, Nötzli, Olukotun, Ré (2017) *EmptyHeaded: A Relational Engine for Graph Processing.* ACM TODS 42(4). Implementación de WCOJ en producción para grafos.

Khamis, Ngo, Suciu (2017) *What Do Shannon-type Inequalities, Submodular Width, and Disjunctive Datalog Have To Do With One Another?* PODS 2017. Generalización del AGM bound a queries con disjunción.

Olteanu, Závodný (2015) *Size Bounds for Factorised Representations of Query Results.* ACM TODS 40(1). Representaciones factorizadas; relevante para entender por qué Yannakakis no alcanza el AGM en general.

Apéndice E

Mapa interno del motor (referencia técnica)

GraphHunter resuelve un problema con muchos nombres: *subgraph isomorphism* sobre un grafo dirigido temporal, donde las consultas (*hipótesis*) describen un patrón estructural y el motor enumera las instancias que aparecen en los logs ingresados. La superficie del DSL es pequeña, pero detrás hay una pila de algoritmos, layouts de memoria y trucos de hot-path que se acumularon a medida que escalamos de cientos de miles a millones de aristas.

Este apéndice es un mapa interno: tres secciones cubriendo (1) los algoritmos que el motor ejecuta y cuándo elige cada uno, (2) las estructuras de memoria que hacen que el camino caliente quepa en L1, y (3) los trucos no obvios que aparecen dispersos en el código. No es un manual del DSL ni una guía de usuario; el lector objetivo es alguien que va a tocar el motor. Para una explicación pedagógica completa con ejercicios, ver los capítulos principales de este libro; este apéndice asume familiaridad con ellos.

Las citas `archivo:linea` apuntan al árbol `core/graph-engine/src/` salvo indicación contraria. Las páginas siguen al cierre del *M-track* (mayo 2026, post-eliminación de `libgraphmatch`).

E.1. Sistemas de algoritmos

El motor implementa una docena de algoritmos distintos para resolver hipótesis. La elección no la hace el usuario: un *planner* examina la forma de la hipótesis, las pistas (*markers*) de nombre y el contexto de ejecución, y rutea a la rutina más barata que es correcta para el caso. Esta sección cataloga el zoo, explica cada operador y describe el árbol de decisión.

E.1.1. Panorama y dispatch

El *planner* (`planner.rs:143-214`) decide la ruta de ejecución a partir de la hipótesis, en cuatro etapas: detectores de *name marker* (textos del estilo `k4-clique` en el nombre del paso), detectores estructurales (cantidad de pasos + homogeneidad de tipos), prueba de aciclicidad-α (Yannakakis), y *fallback* a DFS genérico. Cada etapa devuelve un `DispatchPlan` concreto si su patrón coincide; si no, cede a la siguiente.

La tabla E.1 resume los doce operadores principales con su complejidad, su archivo de origen y la condición que dispara su elección.

E.1.2. DFS genérico y *smart* top-K

El DFS genérico (`graph.rs:802-1450`, entry point `search_temporal_pattern`) es el caballo de tiro. Resuelve cualquier patrón — cadenas, árboles, ciclos cerrados con *binding* explícito — iterando una pila de *frames* sin recursión, lo que evita el costo de *call/ret* y permite reutilizar *scratch* entre llamadas.

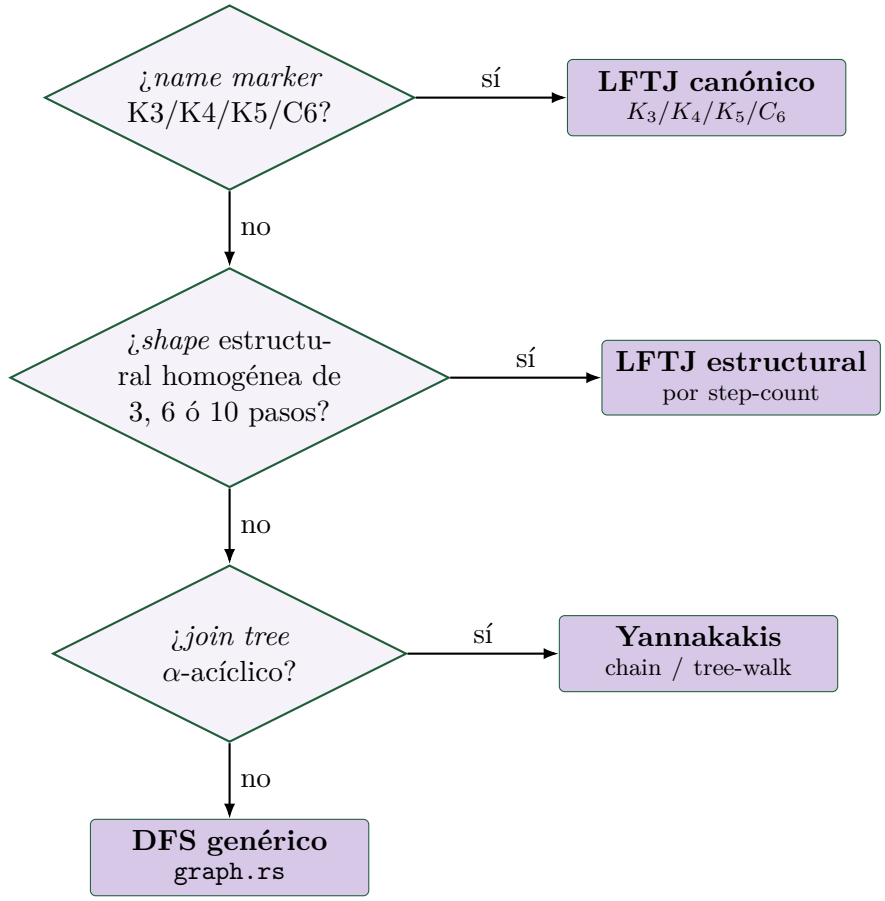


Figura E.1: Árbol de decisión del *planner*. Cada hipótesis baja por la diagonal hasta encontrar la primera ruta que coincide; el DFS genérico es el *fallback* universal. La detección de *name marker* se evalúa primero porque es $O(1)$ en strings y siempre dispara LFTJ aunque la *shape* sea ambigua.

```
struct Frame {
    sid: StrId,             // vértice actual
    step_idx: usize,        // posición en la hipótesis
    edge_lo: usize,         // ventana de aristas válidas
    edge_hi: usize,         //   (precomputada por temporal_range)
    edge_cursor: usize,     // próxima arista a probar
}
// el motor mantiene un Vec<Frame> reutilizable por sesión
```

Listing E.1: Forma simplificada del *frame* del DFS iterativo (`graph.rs`).

La variante *smart* (`graph.rs:1597-1749`) atiende a *top-K* con *anomaly score*: en lugar de devolver todos los *matches* y ordenar después, mantiene un min-heap de tamaño K y poda con una cota admisible (asume que las aristas futuras pueden puntuar 1.0, así nunca elimina un *path* que podría entrar en el *top*). Cuando el *scorer* de anomalía está finalizado, el *planner* prefiere esta ruta porque el peor caso no se degrada y el caso esperado es lineal en |OUT|.

E.1.3. Familia LFTJ canónica

Leapfrog Triejoin (Veldhuizen, ICDT 2014) es el algoritmo *worst-case-optimal join* sobre trie-joins. La idea: cada relación se ve como un trie ordenado, los *joins* se resuelven recorriendo todos los iteradores en *lockstep* con *seek* en lugar de *merge*, y la cota AGM ($|J| \leq |D|^{\rho^*}$) se alcanza con ρ^* calculado por programación lineal sobre el *fractional edge cover*. En GraphHunter implemen-

Operador	Complejidad	Trigger	Origen
DFS genérico	$O(m \cdot \deg^k)$	*fallback*	`graph.rs:802-1450`
DFS smart top-K	$O(\|\text{OUT}\| \log K)$	*scorer* finalizado	`graph.rs:1597-1749`
K_3 LFTJ	$O(m^{1,5})$ AGM	marker o 3 pasos homogéneos	`lftj.rs:345-365`
K_4 LFTJ	$O(m^2)$ AGM	marker o 6 pasos homogéneos	`lftj.rs:550-690`
K_5 LFTJ	$O(m^{2,5})$ AGM	marker o 10 pasos homogéneos	`lftj.rs:692-748`
C_6 LFTJ	$O(m^3)$ AGM	marker C6 (antes que K4)	`lftj.rs:1059-1168`
Yannakakis	$O(\|D\|^{\rho^*} + \|\text{OUT}\|)$	*shape* α-acíclica	`yannakakis.rs:1056-1137`
Semi-join reducer	$O(E)$ por pasada	dentro de Yannakakis	`semijoin.rs:45-100`
k-hop reachability (D5)	$O(4 \cdot E)$ build	precómputo, prune entrada	`khop_reach.rs:86-120`
NLF (B2)	$O(E)$ build, $O(1)$ lookup	precómputo, prune entrada	`nlf.rs:35-95`
SIMD intersect + gallop	$O(\|a\|+\|b\|)$	dentro de LFTJ	`simd_rust.rs:32-265`
Symmetry breaker (B9)	$O(n)$ análisis	detección de palíndromos	`symmetry.rs:30-100`

Tabla E.1: Operadores del motor. Las complejidades de LFTJ son cotas AGM (*worst-case optimal join*); las de DFS son sensibles al grado y se degradan rápido para $k \geq 3$.

tamos cuatro instancias dedicadas:

K_3 (triángulos). `lftj.rs:345-365`, `count_triangles_lftj_simd`. Loop externo sobre fuentes a; para cada vecino b de a, *binary search* de b en el trie para obtener sus vecinos, luego intersección $N(a) \cap N(b)$ con SIMD. La forma *canónica* (`count_triangles_canonical_lftj_simd`) impone $a < b < c$ en orden de `StrId`, eliminando las seis permutaciones de $\text{Aut}(K_3) = S_3$.

K_4 (cliques de cuatro). `lftj.rs:550-690`, paralelizado con Rayon. La cadena canónica $a < b < c < d$ se obtiene aplicando `partition_point` sobre cada intersección sucesiva: ab_isect $= N(a) \cap N(b)$, filtrar a $c > b$, luego $d > c$ en abc_isect.

```
for a in srcs {
    for &b in N(a).iter().filter(|&&b| b > a) {
        let ab = intersect(N(a), N(b));            // SIMD
        for &c in ab.iter().filter(|&&c| c > b) {
            let abc = intersect_with(ab, N(c));    // SIMD
            count += abc.iter().filter(|&&d| d > c).count();
        }
    }
}
// reduce sobre workers Rayon: count total = sum(local counts)
```

Listing E.2: Forma del *inner loop* canónico de K_4 (esquemático).

K_5. `lftj.rs:692-748`. Cuatro intersecciones encadenadas (ab, abc, abcd) y la enumeración de e sobre abcd. $\text{Aut}(K_5) = S_5$, dividir por 120.

C_6 (ciclo dirigido). `lftj.rs:1059-1168`. Distinto de K_4: aunque ambos tienen seis pasos, C_6 es un ciclo no clique. Requiere un trie *forward* (de a a f) y otro *reverse* (predecesores indexados por destino) para cerrar la última arista $f \to a$. La canónica usa *star-from-min*: a es el *vertex* de menor `StrId` en el ciclo, lo que rompe $\text{Aut}(C_6) = \mathbb{Z}/6$ (las seis rotaciones) y reduce el *output* a 1/6 de la cuenta directa — en el extremo denso del *sweep ER*, $\sim 90\times$ más rápido que el *naive*.

E.1.4. Yannakakis para α-acíclicos

Cuando la hipótesis es α-acíclica (cadenas, árboles), el *output* puede ser exponencial pero existe un algoritmo *output-sensitive*: Yannakakis (1981). Tres pasos: (1) reducir a un *join tree* con GYO (*Graham–Yu–Ozsoyoglu ear elimination*), (2) propagar *semi-join* hacia adelante y hacia atrás sobre los *bags*, (3) enumerar el *output* caminando el árbol.

En GraphHunter (`yannakakis.rs:1056-1137`, entry `search_temporal_pattern_yannakakis`) la *shape* `Hypothesis::shape` decide la rama: `Chain` usa `enumerate_chain` (DFS lineal sobre *bags* reducidas), `Graph` usa `enumerate_tree` sobre el *join tree* producido por GYO. El *semi-join* (`semijoin.rs:45-100`) usa *bitsets* densos `Vec<u64>`; *forward* elimina $v \in C_i$ sin predecesor en C_{i-1}, *backward* elimina u sin sucesor en C_{i+1}. El criterio temporal estricto $t_i < t_{i+1}$ NO se aplica en el *semi-join* (es conservador, deja pasar candidatos), sólo durante la enumeración final.

E.1.5. Filtros pre-match

Antes de invocar el operador principal, el motor aplica tres podas baratas:

NLF (B2). `nlf.rs:35-95`. *Neighborhood Label Frequency*: histograma 1-hop por vértice v, un `[u32;9]` con la cuenta de vecinos *outgoing* de cada tipo concreto. `can_match(v, requirement)` chequea $\text{row}_v[t] \geq \text{req}[t]$ para cada tipo. Descarta *outliers* obvios (un `User` sin vecinos `Process` no puede ser raíz de un *spawn chain*) en $O(1)$ por vértice tras un *build* $O(E)$.

k-hop reachability (D5). `khop_reach.rs:86-120`. Bitmap de 16 bits `bits[v]` con los tipos alcanzables desde v en a lo sumo `MAX_DEPTH=4` saltos, más matrices *sparse* `min_first[v][L]` y `max_first[v][L]` con cotas temporales. Cuatro rondas de propagación de etiquetas ($O(4 \cdot E)$ build) producen, para cada v, un resumen «¿este vértice puede llegar a un tipo T dentro de la ventana W en ≤ 4 saltos?». Permite descartar inicios antes de entrar al DFS.

Symmetry breaker (B9). `symmetry.rs:30-100`. Análisis $O(n)$ sobre los pasos: detecta hipótesis cuya secuencia de tipos y relaciones es palíndromo (caso típico: `Process \(\to \) Process \(\to \) Process` con `Spawn` en ambas direcciones). Para esos patrones, fuerza `path[0].index()` < `path[\text {last}].index()` en orden `StrId`, eliminando los *matches* duplicados generados por el automorfismo de reverso.

E.2. Mapeos de memoria

El *matching* se gana o se pierde en el *cache hit rate*. Esta sección describe las nueve estructuras de datos que tocan el *hot path*, qué decisiones de *layout* se tomaron en cada una y cómo encaja todo en la jerarquía de memoria.

E.2.1. Identificadores y string interner

`interner.rs:6-16`. `StrId` es un `u32` en un *newtype*: 4 bytes, `Copy`, `Hash`, `Ord`. El `StringInterner` mantiene un `HashMap<Arc<str>, StrId>` y un `Vec<Arc<str>>` bidireccional; cada string se interna una vez y los consumers reciben un identificador denso. Mover un `StrId` es 4 bytes en lugar de 24 (un `String` es `(ptr, cap, len)`). El *hashing* es trivial (`ahash` sobre 4 bytes vs sobre el *slice* completo). El test estático `assert_eq!(size_of::<StrId>(), 4)` en `interner.rs:237` congela el tamaño.

E.2.2. La estructura `CompactRelation`

`relation.rs:96-111` es la unidad de almacenamiento de una arista en el grafo. `\#[repr(C)]` con campos ordenados de mayor a menor alineación para colapsar el *padding*; el resultado son 32

bytes exactos, justo la mitad de una línea de cache de 64 B. El test que custodia el tamaño está en `relation.rs:186-191`: si una edición accidentalmente engorda la struct (típicamente agregando un campo desalineado), el *assert* falla y previene una regresión de $\sim 20\,\%$ en el *footprint* de un grafo grande.

`StreamEdge` (`streaming.rs:108-123`) replica el *layout* byte-by-byte para que el *matcher* pueda iterar `\&[StreamEdge]` sin traducción de campos por arista. Los 4 bytes de `source_sid` son redundantes (la *owner key* ya implica el *source*), pero el costo ($\sim$ 120 MB sobre 30M aristas) es menor que la *branchy* reescritura del *hot loop*.

E.2.3. Caches columnares (SoA)

`graph.rs:99-110`. `entity_type_tags: Vec<u8>` es un arreglo plano indexado por `StrId.index()`: en la posición i está el `u8` con el tipo de entidad del vértice cuyo `StrId` es i, o `TAG_DEAD = 253` si esa *slot* fue invalidada. Reemplaza un `HashMap<StrId, EntityType>` cuya probe costaba 100–300 ciclos en miss; ahora cada test de tipo es un *single load* desde una región contigua que el *prefetcher* de hardware sigue trivialmente. `other_type_names: HashMap<StrId, String>` acompaña por el lado para los `EntityType::Other(\_)`, que son raros y no entran al *hot path*.

NLF (`nlf.rs:34-106`) usa el mismo principio con un payload mayor: `[u32; 9]` por vértice (uno por cada tipo concreto del DSL), 36 bytes por entidad. A 28M vértices son 1 GiB; cuando el *budget* se excede, el almacenamiento cae automáticamente a un *mmap* de respaldo.

E.2.4. CSR trie y claves `u64` empaquetadas

`lftj.rs:1-72`. El trie de dos niveles para LFTJ se representa como un `Vec<u64>` ordenado, donde cada clave es (`StrId.index() as u64`) | (`bucket as u64`) $\ll 32$. Lo que en una notación más explícita sería un `(source, dest)`, comprimido a una palabra de 64 bits que se ordena trivialmente por `u64::cmp`. Las consultas son *seek* con *galloping* (`simd_rust.rs`, `gallop_ge_u32`): doble exponencial hasta cruzar el *target*, luego *binary search* en la ventana acotada. Cuando el *target* está cerca del cursor (caso típico en LFTJ, donde cada iterador avanza un poco después de su vecino), *galloping* toca $O(\log d)$ líneas de cache vs $O(\log n)$ de un *binary search* pelado.

E.2.5. Streaming, mmap y *hugepages*

`streaming.rs`. Cada vértice tiene una `NeighborList` que es `SmallVec<[StreamEdge; 8]>` en el *inline case* (grado ≤ 8, sin *heap touch*) o un `Vec<StreamEdge>` cuando supera. Para *logs* reales de Sysmon la mayoría de los procesos tienen ≤ 8 aristas *outgoing*, así que el *hot path* suele leer del *small* sin ninguna *indirection*.

Las *spills* a disco usan `mmap_advise.rs:29-51`: `MmapHandle` envuelve `memmap2::Mmap` con verificación de alineación. En Linux, si `GRAPHHUNTER_HUGEPAGES=1` está activo, `try_hugepages` llama a `madvise(\dots , MADV_HUGEPAGE)` y reduce los *TLB stalls* en un 10–30 % sobre grafos $> L2$. En Windows y macOS la rutina es un *no-op stub*: el *feature gate* `cfg(target\_os = "linux")` mantiene `libc` fuera del grafo de dependencias en hosts no-Linux.

E.2.6. `HuntResult = Vec<Arc<str>>`

`graph.rs:25-36`. Hasta abril de 2026, un `HuntResult` era `Vec<String>`: por cada *path* devuelto, el motor materializaba una `String::from(\dots )` por cada nodo. En el caso *cap-saturated* (10 000 *matches*, cada uno con tres nodos) eso son 30 000 *allocations* dominadas por `mimalloc` con un costo medible: 2,38 ms vs 1,06 ms cuando el motor escribía los *paths* en un mock que no allocaba ($-55\,\%$).

La migración (2026-04-30) cambió la *shape* a `Vec<Arc<str>>`. El *interner* ya tenía los `Arc<str>` listos; cada *step* es ahora un `Arc::clone` ($\sim$ 4 bytes de *refcount bump*). El test de propiedad ya valida que el *set* de paths producidos no cambia.

E.3. Trucos del sistema

Esta sección cataloga las decisiones que no son obvias leyendo el código de un tirón: por qué un `unsafe` aquí, una *flag* de Cargo allá, un `cfg` arquitectónico más allá. Cada *trick* viene con su cita y la ganancia que motivó el cambio.

E.3.1. Hot-path

Prefetch *lookahead* 8. `graph.rs:1696-1710`. El *inner loop* del DFS lee `entity_type_tags[edges[idx].dest` para filtrar por tipo. Como las aristas están *scattered* en el CSR, el *prefetcher* de hardware no puede anticipar el patrón. Manualmente, el motor emite un `\_mm\_prefetch::<\_MM\_HINT\_T0>` sobre `edges[idx + 8].dest_sid.index()` antes de procesar la iteración `idx`: cuando llega el turno de la línea pre-fetcheada, ya está caliente en L1 y el *load* cuesta 4 ns en lugar de 100. La constante `LOOKAHEAD = 8` es el resultado de un *sweep* (los hits se redistribuyen entre L1 y L2 a medida que sube; 8 es el *sweet spot* en Zen 3 y Skylake).

SoA en lugar de AoS. `entity_type_tags` es `Vec<u8>`, no `HashMap`. Pequeño detalle, gran efecto.

`SmallVec` *inline cap* = 8. `streaming.rs:93`. La distribución de grados es *long-tailed*: la mediana en Sysmon es ≤ 8. Con `SmallVec<[StreamEdge; 8]>`, el caso común no toca el *heap*; los grados altos siguen funcionando con un `Vec` en la rama larga.

`get_unchecked` con *safety doc*. `simd_rust.rs:59-60,126-127`. El *merge* escalar de dos *slices* ordenados puede saltar *bounds checks* porque el `while i < a.len() \&\& j < b.len()` ya los garantiza. El *unsafe block* viene con un comentario `// SAFETY:` que cita el invariante; quitarlo cuesta $\sim 5\,\%$ en el *scalar fallback*. La versión *checked* se conserva en los tests para mantener la prueba.

E.3.2. SIMD plug-and-play

`simd_rust.rs:32-265`. El motor envía cada intersección sorted-u32 por una *dispatch routine* que decide en *runtime*:

```
pub fn intersect_sorted_u32_count(a: &[u32], b: &[u32]) -> usize {
    if a.len() < 32 || b.len() < 32 {
        return scalar_intersect_count(a, b);  // setup AVX2 no paga
    }
    #[cfg(target_arch = "x86_64")]
    {
        if is_x86_feature_detected!("avx2") {
            // SAFETY: se acaba de comprobar AVX2 en runtime.
            return unsafe { avx2_intersect_count(a, b) };
        }
    }
    scalar_intersect_count(a, b)
}
```

Listing E.3: Dispatch *runtime* de la intersección. AVX2 sólo cuando lo soporta el host.

La función AVX2 está marcada con `\#[target_feature(enable = "avx2")]`: el compilador puede emitir `vpcmpeqd` y `vpmovmskb` dentro del cuerpo, pero la función queda `unsafe` (lo que es correcto: llamarla en una CPU sin AVX2 sería UB). El *wrapper* público es *safe* porque hace el *check* antes. El umbral de 32 elementos por *slice* viene de un *sweep*: por debajo, el *setup* de los registros YMM (~ 30 ciclos) domina sobre el *merge* escalar.

E.3.3. Allocator y locks

`mimalloc` como `\#[global_allocator]`. En `apps/cli/src/main.rs` y `apps/tauri/src-tauri/src/main.rs`. El *system allocator* (msvcrt en Windows, glibc en Linux) cuesta caro en *paths* con muchas *allocations* pequeñas: el DFS *scratch* se recicla, pero los parsers temporales no, y a 250 000 eventos/segundo cada *alloc* cuenta. *mimalloc* reduce $\sim$ 10–20 % ese *overhead*.

`parking_lot::RwLock`. `streaming.rs`. `std::sync::RwLock` en Windows envuelve `SRWLOCK` (kernel-mediated, $\sim$ 200 ns en *uncontended*). `parking_lot` usa *futex* + *CAS spinloop* y baja a < 30 ns. En el *matcher* cada *frame push* adquiere brevemente la `RwLock` de la *streaming snapshot*; sobre 10M *pushes* eso eran $\sim$ 300 ms de ciclos *atomic* netos. Con `parking_lot` bajan a $\sim$ 30 ms y dejan de aparecer en el *flame graph*.

E.3.4. Tricks algorítmicos

***Symmetry breaking* canónico.** Cuando $\mathrm{Aut}(P) = G$ con $|G| > 1$, elegir un representante por *orbit* y descartar los demás. Para K_n el *orbit* es S_n (el *chain* $a < b < c < \ldots$ elige el *representante*); para C_6 es $\mathbb{Z}/6$ (*star-from-min*). En el extremo denso del *sweep ER* ($V = 1000, E = 9913, C_6 = 926763$ instancias) el *naive* $O(m \cdot d^4 \cdot \log d)$ tarda 2,03 s; la canónica con *star-from-min* sobre $\mathbb{Z}/6$ tarda 22,5 ms — $\sim 90\times$.

Planner *short-circuit*. `planner.rs:143-214`. La detección de un *name marker* se hace en $O(1)$ sobre el nombre del *step*; si dispara, se evita construir un CSR trie completo y se rutea directo a la rutina LFTJ correspondiente.

Yannakakis 3-tier cache. `yannakakis.rs`. Un *batch* de *hunts* comparte estructura cuando varias hipótesis empiezan con el mismo *step* (caso ATT&CK: muchos *templates* arrancan con `User -[Auth]-> Host`). El motor mantiene tres niveles de cache:

- *Bag cache* (Tier 2): `HashMap<(StepFingerprint, TimeWindow), Arc<Bag>>`. Las *bags* de candidatos se materializan una sola vez por *batch*.
- *Result cache* (Tier 3a): *paths* resueltos cuando dos hipótesis convergen sobre la misma subcadena.
- *Prefix cache* (Tier 3b): si dos hipótesis comparten un *prefix* (los primeros k *steps*), el segundo arranca desde el estado del primero, ahorrándose k niveles de DFS.

Combinados, sobre un *batch* de 50 hipótesis ATT&CK reales producen $\sim 4\times$ *lift* sobre la ejecución secuencial.

E.3.5. Tricks de memoria

`MADV_HUGEPAGE` Linux gate. `mmap_advise.rs:29-51`. Sólo Linux, sólo cuando `GRAPHHUNTER_HUGEPAGES=1` (la *flag* no es *default-on* porque requiere kernel con THP en `always` o `madvise`). Reduce *TLB stalls* 10–30 % en grafos $> L2$. En Windows y macOS es un *stub no-op*: el `cfg` mantiene `libc` fuera del grafo de dependencias.

`Vec<Arc<str>>` *cap-saturation fix*. La *shape* de `HuntResult` cambió en 2026-04-30 y bajó 55 % sobre patrones tipo-agnósticos.

E.3.6. Build y profile

Thin LTO sin `target-cpu=native`. `core/graph-engine/Cargo.toml` `[profile.release]` y `[profile.bench]`. La bisección de abril 2026 (documentada en `docs/perf/release-profile.md`) probó tres *cells*:

- cell0 (*default*): baseline.
- cell1 (thin LTO): +4–7 % *geomean* sobre cell0.
- cell2 (thin LTO + `target-cpu=native`): *regresión* de 10–15 % en *hunt 10K* y de 25–27 % en *ingest* pequeño (Galaxy Book4 hybrid P/E: el compilador emite ops 512-bit, las E-cores hacen *downclock* y el *throughput* colapsa).

Resultado: thin LTO se queda, `target-cpu=native` se descarta. La regla derivada (registrada como *feedback* de bench-methodology) es «bisectar siempre por env vars, no por edición de archivos» para no *wedge* el *repo* en estados intermedios.

Workspace `opt-level=3` en dev. `apps/tauri/src-tauri/Cargo.toml`. Candle (el backend Phi-3) y la stack `gemm`/`half` corren «usable» sólo con `-O3`. `[profile.dev.package."*"] opt-level = 3` optimiza las dependencias; los *workspace members* (`graph_hunter_local_llm`, `graph_hunter_constrained` `graph_hunter_api`) reciben `opt-level = 3` explícito porque Cargo no los cubre con el *wildcard*. Sin esto, `tauri dev` hangs $\sim$ 9 minutos por inferencia; con esto baja a 10–20 segundos — dentro de 2× del `--release`.

Plug-and-play *default-on*. `apps/cli/src/main.rs`. `GRAPHHUNTER_LFTJ_PLANNER=1`, `GRAPHHUNTER_K4_LFTJ=` `GRAPHHUNTER_YANNAKAKIS=1` se setean al inicio de `main` (single-thread, antes de cualquier *query*). Cualquier usuario que arranque la app o el CLI se beneficia del *cost model* sin tocar *config*. Para deshabilitar, basta exportar la *flag* con `=0`: cero rebuild. La *badge* «`AVX2`» en la barra de estado (vía `simd_rust::isa_label()`) hace visible al usuario qué *path* está activo.

E.3.7. Filosofía: plug-and-play

La regla escrita es *toda mejora de performance debe ser default-on al startup, con un badge visible en la UI*. La justificación: las optimizaciones detrás de un *feature flag* desactivado son código muerto en producción, y los *benchmarks* que las miden mienten sobre el comportamiento real. La consecuencia es que cada cambio en esta sección paga su *tax*: tiene que ser estable enough para encenderse por defecto, y tiene que tener un *indicator* visible cuando está activo (*badge*, *log line* al startup, telemetría). En la práctica esto frena cambios especulativos: si un trick no se anima a salir *default-on*, no entra al árbol.

E.4. Referencias cruzadas a documentos de performance

- `docs/perf/2026-05-07-pre-libgraphmatch-removal.md` — *baseline* pre-migración del FFI C++ al DFS Rust.
- `docs/perf/2026-05-07-post-libgraphmatch-removal.md` — números post-migración (engine 19–28 % más rápido sobre *big graph*).
- `docs/perf/2026-05-07-libgraphmatch-simd-audit.md` — inventario de primitivas SIMD que el FFI C++ exponía y dónde están en Rust hoy.
- `docs/perf/latex/cambios-perf-2026-04.tex` — el M-track de abril 2026 (D5, B9, C2, D2-lite, D1, M3 canonical, D3 caching).
- `docs/perf/release-profile.md` — bisección del *profile* de release (thin LTO sí, native no).

Este apéndice consolida el contenido del libro *internals-systems* (eliminado el 2026-05-13 como parte de la consolidación a cinco PDFs ortogonales: *manual*, *especificación*, *arquitectura*, *algoritmos*, *paper*).